

EmbeddedOS: The Complete Guide

Srikanth Patchava & EmbeddedOS Contributors

April 2026

Contents

1	EmbeddedOS: The Complete Guide	20
1.1	From Bootloader to Browser, Kernel to Space	20
1.2	About the Author	20
1.3	About This Book	20
1.4	Table of Contents	21
1.4.1	Part I — Foundations	21
1.4.2	Part II — Kernel & HAL	21
1.4.3	Part III — Services	22
1.4.4	Part IV — The Ecosystem	23
1.4.5	Part V — Hardware Products	24
1.4.6	Part VI — Applications & Tools	24
1.4.7	Part VII — Advanced Topics	25
1.4.8	Appendices	26
1.5	How to Build This Book	26
1.5.1	Read Online	26
1.5.2	Build Locally (mdBook)	26
1.5.3	Generate PDF	26
1.6	Conventions Used in This Book	26
2	Chapter 1: Introduction to EmbeddedOS	28
2.1	1.1 What Is EmbeddedOS?	28
2.2	1.2 The Vision	28
2.3	1.3 The 18-Repo Ecosystem	30
2.3.1	Dependency Graph	30
2.4	1.4 Architecture at a Glance	31
2.5	1.5 Who Should Read This Book	31
2.5.1	Prerequisites	31
2.6	1.6 The 48 Product Profiles	32
2.7	1.7 Book Organization	32
2.8	1.8 Conventions Used in This Book	33
2.8.1	Return Code Convention	33
2.9	1.9 How to Build and Run Examples	33
2.10	1.10 Getting Help	33
3	Chapter 2: Getting Started	35
3.1	2.1 Prerequisites	35
3.1.1	Installation by Platform	35

3.1.2	Cross-Compilation Toolchains	36
3.2	2.2 Cloning the Repository	36
3.3	2.3 Building from Source	36
3.3.1	Host Build (No Hardware Required)	36
3.3.2	Building with a Product Profile	37
3.4	2.4 Your First Application: Blink GPIO	37
3.4.1	Source Code (<code>examples/blink-gpio/main.c</code>)	37
3.4.2	CMakeLists.txt	38
3.4.3	eos.yaml	38
3.4.4	Build and Run	38
3.4.5	How It Works on the Host	39
3.5	2.5 Quickstart: Host Build	39
3.6	2.6 Quickstart: STM32	40
3.6.1	Hardware	40
3.6.2	Build	40
3.6.3	Flash via OpenOCD	40
3.6.4	Pin Mapping (STM32 Nucleo)	40
3.6.5	Debugging with GDB	40
3.7	2.7 Quickstart: Raspberry Pi 4	41
3.7.1	Build Natively on Pi	41
3.7.2	Cross-Compile from Host	41
3.7.3	GPIO Pin Mapping (BCM)	41
3.8	2.8 Quickstart: nRF52	41
3.8.1	Hardware	41
3.8.2	Prerequisites	42
3.8.3	Build and Flash	42
3.8.4	Verify via Serial	42
3.8.5	Pin Mapping (nRF52840-DK)	42
3.9	2.9 Troubleshooting	42
3.10	2.10 Next Steps	43
4	Chapter 3: Architecture	44
4.1	3.1 Design Principles	44
4.2	3.2 Layered Architecture	45
4.2.1	Layer Responsibilities	45
4.2.2	Information Flow	45
4.3	3.3 The Build System	45
4.3.1	Core Engine (core/)	46
4.3.2	Build Backends	46
4.4	3.4 System Kinds	46
4.4.1	Linux Pipeline	47
4.4.2	RTOS Firmware Pipeline	47
4.4.3	Hybrid Pipeline	47
4.4.4	Configuration Examples	47
4.5	3.5 The Product Profile System	47
4.5.1	How Profiles Work	47
4.5.2	Profile Categories	48
4.5.3	Custom Profiles	48

4.6	3.6 The Layer System	48
4.7	3.7 Three Independent Graphs	49
4.7.1	1. Source Graph – Where Code Comes From	49
4.7.2	2. Build Graph – How Things Are Built	49
4.7.3	3. Output Graph – How Outputs Become Deployable	49
4.8	3.8 Toolchain Definitions	49
4.9	3.9 Configuration Schema Summary	50
4.10	3.10 Summary	50
5	Chapter 4: Hardware Abstraction Layer	51
5.1	4.1 Overview	51
5.2	4.2 HAL Initialization	52
5.2.1	System Utility Functions	52
5.3	4.3 GPIO Interface	53
5.3.1	Configuration Types	53
5.3.2	GPIO API Reference	53
5.3.3	GPIO Interrupt Example	54
5.4	4.4 UART Interface	55
5.4.1	Configuration	55
5.4.2	UART API Reference	55
5.4.3	UART Echo Example	55
5.5	4.5 SPI Interface	56
5.5.1	Configuration	56
5.5.2	SPI Mode Reference	56
5.5.3	SPI API Reference	56
5.5.4	SPI Sensor Read Example	57
5.6	4.6 I2C Interface	57
5.6.1	I2C API Reference	57
5.6.2	I2C Temperature Sensor Example	58
5.7	4.7 Timer Interface	59
5.7.1	Timer API Reference	59
5.7.2	Periodic Timer Example	59
5.8	4.8 Interrupt Control Interface	60
5.9	4.9 Backend Dispatch Pattern	60
5.9.1	The Backend Structure	60
5.9.2	Registering a Backend	61
5.9.3	How Dispatch Works	61
5.10	4.10 Writing a Custom Backend	62
6	Chapter 5: Extended HAL	63
6.1	5.1 Overview	63
6.1.1	Complete Peripheral List	63
6.2	5.2 Analog Peripherals	64
6.2.1	ADC — Analog-to-Digital Converter	64
6.2.2	DAC — Digital-to-Analog Converter	64
6.2.3	PWM — Pulse Width Modulation	65
6.3	5.3 Bus Interfaces	65
6.3.1	CAN — Controller Area Network	65

6.3.2	USB — Universal Serial Bus (Device Mode)	66
6.4	5.4 Networking Interfaces	66
6.4.1	WiFi	66
6.4.2	BLE — Bluetooth Low Energy	67
6.4.3	Cellular / Modem	67
6.5	5.5 Media Interfaces	67
6.5.1	Camera	67
6.5.2	Audio	68
6.5.3	Display	68
6.6	5.6 Sensor Interfaces	68
6.6.1	GNSS (GPS)	68
6.6.2	IMU — Inertial Measurement Unit	69
6.6.3	Radar / Lidar	69
6.7	5.7 System & Storage Peripherals	69
6.7.1	DMA — Direct Memory Access	69
6.7.2	Flash / EEPROM	69
6.7.3	RTC — Real-Time Clock	70
6.7.4	Watchdog Timer	70
6.8	5.8 The Extended Backend vtable	70
6.8.1	Registering an Extended Backend	70
6.8.2	Retrieving the Active Backend	71
6.9	5.9 Conditional Compilation	71
6.10	5.10 Summary	71
7	Chapter 6: RTOS Kernel	73
7.1	6.1 Overview	73
7.1.1	Kernel Configuration Defaults	74
7.1.2	Return Codes	74
7.2	6.2 Kernel Lifecycle	74
7.2.1	Kernel API	75
7.3	6.3 Task Management	75
7.3.1	Task States	75
7.3.2	Task API Reference	75
7.3.3	Example: Multiple Tasks	76
7.3.4	Priority Guidelines	77
7.4	6.4 Mutex	77
7.4.1	Mutex API	77
7.4.2	Example: Protected Shared Counter	78
7.5	6.5 Semaphore	78
7.5.1	Semaphore API	79
7.5.2	Example: Producer-Consumer with Semaphore	79
7.6	6.6 Message Queues	80
7.6.1	Queue API	80
7.6.2	Example: Sensor Data Pipeline	81
7.7	6.7 Software Timers	82
7.7.1	Timer API	82
7.7.2	Example: Watchdog Feed Timer	82
7.8	6.8 Common Patterns	83

7.8.1	Pattern: Timeout with Retry	83
7.8.2	Pattern: Event Loop	83
7.9	6.9 Summary	83
8	Chapter 7: Multicore	85
8.1	7.1 Overview	85
8.1.1	Key Constants	85
8.2	7.2 SMP vs. AMP	85
8.3	7.3 Core Management	86
8.3.1	Core States	86
8.3.2	Core API	86
8.3.3	Example: Booting a Secondary Core	87
8.3.4	Architecture-Specific Core ID Detection	87
8.4	7.4 Spinlocks	88
8.4.1	Spinlock API	88
8.4.2	Architecture-Specific Spin Hints	88
8.4.3	Implementation Detail	89
8.4.4	IRQ-Safe Spinlocks	89
8.5	7.5 Inter-Processor Interrupts (IPI)	89
8.5.1	IPI Types	90
8.5.2	IPI API	90
8.5.3	Example: Cross-Core Function Call	90
8.6	7.6 Shared Memory	90
8.6.1	Shared Memory API	90
8.6.2	Example: Inter-Core Data Sharing	91
8.6.3	Cache Operations by Architecture	91
8.7	7.7 Task Core Affinity	91
8.7.1	Affinity Macros	92
8.8	7.8 Remote Processor Control	92
8.8.1	Remote Processor API	92
8.8.2	Example: AMP Communication (Cortex-A Cortex-M)	92
8.9	7.9 Memory Barriers and Atomics	93
8.9.1	Memory Barriers	93
8.9.2	Barrier Implementation by Architecture	93
8.9.3	Atomic Operations	93
8.9.4	Example: Lock-Free Counter	93
8.10	7.10 Summary	94
9	Chapter 8: Driver Framework	95
9.1	8.1 Overview	95
9.2	8.2 Driver Architecture	95
9.3	8.3 Driver Classes	96
9.4	8.4 Driver Lifecycle	96
9.4.1	State Definitions	97
9.5	8.5 The Driver Structure	97
9.5.1	Match Info Structure	98
9.6	8.6 Driver Registry API	98
9.6.1	Registration and Lookup	98

9.6.2	Lifecycle Operations	98
9.6.3	Introspection	99
9.7	8.7 Writing a Driver: Complete Example	99
9.8	8.8 Power Management	101
9.8.1	System-Wide Suspend/Resume	101
9.8.2	Suspend/Resume Order	102
9.9	8.9 Device Tree Integration	102
9.9.1	Device Tree Structure	102
9.9.2	Key Constants	102
9.9.3	Device Tree API	103
9.9.4	Example: Parsing a Device Tree	103
9.9.5	Example Device Tree Source	104
9.10	8.10 Driver-Device Tree Integration	104
9.11	8.11 The Unified Driver Framework	105
9.11.1	Driver States (Framework)	105
9.11.2	Framework API	105
9.11.3	Framework Lifecycle	106
9.12	8.12 Best Practices	106
9.12.1	Driver Development Guidelines	106
9.12.2	Common Patterns	106
9.13	8.13 Summary	107
10	Chapter 9: Cryptographic Services	108
10.1	9.1 Overview	108
10.2	9.2 Hash Functions — SHA-256	109
10.2.1	9.2.1 Constants and Context	109
10.2.2	9.2.2 Streaming API	109
10.2.3	9.2.3 Convenience Helpers	110
10.3	9.3 Hash Functions — SHA-512	110
10.3.1	9.3.1 SHA-256 vs SHA-512 Comparison	110
10.4	9.4 Cyclic Redundancy Checks — CRC-32 / CRC-64	111
10.4.1	9.4.1 API	111
10.4.2	9.4.2 When to Use CRC vs SHA	111
10.5	9.5 Symmetric Encryption — AES	111
10.5.1	9.5.1 Constants	111
10.5.2	9.5.2 Key Expansion	112
10.5.3	9.5.3 ECB Mode (Single Block)	112
10.5.4	9.5.4 CBC Mode (Chained Blocks)	112
10.6	9.6 Asymmetric Cryptography — RSA	113
10.6.1	9.6.1 Key Structure	113
10.6.2	9.6.2 Sign and Verify	113
10.7	9.7 Elliptic Curve Cryptography — ECDSA P-256	113
10.7.1	9.7.1 Key Structure	114
10.7.2	9.7.2 Sign and Verify	114
10.7.3	9.7.3 RSA vs ECC Comparison	114
10.8	9.8 Algorithm Selection Guide	114
10.9	9.9 Complete Example: Firmware Integrity Pipeline	115
10.10	9.10 Security Considerations	115

10.119.11	API Reference Summary	116
11	Chapter 10: Security Services	117
11.1	10.1 Overview	117
11.2	10.2 Secure Boot	117
11.2.1	10.2.1 Boot Status States	117
11.2.2	10.2.2 Secure Boot Structure	118
11.2.3	10.2.3 Verification Workflow	118
11.2.4	10.2.4 Boot Verification Flow	118
11.3	10.3 Firmware Signing	119
11.3.1	10.3.1 Signed Firmware Structure	119
11.3.2	10.3.2 Signing a Firmware Image	119
11.3.3	10.3.3 Verifying a Signed Firmware	120
11.4	10.4 Key Management (Keystore)	120
11.4.1	10.4.1 Supported Key Types	120
11.4.2	10.4.2 Key Entry and Store	120
11.4.3	10.4.3 Keystore Operations	121
11.4.4	10.4.4 Keystore Limits	121
11.5	10.5 Access Control (ACL)	121
11.5.1	10.5.1 ACL Rule Model	122
11.5.2	10.5.2 ACL Configuration	122
11.5.3	10.5.3 ACL Limits	123
11.6	10.6 Audit and Integrity	123
11.7	10.7 Security Architecture — Putting It All Together	123
11.8	10.8 Best Practices	124
11.9	10.9 API Reference Summary	124
12	Chapter 11: Networking	125
12.1	11.1 Overview	125
12.2	11.2 Socket API	125
12.2.1	11.2.1 Core Types	125
12.2.2	11.2.2 Initialization	126
12.2.3	11.2.3 TCP Client Example	126
12.2.4	11.2.4 TCP Server Example	127
12.2.5	11.2.5 UDP Communication	127
12.2.6	11.2.6 Socket API Reference	128
12.3	11.3 HTTP Client	128
12.3.1	11.3.1 Response Structure	128
12.3.2	11.3.2 GET Request	128
12.3.3	11.3.3 POST Request	129
12.4	11.4 MQTT Client	129
12.4.1	11.4.1 Configuration	129
12.4.2	11.4.2 Publish / Subscribe Example	129
12.4.3	11.4.3 MQTT QoS Levels	130
12.4.4	11.4.4 MQTT API Reference	130
12.5	11.5 mDNS Service Discovery	131
12.5.1	11.5.1 Register a Service	131
12.5.2	11.5.2 Discover a Service	131

12.6	11.6 TLS Integration	131
12.6.1	11.6.1 Enabling TLS	132
12.7	11.7 Network Architecture Patterns	132
12.7.1	11.7.1 Sensor Gateway	132
12.8	11.8 Error Handling	132
12.9	11.9 Complete API Reference	133
13	Chapter 12: Filesystem	134
13.1	12.1 Overview	134
13.2	12.2 Filesystem Types	134
13.3	12.3 Configuration	135
13.3.1	12.3.1 Initialization Example	135
13.4	12.4 Open Flags	135
13.5	12.5 File Operations	136
13.5.1	12.5.1 Write and Read	136
13.5.2	12.5.2 Seek and Tell	136
13.5.3	12.5.3 Truncate	136
13.6	12.6 Directory Operations	137
13.6.1	12.6.1 Directory Entry Structure	137
13.7	12.7 Path Operations	137
13.8	12.8 Filesystem Statistics	138
13.9	12.9 System Limits	138
13.10	12.10 Configuration Persistence Pattern	138
13.11	12.11 Filesystem Type Comparison	139
13.12	12.12 Data Logging Example	139
13.13	12.13 API Reference Summary	140
14	Chapter 13: OTA Updates	141
14.1	13.1 Overview	141
14.2	13.2 OTA State Machine	141
14.3	13.3 A/B Slot Management	142
14.3.1	13.3.1 Slot Operations	142
14.4	13.4 Update Source Configuration	143
14.5	13.5 Update Status	143
14.6	13.6 Complete OTA Workflow	143
14.6.1	13.6.1 Check for Updates	143
14.6.2	13.6.2 Download and Apply	144
14.6.3	13.6.3 Progress Monitoring	145
14.6.4	13.6.4 Status Query	145
14.7	13.7 Rollback	145
14.7.1	13.7.1 Rollback Safety Flow	146
14.8	13.8 OTA with MQTT Trigger	146
14.9	13.9 Security Considerations	146
14.10	13.10 API Reference Summary	147
15	Chapter 14: Sensor & Motor Frameworks	148
15.1	14.1 Overview	148

16 Part I: Sensor Framework	149
16.1 14.2 Sensor Types	149
16.2 14.3 Filter Types	149
16.3 14.4 Sensor Registration	150
16.3.1 14.4.1 Registration Example	150
16.4 14.5 Reading Sensor Data	151
16.4.1 14.5.1 Raw vs Filtered Reads	151
16.5 14.6 Calibration	151
16.5.1 14.6.1 Manual Calibration	152
16.5.2 14.6.2 Auto-Calibration	152
16.6 14.7 Runtime Filter Changes	152
17 Part II: Motor Control Framework	153
17.1 14.8 Motor Control Architecture	153
17.2 14.9 PID Parameters	153
17.3 14.10 Motor Controller Configuration	153
17.3.1 14.10.1 Configuration Example	154
17.4 14.11 Speed Control	154
17.5 14.12 Position Control	154
17.6 14.13 Trajectory Planning	154
17.6.1 14.13.1 Trajectory Example	155
17.7 14.14 Motor Status	155
17.8 14.15 Emergency Stop and Safety	156
17.9 14.16 PID Tuning at Runtime	156
17.10 14.17 Control Loop Integration	156
17.11 14.18 Combined Sensor + Motor Example	157
17.12 14.19 API Reference Summary	158
17.12.1 Sensor API	158
17.12.2 Motor Control API	158
18 Chapter 15: Compatibility Layers	159
18.1 15.1 Overview	159
19 Part I: POSIX Compatibility	160
19.1 15.2 POSIX Threads (pthreads)	160
19.1.1 15.2.1 Thread Attributes	160
19.1.2 15.2.2 Thread Creation and Join	160
19.1.3 15.2.3 Mapping Table: pthreads to EoS Kernel	161
19.2 15.3 POSIX File I/O	161
19.2.1 15.3.1 File Descriptor Model	161
19.2.2 15.3.2 I/O Operations	161
19.2.3 15.3.3 Open Flags	162
19.3 15.4 POSIX IPC	162
19.3.1 15.4.1 Message Queues	162
19.3.2 15.4.2 Pipes	163
19.3.3 15.4.3 Shared Memory	163
20 Part II: VxWorks Compatibility	164

20.1	15.5 VxWorks Task API	164
20.1.1	15.5.1 Task Spawning	164
20.1.2	15.5.2 Task Operations	165
20.1.3	15.5.3 Mapping Table: VxWorks to EoS Kernel	165
20.2	15.6 VxWorks Semaphores	165
20.2.1	15.6.1 Semaphore Types	166
21	Part III: Linux IPC Compatibility	167
21.1	15.7 AF_UNIX Socket Emulation	167
21.1.1	15.7.1 Socket Address	167
21.1.2	15.7.2 Stream Socket Example	167
21.2	15.8 eventfd Emulation	168
21.3	15.9 epoll Emulation	168
21.3.1	15.9.1 epoll Events	169
21.4	15.10 Pipe Emulation	169
21.5	15.11 Configuration Constants	169
21.6	15.12 Porting Strategy	170
21.7	15.13 API Reference Summary	170
21.7.1	POSIX APIs	170
21.7.2	VxWorks APIs	171
21.7.3	Linux IPC APIs	171
22	Chapter 16: UI Framework	173
22.1	16.1 Overview	173
22.2	16.2 Configuration	173
22.2.1	16.2.1 Framebuffer Modes	173
22.2.2	16.2.2 Themes	174
22.2.3	16.2.3 Display Rotation	174
22.2.4	16.2.4 Configuration Structure	174
22.3	16.3 Initialization	174
22.3.1	16.3.1 Basic Setup	174
22.3.2	16.3.2 Main Loop Integration	175
22.3.3	16.3.3 RTOS Task Integration	175
22.4	16.4 Display and Input Accessors	176
22.5	16.5 Theme Switching	176
22.6	16.6 Gesture Recognition	176
22.6.1	16.6.1 Gesture Types	176
22.6.2	16.6.2 Gesture Event Structure	176
22.6.3	16.6.3 Registering Gesture Callbacks	177
22.6.4	16.6.4 Gesture Configuration	178
22.7	16.7 Display Rendering Pipeline	178
22.8	16.8 Internal Driver Callbacks	179
22.9	16.9 LVGL Widget Examples	179
22.9.1	16.9.1 Button with Label	179
22.9.2	16.9.2 Sensor Dashboard	179
22.10	16.10 Memory Considerations	180
22.10.1	16.10.1 Choosing Framebuffer Mode	180
22.11	16.11 Complete Application Example	180

22.12	16.12	API Reference Summary	182
23		Chapter 17: eBoot — The EoS Bootloader	183
23.1	17.1	Introduction	183
23.2	17.2	Architecture Overview	183
	23.2.1	Stage-0: Minimal Hardware Initialization	184
	23.2.2	Stage-1: Full Boot Logic	184
23.3	17.3	Key Features	184
23.4	17.4	Building eBoot	184
	23.4.1	Native Build (Host Testing)	184
	23.4.2	Cross-Compilation for STM32F4	185
	23.4.3	Flashing with eFlash	185
23.5	17.5	A/B Slot Management	185
	23.5.1	Slot State Machine	185
	23.5.2	Boot Policy Rules	185
	23.5.3	Configuration Storage	186
23.6	17.6	Secure Boot Chain	186
	23.6.1	Hash Verification Example	186
23.7	17.7	Firmware Update Pipeline	186
	23.7.1	Update Flow	186
23.8	17.8	Board Port Structure	187
23.9	17.9	Integration with EoS and ebuild	187
23.10	17.10	Debugging and Diagnostics	188
	23.10.1	Boot Log	188
	23.10.2	Failure Recovery	188
23.11	17.11	Testing	188
23.12	17.12	Summary	189
24		Chapter 18: ebuild — The EoS Build System	190
24.1	18.1	Introduction	190
24.2	18.2	Architecture	190
	24.2.1	Supported Input Formats	190
	24.2.2	Generated Configuration Files	191
24.3	18.3	Installation	191
24.4	18.4	Hands-On Examples	191
	24.4.1	Hello World	191
	24.4.2	Multi-Target Build	192
	24.4.3	Hardware Analysis	192
24.5	18.5	The Analyze Pipeline	192
	24.5.1	Step 1: Component Extraction	192
	24.5.2	Step 2: Component Classification	192
	24.5.3	Step 3: Configuration Generation	193
24.6	18.6	Build System Configuration	193
	24.6.1	Build Commands	193
24.7	18.7	Cross-Compilation Support	194
24.8	18.8	Integration with EoS Ecosystem	194
	24.8.1	Typical Workflow	194
24.9	18.9	SDK Generation	194

24.10	18.10	Continuous Integration	195
24.11	18.11	Summary	195
25		Chapter 19: eIPC — Embedded Inter-Process Communication	196
25.1	19.1	Introduction	196
25.2	19.2	Architecture	196
	25.2.1	Core Components	197
	25.2.2	Security Components	197
25.3	19.3	Key Features	197
25.4	19.4	Transport Layer	197
25.5	19.5	The Policy Engine	198
25.6	19.6	Server Implementation	198
	25.6.1	Initialization Flow	198
	25.6.2	EAI Backend Integration	199
	25.6.3	Message Types	199
25.7	19.7	Protocol Format	199
	25.7.1	Protocol Versioning	200
25.8	19.8	Security Model	200
	25.8.1	Authentication	200
	25.8.2	Authorization	200
	25.8.3	Audit Trail	200
25.9	19.9	Health Monitoring	200
25.10	19.10	Deployment Patterns	201
	25.10.1	Pattern 1: Local Bridge (Most Common)	201
	25.10.2	Pattern 2: Distributed	201
25.11	19.11	Building and Testing	201
25.12	19.12	Summary	202
26		Chapter 20: eAI — Embedded AI Layer	203
26.1	20.1	Introduction	203
26.2	20.2	Design Principles	203
26.3	20.3	Two-Tier Architecture	203
	26.3.1	EAI-Min — Lightweight Runtime	204
	26.3.2	EAI-Full — Complete AI Platform	204
26.4	20.4	Inference Engine	204
	26.4.1	Supported Backends	204
	26.4.2	Quantization Pipeline	205
26.5	20.5	Agent Framework	205
	26.5.1	Tool Integration	205
	26.5.2	Agent Configuration	206
26.6	20.6	Security Architecture	206
26.7	20.7	Power-Aware Scheduling	206
	26.7.1	Power Modes	207
26.8	20.8	Integration with EoS	207
26.9	20.9	Model Management	207
	26.9.1	Model Formats	207
	26.9.2	Over-the-Air Model Updates	208
26.10	20.10	Building eAI	208

26.1120.11 Compliance	208
26.1220.12 Summary	209
27 Chapter 21: eNI — Embedded Neural Interface	210
27.1 21.1 Introduction	210
27.2 21.2 Architecture	210
27.2.1 Providers	211
27.3 21.3 Core Processing Pipeline	211
27.3.1 Stage 1: Signal Acquisition	211
27.3.2 Stage 2: Digital Signal Processing	211
27.3.3 EEG Frequency Bands	211
27.3.4 Stage 3: Feature Extraction	211
27.3.5 Stage 4: Classification	212
27.4 21.4 Python SDK	212
27.4.1 ENIProvider Class	212
27.4.2 Key SDK Constants	212
27.4.3 Library Loading	213
27.5 21.5 Native C API	213
27.6 21.6 EIPC Integration	214
27.7 21.7 Build Configurations	214
27.8 21.8 Applications	214
27.9 21.9 Simulator Mode	215
27.1021.10 Summary	215
28 Chapter 22: EoSim — Multi-Architecture Simulation Platform	216
28.1 22.1 Introduction	216
28.2 22.2 Supported Platforms	216
28.2.1 ARM Cortex-M (MCU)	216
28.2.2 ARM Cortex-A (Application Processors)	217
28.2.3 RISC-V	217
28.2.4 Other Architectures	217
28.3 22.3 Simulation Engines	217
28.3.1 Engine Selection	217
28.4 22.4 CLI Reference	218
28.5 22.5 Platform Definition Format	218
28.6 22.6 GUI Dashboard	219
28.6.1 GUI Panels	219
28.7 22.7 CI Integration	220
28.8 22.8 Cluster Simulation	220
28.9 22.9 Specialty Simulators	220
28.1022.10 Architecture	221
28.1122.11 Summary	221
29 Chapter 23: EoStudio — Cross-Platform Design Suite	222
29.1 23.1 Introduction	222
29.2 23.2 The Twelve Editors	222
29.3 23.3 Quick Start	223
29.4 23.4 Code Generation	223

29.4.1	Code Generation Example	223
29.5	23.5 AI and LLM Integration	224
29.5.1	LLM Backend Configuration	224
29.6	23.6 Project Templates	224
29.7	23.7 Architecture	224
29.8	23.8 Plugin System	225
29.9	23.9 Platform Support	225
29.10	23.10 Testing	226
29.11	23.11 Summary	226
30	Chapter 24: eRadar360 — Advanced Driver Awareness System	227
30.1	24.1 Introduction	227
30.2	24.2 Key Specifications	227
30.3	24.3 System Architecture	228
30.3.1	RK3588S Main Processor	228
30.3.2	AWR2944 Radar SoCs (x2)	228
30.3.3	STM32H7B3 Co-Processor	228
30.4	24.4 Why SIW Phased Array?	228
30.5	24.5 Radar Performance	228
30.6	24.6 V2X Communication	229
30.7	24.7 AI False-Alert Suppression	229
30.8	24.8 Laser Detection	229
30.9	24.9 OBD-II Integration	229
30.10	24.10 Electrical Specifications	230
30.10.1	Power	230
30.10.2	Internal Rails	230
30.11	24.11 PCB Design	230
30.11.1	Design Files	230
30.12	24.12 Environmental and Regulatory	230
30.13	24.13 Summary	231
31	Chapter 25: eHealth365 — Wearable Health Monitoring	232
31.1	25.1 Introduction	232
31.2	25.2 System Architecture	232
31.3	25.3 The Two Devices	233
31.3.1	Smart Ring Pro — Vitality Hub	233
31.3.2	Smart Patch Pro — Chemistry Hub	233
31.4	25.4 Health Metrics Coverage	233
31.5	25.5 Hardware Design — Smart Ring Pro	234
31.5.1	Key Components	234
31.5.2	Power Budget	234
31.6	25.6 Hardware Design — Smart Patch Pro	235
31.6.1	Key Components	235
31.6.2	Monthly Blood Cartridge	235
31.7	25.7 Mobile App Architecture	235
31.8	25.8 Sensing Strategy — The Life Flow Pattern	235
31.9	25.9 Data Architecture	236
31.10	25.10 Pricing and Go-To-Market	236

31.10.1	Phased Rollout	236
31.1125.11	EoS Integration	236
31.1225.12	Summary	237
32	Chapter 26: ePAM — Personal Air Mobility	238
32.1	26.1 Introduction	238
32.2	26.2 The Four Vehicles	238
32.3	26.3 Power Architecture	238
32.3.1	The Water/Solar Energy Loop	238
32.4	26.4 Eco Car — Mass Market	239
32.4.1	Manufacturing Cost Breakdown (\$19K-\$32K per unit)	239
32.5	26.5 Urban Drone — City Commuter	239
32.5.1	Key Specifications	239
32.6	26.6 Space Shuttle — Tourism	240
32.6.1	Key Specifications	240
32.7	26.7 Combo Unit — Premium Fleet	240
32.8	26.8 Business Model	240
32.9	26.9 Go-To-Market Roadmap	240
32.10	26.10 Regulatory Requirements	241
32.11	26.11 EoS Integration	241
32.12	26.12 Summary	241
33	Chapter 27: eApps — Unified Marketplace and App Store	243
33.1	27.1 Introduction	243
33.2	27.2 App Categories	243
33.3	27.3 Native App Architecture	244
33.3.1	Building Native Apps	244
33.3.2	Cross-Compilation	244
33.4	27.4 Featured Native Apps	244
33.5	27.5 Mobile Apps (Flutter)	245
33.6	27.6 Web Apps (PWA)	245
33.7	27.7 Browser Extensions	245
33.8	27.8 Developer Tools	246
33.9	27.9 CLI Tools	246
33.10	27.10 Enterprise Deployment	246
33.11	27.11 Live App Store	246
33.12	27.12 CI/CD Pipeline	247
33.13	27.13 Summary	247
34	Chapter 28: eDB — Unified Multi-Model Database	248
34.1	28.1 Introduction	248
34.2	28.2 Three Data Models in One	248
34.2.1	Usage Example	248
34.3	28.3 Architecture	249
34.4	28.4 Quick Start	249
34.4.1	Backend	249
34.4.2	Frontend	250
34.4.3	Browser Standalone	250

34.4.4	Interactive Shell	250
34.5	28.5 Security Features	250
34.5.1	Required Environment Variables	250
34.6	28.6 eBot AI Query Assistant	250
34.7	28.7 REST API	251
34.8	28.8 React Frontend	251
34.9	28.9 Configuration	251
34.10	28.10 Roadmap	252
34.11	28.11 Summary	252
35	Chapter 29: eBrowser — Lightweight Embedded Web Browser	253
35.1	29.1 Introduction	253
35.2	29.2 Quick Start	253
35.3	29.3 Features	254
35.4	29.4 Architecture	254
35.5	29.5 Building from Source	254
35.6	29.6 Embedded Deployment	255
35.6.1	Memory Requirements	255
35.7	29.7 Configuration	255
35.8	29.8 Platform Support	256
35.9	29.9 Summary	256
36	Chapter 30: eOffice — AI-Powered Office Productivity Suite	257
36.1	30.1 Introduction	257
36.2	30.2 The Twelve Applications	257
36.3	30.3 Quick Start	258
36.4	30.4 AI Integration (eBot)	258
36.4.1	Document AI Actions	258
36.4.2	Spreadsheet AI Actions	258
36.4.3	Email AI Actions	259
36.5	30.5 Architecture	259
36.6	30.6 Platform Support	259
36.7	30.7 File Formats	260
36.8	30.8 Security	260
36.9	30.9 Testing	260
36.10	30.10 Summary	261
37	Chapter 31: eVera — Voice-First Multi-Agent AI Assistant	262
37.1	31.1 Introduction	262
37.2	31.2 Multi-Agent Architecture	262
37.2.1	Agent Categories	262
37.3	31.3 Voice Interface	263
37.3.1	Speech Backends	263
37.4	31.4 Tool System	263
37.5	31.5 Quick Start	264
37.6	31.6 Automation Rules	264
37.7	31.7 Platform Support	264
37.8	31.8 Security	265

37.9	31.9 LLM Backends	265
37.1031.10	Summary	265
38	Chapter 32: eStocks — Algorithmic Trading System	267
38.1	32.1 Introduction	267
38.2	32.2 Quick Start	267
38.3	32.3 The 15 Trading Strategies	267
38.4	32.4 Seven-Layer Risk Management	268
38.5	32.5 Production Safety Controls	268
38.6	32.6 Platform Support	269
38.7	32.7 Architecture	269
38.8	32.8 Backtesting	269
38.9	32.9 ML and AI Strategies	270
	38.9.1 LSTM Deep Learning (Strategy 9)	270
	38.9.2 PPO Reinforcement Learning (Strategy 10)	270
	38.9.3 Self-Learning Ensemble (Strategy 11)	270
	38.9.4 Sentiment Analysis (Strategy 12)	270
38.1032.10	CI/CD	270
38.1132.11	Summary	270
39	Chapter 33: Cross-Compilation for EoS	271
39.1	33.1 Introduction	271
39.2	33.2 Supported Toolchains	271
39.3	33.3 Toolchain File Anatomy	271
	39.3.1 Key Elements	272
39.4	33.4 AArch64 Linux Toolchain	272
39.5	33.5 RISC-V Toolchain	273
39.6	33.6 Cross-Compilation Workflow	273
	39.6.1 Step 1: Install the Toolchain	273
	39.6.2 Step 2: Configure with Toolchain File	273
	39.6.3 Step 3: Build	274
	39.6.4 Step 4: Flash	274
39.7	33.7 Multi-Architecture CI	274
39.8	33.8 Linker Scripts	274
39.9	33.9 Binary Size Optimization	275
39.1033.10	Debugging Cross-Compiled Firmware	275
39.1133.11	Summary	276
40	Chapter 34: Testing and Quality Assurance	277
40.1	34.1 Introduction	277
40.2	34.2 Test Infrastructure	277
40.3	34.3 Test Framework	278
40.4	34.4 HAL Testing with Mock Backends	278
40.5	34.5 Kernel Testing	279
40.6	34.6 Fuzz Testing	280
40.7	34.7 Running Tests	280
40.8	34.8 CI Pipeline	281
	40.8.1 Nightly Regression	281

40.9	34.9 Code Quality Standards	281
40.10	34.10 Test Coverage Goals	281
40.11	34.11 Summary	282
41	Chapter 35: Safety and Compliance	283
41.1	35.1 Introduction	283
41.2	35.2 Applicable Standards	283
41.3	35.3 IEC 61508 — Industrial Functional Safety	283
41.3.1	SIL Levels and EoS	284
41.4	35.4 ISO 26262 — Automotive Functional Safety	284
41.4.1	ASIL Levels and EoS Product Profiles	284
41.5	35.5 DO-178C — Aerospace Software Assurance	284
41.5.1	Design Assurance Levels	285
41.6	35.6 Safety-Critical Engineering Practices	285
41.6.1	Memory Safety	285
41.6.2	Deterministic Behavior	285
41.6.3	Defensive Programming	285
41.6.4	Redundancy	286
41.7	35.7 Coding Standards	286
41.7.1	Key Rules	286
41.8	35.8 Traceability	286
41.9	35.9 Configuration Management	286
41.10	35.10 Summary	287
42	Chapter 36: Contributing to EoS	288
42.1	36.1 Introduction	288
42.2	36.2 Prerequisites	288
42.3	36.3 Development Setup	288
42.4	36.4 Contribution Workflow	289
42.5	36.5 Code Guidelines	289
42.5.1	C Style	289
42.5.2	Platform Guards	289
42.5.3	Portability Rules	289
42.6	36.6 Commit Messages	290
42.7	36.7 CI Requirements	290
42.7.1	How to Verify Locally	290
42.8	36.8 Adding New Features	290
42.8.1	New HAL Peripheral	290
42.8.2	New Product Profile	290
42.8.3	New Service	291
42.9	36.9 Pull Request Checklist	291
42.10	36.10 Project Structure	291
42.11	36.11 Reporting Issues	291
42.12	36.12 License	292
42.13	36.13 Summary	292
43	Appendix A: API Quick Reference	293
43.1	Top 50 EoS API Functions	293

43.2	Return Codes	294
44	Appendix B: Product Profiles	295
44.1	All EoS Product Profiles	295
44.2	Usage	296
45	Appendix C: Pin Reference Tables	297
45.1	STM32F407VGT6 (LQFP-100)	297
45.1.1	Common Peripheral Pin Assignments	297
45.1.2	Memory Map	297
45.2	nRF52840 (QFN-73)	298
45.2.1	Common Peripheral Pin Assignments	298
45.2.2	Memory Map	298
45.3	Raspberry Pi 4 (BCM2711)	299
45.3.1	GPIO Header (40-pin)	299
45.3.2	Power Pins	299
46	Appendix D: Glossary	300
47	Appendix E: Bibliography and Standards References	302
47.1	Safety and Functional Safety	302
47.2	Systems and Software Engineering	302
47.3	Software Quality	303
47.4	Security	303
47.5	Coding Standards	303
47.6	Communication and Connectivity	303
47.7	Embedded and Real-Time	303
47.8	Supply Chain and Compliance	304
47.9	Recommended Reading	304

Chapter 1

EmbeddedOS: The Complete Guide

1.1 From Bootloader to Browser, Kernel to Space

Author: Srikanth Patchava & EmbeddedOS Contributors **Version:** 1.0 (April 2026) **License:** CC BY-SA 4.0

“One unified platform from bootloader to browser, from microcontroller to orbit.”

1.2 About the Author

Srikanth Patchava is the creator and lead architect of EmbeddedOS, an open-source embedded operating system ecosystem spanning 18 repositories. With deep expertise in embedded systems, real-time operating systems, hardware design, AI inference, and full-stack development, Srikanth designed EoS to be a universal platform supporting 41 product categories across automotive, aerospace, medical, consumer, and industrial domains.

- GitHub: github.com/embeddedos-org
 - Website: embeddedos-org.github.io
-

1.3 About This Book

This book is the official, comprehensive reference for the entire EmbeddedOS ecosystem. It covers:

- **EoS** — the core embedded operating system (kernel, HAL, services, drivers)
- **eBoot** — the secure bootloader
- **ebuild** — the build system
- **eIPC** — inter-process communication
- **eAI** — embedded AI/ML inference
- **eNI** — neural interface & BCI
- **eApps** — application marketplace

- **EoSim** — multi-architecture simulation
- **EoStudio** — cross-platform design suite
- **eDB** — unified database
- **eBrowser** — embedded web browser
- **eOffice** — office suite
- **eVera** — AI virtual assistant
- **eStocks** — algorithmic trading
- **eHealth365** — wearable health monitoring hardware
- **ePAM** — personal air mobility hardware
- **eRadar360** — automotive radar hardware

Whether you are building a smartwatch, a medical device, an autonomous drone, or a space vehicle, this book provides everything you need.

1.4 Table of Contents

1.4.1 Part I — Foundations

1. [Introduction to EmbeddedOS](#)
 - What is EmbeddedOS?
 - The 18-repository ecosystem
 - Architecture overview
 - Who should read this book
2. [Getting Started](#)
 - Prerequisites and tool installation
 - Building EoS from source
 - Your first “Hello World” on host, STM32, RPi4, nRF52
 - Understanding product profiles
3. [Architecture & Design Philosophy](#)
 - Layered architecture (HAL → Kernel → Services → Apps)
 - The product profile system (41 profiles)
 - Cross-platform portability strategy
 - Standards compliance (ISO 26262, DO-178C, IEC 61508)

1.4.2 Part II — Kernel & HAL

4. [Hardware Abstraction Layer](#)
 - The 33 peripheral interfaces
 - GPIO, UART, SPI, I2C, Timer
 - Backend dispatch pattern
 - Writing a custom HAL backend
5. [Extended HAL — Advanced Peripherals](#)
 - ADC, DAC, PWM, CAN, USB
 - WiFi, BLE, Cellular, NFC
 - Camera, Audio, Display, GPU
 - IMU, GNSS, Radar, Motor
 - The vtable dispatch pattern

6. [RTOS Kernel](#)

- Task management & scheduling
- Mutex, semaphore, message queues
- Software timers
- Priority inversion protection
- Real-time guarantees

7. [Multicore — SMP & AMP](#)

- Core management & identification
- Spinlocks (architecture-aware: ARM, x86, RISC-V)
- Memory barriers & atomics
- Inter-processor interrupts (IPI)
- Shared memory regions
- Task migration & affinity
- Remote processor management

8. [Driver Framework](#)

- Driver probe/remove lifecycle
- Power management hooks
- Device tree integration
- Writing your first driver

1.4.3 Part III — Services

9. [Cryptography](#)

- SHA-256, SHA-512 (full implementation)
- AES encryption
- RSA signing & verification (big-integer arithmetic)
- ECDSA P-256 (elliptic curve)
- CRC checksums

10. [Security](#)

- Keystore & key management
- Access control lists (ACL)
- Secure boot chain
- Integrity monitoring
- Audit logging

11. [Networking](#)

- TCP/UDP sockets
- HTTP client/server
- MQTT client
- mDNS service discovery
- TLS integration

12. [Filesystem & Storage](#)

- Flash filesystem
- File operations API
- Wear leveling
- Secure storage

13. [OTA Updates](#)

- Update lifecycle
- Delta updates

- Rollback protection
 - Secure update verification
14. [Sensor & Motor Services](#)
 - Sensor framework (register, read, calibrate)
 - Motor control with PID
 - Sensor fusion patterns
 15. [Compatibility Layers](#)
 - POSIX (threads, sync, signals, I/O)
 - VxWorks (tasks, semaphores, watchdog, message queues)
 - Linux IPC (D-Bus bridge, shared memory)
 16. [UI Framework](#)
 - LVGL integration
 - Touch input handling
 - Display rendering
 - Building a watchface

1.4.4 Part IV — The Ecosystem

17. [eBoot — Secure Bootloader](#)
 - Stage 0 / Stage 1 boot flow
 - Ed25519 signature verification
 - Board support (24+ boards)
 - Secure boot chain integration
18. [ebuild — Build System](#)
 - CLI commands
 - Build configuration (YAML)
 - Cross-compilation toolchains
 - SDK generation
 - CI/CD integration
19. [eIPC — Inter-Process Communication](#)
 - The EIPC protocol (Go + C)
 - HMAC authentication
 - Service discovery
 - Pub/sub messaging
 - Chat & completion handlers
20. [eAI — Embedded AI](#)
 - AI framework architecture
 - Model loading & inference (EAIM binary format)
 - TFLite Micro integration
 - Secure boot for AI models
 - BCI device integration (OpenBCI)
 - On-device LLM (llama.cpp)
21. [eNI — Neural Interface](#)
 - EEG signal acquisition
 - DSP pipeline (Welch's PSD, band power)
 - ONNX model inference
 - Intent decoding
 - Wireless BCI (BlueZ BLE)

- Python SDK
22. [EoSim — Simulation Platform](#)
 - QEMU integration (QMP, GDB)
 - Native peripheral simulation
 - State bridge architecture
 - Multi-architecture support (52+ platforms)
 23. [EoStudio — Design Suite](#)
 - Platform backends (Tkinter, Web, EoS)
 - 12 specialized editors
 - WebSocket remote rendering
 - LLM integration

1.4.5 Part V — Hardware Products

24. [eRadar360 — Automotive Radar](#)
 - System architecture
 - 10-layer hybrid PCB design
 - Antenna design
 - Power sequencing
 - Manufacturing & bring-up
25. [eHealth365 — Health Monitoring](#)
 - Two-device system (Smart Ring Pro + Smart Patch Pro)
 - Sensor specifications (PPG, CGM, sweat biosensor, bioimpedance)
 - 3D model specifications
 - Monthly blood cartridge system
 - 24-hour sensing schedule
 - Mobile app architecture
 - Battery optimization
 - Business plan & go-to-market
26. [ePAM — Personal Air Mobility](#)
 - Four-vehicle product line
 - Eco Car (solar-hybrid ground vehicle)
 - Urban Drone (eVTOL)
 - Space Shuttle (suborbital) + ULP-SSN avionics
 - Combo Unit (trans-atmospheric)
 - Power architecture (solar + H2 + battery + regen)
 - Tesla-like mobile app
 - Manufacturing cost analysis
 - Regulatory path (FAA, EASA, AST)

1.4.6 Part VI — Applications & Tools

27. [eApps — Marketplace](#)
 - 43 applications
 - App lifecycle management
 - Cross-platform deployment
 - Marketplace API
28. [eDB — Database](#)

- Multi-model database architecture
 - CLI commands
 - Query engine
 - Embedded vs. server modes
29. [eBrowser — Web Browser](#)
 - Browser engine architecture
 - Platform abstraction layer
 - HTML/CSS/JS rendering
 - Embedded device optimization
 30. [eOffice — Office Suite](#)
 - 11 productivity applications
 - Desktop (Electron) architecture
 - Document format support
 31. [eVera — AI Assistant](#)
 - 24+ specialized agents
 - 183+ tools
 - 3D avatar (Three.js)
 - Voice control (3 modes, 19 languages)
 - Plugin architecture
 - Cross-platform (Electron, React Native, web)
 32. [eStocks — Trading System](#)
 - 15 trading strategies
 - 7-layer risk management
 - Backtesting engine
 - Production safety controls
 - Platform integrations (TradingView, IBKR, thinkorswim)

1.4.7 Part VII — Advanced Topics

33. [Cross-Compilation & Toolchains](#)
 - AArch64, ARM hard-float, RISC-V
 - Custom toolchain files
 - Multi-architecture CI/CD
34. [Testing & Quality](#)
 - Unit testing framework
 - Fuzz testing
 - Sanitizers (ASan, UBSan)
 - Valgrind memory checking
 - CI/CD pipeline architecture
35. [Safety & Compliance](#)
 - ISO 26262 (automotive)
 - DO-178C (aerospace)
 - IEC 61508 (industrial)
 - IEC 62304 (medical)
 - Quality management system
36. [Contributing to EmbeddedOS](#)
 - Development workflow
 - Coding standards

- Pull request process
- Community guidelines

1.4.8 Appendices

- [A. API Quick Reference](#)
 - [B. Product Profile Reference](#)
 - [C. Pin Assignment Tables](#)
 - [D. Glossary](#)
 - [E. Bibliography & Standards](#)
-

1.5 How to Build This Book

1.5.1 Read Online

Visit: <https://embeddedos-org.github.io/eos/book/>

1.5.2 Build Locally (mdBook)

```
# Install mdBook
cargo install mdbook

# Build the book
cd docs/book
mdbook build

# Serve locally with live reload
mdbook serve --open
```

1.5.3 Generate PDF

```
# Install pandoc + LaTeX
sudo apt-get install pandoc texlive-xetex texlive-fonts-recommended

# Generate PDF from markdown
cd docs/book
./generate-pdf.sh
# Output: embeddedos-complete-guide.pdf
```

1.6 Conventions Used in This Book

- `code` — function names, file paths, commands
- **Bold** — key terms on first introduction
- *Italic* — emphasis

- `eos_` prefix — all EoS public API functions
- Code blocks with filename headers show source file locations

Copyright (c) 2026 Srikanth Patchava & EmbeddedOS Contributors. Licensed under CC BY-SA 4.0.

Chapter 2

Chapter 1: Introduction to EmbeddedOS

Srikanth Patchava & EmbeddedOS Contributors

2.1 1.1 What Is EmbeddedOS?

EmbeddedOS (EoS) is a **universal embedded operating system framework** designed to let engineers write hardware-independent firmware that compiles and runs on any target — from 8-bit microcontrollers to multi-core application processors, from smartwatches to spacecraft.

At its core, EoS provides:

- A **Hardware Abstraction Layer (HAL)** covering 33 peripheral interfaces
- A lightweight **RTOS kernel** with preemptive multitasking
- A **multicore framework** supporting SMP and AMP topologies
- A **driver framework** with probe/remove lifecycle management
- A **product profile system** with 48 pre-defined hardware configurations
- A **build system** that orchestrates CMake, Ninja, Kbuild, Zephyr, FreeRTOS, and NuttX

EoS follows a single guiding philosophy:

Write once, compile for any hardware. Change the board, not the code.

2.2 1.2 The Vision

The embedded industry suffers from fragmentation. Every vendor ships its own SDK, its own HAL, its own build scripts. Moving a project from one microcontroller family to another often means rewriting 60–80% of the platform code.

EoS eliminates this friction by placing a **stable, vendor-neutral API boundary** between application code and hardware. The same `eos_gpio_write()` call works whether the target is an nRF52840, an STM32H743, a Raspberry Pi 4, or a host PC running Linux.



Figure 2.1: Figure: The Complete EmbeddedOS Ecosystem — 15 products spanning bootloader to browser

```

Your Application Code
(unchanged across all targets)

EoS Public API
hal.h  kernel.h  multicore.h  driver.h

Backend Dispatch Layer
(vtable pointer → platform-specific impl)

Host  STM32  nRF52  RPi4  ESP32  RISC-V  ...
Linux  HAL    HAL    HAL    HAL    HAL

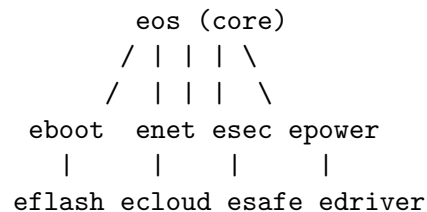
```

2.3 1.3 The 18-Repo Ecosystem

EoS is organized as a family of repositories under the **embeddedos-org** GitHub organization. Each repo is independently versioned and CI-tested.

#	Repository	Purpose
1	eos	Core OS: HAL, kernel, multicore, drivers, services, products
2	eboot	Secure bootloader with verified boot chain
3	ebuild	Build orchestration CLI and package manager
4	ecloud	Cloud connectivity (MQTT, HTTP, OTA)
5	edebug	Debug tools: trace, profiling, crash dump analysis
6	edriver	Vendor-specific driver packs (STM32, nRF, ESP, TI)
7	eflash	Flash programming utilities and algorithms
8	efota	Firmware-Over-The-Air update framework
9	egui	LVGL-based graphical UI framework
10	eiota	IoT protocols: CoAP, LwM2M, Matter
11	eml	Edge ML inference (TFLite Micro, CMSIS-NN)
12	enet	Lightweight TCP/IP stack (lwIP integration)
13	epower	Power management: sleep modes, DVFS, battery
14	esafe	Functional safety (IEC 61508, ISO 26262)
15	esec	Security services: TLS, crypto, secure storage
16	eshell	Interactive command shell for debugging
17	esim	Hardware simulation and virtual device framework
18	etest	Testing framework: unit, integration, HIL

2.3.1 Dependency Graph



|
efota
eiot

All repos depend on **eos** for the core HAL and kernel APIs. Higher-level repos like **efota** and **eiot** depend on networking (**enet**) and security (**esec**).

2.4 1.4 Architecture at a Glance

EoS uses a **layered architecture** with clean separation of concerns:

```

      Applications
    (your firmware / product code)

      Services
networking  OTA  UI  power  security

      RTOS Kernel
tasks  mutex  semaphore  queues  timers

      Hardware Abstraction Layer
GPIO  UART  SPI  I2C  Timer  ADC  ... (×33)

      Platform Backends
STM32  nRF52  RPi  Host-Linux  ESP32  ...
```

Each layer communicates only with its immediate neighbors. Application code never touches hardware registers directly — it always goes through the HAL.

2.5 1.5 Who Should Read This Book

This book is written for:

- **Embedded engineers** who want a portable OS framework
- **Systems architects** evaluating RTOS options for multi-product lines
- **Application developers** building IoT, automotive, medical, or industrial firmware
- **Students** learning embedded systems with a production-quality codebase

2.5.1 Prerequisites

Skill	Level Required
C programming	Intermediate (pointers, structs, function pointers)
Embedded concepts	Basic (GPIO, UART, interrupts)
Build tools	Basic familiarity with CMake
RTOS concepts	Helpful but not required

2.6 1.6 The 48 Product Profiles

EoS ships with 48 pre-defined product profiles in the `products/` directory. Each profile is a C header that enables the appropriate set of HAL peripherals, kernel features, and services for a specific product category.

Category	Profiles
Automotive	automotive, cockpit, ev, autonomous, infotainment
Industrial	industrial, plc, robot, vacuum, printer
Medical	medical, diagnostic, telemedicine, fitness, wearable
Aerospace	aerospace, satellite, ground_control, space_comm, drone
Consumer	mobile, watch, smart_speaker, gaming, xr_headset
Smart Home	smart_home, thermostat, home_camera, voice, smart_tv
Networking	router, gateway, telecom, iot, iptv_stb
Computing	server, desktop, computer, cast_device
Financial	banking, pos, crypto_hw
HMI/Display	hmi, security_cam
Testing	vbox_test, adapter

Selecting a profile at build time is a single CMake flag:

```
cmake -B build -DEOS_PRODUCT=robot
```

This sets the correct `EOS_ENABLE_*` flags to compile only the peripherals your product actually needs, keeping binary size minimal.

2.7 1.7 Book Organization

This book is organized in four parts:

Part	Chapters	Topics
Part I: Foundations	1–3	Introduction, Getting Started, Architecture
Part II: Kernel & HAL	4–8	HAL, Extended HAL, Kernel, Multicore, Drivers
Part III: Services	9–12	Networking, Security, OTA, Power Management
Part IV: Products	13–16	Product Profiles, Testing, Deployment, Contributing

Each chapter includes:

- Conceptual explanation with diagrams
- Complete API reference tables
- Working code examples you can compile and run
- Platform-specific notes where behavior differs

2.8 1.8 Conventions Used in This Book

Convention	Meaning
<code>monospace</code>	Code, file paths, command-line input
bold	First use of an important term
<code>eos_ prefix</code>	All EoS public API functions
<code>EOS_ prefix</code>	All EoS constants and macros
<code>eos*_config_t</code>	Configuration struct passed to <code>*_init()</code>
<code>eos*_handle_t</code>	Opaque handle returned by <code>*_create()</code>

2.8.1 Return Code Convention

All EoS functions that can fail return `int`:

Value	Meaning
0	Success
-1 (<code>EOS_KERN_ERR</code>)	Generic error
-2 (<code>EOS_KERN_TIMEOUT</code>)	Operation timed out
-3 (<code>EOS_KERN_FULL</code>)	Resource full (queue, pool)
-4 (<code>EOS_KERN_EMPTY</code>)	Resource empty
-5 (<code>EOS_KERN_INVALID</code>)	Invalid parameter
-6 (<code>EOS_KERN_NO_MEMORY</code>)	Out of memory

2.9 1.9 How to Build and Run Examples

Every example in this book can be compiled with:

```
cd eos/examples/<example-name>
cmake -B build -DEOS_PRODUCT=iot
cmake --build build
./build/<example-name>
```

For cross-compilation to a specific board:

```
cmake -B build \
  -DCMAKE_TOOLCHAIN_FILE=../../toolchains/arm-none-eabi.cmake \
  -DEOS_PRODUCT=industrial
cmake --build build
```

2.10 1.10 Getting Help

Resource	Location
GitHub Issues	github.com/embeddedos-org/eos/issues
Documentation	<code>docs/</code> directory in each repository
API Reference	<code>docs/api-reference.md</code>

Resource	Location
Architecture Guide	<code>docs/architecture.md</code>
Contributing Guide	<code>CONTRIBUTING.md</code>

Next: [Chapter 2 — Getting Started](#)

Chapter 3

Chapter 2: Getting Started

Srikanth Patchava & EmbeddedOS Contributors

3.1 2.1 Prerequisites

Before building EoS, ensure the following tools are installed:

Tool	Minimum Version	Purpose
CMake	3.16	Build system generator
GCC / Clang	10.x	C compiler
Ninja	1.10	Fast build backend (recommended)
Git	2.x	Source control
Python 3	3.8	Test scripts, tooling

3.1.1 Installation by Platform

Ubuntu / Debian:

```
sudo apt update
sudo apt install cmake gcc g++ git ninja-build python3 python3-pip
```

macOS:

```
brew install cmake ninja python3
# Xcode Command Line Tools provide GCC/Clang
xcode-select --install
```

Windows (PowerShell):

```
winget install CMake.CMake
winget install Ninja-build.Ninja
# Install MinGW-w64 or Visual Studio Build Tools for C compiler
```

3.1.2 Cross-Compilation Toolchains

For targeting embedded hardware, install the appropriate cross-compiler:

Target	Toolchain	Install Command
ARM Cortex-M	arm-none-eabi-gcc	<code>sudo apt install gcc-arm-none-eabi</code>
ARM64 Linux	aarch64-linux-gnu-gcc	<code>sudo apt install gcc-aarch64-linux-gnu</code>
RISC-V 64	riscv64-linux-gnu-gcc	<code>sudo apt install gcc-riscv64-linux-gnu</code>

3.2 2.2 Cloning the Repository

```
git clone https://github.com/embeddedos-org/eos.git
cd eos
```

The repository structure:

```
eos/
  hal/                # Hardware Abstraction Layer
    include/eos/      #  hal.h, hal_extended.h
  kernel/             # RTOS kernel
    include/eos/      #  kernel.h, multicore.h
    src/              #  kernel.c, multicore.c
  drivers/            # Driver framework
    include/eos/      #  driver.h
    src/              #  driver.c, driver_framework.c
    devicetree/       #  devicetree.h, devicetree.c
  products/           # 48 product profile headers
  examples/           # Working example applications
  backends/           # Platform-specific HAL implementations
  boards/             # Board support packages
  toolchains/         # CMake toolchain files
  tests/              # Unit and integration tests
  docs/               # Documentation
  CMakeLists.txt      # Top-level build configuration
```

3.3 2.3 Building from Source

3.3.1 Host Build (No Hardware Required)

The fastest way to start. The host build uses a Linux/POSIX HAL backend that simulates peripherals via printf and stdin.

```
# Configure
cmake -B build -G Ninja

# Build
cmake --build build
```

```
# Run tests
cmake -B build -DEOS_BUILD_TESTS=ON
cmake --build build
ctest --test-dir build --output-on-failure
```

3.3.2 Building with a Product Profile

```
# Build for a specific product category
cmake -B build -DEOS_PRODUCT=iot -G Ninja
cmake --build build
```

The EOS_PRODUCT flag includes the matching header from `products/` (e.g., `products/iot.h`), which defines the appropriate EOS_ENABLE_* flags.

3.4 2.4 Your First Application: Blink GPIO

The classic “Hello World” of embedded systems. This example toggles an LED every 500ms.

3.4.1 Source Code (examples/blink-gpio/main.c)

```
#include <eos/hal.h>
#include <stdio.h>

#define LED_PIN 13

int main(void)
{
    eos_hal_init();

    eos_gpio_config_t led_cfg = {
        .pin    = LED_PIN,
        .mode   = EOS_GPIO_OUTPUT,
        .pull   = EOS_GPIO_PULL_NONE,
        .speed  = EOS_GPIO_SPEED_LOW,
    };
    eos_gpio_init(&led_cfg);

    printf("[blink] Starting LED blink on pin %d\n", LED_PIN);

    while (1) {
        eos_gpio_write(LED_PIN, true);
        printf("[blink] LED ON\n");
        eos_delay_ms(500);

        eos_gpio_write(LED_PIN, false);
        printf("[blink] LED OFF\n");
        eos_delay_ms(500);
    }
}
```

```

    }

    return 0;
}

```

3.4.2 CMakeLists.txt

```

cmake_minimum_required(VERSION 3.16)
project(blink-gpio LANGUAGES C)

get_filename_component(EOS_ROOT "${CMAKE_CURRENT_SOURCE_DIR}/../.." ABSOLUTE)
add_subdirectory(${EOS_ROOT} ${CMAKE_BINARY_DIR}/eos)

add_executable(blink-gpio main.c)
target_link_libraries(blink-gpio PRIVATE eos_hal)
target_include_directories(blink-gpio PRIVATE
    ${EOS_ROOT}/hal/include
    ${EOS_ROOT}/include
)

```

3.4.3 eos.yaml

```

project:
  name: blink-gpio
  version: 0.1.0

workspace:
  backend: ninja
  build_dir: .eos/build

toolchain:
  target: host

system:
  kind: baremetal
  entry: main.c
  output: blink-gpio

```

3.4.4 Build and Run

```

cd examples/blink-gpio
cmake -B build -DEOS_PRODUCT=iot
cmake --build build
./build/blink-gpio

```

Expected output:

```
[blink] Starting LED blink on pin 13
[blink] LED ON
[blink] LED OFF
[blink] LED ON
...
```

3.4.5 How It Works on the Host

On a host build, the HAL backend maps peripheral calls to POSIX equivalents:

HAL Function	Host Behavior
<code>eos_gpio_write(pin, val)</code>	Prints GPIO <pin>: HIGH/LOW
<code>eos_gpio_read(pin)</code>	Returns simulated value
<code>eos_uart_write(port, data, len)</code>	Writes to stdout
<code>eos_uart_read(port, buf, len, t)</code>	Reads from stdin with timeout
<code>eos_delay_ms(ms)</code>	Calls <code>usleep()</code> / <code>Sleep()</code>
<code>eos_get_tick_ms()</code>	Uses <code>clock_gettime()</code> / <code>QueryPerformanceCounter()</code>

3.5 2.5 Quickstart: Host Build

The host build is the recommended starting point for all developers.

```
git clone https://github.com/embeddedos-org/eos.git
cd eos/examples/blink-gpio

cmake -B build -DEOS_PRODUCT=iot
cmake --build build
./build/blink-gpio
```

To build and run the full test suite:

```
cd eos
cmake -B build -DEOS_BUILD_TESTS=ON
cmake --build build
ctest --test-dir build --output-on-failure
```

Try all included examples:

```
cd examples
for example in blink-gpio uart-echo multitask-rtos posix-app multicore-amp; do
  echo "=== Building $example ==="
  cd $example
  cmake -B build -DEOS_PRODUCT=iot
  cmake --build build
  cd ..
done
```


3.6 2.6 Quickstart: STM32

3.6.1 Hardware

- **Board:** STM32H743-Nucleo, STM32F4-Discovery, or any STM32 Nucleo/Discovery
- **Debugger:** Built-in ST-Link
- **Interface:** USB for power/programming, UART2 for serial output

3.6.2 Build

```
cd eos/examples/blink-gpio

cmake -B build \
  -DCMAKE_TOOLCHAIN_FILE=../../toolchains/arm-none-eabi.cmake \
  -DEOS_PRODUCT=industrial
cmake --build build
```

3.6.3 Flash via OpenOCD

```
# STM32H7 Nucleo:
openocd -f interface/stlink.cfg -f target/stm32h7x.cfg \
  -c "program build/blink-gpio.bin 0x08000000 verify reset exit"
```

3.6.4 Pin Mapping (STM32 Nucleo)

EoS Function	STM32 Pin	Nucleo Label
LED	PA5	LD2 (green)
UART2 TX	PA2	D1
UART2 RX	PA3	D0
Button	PC13	B1 (blue)
SPI1 SCK	PA5	D13
I2C1 SDA	PB9	D14
I2C1 SCL	PB8	D15

3.6.5 Debugging with GDB

```
# Terminal 1: Start OpenOCD
openocd -f interface/stlink.cfg -f target/stm32h7x.cfg

# Terminal 2: Connect GDB
arm-none-eabi-gdb build/blink-gpio.elf
(gdb) target remote :3333
(gdb) monitor reset halt
(gdb) load
(gdb) break main
(gdb) continue
```

3.7 2.7 Quickstart: Raspberry Pi 4

3.7.1 Build Natively on Pi

```
sudo apt install cmake gcc g++ git
git clone https://github.com/embeddedos-org/eos.git
cd eos/examples/blink-gpio

cmake -B build -DEOS_PRODUCT=gateway
cmake --build build
./build/blink-gpio
```

3.7.2 Cross-Compile from Host

```
sudo apt install gcc-aarch64-linux-gnu
cd eos/examples/blink-gpio

cmake -B build \
  -DCMAKE_C_COMPILER=aarch64-linux-gnu-gcc \
  -DEOS_PRODUCT=gateway
cmake --build build

scp build/blink-gpio pi@raspberrypi.local:~/
```

3.7.3 GPIO Pin Mapping (BCM)

BCM GPIO	Physical Pin	Common Use
2	3	I2C SDA
3	5	I2C SCL
4	7	General GPIO
14	8	UART TX
15	10	UART RX
17	11	General GPIO
18	12	PWM

For real GPIO control on the Pi, run with `sudo`:

```
sudo ./build/blink-gpio
```

3.8 2.8 Quickstart: nRF52

3.8.1 Hardware

- **Board:** nRF52840-DK (PCA10056) or nRF52832-DK (PCA10040)
- **Debugger:** Built-in J-Link

3.8.2 Prerequisites

```
sudo apt install gcc-arm-none-eabi
# Install Nordic nRF Command Line Tools from nordicsemi.com
```

3.8.3 Build and Flash

```
cd eos/examples/blink-gpio

cmake -B build \
  -DCMAKE_TOOLCHAIN_FILE=../../toolchains/arm-none-eabi.cmake \
  -DEOS_PRODUCT=iot
cmake --build build

# Flash via J-Link
nrfjprog --program build/blink-gpio.hex --chiperase --verify
nrfjprog --reset
```

3.8.4 Verify via Serial

```
screen /dev/ttyACM0 115200
```

3.8.5 Pin Mapping (nRF52840-DK)

EoS Pin	nRF52840 GPIO	Board Label	Function
13	P0.13	LED1	User LED
14	P0.14	LED2	User LED
15	P0.15	LED3	User LED
16	P0.16	LED4	User LED
11	P0.11	Button 1	User button
6	P0.06	UART TX	Debug UART
8	P0.08	UART RX	Debug UART

3.9 2.9 Troubleshooting

Problem	Solution
CMake version too old	Upgrade: <code>pip install cmake --upgrade</code>
arm-none-eabi-gcc not found	Install: <code>sudo apt install gcc-arm-none-eabi</code>
Host build: no output	Check that <code>eos_hal_init()</code> is called first
STM32: flash fails	Try erasing first, check USB cable
nRF52: device locked	Run <code>nrfjprog --recover</code>
RPi4: GPIO permission denied	Run with <code>sudo</code> or add user to <code>gpio</code> group
Link errors	Ensure <code>target_link_libraries</code> includes <code>eos_hal</code>

3.10 2.10 Next Steps

Now that you have a working build, continue with:

- **Chapter 3** — Understand the layered architecture
- **Chapter 4** — Deep dive into the HAL API
- **Chapter 6** — Build multitasking applications with the kernel

Next: [Chapter 3 — Architecture](#)

Chapter 4

Chapter 3: Architecture

Srikanth Patchava & EmbeddedOS Contributors

4.1 3.1 Design Principles

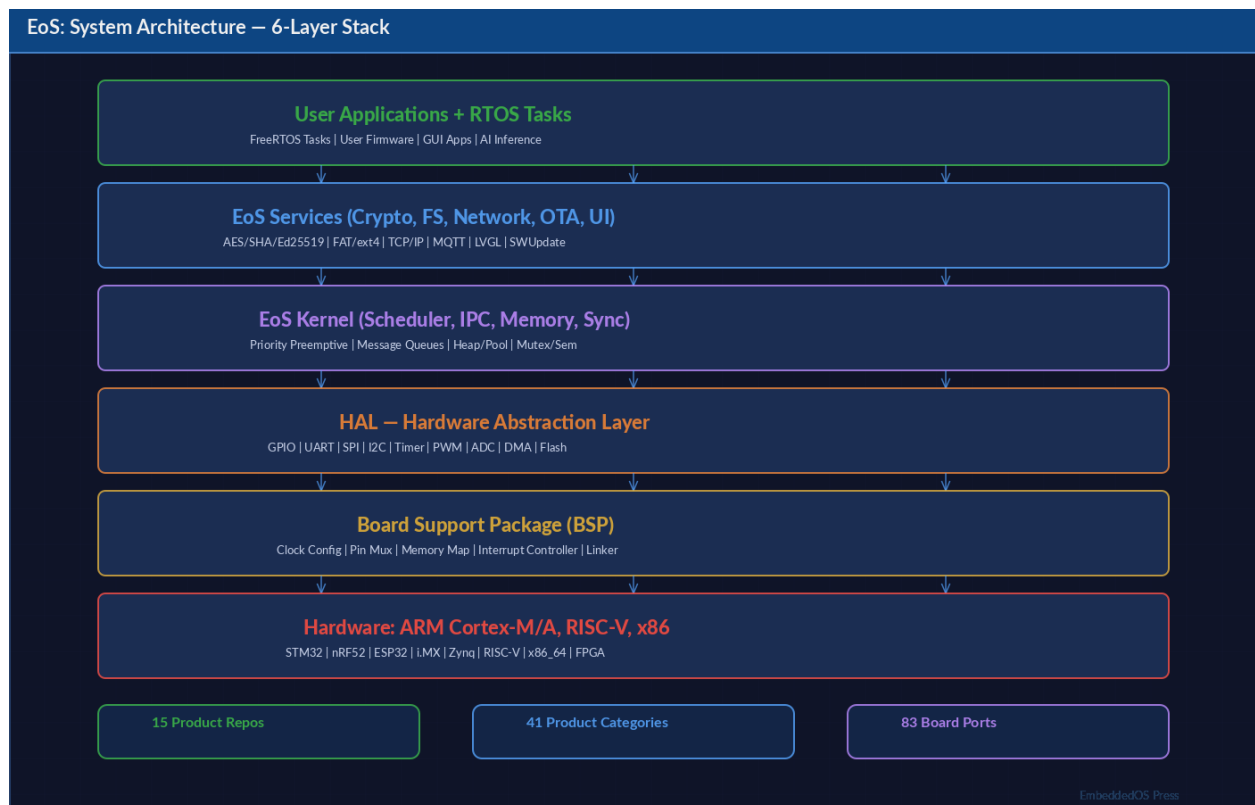


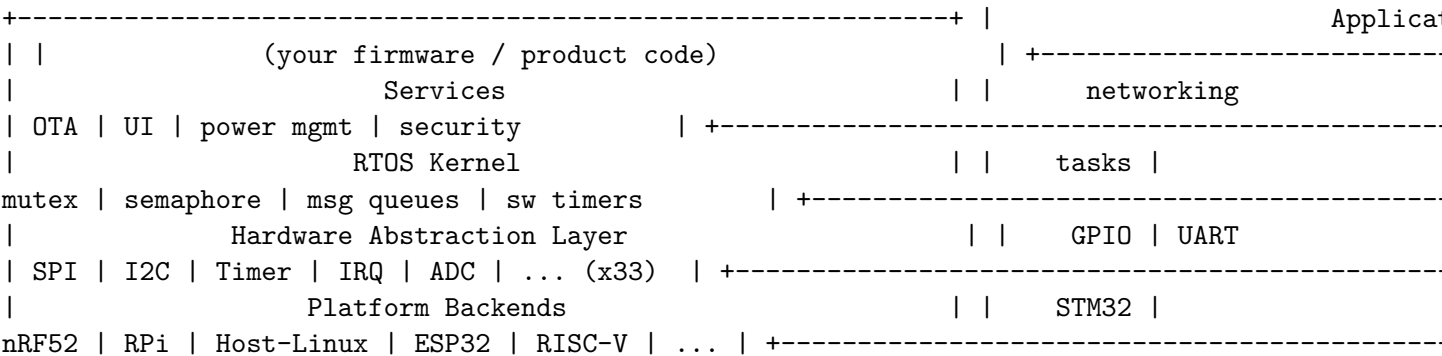
Figure 4.1: Figure: EoS 6-Layer System Architecture — from hardware through kernel to applications

EoS is built on six foundational principles:

- 1. **Simplicity over completeness** – Easy to learn, easy to extend
- 2. **Reuse existing tools** – Never rewrite build systems; wrap them
- 3. **Build incrementally** – Cache aggressively, rebuild only what changed
- 4. **Developer experience first** – Clear errors, helpful logs, fast feedback
- 5. **Separate system kinds** – Linux and RTOS are peers, not subsets
- 6. **RTOS is a first-class citizen** – Not squeezed into Linux abstractions

4.2 3.2 Layered Architecture

EoS uses a strict layered architecture. Each layer communicates only with its immediate neighbors through well-defined C interfaces.



4.2.1 Layer Responsibilities

Layer	Header(s)	Responsibility
Application	User code	Product-specific business logic
Services	services/*.h	Cross-cutting concerns (OTA, networking, UI)
Kernel	kernel.h, multicore.h	Task scheduling, synchronization, IPC
HAL	hal.h, hal_extended.h	Vendor-neutral peripheral access
Backend	backend.h	Platform-specific hardware drivers

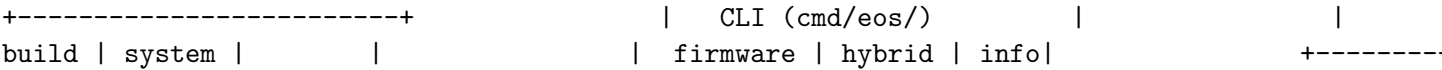
4.2.2 Information Flow

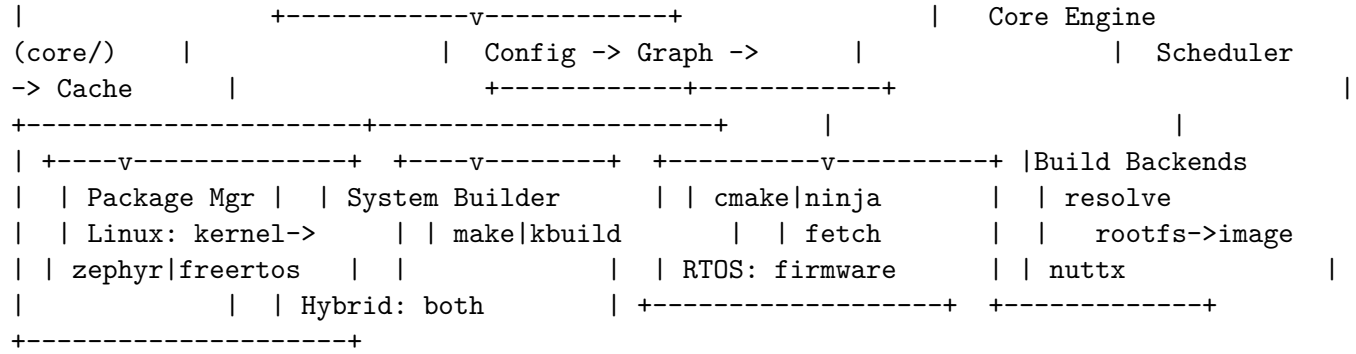
Application | v calls eos_gpio_write(pin, true) HAL --> dispatches
via backend vtable | v backend->gpio_write(pin, true) Backend -->
writes to hardware register (or printf on host)

The key insight: the HAL API is a **stable contract**. Backends can change (new chip support, bug fixes) without affecting application code.

4.3 3.3 The Build System

EoS uses CMake as its meta-build system, but it orchestrates multiple native build systems depending on the target.





4.3.1 Core Engine (core/)

The core engine handles configuration parsing, dependency resolution, and build scheduling:

Module	File	Purpose
Types	ypes.h	EosArch, EosBuildType, EosSystemKind, EosRtosProvider
Error	error.h	Error codes and macros
Logger	log.h/c	Leveled logging with color output
Config	config.h/c	YAML config parser (no external deps)
Graph	graph.h/c	DAG with topological sort
Scheduler	scheduler.h/c	Build execution across the dependency graph
Cache	cache.h/c	Hash-based rebuild detection

4.3.2 Build Backends

EoS never rewrites build logic – it orchestrates existing build systems through thin adapter modules:

Backend	Use Case	Invocation
CMake	First-party C/C++ code	cmake -B build && cmake --build build
Ninja	Direct ninja invocation	ininja -C build
Make	Legacy Unix packages	make -j
Kbuild	Linux kernel, U-Boot	make defconfig && make
Zephyr	Zephyr RTOS apps	west build
FreeRTOS	FreeRTOS apps	CMake/Make + vendor SDK
NuttX	Apache NuttX	make menuconfig && make

4.4 3.4 System Kinds

EoS supports three distinct system kinds, each with its own build pipeline:

Kind	Pipeline	CLI Command
Linux	build -> package -> rootfs -> kernel -> image	os system
RTOS	build -> package -> firmware	os firmware

Kind	Pipeline	CLI Command
Hybrid	Linux pipeline + RTOS firmware(s)	os hybrid

4.4.1 Linux Pipeline

Source -> Build -> Package -> Rootfs -> Kernel -> Image
 FHS skeleton install packages
 init create users

4.4.2 RTOS Firmware Pipeline

Source -> Configure SDK -> Cross-compile -> Firmware Binary
 .bin / .elf / .hex / .uf2

4.4.3 Hybrid Pipeline

For multi-core SoCs (e.g., STM32MP1 with Cortex-A + Cortex-M):

Phase 1: Build Linux system (kernel + rootfs + image) Phase 2: Build each RTOS
 firmware target Phase 3: Combine outputs for multi-core deployment

4.4.4 Configuration Examples

RTOS Firmware (eos.yaml):

```
'yaml system: kind: rtos rtos: provider: freertos board: stm32f4 entry: apps/realtime-io output:
firmware.bin format: bin
```

```
toolchain: target: arm-none-eabi '
```

Hybrid System (eos.yaml):

```
'yaml system: kind: hybrid linux: provider: buildroot rtos: - provider: freertos core: m4 board:
stm32mp1-m4 entry: apps/realtime-io output: firmware-m4.bin
```

```
toolchain: linux: target: aarch64-linux-gnu rtos: target: arm-none-eabi '
```

4.5 3.5 The Product Profile System

IoS ships with 48 product profiles in the products/ directory. Each profile is a C header that defines EOS_ENABLE_* preprocessor flags to select which HAL peripherals, kernel features, and services are compiled into the firmware.

4.5.1 How Profiles Work

When you pass -DEOS_PRODUCT=robot to CMake, the build system includes products/robot.h. This header might contain:

```
c // products/robot.h #define EOS_ENABLE_GPIO      1 #define EOS_ENABLE_UART      1
#define EOS_ENABLE_SPI      1 #define EOS_ENABLE_I2C      1 #define EOS_ENABLE_PWM
1 #define EOS_ENABLE_ADC      1 #define EOS_ENABLE_MOTOR      1 #define EOS_ENABLE_IMU
```



```
1 #define EOS_ENABLE_CAMERA    1 #define EOS_ENABLE_WIFI    1 #define EOS_ENABLE_MULTICORE
1
```

4.5.2 Profile Categories

Category	Count	Profiles
Automotive and Transport	5	automotive, cockpit, ev, autonomous, infotainment
Industrial and Manufacturing	5	industrial, plc, robot, vacuum, printer
Medical and Health	5	medical, diagnostic, telemedicine, fitness, wearable
Space and Aerospace	5	aerospace, satellite, ground_control, space_comm, drone
Consumer Electronics	5	mobile, watch, smart_speaker, gaming, xr_headset
Smart Home and IoT	5	smart_home, thermostat, home_camera, voice, smart_tv
Networking and Telecom	5	router, gateway, telecom, iot, iptv_stb
Computing and Server	4	server, desktop, computer, cast_device
Financial	3	banking, pos, crypto_hw
HMI and Security	2	hmi, security_cam
Testing and Development	4	vbox_test, adapter, ai_edge, tv_os

4.5.3 Custom Profiles

Create your own profile by adding a header to products/:

```
'c // products/my_device.h #ifndef EOS_PRODUCT_MY_DEVICE_H #define EOS_PRODUCT_MY_DEVICE_H
#define EOS_ENABLE_GPIO 1 #define EOS_ENABLE_UART 1 #define EOS_ENABLE_BLE
1 #define EOS_ENABLE_DISPLAY 1 #define EOS_ENABLE_TOUCH 1 #define EOS_ENABLE_HAPTICS
1
#endif '
```

Then build with:

```
"bash cmake -B build -DEOS_PRODUCT=my_device
```

4.6 3.6 The Layer System

EoS uses six stable layer types for configuration management:

Layer	Purpose	Example
Core	Default build rules, package schemas	Always included
BSP	Board-specific: kernel, bootloader, device tree	boards/stm32h7/
Distro	Package selection, init system, users	layers/minimal/
Vendor	SDK integrations, proprietary components	layers/nordic/
Product	Final config, branding, release images	products/robot.h
RTOS	RTOS kernel config, middleware, scheduler	layers/freertos/

Layers are stacked in eos.yaml:

```
yaml layers:  - layers/core    - layers/bsp/stm32h7    - layers/distro/minimal    -
layers/vendor/nordic
```

4.7 3.7 Three Independent Graphs

EoS internally maintains three graph types to separate concerns:

4.7.1 1. Source Graph – Where Code Comes From

```
git clone -> tarball download -> local path      +-- checksum verification      +--
cache lookup
```

4.7.2 2. Build Graph – How Things Are Built

```
zlib --> openssl --> curl --> application  (make)  (cmake)  (cmake)  (cmake)
```

A DAG (Directed Acyclic Graph) with topological sort determines build order.

4.7.3 3. Output Graph – How Outputs Become Deployable

```
Linux:  kernel + rootfs -> disk image (.raw, .qcow2) RTOS:  firmware binary
-> flash layout (.bin, .hex) Hybrid: Linux image + RTOS firmware(s) -> combined
package
```

4.8 3.8 Toolchain Definitions

Toolchains are YAML-based definitions specifying CC, CXX, AR, sysroot, and flags:

Toolchain File	Target Architecture
aarch64-linux-gnu.cmake	ARM64 Linux
x86_64-linux-gnu.cmake	x86_64 Linux
iscv64-linux-gnu.cmake	RISC-V 64 Linux
arm-none-eabi.cmake	ARM Cortex-M/R bare-metal

Usage:

```
“bash cmake -B build -DCMAKE_TOOLCHAIN_FILE=toolchains/arm-none-eabi.cmake
```

4.9 3.9 Configuration Schema Summary

The eos.yaml file drives everything. Key sections:

‘yaml project: name: # Project name version: # Semantic version

workspace: backend: <cmake|ninja|make> build_dir: cache_dir:

toolchain: target: # e.g., aarch64-linux-gnu

packages: - name: version: source: build: type: <cmake|ninja|make|kbuild> deps: -

system: kind: <linux|rtos|hybrid> ‘

4.10 3.10 Summary

Concept	Key Takeaway
Layered architecture	HAL -> Kernel -> Services -> Apps
Backend dispatch	vtable pointer swaps platform at link time
System kinds	Linux, RTOS, and Hybrid are equal peers
Product profiles	48 pre-built, or create your own
Build orchestration	CMake wraps Ninja, Make, Kbuild, Zephyr, etc.
Three graphs	Source, Build, Output – independently managed

Next: Chapter 4 – Hardware Abstraction Layer

Chapter 5

Chapter 4: Hardware Abstraction Layer

Srikanth Patchava & EmbeddedOS Contributors

5.1 4.1 Overview

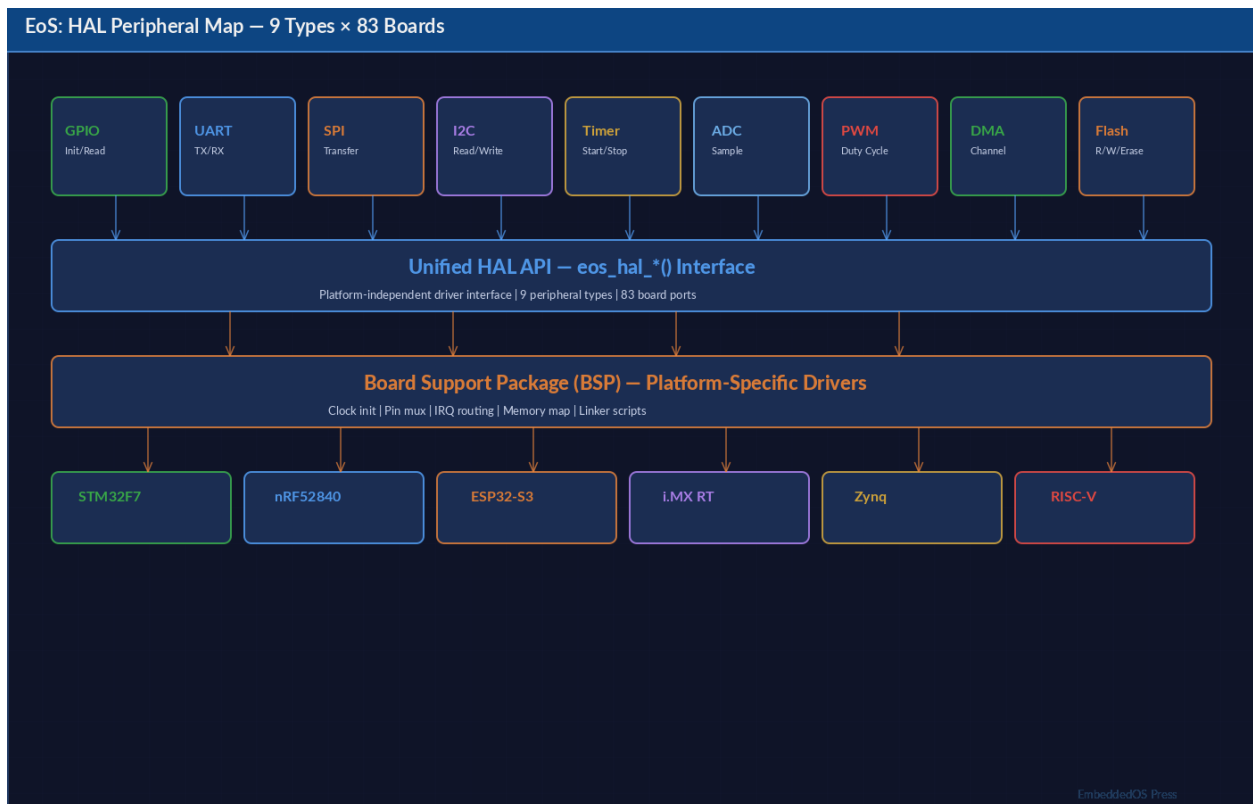


Figure 5.1: Figure: EoS HAL Peripheral Map — 9 peripheral types across 83 board ports

The Hardware Abstraction Layer (HAL) is the foundation of EoS portability. It provides a **vendor-neutral C API** for accessing hardware peripherals, so application code remains unchanged across all supported platforms.

The core HAL (`hal.h`) covers six peripheral families plus system utilities:

Category	Interfaces
System	<code>eos_hal_init</code> , <code>eos_hal_deinit</code> , <code>eos_delay_ms</code> , <code>eos_get_tick_ms</code>
GPIO	Digital I/O, interrupt callbacks
UART	Serial communication
SPI	Synchronous serial (full-duplex)
I2C	Two-wire serial bus
Timer	Hardware timers with callbacks
Interrupt	Global IRQ control, handler registration

An additional 27 peripheral interfaces are provided in `hal_extended.h` (Chapter 5).

5.2 4.2 HAL Initialization

Every EoS application must call `eos_hal_init()` before using any peripheral:

```
#include <eos/hal.h>

int main(void)
{
    int rc = eos_hal_init();
    if (rc != 0) {
        // HAL initialization failed
        return rc;
    }

    // ... use HAL APIs ...

    eos_hal_deinit();
    return 0;
}
```

5.2.1 System Utility Functions

Function	Signature	Description
<code>eos_hal_init</code>	<code>int eos_hal_init(void)</code>	Initialize HAL subsystem
<code>eos_hal_deinit</code>	<code>void eos_hal_deinit(void)</code>	Release all HAL resources
<code>eos_delay_ms</code>	<code>void eos_delay_ms(uint32_t ms)</code>	Blocking delay in milliseconds
<code>eos_get_tick_ms</code>	<code>uint32_t eos_get_tick_ms(void)</code>	Tick count since HAL init

5.3 4.3 GPIO Interface

GPIO (General-Purpose Input/Output) is the most fundamental peripheral. EoS provides a rich GPIO API with mode configuration, pull resistors, speed settings, and interrupt support.

5.3.1 Configuration Types

```
typedef enum {
    EOS_GPIO_INPUT    = 0,    // Digital input
    EOS_GPIO_OUTPUT    = 1,    // Push-pull output
    EOS_GPIO_AF        = 2,    // Alternate function
    EOS_GPIO_ANALOG    = 3,    // Analog mode (for ADC/DAC)
} eos_gpio_mode_t;

typedef enum {
    EOS_GPIO_PULL_NONE = 0,    // No pull resistor
    EOS_GPIO_PULL_UP   = 1,    // Internal pull-up
    EOS_GPIO_PULL_DOWN = 2,    // Internal pull-down
} eos_gpio_pull_t;

typedef enum {
    EOS_GPIO_SPEED_LOW    = 0,    // Low frequency
    EOS_GPIO_SPEED_MEDIUM = 1,    // Medium frequency
    EOS_GPIO_SPEED_HIGH   = 2,    // High frequency
    EOS_GPIO_SPEED_VHIGH  = 3,    // Very high frequency
} eos_gpio_speed_t;

typedef struct {
    uint16_t    pin;            // Pin number
    eos_gpio_mode_t mode;        // Input/Output/AF/Analog
    eos_gpio_pull_t pull;        // Pull-up/down/none
    eos_gpio_speed_t speed;      // Output speed
    uint8_t     af_num;         // Alternate function number
} eos_gpio_config_t;
```

5.3.2 GPIO API Reference

Function	Signature	Description
<code>eos_gpio_init</code>	<code>int eos_gpio_init(const eos_gpio_config_t *cfg)</code>	Configure a GPIO pin
<code>eos_gpio_deinit</code>	<code>void eos_gpio_deinit(uint16_t pin)</code>	Release a GPIO pin
<code>eos_gpio_write</code>	<code>void eos_gpio_write(uint16_t pin, bool value)</code>	Set output high/low

Function	Signature	Description
<code>eos_gpio_read</code>	<code>bool eos_gpio_read(uint16_t pin)</code>	Read input state
<code>eos_gpio_toggle</code>	<code>void eos_gpio_toggle(uint16_t pin)</code>	Toggle output state
<code>eos_gpio_set_irq</code>	<code>int eos_gpio_set_irq(uint16_t pin, eos_gpio_edge_t edge, eos_gpio_callback_t cb, void *ctx)</code>	Register interrupt

5.3.3 GPIO Interrupt Example

```

void button_handler(uint16_t pin, void *ctx)
{
    bool *pressed = (bool *)ctx;
    *pressed = true;
    printf("Button pressed on pin %d\n", pin);
}

int main(void)
{
    eos_hal_init();

    // Configure button as input with pull-up
    eos_gpio_config_t btn_cfg = {
        .pin = 11,
        .mode = EOS_GPIO_INPUT,
        .pull = EOS_GPIO_PULL_UP,
    };
    eos_gpio_init(&btn_cfg);

    // Register falling-edge interrupt
    static bool pressed = false;
    eos_gpio_set_irq(11, EOS_GPIO_EDGE_FALLING, button_handler, &pressed);

    while (!pressed) {
        eos_delay_ms(10);
    }
    return 0;
}

```

5.4 4.4 UART Interface

UART (Universal Asynchronous Receiver-Transmitter) provides serial communication.

5.4.1 Configuration

```
typedef struct {
    uint8_t      port;          // UART port number (0, 1, 2, ...)
    uint32_t      baudrate;     // Baud rate (9600, 115200, etc.)
    uint8_t      data_bits;    // 7, 8, or 9
    eos_uart_parity_t parity;   // NONE, EVEN, ODD
    eos_uart_stop_t stop_bits;  // 1 or 2 stop bits
} eos_uart_config_t;
```

5.4.2 UART API Reference

Function	Signature	Description
<code>eos_uart_init</code>	<code>int eos_uart_init(const eos_uart_config_t *cfg)</code>	Initialize UART port
<code>eos_uart_deinit</code>	<code>void eos_uart_deinit(uint8_t port)</code>	Release UART port
<code>eos_uart_write</code>	<code>int eos_uart_write(uint8_t port, const uint8_t *data, size_t len)</code>	Transmit data
<code>eos_uart_read</code>	<code>int eos_uart_read(uint8_t port, uint8_t *data, size_t len, uint32_t timeout_ms)</code>	Receive with timeout
<code>eos_uart_set_rx_callback</code>	<code>int eos_uart_set_rx_callback(uint8_t port, eos_uart_rx_callback_t cb, void *ctx)</code>	Register RX callback

5.4.3 UART Echo Example

```
#include <eos/hal.h>

int main(void)
{
    eos_hal_init();

    eos_uart_config_t uart_cfg = {
```



```

        .port      = 0,
        .baudrate  = 115200,
        .data_bits = 8,
        .parity    = EOS_UART_PARITY_NONE,
        .stop_bits = EOS_UART_STOP_1,
    };
    eos_uart_init(&uart_cfg);

    uint8_t buf[64];
    while (1) {
        int n = eos_uart_read(0, buf, sizeof(buf), 1000);
        if (n > 0) {
            eos_uart_write(0, buf, n); // Echo back
        }
    }
}

```

5.5 4.5 SPI Interface

SPI (Serial Peripheral Interface) is a full-duplex synchronous serial bus.

5.5.1 Configuration

```

typedef struct {
    uint8_t      port;           // SPI port number
    uint32_t     clock_hz;      // Clock frequency
    eos_spi_mode_t mode;        // Mode 0-3 (CPOL/CPHA)
    uint8_t      bits_per_word; // Usually 8
    uint16_t     cs_pin;        // Chip-select GPIO pin
} eos_spi_config_t;

```

5.5.2 SPI Mode Reference

Mode	CPOL	CPHA	Clock Idle	Data Sampled
0	0	0	Low	Rising edge
1	0	1	Low	Falling edge
2	1	0	High	Falling edge
3	1	1	High	Rising edge

5.5.3 SPI API Reference

Function	Signature	Description
<code>eos_spi_init</code>	<code>int eos_spi_init(const eos_spi_config_t *cfg)</code>	Initialize SPI port

Function	Signature	Description
<code>eos_spi_deinit</code>	<code>void eos_spi_deinit(uint8_t port)</code>	Release SPI port
<code>eos_spi_transfer</code>	<code>int eos_spi_transfer(uint8_t port, const uint8_t *tx, uint8_t *rx, size_t len)</code>	Full-duplex transfer
<code>eos_spi_write</code>	<code>int eos_spi_write(uint8_t port, const uint8_t *data, size_t len)</code>	Write only
<code>eos_spi_read</code>	<code>int eos_spi_read(uint8_t port, uint8_t *data, size_t len)</code>	Read only

5.5.4 SPI Sensor Read Example

```
uint8_t spi_read_register(uint8_t reg)
{
    uint8_t tx[2] = { reg | 0x80, 0x00 }; // Read bit + register
    uint8_t rx[2] = { 0 };

    eos_spi_transfer(0, tx, rx, 2);
    return rx[1];
}
```

5.6 4.6 I2C Interface

I2C (Inter-Integrated Circuit) is a two-wire bus for communicating with sensors, EEPROMs, and other peripherals.

5.6.1 I2C API Reference

Function	Signature	Description
<code>eos_i2c_init</code>	<code>int eos_i2c_init(const eos_i2c_config_t *cfg)</code>	Initialize I2C port
<code>eos_i2c_deinit</code>	<code>void eos_i2c_deinit(uint8_t port)</code>	Release I2C port
<code>eos_i2c_write</code>	<code>int eos_i2c_write(uint8_t port, uint16_t addr, const uint8_t *data, size_t len)</code>	Write to device

Function	Signature	Description
<code>eos_i2c_read</code>	<code>int eos_i2c_read(uint8_t port, uint16_t addr, uint8_t *data, size_t len)</code>	Read from device
<code>eos_i2c_write_reg</code>	<code>int eos_i2c_write_reg(uint8_t port, uint16_t addr, uint8_t reg, const uint8_t *data, size_t len)</code>	Write to register
<code>eos_i2c_read_reg</code>	<code>int eos_i2c_read_reg(uint8_t port, uint16_t addr, uint8_t reg, uint8_t *data, size_t len)</code>	Read from register

5.6.2 I2C Temperature Sensor Example

```
#include <eos/hal.h>
#include <stdio.h>

#define TMP102_ADDR 0x48
#define TMP102_TEMP 0x00

float read_temperature(void)
{
    uint8_t raw[2];
    eos_i2c_read_reg(0, TMP102_ADDR, TMP102_TEMP, raw, 2);

    int16_t temp = (raw[0] << 4) | (raw[1] >> 4);
    if (temp & 0x800) temp |= 0xF000; // Sign extend
    return temp * 0.0625f;
}

int main(void)
{
    eos_hal_init();

    eos_i2c_config_t i2c_cfg = {
        .port      = 0,
        .clock_hz   = 400000, // 400 kHz (Fast mode)
        .own_addr   = 0,
    };
    eos_i2c_init(&i2c_cfg);
}
```

```

while (1) {
    float temp = read_temperature();
    printf("Temperature: %.2f C\n", temp);
    eos_delay_ms(1000);
}
}

```

5.7 4.7 Timer Interface

Hardware timers provide precise periodic or one-shot callbacks.

5.7.1 Timer API Reference

Function	Signature	Description
<code>eos_timer_init</code>	<code>int eos_timer_init(const eos_timer_config_t *cfg)</code>	Configure timer
<code>eos_timer_deinit</code>	<code>void eos_timer_deinit(uint8_t timer_id)</code>	Release timer
<code>eos_timer_start</code>	<code>int eos_timer_start(uint8_t timer_id)</code>	Start counting
<code>eos_timer_stop</code>	<code>int eos_timer_stop(uint8_t timer_id)</code>	Stop counting
<code>eos_timer_get_count</code>	<code>uint32_t eos_timer_get_count(uint8_t timer_id)</code>	Read counter value

5.7.2 Periodic Timer Example

```

void heartbeat(uint8_t timer_id, void *ctx)
{
    uint32_t *count = (uint32_t *)ctx;
    (*count)++;
    printf("Heartbeat #%u\n", *count);
}

int main(void)
{
    eos_hal_init();

    static uint32_t beat_count = 0;
    eos_timer_config_t tmr = {
        .timer_id = 0,

```

```

        .period_us    = 1000000,  // 1 second
        .auto_reload = true,
        .callback     = heartbeat,
        .ctx          = &beat_count,
    };
    eos_timer_init(&tmr);
    eos_timer_start(0);

    while (beat_count < 10) {
        eos_delay_ms(100);
    }
    eos_timer_stop(0);
}

```

5.8 4.8 Interrupt Control Interface

Global interrupt management for critical sections:

Function	Signature	Description
<code>eos_irq_disable</code>	<code>void</code> <code>eos_irq_disable(void)</code>	Disable all interrupts
<code>eos_irq_enable</code>	<code>void</code> <code>eos_irq_enable(void)</code>	Enable all interrupts
<code>eos_irq_register</code>	<code>int</code> <code>eos_irq_register(uint16_t irq, void (*handler)(void), uint8_t priority)</code>	Register ISR
<code>eos_irq_unregister</code>	<code>void</code> <code>eos_irq_unregister(uint16_t irq)</code>	Remove ISR
<code>eos_irq_set_priority</code>	<code>void</code> <code>eos_irq_set_priority(uint16_t irq, uint8_t priority)</code>	Change priority

5.9 4.9 Backend Dispatch Pattern

The HAL achieves portability through a **backend vtable pattern**. Each platform implements an `eos_hal_backend_t` struct with function pointers for every HAL operation.

5.9.1 The Backend Structure

```

typedef struct {
    const char *name;

```

```

// System
int (*init)(void);
void (*deinit)(void);
void (*delay_ms)(uint32_t ms);
uint32_t (*get_tick_ms)(void);

// GPIO
int (*gpio_init)(const eos_gpio_config_t *cfg);
void (*gpio_deinit)(uint16_t pin);
void (*gpio_write)(uint16_t pin, bool value);
bool (*gpio_read)(uint16_t pin);
void (*gpio_toggle)(uint16_t pin);
int (*gpio_set_irq)(uint16_t pin, eos_gpio_edge_t edge,
                    eos_gpio_callback_t cb, void *ctx);

// UART, SPI, I2C, Timer, Interrupt ...
// (function pointers for each peripheral)
} eos_hal_backend_t;

```

5.9.2 Registering a Backend

At startup, the platform-specific code registers its backend:

```

// backends/stm32/stm32_hal.c
static const eos_hal_backend_t stm32_backend = {
    .name      = "stm32",
    .init      = stm32_hal_init,
    .deinit    = stm32_hal_deinit,
    .delay_ms  = stm32_delay_ms,
    .get_tick_ms = stm32_get_tick_ms,
    .gpio_init = stm32_gpio_init,
    .gpio_write = stm32_gpio_write,
    .gpio_read  = stm32_gpio_read,
    .gpio_toggle = stm32_gpio_toggle,
    // ... all other operations
};

void stm32_register(void)
{
    eos_hal_register_backend(&stm32_backend);
}

```

5.9.3 How Dispatch Works

When application code calls `eos_gpio_write(pin, true)`, the HAL dispatches:

```

eos_gpio_write(pin, true)
    → backend->gpio_write(pin, true)

```

```
→ stm32_gpio_write(pin, true)
→ HAL_GPIO_WritePin(GPIOx, pin_mask, GPIO_PIN_SET)
```

5.10 4.10 Writing a Custom Backend

To port EoS to a new platform:

1. Create `backends/my_platform/my_hal.c`
2. Implement all function pointers in `eos_hal_backend_t`
3. Call `eos_hal_register_backend(&my_backend)` during platform init
4. Add the backend source to `CMakeLists.txt`

Minimal skeleton:

```
#include <eos/hal.h>

static int my_init(void) { /* init clocks, peripherals */ return 0; }
static void my_deinit(void) { /* cleanup */ }
static void my_delay_ms(uint32_t ms) { /* platform delay */ }
static uint32_t my_get_tick_ms(void) { /* return ms counter */ return 0; }

static int my_gpio_init(const eos_gpio_config_t *cfg) { return 0; }
static void my_gpio_write(uint16_t pin, bool val) { /* write register */ }
static bool my_gpio_read(uint16_t pin) { return false; }
static void my_gpio_toggle(uint16_t pin) { /* toggle register */ }
// ... implement remaining peripherals ...

static const eos_hal_backend_t my_backend = {
    .name      = "my_platform",
    .init      = my_init,
    .deinit    = my_deinit,
    .delay_ms  = my_delay_ms,
    .get_tick_ms = my_get_tick_ms,
    .gpio_init = my_gpio_init,
    .gpio_write = my_gpio_write,
    .gpio_read  = my_gpio_read,
    .gpio_toggle = my_gpio_toggle,
};

void my_platform_register(void)
{
    eos_hal_register_backend(&my_backend);
}
```

Next: [Chapter 5 — Extended HAL](#)

Chapter 6

Chapter 5: Extended HAL

Srikanth Patchava & EmbeddedOS Contributors

6.1 5.1 Overview

The Extended HAL (`hal_extended.h`) provides 27 additional peripheral interfaces beyond the core GPIO, UART, SPI, I2C, and Timer covered in Chapter 4. Each peripheral is **conditionally compiled** based on `EOS_ENABLE_*` flags set by the product profile.

6.1.1 Complete Peripheral List

#	Peripheral	Enable Flag	Category
1	ADC	<code>EOS_ENABLE_ADC</code>	Analog
2	DAC	<code>EOS_ENABLE_DAC</code>	Analog
3	PWM	<code>EOS_ENABLE_PWM</code>	Analog
4	CAN	<code>EOS_ENABLE_CAN</code>	Bus
5	USB	<code>EOS_ENABLE_USB</code>	Bus
6	Ethernet	<code>EOS_ENABLE_ETHERNET</code>	Networking
7	WiFi	<code>EOS_ENABLE_WIFI</code>	Networking
8	BLE	<code>EOS_ENABLE_BLE</code>	Networking
9	Camera	<code>EOS_ENABLE_CAMERA</code>	Media
10	Audio	<code>EOS_ENABLE_AUDIO</code>	Media
11	Display	<code>EOS_ENABLE_DISPLAY</code>	Media
12	Motor	<code>EOS_ENABLE_MOTOR</code>	Actuators
13	GNSS	<code>EOS_ENABLE_GNSS</code>	Sensors
14	IMU	<code>EOS_ENABLE_IMU</code>	Sensors
15	Touch	<code>EOS_ENABLE_TOUCH</code>	Input
16	RTC	<code>EOS_ENABLE_RTC</code>	Timekeeping
17	DMA	<code>EOS_ENABLE_DMA</code>	System
18	Flash	<code>EOS_ENABLE_FLASH</code>	Storage
19	WDT	<code>EOS_ENABLE_WDT</code>	System
20	NFC	<code>EOS_ENABLE_NFC</code>	Wireless

#	Peripheral	Enable Flag	Category
21	IR	<code>EOS_ENABLE_IR</code>	Wireless
22	Cellular	<code>EOS_ENABLE_CELLULAR</code>	Networking
23	Radar	<code>EOS_ENABLE_RADAR</code>	Sensors
24	GPU	<code>EOS_ENABLE_GPU</code>	Graphics
25	HDMI	<code>EOS_ENABLE_HDMI</code>	Graphics
26	PCIe	<code>EOS_ENABLE_PCIE</code>	Bus
27	SDIO	<code>EOS_ENABLE_SDIO</code>	Storage
28	Haptics	<code>EOS_ENABLE_HAPTICS</code>	Actuators

6.2 5.2 Analog Peripherals

6.2.1 ADC — Analog-to-Digital Converter

```
typedef struct {
    uint8_t channel;           // ADC channel number
    uint8_t resolution_bits;   // 8, 10, 12, or 16
    uint32_t sample_rate_hz;   // Samples per second
} eos_adc_config_t;
```

Function	Description
<code>eos_adc_init(cfg)</code>	Initialize ADC channel
<code>eos_adc_deinit(ch)</code>	Release ADC channel
<code>eos_adc_read(ch)</code>	Read raw ADC value
<code>eos_adc_read_mv(ch)</code>	Read in millivolts

Example: Reading a potentiometer

```
eos_adc_config_t adc = {
    .channel      = 0,
    .resolution_bits = 12,
    .sample_rate_hz = 1000,
};
eos_adc_init(&adc);

uint32_t raw = eos_adc_read(0);           // 0-4095 for 12-bit
uint32_t mv  = eos_adc_read_mv(0);        // Millivolts
printf("ADC: raw=%u voltage=%u mV\n", raw, mv);
```

6.2.2 DAC — Digital-to-Analog Converter

Function	Description
<code>eos_dac_init(cfg)</code>	Initialize DAC channel
<code>eos_dac_deinit(ch)</code>	Release DAC channel

Function	Description
<code>eos_dac_write(ch, val)</code>	Write raw DAC value
<code>eos_dac_write_mv(ch, mv)</code>	Write in millivolts

6.2.3 PWM — Pulse Width Modulation

```
typedef struct {
    uint8_t  channel;           // PWM channel
    uint32_t frequency_hz;      // PWM frequency
    uint16_t duty_pct_x10;      // 0-1000 (0.0%-100.0%)
} eos_pwm_config_t;
```

Function	Description
<code>eos_pwm_init(cfg)</code>	Initialize PWM channel
<code>eos_pwm_set_duty(ch, pct)</code>	Set duty cycle (0–1000)
<code>eos_pwm_set_freq(ch, hz)</code>	Change frequency
<code>eos_pwm_start(ch)</code>	Start PWM output
<code>eos_pwm_stop(ch)</code>	Stop PWM output

Example: LED dimming

```
eos_pwm_config_t pwm = {
    .channel      = 0,
    .frequency_hz = 1000,
    .duty_pct_x10 = 500, // 50.0%
};
eos_pwm_init(&pwm);
eos_pwm_start(0);

// Fade from 0% to 100%
for (uint16_t d = 0; d <= 1000; d += 10) {
    eos_pwm_set_duty(0, d);
    eos_delay_ms(20);
}
```

6.3 5.3 Bus Interfaces

6.3.1 CAN — Controller Area Network

```
typedef struct {
    uint32_t id;           // CAN identifier
    bool     extended;     // true = 29-bit, false = 11-bit
    bool     rtr;          // Remote Transmission Request
    uint8_t  dlc;          // Data Length Code (0-8)
```

```
uint8_t data[8];    // Payload
} eos_can_msg_t;
```

Function	Description
<code>eos_can_init(cfg)</code>	Initialize CAN port
<code>eos_can_send(port, msg)</code>	Transmit CAN frame
<code>eos_can_receive(port, msg, timeout)</code>	Receive CAN frame
<code>eos_can_set_filter(port, id, mask)</code>	Set acceptance filter

6.3.2 USB — Universal Serial Bus (Device Mode)

Function	Description
<code>eos_usb_init(cfg)</code>	Initialize USB device
<code>eos_usb_write(port, ep, data, len)</code>	Write to endpoint
<code>eos_usb_read(port, ep, buf, len, timeout)</code>	Read from endpoint
<code>eos_usb_is_connected(port)</code>	Check connection status

6.4 5.4 Networking Interfaces

6.4.1 WiFi

```
typedef struct {
    char            ssid[33];
    char            password[65];
    eos_wifi_security_t security; // OPEN, WPA2, WPA3
} eos_wifi_config_t;
```

Complete WiFi workflow:

```
eos_wifi_init();

eos_wifi_config_t wifi = {
    .ssid      = "MyNetwork",
    .password  = "secret123",
    .security  = EOS_WIFI_SEC_WPA2,
};

eos_wifi_connect(&wifi);

// Wait for connection
while (!eos_wifi_is_connected()) {
    eos_delay_ms(100);
}

uint32_t ip;
eos_wifi_get_ip(&ip);
```

```
printf("Connected! IP: %d.%d.%d.%d\n",
      ip & 0xFF, (ip >> 8) & 0xFF,
      (ip >> 16) & 0xFF, (ip >> 24) & 0xFF);
```

6.4.2 BLE — Bluetooth Low Energy

Function	Description
<code>eos_ble_init(cfg)</code>	Initialize BLE stack
<code>eos_ble_advertise_start()</code>	Start advertising
<code>eos_ble_advertise_stop()</code>	Stop advertising
<code>eos_ble_connect(addr)</code>	Connect to peripheral
<code>eos_ble_disconnect()</code>	Disconnect
<code>eos_ble_is_connected()</code>	Check connection
<code>eos_ble_send(data, len)</code>	Transmit data
<code>eos_ble_set_rx_callback(cb, ctx)</code>	Register RX handler

6.4.3 Cellular / Modem

Supports 2G through 5G, NB-IoT, and LTE Cat-M:

```
typedef enum {
    EOS_CELL_2G      = 0,
    EOS_CELL_3G      = 1,
    EOS_CELL_4G      = 2,
    EOS_CELL_5G      = 3,
    EOS_CELL_NB      = 4,  // NB-IoT
    EOS_CELL_CAT_M   = 5,  // LTE Cat-M
} eos_cellular_tech_t;
```

Function	Description
<code>eos_cellular_init(cfg)</code>	Initialize modem
<code>eos_cellular_connect()</code>	Establish data connection
<code>eos_cellular_get_status(status)</code>	Get signal/registration info
<code>eos_cellular_send(data, len)</code>	Send data
<code>eos_cellular_sms_send(number, msg)</code>	Send SMS

6.5 5.5 Media Interfaces

6.5.1 Camera

```
typedef struct {
    uint8_t      id;
    uint16_t     width;
    uint16_t     height;
    eos_camera_format_t format;  // RGB565, RGB888, YUV422, JPEG, GRAY8
```

```

    uint8_t      fps;
} eos_camera_config_t;

typedef struct {
    uint8_t      *data;
    size_t       size;
    uint16_t     width, height;
    eos_camera_format_t format;
    uint32_t     timestamp_ms;
} eos_camera_frame_t;

```

6.5.2 Audio

Function	Description
<code>eos_audio_init(cfg)</code>	Initialize audio codec
<code>eos_audio_play(id, data, len)</code>	Play audio buffer
<code>eos_audio_record(id, buf, len, timeout)</code>	Record audio
<code>eos_audio_set_volume(id, pct)</code>	Set volume (0–100%)
<code>eos_audio_mute(id, mute)</code>	Mute/unmute

6.5.3 Display

```

int eos_display_init(const eos_display_config_t *cfg);
int eos_display_draw_pixel(uint8_t id, uint16_t x, uint16_t y, uint32_t color);
int eos_display_draw_rect(uint8_t id, uint16_t x, uint16_t y,
                          uint16_t w, uint16_t h, uint32_t color);
int eos_display_draw_bitmap(uint8_t id, uint16_t x, uint16_t y,
                            uint16_t w, uint16_t h, const uint8_t *data);
int eos_display_flush(uint8_t id);
int eos_display_clear(uint8_t id, uint32_t color);
int eos_display_set_brightness(uint8_t id, uint8_t brightness_pct);

```

6.6 5.6 Sensor Interfaces

6.6.1 GNSS (GPS)

```

typedef struct {
    double      latitude;
    double      longitude;
    float       altitude_m;
    float       speed_mps;
    float       heading_deg;
    uint8_t     satellites;
    bool        fix_valid;
    uint32_t    utc_time;
}

```

```
} eos_gnss_position_t;
```

6.6.2 IMU — Inertial Measurement Unit

```
typedef struct { float x, y, z; } eos_imu_vec3_t;

int eos_imu_read_accel(uint8_t id, eos_imu_vec3_t *accel);
int eos_imu_read_gyro(uint8_t id, eos_imu_vec3_t *gyro);
int eos_imu_read_mag(uint8_t id, eos_imu_vec3_t *mag);
int eos_imu_read_temp(uint8_t id, float *temp_c);
```

6.6.3 Radar / Lidar

Supports ultrasonic, LiDAR, mmWave, and Time-of-Flight sensors:

```
typedef struct {
    float    distance_mm;
    float    velocity_mps;
    float    angle_deg;
    uint8_t  signal_strength;
    bool     valid;
} eos_radar_measurement_t;
```

6.7 5.7 System & Storage Peripherals

6.7.1 DMA — Direct Memory Access

```
typedef struct {
    uint8_t          channel;
    eos_dma_direction_t direction; // MEM_TO_MEM, MEM_TO_PERIPH, PERIPH_TO_MEM
    eos_dma_data_width_t data_width; // 8, 16, or 32 bit
    bool             circular;
    uint8_t          priority; // 0=low, 3=very high
} eos_dma_config_t;
```

6.7.2 Flash / EEPROM

Function	Description
<code>eos_flash_init(cfg)</code>	Initialize flash device
<code>eos_flash_read(id, addr, buf, len)</code>	Read from flash
<code>eos_flash_write(id, addr, data, len)</code>	Write to flash
<code>eos_flash_erase_sector(id, addr)</code>	Erase a sector
<code>eos_flash_erase_chip(id)</code>	Full chip erase
<code>eos_flash_get_info(id, info)</code>	Get flash geometry

6.7.3 RTC — Real-Time Clock

```
typedef struct {
    uint16_t year;
    uint8_t month, day;
    uint8_t hour, minute, second;
    uint8_t weekday; // 0=Sun, 6=Sat
} eos_rtc_time_t;
```

6.7.4 Watchdog Timer

Function	Description
<code>eos_wdt_init(cfg)</code>	Configure watchdog
<code>eos_wdt_start()</code>	Start watchdog
<code>eos_wdt_feed()</code>	Reset/kick watchdog counter
<code>eos_wdt_stop()</code>	Stop watchdog

6.8 5.8 The Extended Backend vtable

Hardware vendors implement the `eos_hal_ext_backend_t` vtable to provide their own drivers for extended peripherals.

```
typedef struct {
    const char *name;

    // BLE operations (conditionally compiled)
    int (*ble_init)(const eos_ble_config_t *cfg);
    void (*ble_deinit)(void);
    int (*ble_advertise_start)(void);
    int (*ble_send)(const uint8_t *data, size_t len);

    // WiFi operations
    int (*wifi_init)(const eos_wifi_config_t *cfg);
    int (*wifi_connect)(const char *ssid, const char *password);

    // Camera, Display, Touch, Motor, GNSS, IMU, Audio, GPU, HDMI...
    // (each peripheral has its own set of function pointers)
} eos_hal_ext_backend_t;
```

6.8.1 Registering an Extended Backend

```
static const eos_hal_ext_backend_t my_nrf_ble = {
    .name = "nrf52_ble",
    .ble_init = nrf_ble_init,
    .ble_deinit = nrf_ble_deinit,
    .ble_advertise_start = nrf_ble_adv_start,
```

```

    .ble_advertise_stop = nrf_ble_adv_stop,
    .ble_send           = nrf_ble_send,
    .ble_set_rx_callback = nrf_ble_set_rx_cb,
};

void nrf_platform_init(void)
{
    eos_hal_register_ext_backend(&my_nrf_ble);
}

```

Multiple backends can be registered — later registrations override earlier ones for the same peripheral. This allows vendor-specific optimizations while maintaining the standard API.

6.8.2 Retrieving the Active Backend

```

const eos_hal_ext_backend_t *ext = eos_hal_get_ext_backend();
if (ext && ext->ble_init) {
    ext->ble_init(&ble_cfg);
}

```

6.9 5.9 Conditional Compilation

Each extended peripheral is guarded by an `EOS_ENABLE_*` preprocessor flag. If the flag is not defined (or is 0), the peripheral code is completely excluded from the binary.

This is critical for resource-constrained targets where every kilobyte matters.

Example: A product that only needs BLE and Display

```

// products/my_wearable.h
#define EOS_ENABLE_BLE      1
#define EOS_ENABLE_DISPLAY 1
// All other EOS_ENABLE_* are implicitly 0

```

Only BLE and Display code is compiled. CAN, USB, Camera, GPU, etc. add zero bytes.

6.10 5.10 Summary

The Extended HAL gives EoS coverage across the full spectrum of embedded hardware:

Category	Peripherals
Analog	ADC, DAC, PWM
Bus	CAN, USB, PCIe, SDIO
Networking	WiFi, BLE, Ethernet, Cellular
Media	Camera, Audio, Display, HDMI
Sensors	IMU, GNSS, Radar
Input	Touch, NFC, IR
System	DMA, Flash, RTC, WDT

Category	Peripherals
Actuators	Motor, Haptics
Graphics	GPU, HDMI

Next: [Chapter 6 — RTOS Kernel](#)

Chapter 7

Chapter 6: RTOS Kernel

Srikanth Patchava & EmbeddedOS Contributors

7.1 6.1 Overview

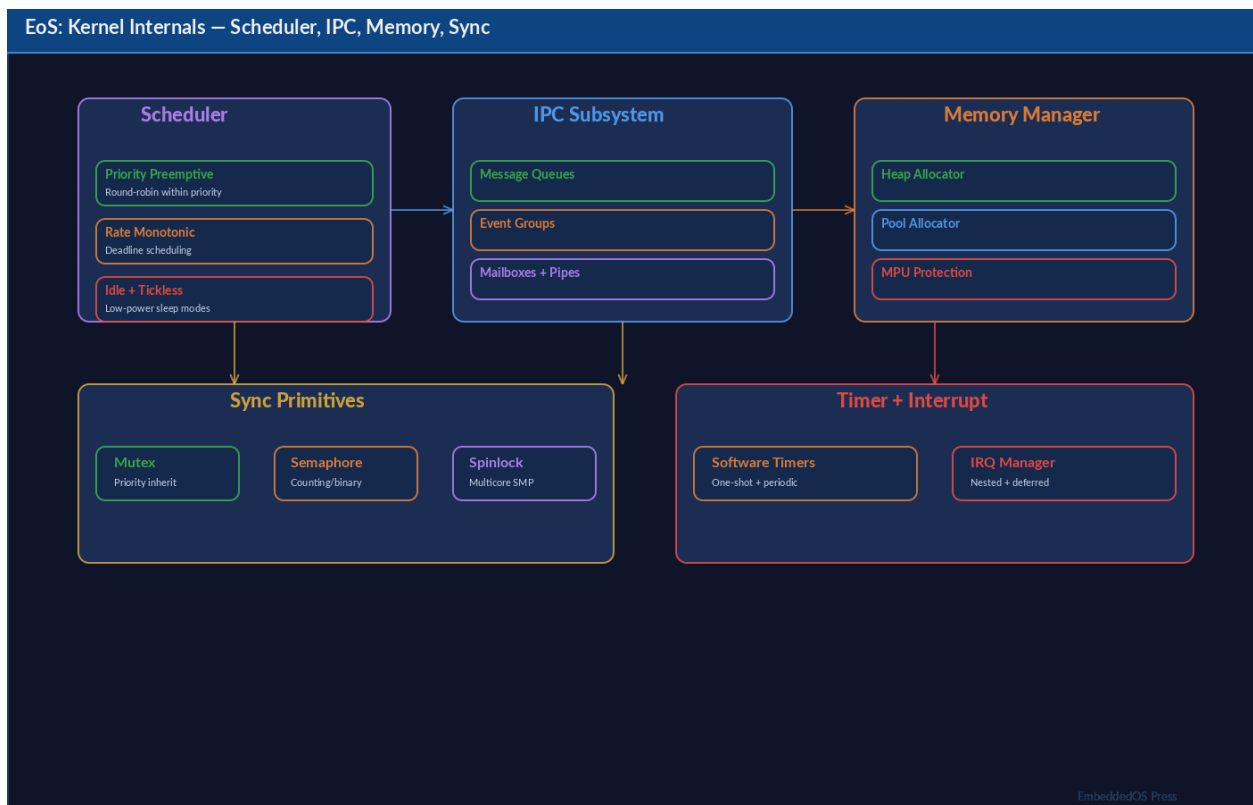


Figure 7.1: Figure: EoS Kernel Internals — scheduler, IPC, memory management, sync primitives

The EoS kernel is a lightweight, preemptive real-time operating system designed for resource-constrained embedded systems. It provides:

- **Task management** with priority-based preemptive scheduling
- **Mutexes** for mutual exclusion with priority inheritance
- **Counting semaphores** for resource counting and signaling
- **Message queues** for inter-task communication
- **Software timers** for deferred execution

All kernel objects use lightweight 8-bit handles for minimal memory overhead.

7.1.1 Kernel Configuration Defaults

Macro	Default	Description
<code>EOS_MAX_TASKS</code>	16	Maximum concurrent tasks
<code>EOS_MAX_MutexES</code>	8	Maximum mutexes
<code>EOS_MAX_SEMAPHORES</code>	8	Maximum semaphores
<code>EOS_MAX_QUEUES</code>	8	Maximum message queues
<code>EOS_MAX_TIMERS</code>	8	Maximum software timers
<code>EOS_WAIT_FOREVER</code>	0xFFFFFFFF	Infinite timeout
<code>EOS_NO_WAIT</code>	0	Non-blocking operation

7.1.2 Return Codes

Code	Value	Meaning
<code>EOS_KERN_OK</code>	0	Success
<code>EOS_KERN_ERR</code>	-1	Generic error
<code>EOS_KERN_TIMEOUT</code>	-2	Operation timed out
<code>EOS_KERN_FULL</code>	-3	Queue/pool is full
<code>EOS_KERN_EMPTY</code>	-4	Queue/pool is empty
<code>EOS_KERN_INVALID</code>	-5	Invalid parameter
<code>EOS_KERN_NO_MEMORY</code>	-6	Out of memory

7.2 6.2 Kernel Lifecycle

```
#include <eos/kernel.h>

int main(void)
{
    eos_hal_init();
    eos_kernel_init();

    // Create tasks, mutexes, queues, etc.
    eos_task_create("main_task", main_task_fn, NULL, 5, 1024);

    // Start the scheduler - this never returns
    eos_kernel_start();
}
```

```

    return 0; // Never reached
}

```

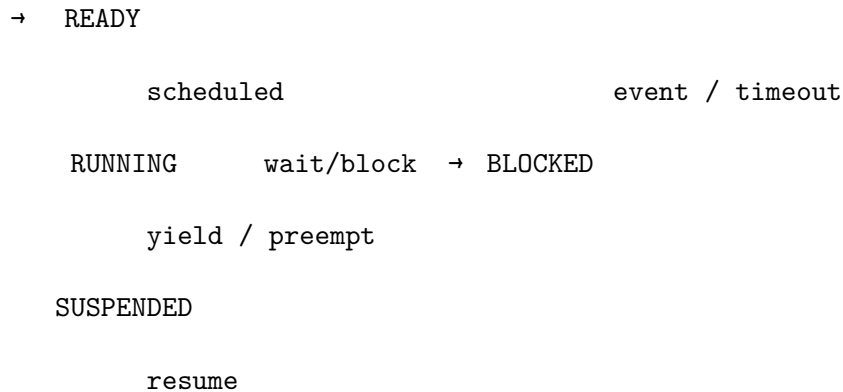
7.2.1 Kernel API

Function	Signature	Description
<code>eos_kernel_init</code>	<code>int</code> <code>eos_kernel_init(void)</code>	Initialize kernel data structures
<code>eos_kernel_start</code>	<code>void</code> <code>eos_kernel_start(void)</code>	Start scheduler (does not return)
<code>eos_kernel_is_running</code>	<code>bool</code> <code>eos_kernel_is_running(void)</code>	Check if scheduler is active
<code>eos_kernel_tick</code>	<code>void</code> <code>eos_kernel_tick(void)</code>	Called from timer ISR

7.3 6.3 Task Management

Tasks are the fundamental unit of execution. Each task has its own stack, priority level, and entry function.

7.3.1 Task States



State	Value	Description
<code>EOS_TASK_READY</code>	0	Eligible to run
<code>EOS_TASK_RUNNING</code>	1	Currently executing
<code>EOS_TASK_BLOCKED</code>	2	Waiting on resource
<code>EOS_TASK_SUSPENDED</code>	3	Manually suspended
<code>EOS_TASK_DELETED</code>	4	Marked for cleanup

7.3.2 Task API Reference

Function	Signature	Description
<code>eos_task_create</code>	int <code>eos_task_create(const char *name, eos_task_func_t entry, void *arg, uint8_t priority, uint32_t stack_size)</code>	Create a new task
<code>eos_task_delete</code>	int <code>eos_task_delete(eos_task_handle_t h)</code>	Delete a task
<code>eos_task_suspend</code>	int <code>eos_task_suspend(eos_task_handle_t h)</code>	Suspend a task
<code>eos_task_resume</code>	int <code>eos_task_resume(eos_task_handle_t h)</code>	Resume a suspended task
<code>eos_task_yield</code>	void <code>eos_task_yield(void)</code>	Yield to other ready tasks
<code>eos_task_delay_ms</code>	void <code>eos_task_delay_ms(uint32_t ms)</code>	Sleep for N milliseconds
<code>eos_task_get_current</code>	<code>eos_task_handle_t</code> <code>eos_task_get_current(void)</code>	Get current task handle
<code>eos_task_get_state</code>	<code>eos_task_state_t</code> <code>eos_task_get_state(eos_task_handle_t h)</code>	Query task state
<code>eos_task_get_name</code>	const char <code>*eos_task_get_name(eos_task_handle_t h)</code>	Get task name

7.3.3 Example: Multiple Tasks

```

#include <eos/hal.h>
#include <eos/kernel.h>
#include <stdio.h>

void sensor_task(void *arg)
{
    while (1) {
        printf("[sensor] Reading sensor data...\n");
        eos_task_delay_ms(1000);
    }
}

void display_task(void *arg)

```

```

{
    while (1) {
        printf("[display] Updating display...\n");
        eos_task_delay_ms(500);
    }
}

void comms_task(void *arg)
{
    while (1) {
        printf("[comms] Sending data...\n");
        eos_task_delay_ms(2000);
    }
}

int main(void)
{
    eos_hal_init();
    eos_kernel_init();

    eos_task_create("sensor", sensor_task, NULL, 5, 1024);
    eos_task_create("display", display_task, NULL, 4, 1024);
    eos_task_create("comms", comms_task, NULL, 3, 2048);

    eos_kernel_start();
    return 0;
}

```

7.3.4 Priority Guidelines

Priority	Use Case
0 (lowest)	Idle / background tasks
1–3	Low priority: logging, diagnostics
4–6	Normal: application logic, UI
7–9	High: communication, sensor sampling
10+	Critical: safety, motor control, ISR-driven

7.4 6.4 Mutex

Mutexes provide mutual exclusion for protecting shared resources.

7.4.1 Mutex API

Function	Signature	Description
<code>eos_mutex_create</code>	<code>int eos_mutex_create(eos_mutex_handle_t *out)</code>	Create mutex
<code>eos_mutex_lock</code>	<code>int eos_mutex_lock(eos_mutex_handle_t h, uint32_t timeout_ms)</code>	Acquire lock
<code>eos_mutex_unlock</code>	<code>int eos_mutex_unlock(eos_mutex_handle_t h)</code>	Release lock
<code>eos_mutex_delete</code>	<code>int eos_mutex_delete(eos_mutex_handle_t h)</code>	Destroy mutex

7.4.2 Example: Protected Shared Counter

```
static eos_mutex_handle_t counter_lock;
static volatile uint32_t shared_counter = 0;

void increment_task(void *arg)
{
    while (1) {
        eos_mutex_lock(counter_lock, EOS_WAIT_FOREVER);
        shared_counter++;
        printf("Counter: %u\n", shared_counter);
        eos_mutex_unlock(counter_lock);
        eos_task_delay_ms(100);
    }
}

int main(void)
{
    eos_hal_init();
    eos_kernel_init();

    eos_mutex_create(&counter_lock);
    eos_task_create("inc1", increment_task, NULL, 5, 512);
    eos_task_create("inc2", increment_task, NULL, 5, 512);

    eos_kernel_start();
    return 0;
}
```

7.5 6.5 Semaphore

Counting semaphores are used for resource counting and task synchronization.

7.5.1 Semaphore API

Function	Signature	Description
<code>eos_sem_create</code>	<code>int eos_sem_create(eos_sem_handle_t *out, uint32_t initial, uint32_t max)</code>	Create semaphore
<code>eos_sem_wait</code>	<code>int eos_sem_wait(eos_sem_handle_t h, uint32_t timeout_ms)</code>	Decrement (block if 0)
<code>eos_sem_post</code>	<code>int eos_sem_post(eos_sem_handle_t h)</code>	Increment
<code>eos_sem_delete</code>	<code>int eos_sem_delete(eos_sem_handle_t h)</code>	Destroy semaphore
<code>eos_sem_get_count</code>	<code>uint32_t eos_sem_get_count(eos_sem_handle_t h)</code>	Current count

7.5.2 Example: Producer-Consumer with Semaphore

```
static eos_sem_handle_t data_ready;
static int sensor_value = 0;

void producer(void *arg)
{
    while (1) {
        sensor_value = eos_adc_read(0);    // Read sensor
        eos_sem_post(data_ready);          // Signal consumer
        eos_task_delay_ms(100);
    }
}

void consumer(void *arg)
{
    while (1) {
        eos_sem_wait(data_ready, EOS_WAIT_FOREVER);
        printf("Sensor: %d\n", sensor_value);
    }
}

int main(void)
{
    eos_hal_init();
    eos_kernel_init();
}
```



```

eos_sem_create(&data_ready, 0, 1); // Binary semaphore
eos_task_create("producer", producer, NULL, 6, 512);
eos_task_create("consumer", consumer, NULL, 5, 512);

eos_kernel_start();
return 0;
}

```

7.6 6.6 Message Queues

Message queues enable type-safe inter-task communication by passing fixed-size data items between tasks.

7.6.1 Queue API

Function	Signature	Description
<code>eos_queue_create</code>	<code>int eos_queue_create(eos_queue_handle_t *out, size_t item_size, uint32_t capacity)</code>	Create queue
<code>eos_queue_send</code>	<code>int eos_queue_send(eos_queue_handle_t h, const void *item, uint32_t timeout_ms)</code>	Enqueue item
<code>eos_queue_receive</code>	<code>int eos_queue_receive(eos_queue_handle_t h, void *item, uint32_t timeout_ms)</code>	Dequeue item
<code>eos_queue_peek</code>	<code>int eos_queue_peek(eos_queue_handle_t h, void *item)</code>	Peek without removing
<code>eos_queue_delete</code>	<code>int eos_queue_delete(eos_queue_handle_t h)</code>	Destroy queue
<code>eos_queue_count</code>	<code>uint32_t eos_queue_count(eos_queue_handle_t h)</code>	Items in queue
<code>eos_queue_is_full</code>	<code>bool eos_queue_is_full(eos_queue_handle_t h)</code>	Check if full
<code>eos_queue_is_empty</code>	<code>bool eos_queue_is_empty(eos_queue_handle_t h)</code>	Check if empty

7.6.2 Example: Sensor Data Pipeline

```
typedef struct {
    uint16_t sensor_id;
    int32_t value;
    uint32_t timestamp;
} sensor_event_t;

static eos_queue_handle_t event_queue;

void sensor_task(void *arg)
{
    while (1) {
        sensor_event_t evt = {
            .sensor_id = 1,
            .value      = eos_adc_read(0),
            .timestamp  = eos_get_tick_ms(),
        };
        eos_queue_send(event_queue, &evt, EOS_WAIT_FOREVER);
        eos_task_delay_ms(100);
    }
}

void logger_task(void *arg)
{
    sensor_event_t evt;
    while (1) {
        if (eos_queue_receive(event_queue, &evt, 5000) == EOS_KERN_OK) {
            printf("[%u] Sensor %u = %d\n",
                   evt.timestamp, evt.sensor_id, evt.value);
        } else {
            printf("No data for 5 seconds\n");
        }
    }
}

int main(void)
{
    eos_hal_init();
    eos_kernel_init();

    eos_queue_create(&event_queue, sizeof(sensor_event_t), 16);
    eos_task_create("sensor", sensor_task, NULL, 6, 1024);
    eos_task_create("logger", logger_task, NULL, 4, 1024);

    eos_kernel_start();
    return 0;
}
```

```
}
```

7.7 6.7 Software Timers

Software timers execute callbacks at specified intervals without requiring a dedicated task or hardware timer.

7.7.1 Timer API

Function	Signature	Description
<code>eos_swtimer_create</code>	<code>int eos_swtimer_create(eos_swtimer_handle_t *out, const char *name, uint32_t period_ms, bool auto_reload, eos_swtimer_callback_t cb, void *ctx)</code>	Create timer
<code>eos_swtimer_start</code>	<code>int eos_swtimer_start(eos_swtimer_handle_t h)</code>	Start timer
<code>eos_swtimer_stop</code>	<code>int eos_swtimer_stop(eos_swtimer_handle_t h)</code>	Stop timer
<code>eos_swtimer_reset</code>	<code>int eos_swtimer_reset(eos_swtimer_handle_t h)</code>	Reset period
<code>eos_swtimer_delete</code>	<code>int eos_swtimer_delete(eos_swtimer_handle_t h)</code>	Destroy timer

7.7.2 Example: Watchdog Feed Timer

```
void wdt_feed_cb(eos_swtimer_handle_t h, void *ctx)
{
    eos_wdt_feed();
    printf("Watchdog fed at %u ms\n", eos_get_tick_ms());
}

int main(void)
{
    eos_hal_init();
    eos_kernel_init();

    eos_swtimer_handle_t wdt_timer;
    eos_swtimer_create(&wdt_timer, "wdt_feed", 500, true,
```

```

        wdt_feed_cb, NULL);
eos_swtimer_start(wdt_timer);

eos_kernel_start();
return 0;
}

```

7.8 6.8 Common Patterns

7.8.1 Pattern: Timeout with Retry

```

int send_with_retry(eos_queue_handle_t q, const void *item, int max_retries)
{
    for (int i = 0; i < max_retries; i++) {
        int rc = eos_queue_send(q, item, 100); // 100ms timeout
        if (rc == EOS_KERN_OK) return 0;
        if (rc != EOS_KERN_TIMEOUT) return rc;
    }
    return EOS_KERN_TIMEOUT;
}

```

7.8.2 Pattern: Event Loop

```

void app_task(void *arg)
{
    app_event_t evt;
    while (1) {
        if (eos_queue_receive(app_queue, &evt, EOS_WAIT_FOREVER) == EOS_KERN_OK) {
            switch (evt.type) {
                case EVT_BUTTON: handle_button(&evt); break;
                case EVT_SENSOR: handle_sensor(&evt); break;
                case EVT_TIMER:  handle_timer(&evt);  break;
            }
        }
    }
}

```

7.9 6.9 Summary

Primitive	Use Case	Typical Capacity
Task	Independent execution unit	16 max
Mutex	Protect shared resources	8 max
Semaphore	Counting/signaling	8 max
Queue	Inter-task data passing	8 queues, variable depth
SW Timer	Deferred callbacks	8 max

Next: [Chapter 7 — Multicore](#)

Chapter 8

Chapter 7: Multicore

Srikanth Patchava & EmbeddedOS Contributors

8.1 7.1 Overview

Modern embedded SoCs frequently include multiple processor cores. EoS provides a comprehensive multicore framework supporting both **SMP** (Symmetric Multi-Processing) and **AMP** (Asymmetric Multi-Processing) topologies.

The multicore API (`multicore.h`) provides:

- **Core management** — start, stop, and query cores
- **Spinlocks** — low-level multi-core synchronization
- **Inter-Processor Interrupts (IPI)** — cross-core signaling
- **Shared memory** — named memory regions with cache management
- **Task core affinity** — bind tasks to specific cores
- **Remote processor control** — manage heterogeneous co-processors
- **Memory barriers and atomics** — architecture-portable primitives

All multicore functionality is conditionally compiled under `EOS_ENABLE_MULTICORE`.

8.1.1 Key Constants

Macro	Value	Description
<code>EOS_MAX_CORES</code>	16	Maximum supported cores
<code>EOS_MAX_IPC_CHANNELS</code>	8	Maximum IPC channels

8.2 7.2 SMP vs. AMP

Feature	SMP	AMP
OS image	Same on all cores	Different per core
Memory	Shared address space	May be separate

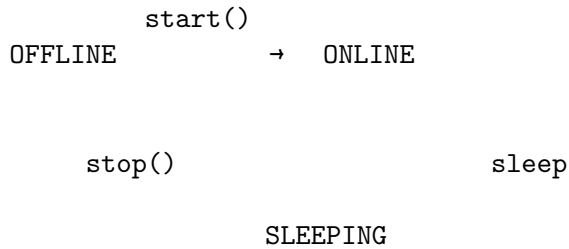
Feature	SMP	AMP
Scheduling	Global scheduler	Per-core schedulers
Use case	Linux-class SoCs	Heterogeneous (A-core + M-core)
EoS mode	EOS_MP_SMP	EOS_MP_AMP

```
// Initialize for SMP
eos_multicore_init(EOS_MP_SMP);

// Initialize for AMP (e.g., STM32MP1: Cortex-A7 + Cortex-M4)
eos_multicore_init(EOS_MP_AMP);
```

8.3 7.3 Core Management

8.3.1 Core States



State	Value	Description
EOS_CORE_OFFLINE	0	Core powered down or not started
EOS_CORE_ONLINE	1	Core executing code
EOS_CORE_SLEEPING	2	Core in low-power sleep
EOS_CORE_HALTED	3	Core stopped due to error

8.3.2 Core API

Function	Signature	Description
eos_multicore_init	int eos_multicore_init(eos_mp_mode_t mode)	Initialize multicore subsystem
eos_multicore_deinit	void eos_multicore_deinit(void)	Shutdown multicore
eos_core_count	uint8_t eos_core_count(void)	Number of active cores
eos_core_id	uint8_t eos_core_id(void)	Current core ID

Function	Signature	Description
<code>eos_core_start</code>	int <code>eos_core_start(uint8_t id, eos_core_entry_fn entry, void *arg)</code>	Boot a secondary core
<code>eos_core_stop</code>	int <code>eos_core_stop(uint8_t id)</code>	Stop a core (cannot stop core 0)
<code>eos_core_get_info</code>	int <code>eos_core_get_info(uint8_t id, eos_core_info_t *info)</code>	Query core state

8.3.3 Example: Booting a Secondary Core

```

void core1_main(void *arg)
{
    printf("Core %d is alive! arg=%p\n", eos_core_id(), arg);

    while (1) {
        // Core 1 workload
        eos_delay_ms(1000);
    }
}

int main(void)
{
    eos_hal_init();
    eos_multicore_init(EOS_MP_SMP);

    printf("Core 0 starting core 1...\n");
    eos_core_start(1, core1_main, NULL);

    printf("Cores online: %d\n", eos_core_count());

    while (1) {
        eos_delay_ms(1000);
    }
}

```

8.3.4 Architecture-Specific Core ID Detection

The `eos_core_id()` function uses inline assembly for each architecture:

Architecture	Instruction	Register
AArch64	<code>mrs x0, mpidr_el1</code>	MPIDR_EL1[7:0]
ARMv7	<code>mrc p15, 0, r0, c0, c0, 5</code>	MPIDR[1:0]
x86_64	<code>cpuid (leaf 1)</code>	EBX[31:24] (APIC ID)
RISC-V	<code>csrr a0, mhartid</code>	mhartid

8.4 7.4 Spinlocks

Spinlocks are the lowest-level synchronization primitive for multi-core systems. They busy-wait and should only be held for very short durations.

8.4.1 Spinlock API

Function	Signature	Description
<code>eos_spin_init</code>	<code>void eos_spin_init(eos_spinlock_t *lock)</code>	Initialize spinlock
<code>eos_spin_lock</code>	<code>void eos_spin_lock(eos_spinlock_t *lock)</code>	Acquire (busy-wait)
<code>eos_spin_trylock</code>	<code>bool eos_spin_trylock(eos_spinlock_t *lock)</code>	Try acquire (non-blocking)
<code>eos_spin_unlock</code>	<code>void eos_spin_unlock(eos_spinlock_t *lock)</code>	Release
<code>eos_spin_lock_irqsave</code>	<code>void eos_spin_lock_irqsave(eos_spinlock_t *lock, uint32_t *flags)</code>	Acquire + disable IRQs
<code>eos_spin_unlock_irqrestore</code>	<code>void eos_spin_unlock_irqrestore(eos_spinlock_t *lock, uint32_t flags)</code>	Release + restore IRQs

8.4.2 Architecture-Specific Spin Hints

The spinlock implementation uses architecture-specific wait hints to reduce power consumption during contention:

Architecture	Lock Wait Hint	Unlock Wake Hint
ARM/AArch64	<code>wfe</code> (Wait For Event)	<code>sev</code> (Send Event)
x86	<code>pause</code>	(none needed)
RISC-V	(spin only)	(none)

8.4.3 Implementation Detail

```
void eos_spin_lock(eos_spinlock_t *lock)
{
    while (__atomic_exchange_n(lock, 1, __ATOMIC_ACQUIRE)) {
        #if defined(__aarch64__) || defined(__arm__)
            __asm__ volatile("wfe");    // Power-efficient wait
        #elif defined(__x86_64__)
            __asm__ volatile("pause"); // Reduce pipeline stall
        #endif
    }
}

void eos_spin_unlock(eos_spinlock_t *lock)
{
    __atomic_store_n(lock, 0, __ATOMIC_RELEASE);
    #if defined(__aarch64__) || defined(__arm__)
        __asm__ volatile("sev"); // Wake waiting cores
    #endif
}
```

8.4.4 IRQ-Safe Spinlocks

When a spinlock protects data accessed from both task and interrupt context, use the IRQ-safe variants:

```
eos_spinlock_t data_lock = EOS_SPINLOCK_INIT;
uint32_t flags;

eos_spin_lock_irqsave(&data_lock, &flags);
// Critical section - interrupts disabled, lock held
shared_data++;
eos_spin_unlock_irqrestore(&data_lock, flags);
// Interrupts restored to previous state
```

The IRQ save/restore is also architecture-specific:

Architecture	Save	Restore
AArch64	mrs x0, daif; msr daifset, #0xF	msr daif, x0
ARMv7	mrs r0, cpsr; cpsid if	msr cpsr_c, r0
x86	pushf; pop; cli	sti (if IF was set)
RISC-V	csrr; csrrc mstatus, 0x8	csrs mstatus, 0x8

8.5 7.5 Inter-Processor Interrupts (IPI)

IPIs allow one core to signal another for scheduling, function calls, or user-defined events.

8.5.1 IPI Types

Type	Value	Purpose
<code>EOS_IPI_RESCHEDULE</code>	0	Trigger scheduler on target core
<code>EOS_IPI_CALL_FUNC</code>	1	Execute a function on target core
<code>EOS_IPI_STOP</code>	2	Request core shutdown
<code>EOS_IPI_USER</code>	3	User-defined event

8.5.2 IPI API

Function	Description
<code>eos_ipi_send(target, type, data)</code>	Send IPI to a specific core
<code>eos_ipi_broadcast(type, data)</code>	Send IPI to all other online cores
<code>eos_ipi_register_handler(type, handler, ctx)</code>	Register IPI handler

8.5.3 Example: Cross-Core Function Call

```
void ipi_handler(uint8_t from_core, uint32_t data, void *ctx)
{
    printf("Core %d received IPI from core %d, data=%u\n",
           eos_core_id(), from_core, data);
}

int main(void)
{
    eos_multicore_init(EOS_MP_SMP);

    eos_ipi_register_handler(EOS_IPI_USER, ipi_handler, NULL);
    eos_core_start(1, core1_main, NULL);

    // Send user IPI to core 1
    eos_ipi_send(1, EOS_IPI_USER, 42);

    // Broadcast to all other cores
    eos_ipi_broadcast(EOS_IPI_RESCHEDULE, 0);
}
```

8.6 7.6 Shared Memory

Named shared memory regions enable data exchange between cores with explicit cache management.

8.6.1 Shared Memory API

Function	Description
<code>eos_shmem_create(cfg, region)</code>	Create a named shared region
<code>eos_shmem_open(name, region)</code>	Open an existing region by name
<code>eos_shmem_close(region)</code>	Close a region
<code>eos_shmem_flush(region)</code>	Clean + invalidate cache lines
<code>eos_shmem_invalidate(region)</code>	Invalidate cache lines

8.6.2 Example: Inter-Core Data Sharing

```
// Core 0: Create shared buffer
uint8_t shared_buf[256] __attribute__((aligned(64)));

eos_shmem_config_t cfg = {
    .name    = "sensor_data",
    .base    = shared_buf,
    .size    = sizeof(shared_buf),
    .cached  = true,
};
eos_shmem_region_t region;
eos_shmem_create(&cfg, &region);

// Write data and flush
memcpy(shared_buf, sensor_data, sizeof(sensor_data));
eos_shmem_flush(&region); // Push to main memory

// Core 1: Open and read
eos_shmem_region_t remote;
eos_shmem_open("sensor_data", &remote);
eos_shmem_invalidate(&remote); // Pull from main memory
memcpy(local_buf, remote.addr, remote.size);
```

8.6.3 Cache Operations by Architecture

Architecture	Flush (Clean+Invalidate)	Invalidate
AArch64	dc civac per cache line + dsb sy	dc ivac + dsb sy
ARMv7	mcr p15 DCCIMVAC + dsb	mcr p15 DCIMVAC + dsb
x86	clflush + mfence	clflush + mfence

8.7 7.7 Task Core Affinity

Bind tasks to specific cores using affinity masks:

```
// Pin task to core 0 only
eos_task_set_affinity(task_id, EOS_CORE_MASK(0));
```

```

// Allow task on cores 0 and 1
eos_task_set_affinity(task_id, EOS_CORE_MASK(0) | EOS_CORE_MASK(1));

// Allow any core
eos_task_set_affinity(task_id, EOS_CORE_MASK_ALL);

// Migrate task to core 2
eos_task_migrate(task_id, 2);

```

8.7.1 Affinity Macros

Macro	Value	Meaning
<code>EOS_CORE_MASK(c)</code>	<code>1U << c</code>	Single core mask
<code>EOS_CORE_MASK_ALL</code>	<code>0xFFFFFFFF</code>	Run on any core
<code>EOS_CORE_MASK_NONE</code>	<code>0x00000000</code>	No core (invalid)

8.8 7.8 Remote Processor Control

For AMP systems with heterogeneous co-processors (e.g., Cortex-A + Cortex-M):

8.8.1 Remote Processor API

Function	Description
<code>eos_rproc_init(cfg)</code>	Configure remote processor
<code>eos_rproc_start(id)</code>	Boot remote processor
<code>eos_rproc_stop(id)</code>	Stop remote processor
<code>eos_rproc_get_state(id)</code>	Query state (OFFLINE/RUNNING/CRASHED)
<code>eos_rproc_send(id, data, len)</code>	Send message via shared-memory mailbox
<code>eos_rproc_set_rx_callback(id, cb, ctx)</code>	Register receive callback

8.8.2 Example: AMP Communication (Cortex-A Cortex-M)

```

// Main core (Cortex-A): Send command to M-core
eos_rproc_config_t m4_cfg = {
    .id          = 0,
    .name         = "cortex-m4",
    .firmware_addr = 0x10000000,
    .firmware_size = 64 * 1024,
};
eos_rproc_init(&m4_cfg);
eos_rproc_start(0);

// Send a command
uint8_t cmd[] = { 0x01, 0x42, 0x00, 0x10 };

```

```

eos_rproc_send(0, cmd, sizeof(cmd));

// Receive responses
void rx_handler(uint8_t rproc_id, const void *data, size_t len, void *ctx)
{
    printf("Received %zu bytes from rproc %d\n", len, rproc_id);
}
eos_rproc_set_rx_callback(0, rx_handler, NULL);

```

8.9 7.9 Memory Barriers and Atomics

8.9.1 Memory Barriers

Function	Instruction	Purpose
<code>eos_dmb()</code>	<code>dmb</code> / <code>mfence</code> / <code>fence rw,rw</code>	Data Memory Barrier
<code>eos_dsb()</code>	<code>dsb</code> / <code>mfence</code> / <code>fence iorw,iorw</code>	Data Synchronization Barrier
<code>eos_isb()</code>	<code>isb</code> / <code>(nop)</code> / <code>fence.i</code>	Instruction Synchronization Barrier

8.9.2 Barrier Implementation by Architecture

Barrier	AArch64	ARMv7	x86	RISC-V
DMB	<code>dmb sy</code>	<code>dmb</code>	<code>mfence</code>	<code>fence rw, rw</code>
DSB	<code>dsb sy</code>	<code>dsb</code>	<code>mfence</code>	<code>fence iorw, iorw</code>
ISB	<code>isb</code>	<code>isb</code>	(compiler barrier)	<code>fence.i</code>

8.9.3 Atomic Operations

Function	Description
<code>eos_atomic_add(ptr, val)</code>	Atomic fetch-and-add
<code>eos_atomic_sub(ptr, val)</code>	Atomic fetch-and-subtract
<code>eos_atomic_cas(ptr, expected, desired)</code>	Compare-and-swap
<code>eos_atomic_load(ptr)</code>	Atomic load
<code>eos_atomic_store(ptr, val)</code>	Atomic store

All atomics use `__ATOMIC_SEQ_CST` ordering for maximum safety.

8.9.4 Example: Lock-Free Counter

```

volatile int32_t request_count = 0;

void handle_request(void)
{

```

```

int32_t old = eos_atomic_add(&request_count, 1);
printf("Request #%d (total: %d)\n", old + 1,
      eos_atomic_load(&request_count));
}

```

8.10 7.10 Summary

Feature	API	Use Case
Core mgmt	<code>eos_core_start/stop</code>	Boot secondary cores
Spinlocks	<code>eos_spin_lock/unlock</code>	Short critical sections
IPI	<code>eos_ipi_send/broadcast</code>	Cross-core signaling
Shared mem	<code>eos_shmem_create/open</code>	Inter-core data exchange
Affinity	<code>eos_task_set_affinity</code>	Pin tasks to cores
Remote proc	<code>eos_rproc_init/start</code>	AMP co-processor control
Barriers	<code>eos_dmb/dsb/isb</code>	Memory ordering
Atomics	<code>eos_atomic_add/cas</code>	Lock-free data structures

Next: [Chapter 8 — Driver Framework](#)

Chapter 9

Chapter 8: Driver Framework

Srikanth Patchava & EmbeddedOS Contributors

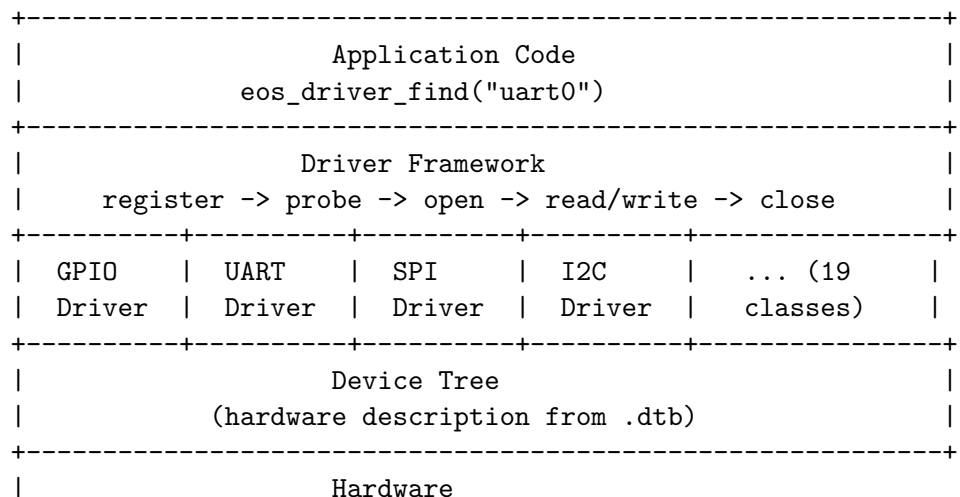
9.1 8.1 Overview

The EoS driver framework provides a structured approach to hardware driver development with a well-defined lifecycle, power management integration, and device tree support.

The framework consists of three components:

Component	Header	Purpose
Driver Model	<code>driver.h</code>	Registration, probe/remove lifecycle, matching
Driver Framework	<code>drivers.h</code>	Unified open/close/read/write/ioctl interface
Device Tree	<code>devicetree.h</code>	Hardware description parsing (.dtb)

9.2 8.2 Driver Architecture



+-----+

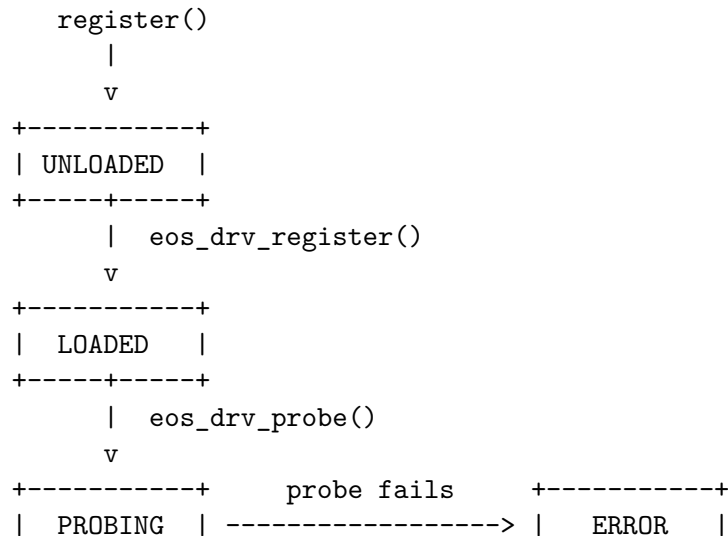
9.3 8.3 Driver Classes

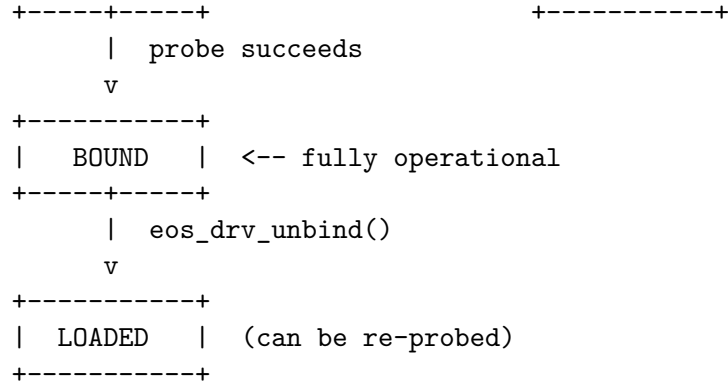
EoS defines 19 driver classes covering all peripheral types:

Class Enum	Value	Category
EOS_DRV_CLASS_GPIO	0	Digital I/O
EOS_DRV_CLASS_UART	1	Serial
EOS_DRV_CLASS_SPI	2	Serial
EOS_DRV_CLASS_I2C	3	Serial
EOS_DRV_CLASS_CAN	4	Bus
EOS_DRV_CLASS_PWM	5	Analog
EOS_DRV_CLASS_ADC	6	Analog
EOS_DRV_CLASS_DAC	7	Analog
EOS_DRV_CLASS_TIMER	8	Timing
EOS_DRV_CLASS_DMA	9	System
EOS_DRV_CLASS_ETHERNET	10	Network
EOS_DRV_CLASS_USB	11	Bus
EOS_DRV_CLASS_SDIO	12	Storage
EOS_DRV_CLASS_DISPLAY	13	Graphics
EOS_DRV_CLASS_SENSOR	14	Input
EOS_DRV_CLASS_STORAGE	15	Storage
EOS_DRV_CLASS_NETWORK	16	Network
EOS_DRV_CLASS_AUDIO	17	Media
EOS_DRV_CLASS_CAMERA	18	Media
EOS_DRV_CLASS_CUSTOM	0xFF	User-defined

9.4 8.4 Driver Lifecycle

Every driver follows a well-defined state machine:





9.4.1 State Definitions

State	Value	Description
<code>EOS_DRV_UNLOADED</code>	0	Not registered
<code>EOS_DRV_LOADED</code>	1	Registered but not bound to hardware
<code>EOS_DRV_PROBING</code>	2	Currently executing probe function
<code>EOS_DRV_BOUND</code>	3	Successfully bound, operational
<code>EOS_DRV_ERROR</code>	4	Probe failed

9.5 8.5 The Driver Structure

The `EosDriver` structure is the central data type. Each driver instance contains identification, operations, matching info, and runtime data:

```

typedef struct eos_driver {
    // Identification
    char      name[EOS_DRV_NAME_MAX]; // Unique driver name
    uint32_t  version;                // Driver version
    EosDrvClass class_id;              // Peripheral class
    EosDrvState state;                 // Current lifecycle state

    // Hardware matching
    EosDrvMatchInfo match;             // vendor/device/compat

    // Operations (function pointers)
    int  (*probe)(struct eos_driver *drv, void *platform_data);
    void (*remove)(struct eos_driver *drv);
    int  (*suspend)(struct eos_driver *drv);
    int  (*resume)(struct eos_driver *drv);
    int  (*ioctl)(struct eos_driver *drv, uint32_t cmd, void *arg);

    // Runtime data
    void *priv_data; // Driver-private data
    void *platform_data; // Platform-specific data
    int  ref_count; // Reference count

```

```

    int      irq_num;           // Assigned IRQ number
    uint32_t base_addr;        // Memory-mapped base address
    uint32_t reg_size;         // Register region size
} EosDriver;

```

9.5.1 Match Info Structure

```

typedef struct {
    uint16_t vendor_id;        // Vendor identifier
    uint16_t device_id;       // Device identifier
    uint32_t compat_hash;      // Hash of compatibility string
    char     compat_str[32];    // e.g., "st,stm32-uart"
} EosDrvMatchInfo;

```

9.6 8.6 Driver Registry API

The driver registry manages all registered drivers:

9.6.1 Registration and Lookup

Function	Signature	Description
eos_drv_init	int eos_drv_init(EosDrvRegistry *reg)	Initialize registry
eos_drv_shutdown	void eos_drv_shutdown(EosDrvRegistry *reg)	Unbind all, shutdown
eos_drv_register	int eos_drv_register(EosDrvRegistry *reg, EosDriver *drv)	Register a driver
eos_drv_unregister	int eos_drv_unregister(EosDrvRegistry *reg, const char *name)	Unregister by name
eos_drv_find	EosDriver *eos_drv_find(EosDrvRegistry *reg, const char *name)	Find by name
eos_drv_find_by_class	EosDriver *eos_drv_find_by_class(reg, cls, idx)	Find by class
eos_drv_find_by_match	EosDriver *eos_drv_find_by_match(reg, match)	Find by match info

9.6.2 Lifecycle Operations

Function	Description
<code>eos_drv_probe(reg, name, platform_data)</code>	Probe a specific driver
<code>eos_drv_probe_all(reg)</code>	Probe all loaded drivers
<code>eos_drv_unbind(reg, name)</code>	Unbind a bound driver

9.6.3 Introspection

Function	Description
<code>eos_drv_count(reg)</code>	Total registered drivers
<code>eos_drv_count_by_class(reg, class)</code>	Count by peripheral class
<code>eos_drv_dump(reg)</code>	Print registry to stderr
<code>eos_drv_ioctl(reg, name, cmd, arg)</code>	Send ioctl to named driver

9.7 8.7 Writing a Driver: Complete Example

Here is a complete example of an I2C temperature sensor driver:

```
#include <eos/driver.h>
#include <stdio.h>

#define TMP102_TEMP_REG 0x00
#define TMP102_CONF_REG 0x01

typedef struct {
    uint8_t i2c_port;
    uint16_t i2c_addr;
    float last_temp;
} tmp102_data_t;

// --- Probe: initialize the hardware ---
static int tmp102_probe(EosDriver *drv, void *platform_data)
{
    tmp102_data_t *data = (tmp102_data_t *)drv->priv_data;
    if (!data) return -1;

    // Configure sensor: 12-bit resolution, continuous mode
    uint8_t conf[2] = { 0x60, 0xA0 };
    eos_i2c_write_reg(data->i2c_port, data->i2c_addr,
                      TMP102_CONF_REG, conf, 2);

    printf("[tmp102] Probed at I2C %d:0x%02X\n",
           data->i2c_port, data->i2c_addr);
    return 0;
}
```

```

// --- Remove: cleanup ---
static void tmp102_remove(EosDriver *drv)
{
    printf("[tmp102] Removed\n");
}

// --- Suspend: put sensor in shutdown mode ---
static int tmp102_suspend(EosDriver *drv)
{
    tmp102_data_t *data = (tmp102_data_t *)drv->priv_data;
    uint8_t conf[2] = { 0x61, 0xA0 }; // SD bit set
    eos_i2c_write_reg(data->i2c_port, data->i2c_addr,
                      TMP102_CONF_REG, conf, 2);

    return 0;
}

// --- Resume: wake sensor ---
static int tmp102_resume(EosDriver *drv)
{
    tmp102_data_t *data = (tmp102_data_t *)drv->priv_data;
    uint8_t conf[2] = { 0x60, 0xA0 }; // SD bit clear
    eos_i2c_write_reg(data->i2c_port, data->i2c_addr,
                      TMP102_CONF_REG, conf, 2);

    return 0;
}

// --- Ioctl: read temperature ---
#define TMP102_IOCTL_READ_TEMP 0x01

static int tmp102_ioctl(EosDriver *drv, uint32_t cmd, void *arg)
{
    if (cmd != TMP102_IOCTL_READ_TEMP || !arg) return -1;

    tmp102_data_t *data = (tmp102_data_t *)drv->priv_data;
    uint8_t raw[2];
    eos_i2c_read_reg(data->i2c_port, data->i2c_addr,
                     TMP102_TEMP_REG, raw, 2);

    int16_t temp = (raw[0] << 4) | (raw[1] >> 4);
    if (temp & 0x800) temp |= 0xF000;

    float *result = (float *)arg;
    *result = temp * 0.0625f;
    data->last_temp = *result;
    return 0;
}

```

```

// --- Driver definition ---
static tmp102_data_t sensor_data = {
    .i2c_port = 0,
    .i2c_addr = 0x48,
};

static EosDriver tmp102_driver = {
    .name      = "tmp102",
    .version   = 1,
    .class_id  = EOS_DRV_CLASS_SENSOR,
    .match     = {
        .compat_str = "ti,tmp102",
    },
    .probe     = tmp102_probe,
    .remove    = tmp102_remove,
    .suspend   = tmp102_suspend,
    .resume    = tmp102_resume,
    .ioctl     = tmp102_ioctl,
    .priv_data = &sensor_data,
};

// --- Usage ---
void app_init(void)
{
    EosDrvRegistry registry;
    eos_drv_init(&registry);

    eos_drv_register(&registry, &tmp102_driver);
    eos_drv_probe(&registry, "tmp102", NULL);

    float temp;
    eos_drv_ioctl(&registry, "tmp102", TMP102_IOCTL_READ_TEMP, &temp);
    printf("Temperature: %.2f C\n", temp);
}

```

9.8 8.8 Power Management

The driver framework integrates power management through `suspend` and `resume` callbacks.

9.8.1 System-Wide Suspend/Resume

```

// Suspend all bound drivers (in reverse registration order)
int suspended = eos_drv_suspend_all(&registry);
printf("%d drivers suspended\n", suspended);

// Enter low-power mode...

```

```
// Resume all drivers (in registration order)
int resumed = eos_drv_resume_all(&registry);
printf("%d drivers resumed\n", resumed);
```

9.8.2 Suspend/Resume Order

Suspend processes drivers in **reverse registration order** (last registered suspends first). Resume processes in **forward order**. This ensures dependencies are respected:

Registration: gpio -> i2c -> sensor -> display

Suspend order: display -> sensor -> i2c -> gpio

Resume order: gpio -> i2c -> sensor -> display

9.9 8.9 Device Tree Integration

EoS includes a device tree parser for describing hardware topology in a vendor-neutral format, compatible with the standard Devicetree Specification.

9.9.1 Device Tree Structure

```
typedef struct eos_dt_node {
    char            name[EOS_DT_NAME_MAX];
    EosDtProp       props[EOS_DT_MAX_PROPS];
    int             prop_count;
    struct eos_dt_node *parent;
    struct eos_dt_node *children[16];
    int             child_count;
    uint32_t        phandle;
} EosDtNode;

typedef struct {
    EosDtNode nodes[EOS_DT_MAX_NODES];
    int node_count;
    EosDtNode *root;
    uint32_t version;
    uint32_t boot_cpuid;
    char model[64];
    char compatible[128];
} EosDeviceTree;
```

9.9.2 Key Constants

Macro	Value	Description
EOS_DT_MAX_NODES	128	Maximum nodes in tree
EOS_DT_MAX_PROPS	16	Maximum properties per node
EOS_DT_NAME_MAX	64	Maximum name length

Macro	Value	Description
EOS_DT_PROP_MAX	256	Maximum property data size
EOS_DT_MAGIC	0xD00DFEED	DTB magic number

9.9.3 Device Tree API

Function	Description
<code>eos_dt_parse(dt, dtb, size)</code>	Parse a flattened device tree blob
<code>eos_dt_find(dt, path)</code>	Find node by path
<code>eos_dt_find_compatible(dt, compat)</code>	Find node by compatible string
<code>eos_dt_find_by_phandle(dt, phandle)</code>	Find node by phandle
<code>eos_dt_get_prop(node, name)</code>	Get raw property
<code>eos_dt_get_u32(node, name, val)</code>	Read 32-bit property
<code>eos_dt_get_string(node, name, buf, len)</code>	Read string property
<code>eos_dt_get_reg(node, addr, size)</code>	Read reg property (base + size)
<code>eos_dt_get_irq(node, index)</code>	Read interrupt number

9.9.4 Example: Parsing a Device Tree

```
EosDeviceTree dt;
extern const uint8_t _dtb_start[];
extern const uint32_t _dtb_size;

// Parse the compiled device tree blob
eos_dt_parse(&dt, _dtb_start, _dtb_size);

// Find UART node
EosDtNode *uart = eos_dt_find(&dt, "/soc/uart@40004000");
if (uart) {
    uint32_t base, size;
    eos_dt_get_reg(uart, &base, &size);
    printf("UART base=0x%08X size=0x%X\n", base, size);

    int irq = eos_dt_get_irq(uart, 0);
    printf("UART IRQ=%d\n", irq);
}

// Find all nodes with a specific compatible string
EosDtNode *sensor = eos_dt_find_compatible(&dt, "ti,tmp102");
if (sensor) {
    uint32_t addr;
    eos_dt_get_u32(sensor, "reg", &addr);
    printf("TMP102 at I2C address 0x%02X\n", addr);
}
```


9.9.5 Example Device Tree Source

```
/ {
    model = "EoS Dev Board";
    compatible = "eos,devboard";

    soc {
        uart@40004000 {
            compatible = "st,stm32-uart";
            reg = <0x40004000 0x400>;
            interrupts = <38>;
            clock-frequency = <84000000>;
            status = "okay";
        };

        i2c@40005400 {
            compatible = "st,stm32-i2c";
            reg = <0x40005400 0x400>;
            clock-frequency = <400000>;
            status = "okay";

            tmp102@48 {
                compatible = "ti,tmp102";
                reg = <0x48>;
            };
        };

        gpio@40020000 {
            compatible = "st,stm32-gpio";
            reg = <0x40020000 0x400>;
            gpio-cells = <2>;
        };
    };
};
```

9.10 8.10 Driver-Device Tree Integration

Combine the driver framework with device tree for automatic hardware discovery:

```
void auto_probe_drivers(EosDrvRegistry *reg, EosDeviceTree *dt)
{
    for (int i = 0; i < eos_drv_count(reg); i++) {
        EosDriver *drv = reg->drivers[i];
        if (!drv || drv->state != EOS_DRV_LOADED) continue;

        // Match driver against device tree
        EosDtNode *node = eos_dt_find_compatible(
            dt, drv->match.compat_str);
    }
}
```

```

if (node) {
    uint32_t base, size;
    eos_dt_get_reg(node, &base, &size);

    drv->base_addr = base;
    drv->reg_size  = size;
    drv->irq_num   = eos_dt_get_irq(node, 0);

    eos_drv_probe(reg, drv->name, node);
    printf("Auto-probed %s at 0x%08X\n", drv->name, base);
}
}
}

```

9.11 8.11 The Unified Driver Framework

In addition to the registry model, EoS provides a higher-level driver framework with a standard open/close/read/write/ioctl interface, similar to POSIX file operations.

9.11.1 Driver States (Framework)

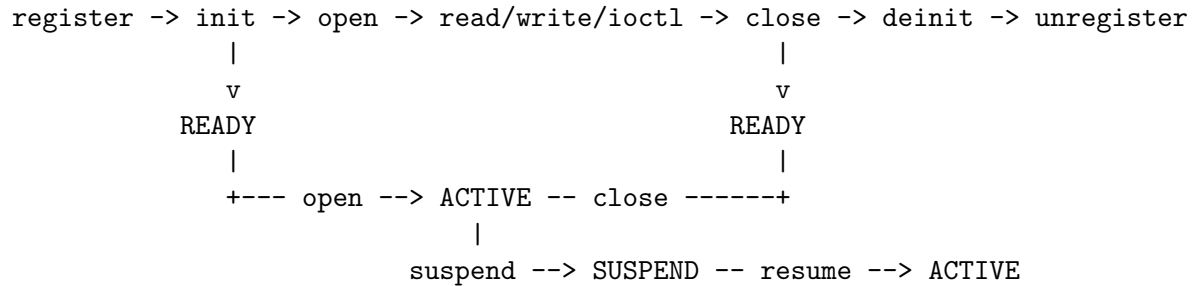
State	Value	Description
EOS_DRV_STATE_UNINIT	0	Not initialized
EOS_DRV_STATE_READY	1	Initialized, ready to open
EOS_DRV_STATE_ACTIVE	2	Opened and operational
EOS_DRV_STATE_SUSPEND	3	Suspended (low power)
EOS_DRV_STATE_ERROR	4	Error state

9.11.2 Framework API

Function	Description
<code>eos_driver_register(drv)</code>	Register a driver
<code>eos_driver_unregister(name)</code>	Unregister by name
<code>eos_driver_find(name)</code>	Find by name
<code>eos_driver_find_by_type(type, instance)</code>	Find by type and instance
<code>eos_driver_init_all()</code>	Initialize all registered drivers
<code>eos_driver_deinit_all()</code>	Deinitialize all drivers
<code>eos_driver_open(drv)</code>	Open a driver for use
<code>eos_driver_close(drv)</code>	Close a driver
<code>eos_driver_read(drv, buf, len)</code>	Read data from driver
<code>eos_driver_write(drv, buf, len)</code>	Write data to driver
<code>eos_driver_ioctl(drv, cmd, arg)</code>	Device-specific control
<code>eos_driver_suspend_all()</code>	Suspend all active drivers
<code>eos_driver_resume_all()</code>	Resume all suspended drivers
<code>eos_driver_count()</code>	Total registered drivers

Function	Description
<code>eos_driver_list(out, max, count)</code>	List all drivers

9.11.3 Framework Lifecycle



9.12 8.12 Best Practices

9.12.1 Driver Development Guidelines

Guideline	Rationale
Always implement <code>probe</code> and <code>remove</code>	Clean lifecycle management
Implement <code>suspend/resume</code> for battery devices	Enables system-wide power management
Use <code>priv_data</code> for driver state	Keeps the driver struct generic
Set <code>match.compat_str</code>	Enables device tree auto-matching
Check <code>state</code> before operations	Prevents use-after-free
Use <code>ioctl</code> for device-specific commands	Keeps the core API clean

9.12.2 Common Patterns

Resource acquisition in probe:

```

static int my_probe(EosDriver *drv, void *pdata)
{
    // 1. Allocate private data
    my_data_t *data = malloc(sizeof(my_data_t));
    if (!data) return -1;

    // 2. Map registers
    data->regs = (volatile uint32_t *)drv->base_addr;

    // 3. Configure hardware
    data->regs[CTRL_REG] = ENABLE_BIT;

    // 4. Register interrupt
    eos_irq_register(drv->irq_num, my_isr, 5);
}

```

```

    drv->priv_data = data;
    return 0;
}

```

Clean removal:

```

static void my_remove(EosDriver *drv)
{
    my_data_t *data = (my_data_t *)drv->priv_data;

    // 1. Disable hardware
    data->regs[CTRL_REG] = 0;

    // 2. Unregister interrupt
    eos_irq_unregister(drv->irq_num);

    // 3. Free private data
    free(data);
    drv->priv_data = NULL;
}

```

9.13 8.13 Summary

Component	Purpose
EosDriver struct	Driver identity, operations, and runtime data
EosDrvRegistry	Centralized driver management
Probe/Remove	Hardware binding lifecycle
Suspend/Resume	System-wide power management
Device Tree	Vendor-neutral hardware description
Match Info	Automatic driver-to-device binding
Framework API	POSIX-like open/close/read/write interface

The driver framework ties together the HAL (Chapter 4-5), the kernel (Chapter 6), and the multi-core subsystem (Chapter 7) into a coherent model for managing hardware across the full range of EoS-supported platforms.

End of Part II: Kernel and HAL

Chapter 10

Chapter 9: Cryptographic Services

Author: Srikanth Patchava & EmbeddedOS Contributors

10.1 9.1 Overview

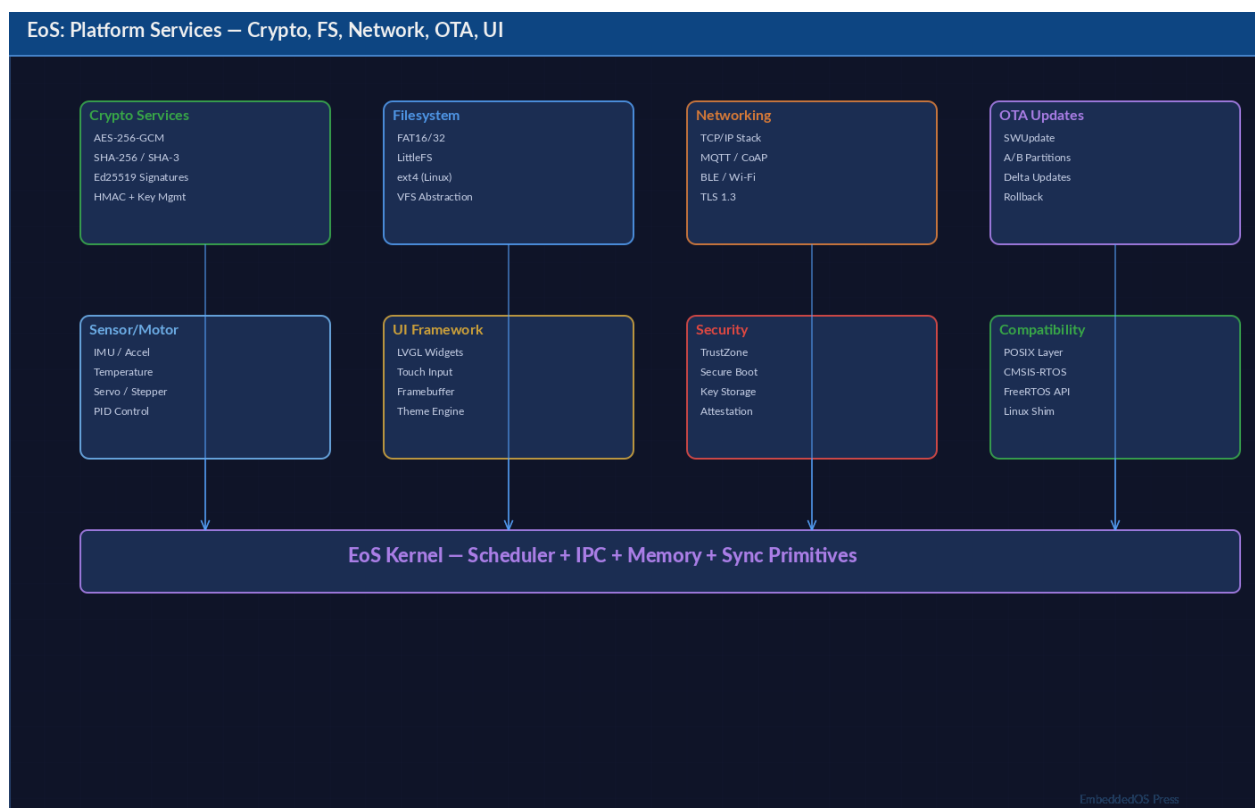
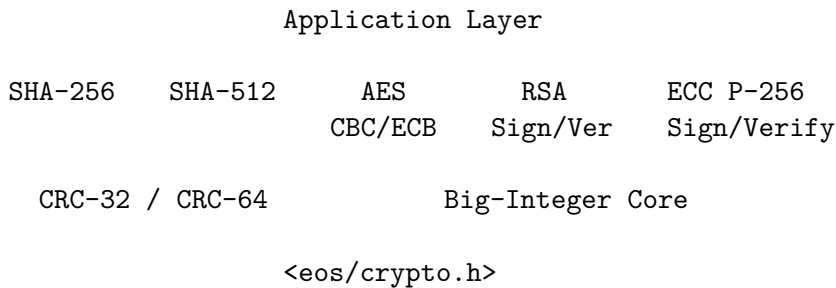


Figure 10.1: Figure: EoS Platform Services — crypto, filesystem, networking, OTA, UI, security

The EoS cryptographic subsystem (`eos/crypto.h`) provides hash functions, checksums, symmetric encryption, and asymmetric key operations suitable for both build-time integrity verification and

runtime security. Every algorithm is implemented in portable C with no external dependencies — critical for bare-metal and resource-constrained targets.



10.2 9.2 Hash Functions — SHA-256

SHA-256 produces a 256-bit (32-byte) digest and is the primary hash used throughout EoS for firmware signing, secure boot, and OTA verification.

10.2.1 9.2.1 Constants and Context

Constant	Value	Description
EOS_SHA256_DIGEST_SIZE	32	Digest size in bytes
EOS_SHA256_BLOCK_SIZE	64	Internal block size
EOS_SHA256_HEX_SIZE	65	Hex string + NUL

```
typedef struct {
    uint32_t state[8]; /* Running hash state (H0-H7) */
    uint8_t buf[64]; /* Partial block buffer */
    uint64_t total; /* Total bytes hashed */
} EosSha256;
```

10.2.2 9.2.2 Streaming API

The streaming interface processes data in arbitrarily sized chunks:

```
EosSha256 ctx;
uint8_t digest[EOS_SHA256_DIGEST_SIZE];
char hex[EOS_SHA256_HEX_SIZE];

eos_sha256_init(&ctx);
eos_sha256_update(&ctx, block1, sizeof(block1));
eos_sha256_update(&ctx, block2, sizeof(block2));
eos_sha256_final(&ctx, digest);
eos_sha256_hex(digest, hex);

printf("SHA-256: %s\n", hex);
```

10.2.3 9.2.3 Convenience Helpers

For common one-shot operations:

```
/* Hash a memory buffer in one call */
char hex[EOS_SHA256_HEX_SIZE];
eos_sha256_data(firmware_blob, firmware_size, hex);

/* Hash a file on the filesystem */
char file_hex[EOS_SHA256_HEX_SIZE];
if (eos_sha256_file("/flash/app.bin", file_hex) == 0) {
    printf("File hash: %s\n", file_hex);
}
```

10.3 9.3 Hash Functions — SHA-512

SHA-512 uses 64-bit arithmetic and produces a 512-bit (64-byte) digest. It is preferred when higher collision resistance is needed or the target CPU has native 64-bit ALU support.

Constant	Value	Description
EOS_SHA512_DIGEST_SIZE	64	Digest size in bytes
EOS_SHA512_BLOCK_SIZE	128	Internal block size
EOS_SHA512_HEX_SIZE	129	Hex string + NUL

```
typedef struct {
    uint64_t state[8]; /* Running hash state */
    uint8_t buf[128]; /* Partial block */
    uint64_t total; /* Total bytes hashed */
} EosSha512;
```

The API mirrors SHA-256:

```
EosSha512 ctx;
uint8_t digest[EOS_SHA512_DIGEST_SIZE];
char hex[EOS_SHA512_HEX_SIZE];

eos_sha512_init(&ctx);
eos_sha512_update(&ctx, data, len);
eos_sha512_final(&ctx, digest);
eos_sha512_hex(digest, hex);
```

10.3.1 9.3.1 SHA-256 vs SHA-512 Comparison

Property	SHA-256	SHA-512
Digest size	32 bytes	64 bytes
Block size	64 bytes	128 bytes
Word size	32-bit	64-bit

Property	SHA-256	SHA-512
Rounds	64	80
Context RAM	~112 bytes	~216 bytes
Best for	Cortex-M	64-bit cores

10.4 9.4 Cyclic Redundancy Checks — CRC-32 / CRC-64

CRC functions provide fast error-detection checksums for data integrity without the computational overhead of cryptographic hashes.

10.4.1 9.4.1 API

```
/* Incremental CRC-32 */
uint32_t crc = 0;
crc = eos_crc32(crc, chunk1, len1);
crc = eos_crc32(crc, chunk2, len2);
printf("CRC-32: 0x%08X\n", crc);

/* File CRC-32 */
uint32_t file_crc = eos_crc32_file("/flash/config.bin");

/* CRC-64 for large datasets */
uint64_t crc64 = 0;
crc64 = eos_crc64(crc64, data, data_len);
```

10.4.2 9.4.2 When to Use CRC vs SHA

Criterion	CRC-32/64	SHA-256
Speed	Very fast	Moderate
Security	None (detectable)	Cryptographic
Use case	Bus errors, flash	Firmware signing
Collision resist.	Weak	Strong

10.5 9.5 Symmetric Encryption — AES

EoS provides AES-128 and AES-256 in both ECB (single-block) and CBC (chained) modes.

10.5.1 9.5.1 Constants

Constant	Value	Description
EOS_AES_BLOCK_SIZE	16	Block size in bytes
EOS_AES128_KEY_SIZE	16	AES-128 key size
EOS_AES256_KEY_SIZE	32	AES-256 key size
EOS_AES_MAX_ROUNDS	14	Max key-schedule rounds

10.5.2 9.5.2 Key Expansion

```
typedef struct {  
    uint32_t rk[60];    /* Expanded round keys */  
    int nr;             /* Number of rounds */  
} EosAesCtx;
```

Initialize the context once, then encrypt/decrypt multiple blocks:

```
EosAesCtx aes;  
uint8_t key[EOS_AES256_KEY_SIZE] = { /* 32 bytes */ };  
  
eos_aes_init(&aes, key, 256); /* 128 or 256 */
```

10.5.3 9.5.3 ECB Mode (Single Block)

ECB encrypts each 16-byte block independently — suitable only for single-block operations like key wrapping:

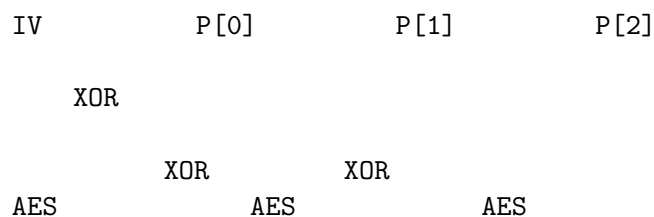
```
uint8_t plaintext[16] = "Hello, AES-256!";  
uint8_t ciphertext[16];  
uint8_t decrypted[16];  
  
eos_aes_encrypt_block(&aes, plaintext, ciphertext);  
eos_aes_decrypt_block(&aes, ciphertext, decrypted);
```

Warning: Never use ECB for multi-block data — identical plaintext blocks produce identical ciphertext, leaking patterns.

10.5.4 9.5.4 CBC Mode (Chained Blocks)

CBC chains blocks through an IV, providing semantic security:

```
uint8_t iv[16] = { /* random 16 bytes */ };  
uint8_t data[64]; /* must be multiple of 16 */  
uint8_t enc[64], dec[64];  
  
eos_aes_cbc_encrypt(&aes, iv, data, enc, sizeof(data));  
eos_aes_cbc_decrypt(&aes, iv, enc, dec, sizeof(enc));
```



C[0]

C[1]

10.6 9.6 Asymmetric Cryptography — RSA

EoS includes a minimal RSA implementation for firmware signing and verification with SHA-256 digests.

10.6.1 9.6.1 Key Structure

```
#define EOS_RSA_MAX_KEY_BYTES 512

typedef struct {
    uint8_t n[EOS_RSA_MAX_KEY_BYTES];    /* Modulus */
    uint8_t e[EOS_RSA_MAX_KEY_BYTES];    /* Public exponent */
    uint8_t d[EOS_RSA_MAX_KEY_BYTES];    /* Private exponent */
    int key_bits;                          /* 2048 or 4096 */
    int has_private;                       /* 1 if d is valid */
} EosRsaKey;
```

10.6.2 9.6.2 Sign and Verify

```
EosRsaKey key;
/* ... load key from keystore ... */

uint8_t hash[32];
eos_sha256_data(firmware, fw_len, NULL);
/* compute raw digest into hash[] */

uint8_t sig[512];
size_t sig_len = sizeof(sig);

/* Sign (requires private key) */
int rc = eos_rsa_sign_sha256(&key, hash, sig, &sig_len);

/* Verify (public key only) */
rc = eos_rsa_verify_sha256(&key, hash, sig, sig_len);
if (rc == 0) {
    printf("Signature valid\n");
}
```

10.7 9.7 Elliptic Curve Cryptography — ECDSA P-256

ECC provides equivalent security to RSA with much smaller keys, making it ideal for constrained devices.

10.7.1 9.7.1 Key Structure

```
typedef struct {
    uint8_t pub[64];    /* Uncompressed public key (x, y) */
    uint8_t priv[32];   /* Private scalar */
    int curve;          /* Curve identifier (P-256 = 0) */
} EosEccKey;
```

10.7.2 9.7.2 Sign and Verify

```
EosEccKey ecc_key;
/* ... load or generate key ... */

uint8_t hash[32];
uint8_t sig[72]; /* DER-encoded ECDSA signature */
size_t sig_len = sizeof(sig);

int rc = eos_ecc_sign(&ecc_key, hash, 32, sig, &sig_len);

rc = eos_ecc_verify(&ecc_key, hash, 32, sig, sig_len);
```

10.7.3 9.7.3 RSA vs ECC Comparison

Property	RSA-2048	ECDSA P-256
Key size (bits)	2048	256
Signature size	256 bytes	~64 bytes
RAM usage	~2 KB	~200 bytes
Verify speed	Moderate	Faster
Standard	PKCS#1 v2.1	FIPS 186-4

10.8 9.8 Algorithm Selection Guide

What do you need?

Integrity Check		Encrypt Data	Authenticate / Sign	
CRC-32	SHA-256	AES	RSA	ECC P-256

(fast) (secure) CBC (compat) (compact)

10.9 9.9 Complete Example: Firmware Integrity Pipeline

```
#include <eos/crypto.h>
#include <stdio.h>

int verify_firmware(const char *fw_path, const EosRsaKey *pub_key)
{
    /* Step 1: Compute SHA-256 of firmware image */
    char hex[EOS_SHA256_HEX_SIZE];
    if (eos_sha256_file(fw_path, hex) != 0) {
        return -1;
    }
    printf("Firmware SHA-256: %s\n", hex);

    /* Step 2: Also verify CRC for transport errors */
    uint32_t crc = eos_crc32_file(fw_path);
    printf("CRC-32: 0x%08X\n", crc);

    /* Step 3: Verify RSA signature */
    uint8_t digest[EOS_SHA256_DIGEST_SIZE];
    EosSha256 ctx;
    eos_sha256_init(&ctx);
    /* ... feed firmware data ... */
    eos_sha256_final(&ctx, digest);

    uint8_t sig[512];
    size_t sig_len = 256;
    /* ... load sig from .sig file ... */

    if (eos_rsa_verify_sha256(pub_key, digest, sig, sig_len) != 0) {
        printf("ERROR: Signature verification failed!\n");
        return -2;
    }

    printf("Firmware verified successfully.\n");
    return 0;
}
```

10.10 9.10 Security Considerations

1. **Key storage** — Never store private keys in plaintext flash. Use the keystore service (Chapter 10) with encrypted storage.
2. **Side channels** — The software AES implementation is not constant-time on all architectures.

For high-security targets, consider hardware AES.

3. **Random numbers** — EoS does not currently bundle a CSPRNG. Seed keys from a hardware entropy source (TRNG) where available.
4. **Algorithm agility** — Use ECC P-256 for new designs. RSA is provided for backward compatibility with existing PKI infrastructure.

10.11 9.11 API Reference Summary

Function	Description
<code>eos_sha256_init</code>	Initialize SHA-256 context
<code>eos_sha256_update</code>	Feed data into hash
<code>eos_sha256_final</code>	Finalize and output digest
<code>eos_sha256_hex</code>	Convert digest to hex string
<code>eos_sha256_data</code>	One-shot hash of buffer
<code>eos_sha256_file</code>	One-shot hash of file
<code>eos_sha512_init/update/final</code>	SHA-512 streaming API
<code>eos_sha512_hex</code>	SHA-512 digest to hex
<code>eos_crc32</code>	Incremental CRC-32
<code>eos_crc32_file</code>	CRC-32 of a file
<code>eos_crc64</code>	Incremental CRC-64
<code>eos_aes_init</code>	Expand AES key schedule
<code>eos_aes_encrypt_block</code>	Encrypt single 16-byte block
<code>eos_aes_decrypt_block</code>	Decrypt single 16-byte block
<code>eos_aes_cbc_encrypt</code>	AES-CBC encryption
<code>eos_aes_cbc_decrypt</code>	AES-CBC decryption
<code>eos_rsa_sign_sha256</code>	RSA-SHA256 signature
<code>eos_rsa_verify_sha256</code>	RSA-SHA256 verification
<code>eos_ecc_sign</code>	ECDSA signature
<code>eos_ecc_verify</code>	ECDSA verification

Next: [Chapter 10 — Security Services](#)

Chapter 11

Chapter 10: Security Services

Author: Srikanth Patchava & EmbeddedOS Contributors

11.1 10.1 Overview

The EoS security subsystem (`eos/security.h`) provides four pillars of embedded security: **secure boot**, **firmware signing**, **key management**, and **access control**. Together these services form a chain of trust from power-on through runtime operation.

Application			
Secure Boot Verify	Firmware Signing	Key Store (AES/RSA/ ECC keys)	Access Control (ACL Rules)
<eos/security.h>			
<eos/crypto.h>			

11.2 10.2 Secure Boot

Secure boot verifies the integrity and authenticity of firmware images before they execute. The EoS boot verifier checks a cryptographic hash and validates a digital signature against a trusted public key.

11.2.1 10.2.1 Boot Status States

State	Value	Description
EOS_BOOT_UNVERIFIED	0	Image has not been checked
EOS_BOOT_VERIFIED	1	Image passed verification

State	Value	Description
EOS_BOOT_FAILED	2	Verification failed
EOS_BOOT_SKIPPED	3	Verification intentionally skipped

11.2.2 10.2.2 Secure Boot Structure

```
typedef struct {
    char image_path[512];      /* Path to firmware image */
    char sig_path[512];        /* Path to detached signature */
    char pubkey_path[512];     /* Path to public key file */
    char hash_algo[32];        /* Hash algorithm name */
    EosBootStatus status;      /* Current verification status */
    char hash[129];            /* Computed hash (hex string) */
} EosSecureBoot;
```

11.2.3 10.2.3 Verification Workflow

```
#include <eos/security.h>

int boot_verify(void)
{
    EosSecureBoot sb;
    eos_secureboot_init(&sb);

    int rc = eos_secureboot_verify_image(
        &sb,
        "/flash/app.bin",      /* firmware image */
        "/flash/app.bin.sig",  /* detached sig */
        "/flash/boot_pubkey.pem" /* trusted pub key */
    );

    eos_secureboot_dump(&sb);    /* print status */

    if (sb.status == EOS_BOOT_VERIFIED) {
        printf("Boot: image verified - hash: %s\n", sb.hash);
        return 0;
    }
    printf("Boot: VERIFICATION FAILED\n");
    return -1;
}
```

11.2.4 10.2.4 Boot Verification Flow

Power-On

```
Load image
from flash
```

```
Compute hash      Load .sig
(SHA-256)         from flash
```

```
eos_secureboot_verify_image()
RSA/ECC verify(hash, sig, pk)
```

```
VERIFIED          FAILED
Jump to app       Halt / fallback
```

11.3 10.3 Firmware Signing

The firmware signing API generates signed firmware packages for distribution and OTA updates.

11.3.1 10.3.1 Signed Firmware Structure

```
typedef struct {
    char input_path[512];      /* Source firmware path      */
    char output_path[512];     /* Signed output path        */
    char key_path[512];        /* Private key path           */
    char hash[129];            /* Computed SHA-256 hash     */
    char signature[1024];      /* Generated signature bytes  */
    size_t sig_len;            /* Signature length           */
    int signed_ok;             /* 1 if signing succeeded     */
} EosSignedFirmware;
```

11.3.2 10.3.2 Signing a Firmware Image

```
EosSignedFirmware sf;
int rc = eos_firmware_sign(
    &sf,
    "/build/firmware.bin",      /* input image      */
    "/keys/signing_key.pem",    /* private key      */
    "/release/firmware.signed" /* output package   */
);
```



```
if (rc == 0 && sf.signed_ok) {
    printf("Signed: hash=%s sig_len=%zu\n", sf.hash, sf.sig_len);
}
```

11.3.3 10.3.3 Verifying a Signed Firmware

```
int rc = eos_firmware_verify_sig(
    "/release/firmware.signed",
    "/release/firmware.sig",
    "/keys/public_key.pem"
);
/* rc == 0 means valid */
```

11.4 10.4 Key Management (Keystore)

The keystore provides persistent, typed storage for cryptographic keys. It supports AES, RSA, and ECC key types with a simple ID-based lookup.

11.4.1 10.4.1 Supported Key Types

Enum	Algorithm	Typical Key Size
EOS_KEY_AES128	AES-128	16 bytes
EOS_KEY_AES256	AES-256	32 bytes
EOS_KEY_RSA2048	RSA 2048	~256 bytes (n)
EOS_KEY_RSA4096	RSA 4096	~512 bytes (n)
EOS_KEY_ECC_P256	ECDSA P-256	32 bytes (priv)
EOS_KEY_ECC_P384	ECDSA P-384	48 bytes (priv)

11.4.2 10.4.2 Key Entry and Store

```
typedef struct {
    char id[EOS_KEY_MAX_ID];           /* Key identifier string */
    EosKeyType type;                   /* Algorithm type */
    uint8_t data[512];                 /* Raw key material */
    size_t data_len;                   /* Actual key length */
    int active;                        /* 1 = active, 0 = revoked */
} EosKeyEntry;

typedef struct {
    EosKeyEntry keys[EOS_KEY_MAX_KEYS]; /* Key slots (max 32) */
    int count;                          /* Current key count */
    char store_path[512];               /* Persistent storage */
} EosKeyStore;
```

11.4.3 10.4.3 Keystore Operations

```
#include <eos/security.h>

void keystore_example(void)
{
    EosKeyStore ks;
    eos_keystore_init(&ks, "/flash/keystore.bin");

    /* Load existing keys from flash */
    eos_keystore_load(&ks);

    /* Add a new AES-256 key */
    uint8_t aes_key[32] = { /* key material */ };
    eos_keystore_add(&ks, "sensor-encryption",
                     EOS_KEY_AES256, aes_key, 32);

    /* Add an ECC P-256 private key */
    uint8_t ecc_priv[32] = { /* private scalar */ };
    eos_keystore_add(&ks, "ota-signing",
                     EOS_KEY_ECC_P256, ecc_priv, 32);

    /* Look up a key by ID */
    const EosKeyEntry *entry = eos_keystore_find(&ks, "sensor-encryption");
    if (entry && entry->active) {
        printf("Found key: type=%d len=%zu\n", entry->type, entry->data_len);
    }

    /* Persist to flash */
    eos_keystore_save(&ks);

    /* Debug dump */
    eos_keystore_dump(&ks);
}
```

11.4.4 10.4.4 Keystore Limits

Constant	Value	Description
EOS_KEY_MAX_ID	64	Max key identifier length
EOS_KEY_MAX_KEYS	32	Max keys in one store

11.5 10.5 Access Control (ACL)

The ACL subsystem implements a subject–resource–permission model for controlling access to system resources at runtime.

11.5.1 10.5.1 ACL Rule Model

Subject	Resource	Permission	ALLOW/DENY
"app:gps"	"/dev/uart1"	"write"	

Each ACL rule is a tuple of (subject, resource, permission, action):

```
typedef struct {
    char subject[128];      /* Who: task name, module ID */
    char resource[256];     /* What: device path, API name */
    char permission[64];    /* How: "read", "write", "exec" */
    EosAclAction action;    /* EOS_ACL_ALLOW or DENY */
} EosAclRule;
```

11.5.2 10.5.2 ACL Configuration

```
#include <eos/security.h>

void configure_acl(void)
{
    EosAcl acl;
    eos_acl_init(&acl);

    /* Allow GPS app to read UART */
    eos_acl_add_rule(&acl, "app:gps", "/dev/uart1",
                    "read", EOS_ACL_ALLOW);

    /* Allow OTA service to write flash */
    eos_acl_add_rule(&acl, "svc:ota", "/dev/flash",
                    "write", EOS_ACL_ALLOW);

    /* Deny untrusted modules from accessing keys */
    eos_acl_add_rule(&acl, "app:*", "/keystore",
                    "read", EOS_ACL_DENY);

    /* Check access at runtime */
    EosAclAction result = eos_acl_check(
        &acl, "app:gps", "/dev/uart1", "read"
    );

    if (result == EOS_ACL_ALLOW) {
        printf("Access granted\n");
    } else {
        printf("Access denied\n");
    }

    eos_acl_dump(&acl);    /* Print all rules */
}
```

```
}
```

11.5.3 10.5.3 ACL Limits

Constant	Value	Description
<code>EOS_ACL_MAX_RULES</code>	64	Maximum rules per ACL

11.6 10.6 Audit and Integrity

While EoS does not include a dedicated audit log service, the building blocks enable audit-style integrity monitoring:

```
/* Periodic integrity check example */
void integrity_monitor(void)
{
    char current_hash[EOS_SHA256_HEX_SIZE];
    eos_sha256_file("/flash/app.bin", current_hash);

    /* Compare against known-good hash stored at build time */
    extern const char expected_hash[];
    if (strcmp(current_hash, expected_hash) != 0) {
        printf("ALERT: firmware tampered!\n");
        /* Trigger secure reboot or lockdown */
    }
}
```

11.7 10.7 Security Architecture — Putting It All Together

```

    Boot ROM
    (immutable root of trust)

    verify

    Bootloader (Stage 1)
    eos_secureboot_verify_image(stage2)

    verify

    Application (Stage 2)
    eos_keystore_load() → keys available
    eos_acl_init()      → access policy active
```

11.8 10.8 Best Practices

1. **Root of trust** — Store the boot verification public key in OTP (one-time programmable) memory or ROM, never in writable flash.
2. **Key rotation** — Use the keystore `active` flag to phase out old keys. Add a new key, mark the old one inactive, then `eos_keystore_save`.
3. **Least privilege** — Start with `EOS_ACL_DENY` as default and explicitly allow only required access paths.
4. **Defense in depth** — Combine secure boot (authenticity) with runtime integrity checks (periodic SHA-256 verification).
5. **Secure erasure** — Zero key material in RAM after use with `memset(key, 0, sizeof(key))` to limit exposure windows.

11.9 10.9 API Reference Summary

Function	Description
<code>eos_secureboot_init</code>	Initialize secure boot context
<code>eos_secureboot_verify_image</code>	Verify image hash + signature
<code>eos_secureboot_dump</code>	Print boot verification status
<code>eos_firmware_sign</code>	Sign a firmware image
<code>eos_firmware_verify_sig</code>	Verify firmware signature
<code>eos_keystore_init</code>	Initialize keystore
<code>eos_keystore_add</code>	Add key to store
<code>eos_keystore_find</code>	Look up key by ID
<code>eos_keystore_save</code>	Persist keystore to flash
<code>eos_keystore_load</code>	Load keystore from flash
<code>eos_keystore_dump</code>	Print all key entries
<code>eos_acl_init</code>	Initialize ACL
<code>eos_acl_add_rule</code>	Add access control rule
<code>eos_acl_check</code>	Check subject-resource permission
<code>eos_acl_dump</code>	Print all ACL rules

Next: [Chapter 11 — Networking](#)

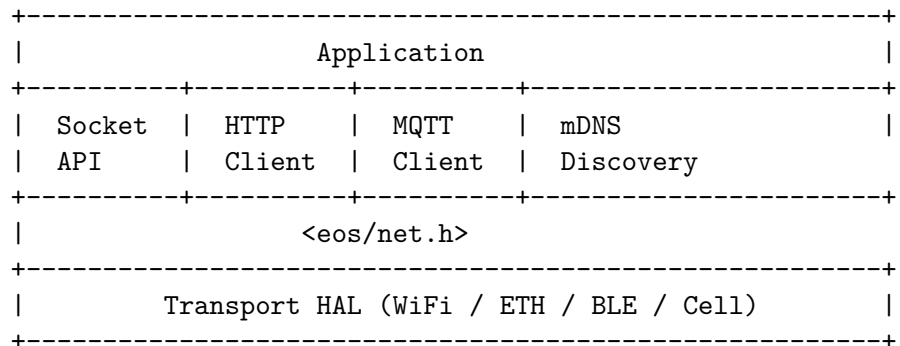
Chapter 12

Chapter 11: Networking

Author: Srikanth Patchava & EmbeddedOS Contributors

12.1 11.1 Overview

The EoS networking layer (eos/net.h) provides a transport-agnostic API covering sockets, HTTP, MQTT, and mDNS service discovery. It works over WiFi, Ethernet, BLE, or cellular transports through a unified interface gated by the `EOS_ENABLE_NET` configuration flag.



12.2 11.2 Socket API

The socket API provides BSD-style TCP/UDP communication primitives.

12.2.1 11.2.1 Core Types

```
typedef enum {
    EOS_NET_TCP = 0,
    EOS_NET_UDP = 1,
} eos_net_proto_t;
```

```
typedef struct {
```

```

    uint32_t ip;          /* Network byte order */
    uint16_t port;
} eos_net_addr_t;

typedef int eos_socket_t;
#define EOS_SOCKET_INVALID (-1)

```

12.2.2 11.2.2 Initialization

```

#include <eos/net.h>

/* Initialize the network stack -- call once at startup */
int rc = eos_net_init();
if (rc != 0) {
    printf("Network init failed: %d\n", rc);
}

/* Shutdown at teardown */
eos_net_deinit();

```

12.2.3 11.2.3 TCP Client Example

```

void tcp_client_example(void)
{
    /* Resolve hostname */
    uint32_t ip;
    eos_net_resolve("api.example.com", &ip);

    /* Create TCP socket */
    eos_socket_t sock = eos_net_socket(EOS_NET_TCP);
    if (sock == EOS_SOCKET_INVALID) return;

    /* Connect */
    eos_net_addr_t addr = { .ip = ip, .port = 8080 };
    eos_net_connect(sock, &addr);

    /* Send request */
    const char *msg = "GET /status HTTP/1.0\r\n\r\n";
    eos_net_send(sock, msg, strlen(msg));

    /* Receive response */
    char buf[512];
    int n = eos_net_recv(sock, buf, sizeof(buf), 5000);
    if (n > 0) {
        buf[n] = 0;
        printf("Response: %s\n", buf);
    }
}

```

```

    }

    eos_net_close(sock);
}

```

12.2.4 11.2.4 TCP Server Example

```

void tcp_server_example(void)
{
    eos_socket_t srv = eos_net_socket(EOS_NET_TCP);
    eos_net_bind(srv, 80);
    eos_net_listen(srv, 4);

    while (1) {
        eos_net_addr_t client;
        eos_socket_t conn = eos_net_accept(srv, &client);
        if (conn == EOS_SOCKET_INVALID) continue;

        char buf[256];
        int n = eos_net_recv(conn, buf, sizeof(buf), 3000);
        if (n > 0) {
            const char *resp = "HTTP/1.0 200 OK\r\n\r\nHello";
            eos_net_send(conn, resp, strlen(resp));
        }
        eos_net_close(conn);
    }
}

```

12.2.5 11.2.5 UDP Communication

```

void udp_example(void)
{
    eos_socket_t sock = eos_net_socket(EOS_NET_UDP);
    eos_net_bind(sock, 5000);    /* Bind for receiving */

    /* Send a datagram */
    eos_net_addr_t dest = { .ip = 0xC0A80001, .port = 5001 };
    eos_net_sendto(sock, "ping", 4, &dest);

    /* Receive a datagram */
    char buf[128];
    eos_net_addr_t src;
    int n = eos_net_recvfrom(sock, buf, sizeof(buf), &src, 2000);
    if (n > 0) {
        printf("Got %d bytes from port %u\n", n, src.port);
    }
}

```



```

    eos_net_close(sock);
}

```

12.2.6 11.2.6 Socket API Reference

Function	Description
<code>eos_net_init</code>	Initialize network stack
<code>eos_net_deinit</code>	Shutdown network stack
<code>eos_net_socket</code>	Create TCP or UDP socket
<code>eos_net_connect</code>	Connect to remote address
<code>eos_net_bind</code>	Bind socket to local port
<code>eos_net_listen</code>	Mark socket as passive (server)
<code>eos_net_accept</code>	Accept incoming connection
<code>eos_net_send</code>	Send data on connected socket
<code>eos_net_recv</code>	Receive with timeout (ms)
<code>eos_net_sendto</code>	Send UDP datagram
<code>eos_net_recvfrom</code>	Receive UDP datagram with source addr
<code>eos_net_close</code>	Close socket
<code>eos_net_resolve</code>	DNS hostname resolution

12.3 11.3 HTTP Client

The built-in HTTP client handles GET and POST requests with automatic memory management for response bodies.

12.3.1 11.3.1 Response Structure

```

typedef struct {
    int      status_code;      /* HTTP status (200, 404, etc.) */
    uint8_t *body;            /* Heap-allocated body          */
    size_t   body_len;        /* Body length in bytes         */
    char     content_type[64]; /* Content-Type header value    */
} eos_http_response_t;

```

12.3.2 11.3.2 GET Request

```

eos_http_response_t resp;
int rc = eos_http_get("http://192.168.1.100/api/status", &resp);

if (rc == 0 && resp.status_code == 200) {
    printf("Content-Type: %s\n", resp.content_type);
    printf("Body (%zu bytes): %.*s\n",
           resp.body_len, (int)resp.body_len, resp.body);
}
eos_http_response_free(&resp); /* Always free! */

```

12.3.3 11.3.3 POST Request

```
const char *json = "{\"sensor\":\"temp\",\"value\":23.5}";
eos_http_response_t resp;

int rc = eos_http_post(
    "http://192.168.1.100/api/data",
    json, strlen(json),
    "application/json",
    &resp
);

if (rc == 0) {
    printf("POST status: %d\n", resp.status_code);
}
eos_http_response_free(&resp);
```

12.4 11.4 MQTT Client

MQTT is the primary protocol for IoT telemetry and command channels. EoS provides a full MQTT 3.1.1 client with QoS 0/1/2 support.

12.4.1 11.4.1 Configuration

```
typedef struct {
    char    broker_host[128];
    uint16_t broker_port;      /* Default: 1883 */
    char    client_id[64];
    char    username[64];
    char    password[64];
    uint16_t keepalive_sec;    /* Default: 60 */
} eos_mqtt_config_t;
```

12.4.2 11.4.2 Publish / Subscribe Example

```
/* Message callback */
void on_message(const char *topic, const uint8_t *payload,
               size_t len, void *ctx)
{
    printf("[%s] %.s\n", topic, (int)len, payload);
}

void mqtt_example(void)
{
    eos_mqtt_config_t cfg = {
        .broker_host = "192.168.1.50",
    }
```

```

    .broker_port = 1883,
    .client_id   = "eos-device-01",
    .keepalive_sec = 30,
};

eos_mqtt_handle_t mqtt = eos_mqtt_connect(&cfg);
if (mqtt == EOS_MQTT_INVALID) return;

/* Subscribe to command topic */
eos_mqtt_subscribe(mqtt, "device/cmd", 1, on_message, NULL);

/* Publish telemetry */
const char *data = "{\"temp\":22.5}";
eos_mqtt_publish(mqtt, "device/telemetry",
                 data, strlen(data), 0);

/* Event loop -- call periodically */
for (int i = 0; i < 100; i++) {
    eos_mqtt_loop(mqtt, 1000);
}

eos_mqtt_unsubscribe(mqtt, "device/cmd");
eos_mqtt_disconnect(mqtt);
}

```

12.4.3 11.4.3 MQTT QoS Levels

QoS	Name	Guarantee	Use Case
0	At most once	Fire-and-forget	Telemetry streams
1	At least once	Acknowledged delivery	Sensor readings
2	Exactly once	Four-step handshake	Commands, billing

12.4.4 11.4.4 MQTT API Reference

Function	Description
<code>eos_mqtt_connect</code>	Connect to broker
<code>eos_mqtt_disconnect</code>	Graceful disconnect
<code>eos_mqtt_publish</code>	Publish message to topic
<code>eos_mqtt_subscribe</code>	Subscribe with callback
<code>eos_mqtt_unsubscribe</code>	Unsubscribe from topic
<code>eos_mqtt_loop</code>	Process network events (timeout)

12.5 11.5 mDNS Service Discovery

Multicast DNS enables zero-configuration service discovery on local networks without a central DNS server.

12.5.1 11.5.1 Register a Service

```
eos_mdns_init();

/* Advertise an HTTP server */
eos_mdns_register("EoS-Device", "_http._tcp", 80);

/* Advertise an MQTT broker */
eos_mdns_register("EoS-Broker", "_mqtt._tcp", 1883);
```

12.5.2 11.5.2 Discover a Service

```
eos_net_addr_t addr;
int rc = eos_mdns_resolve("_http._tcp", &addr, 5000);

if (rc == 0) {
    printf("Found service at %u.%u.%u.%u:%u\n",
           (addr.ip >> 24) & 0xFF, (addr.ip >> 16) & 0xFF,
           (addr.ip >> 8) & 0xFF, addr.ip & 0xFF,
           addr.port);
}

eos_mdns_deinit();
```

12.6 11.6 TLS Integration

EoS networking supports TLS through compile-time integration with mbedTLS or wolfSSL. When `EOS_ENABLE_TLS` is defined, the HTTP and MQTT clients automatically negotiate TLS connections for https URLs and MQTT port 8883.

+-----+			
	Application		
+-----+			
	HTTP	MQTT	
	(TLS)	(TLS)	
+-----+			
	mbedTLS / wolfSSL		
+-----+			
	EoS Socket Layer		
+-----+			

12.6.1 11.6.1 Enabling TLS

In eos_config.h:

```
#define EOS_ENABLE_NET      1
#define EOS_ENABLE_TLS      1
#define EOS_TLS_BACKEND     EOS_TLS_MBEDTLS /* or EOS_TLS_WOLFSSL */
```

12.7 11.7 Network Architecture Patterns

12.7.1 11.7.1 Sensor Gateway

```
void sensor_gateway(void)
{
    eos_net_init();

    /* Connect to cloud MQTT broker */
    eos_mqtt_config_t cfg = {
        .broker_host = "cloud.example.com",
        .broker_port = 8883, /* TLS */
        .client_id   = "gateway-01",
    };
    eos_mqtt_handle_t mqtt = eos_mqtt_connect(&cfg);

    /* Register for local mDNS discovery */
    eos_mdns_init();
    eos_mdns_register("Gateway", "_eos._tcp", 9000);

    /* Publish sensor data periodically */
    while (1) {
        char payload[128];
        snprintf(payload, sizeof(payload),
                 "{\"temp\":%.1f,\"hum\":%.1f}",
                 read_temp(), read_hum());

        eos_mqtt_publish(mqtt, "sensors/env",
                        payload, strlen(payload), 1);
        eos_mqtt_loop(mqtt, 1000);
    }
}
```

12.8 11.8 Error Handling

All networking functions return negative error codes on failure:

Return Value	Meaning
0	Success

Return Value	Meaning
-1	General error
-2	Timeout
-3	Connection refused
-4	DNS resolution failed
-5	TLS handshake failed

12.9 11.9 Complete API Reference

Function	Description
<code>eos_net_init</code>	Initialize network stack
<code>eos_net_deinit</code>	Shutdown network stack
<code>eos_net_socket</code>	Create socket (TCP/UDP)
<code>eos_net_connect</code>	Connect to remote host
<code>eos_net_bind</code>	Bind to local port
<code>eos_net_listen</code>	Listen for connections
<code>eos_net_accept</code>	Accept incoming connection
<code>eos_net_send</code>	Send data
<code>eos_net_recv</code>	Receive data with timeout
<code>eos_net_sendto</code>	Send UDP datagram
<code>eos_net_recvfrom</code>	Receive UDP datagram
<code>eos_net_close</code>	Close socket
<code>eos_net_resolve</code>	DNS resolution
<code>eos_http_get</code>	HTTP GET request
<code>eos_http_post</code>	HTTP POST request
<code>eos_http_response_free</code>	Free HTTP response memory
<code>eos_mqtt_connect</code>	Connect to MQTT broker
<code>eos_mqtt_disconnect</code>	Disconnect from broker
<code>eos_mqtt_publish</code>	Publish to topic
<code>eos_mqtt_subscribe</code>	Subscribe to topic
<code>eos_mqtt_unsubscribe</code>	Unsubscribe from topic
<code>eos_mqtt_loop</code>	Process MQTT events
<code>eos_mdns_init</code>	Initialize mDNS
<code>eos_mdns_deinit</code>	Shutdown mDNS
<code>eos_mdns_register</code>	Register mDNS service
<code>eos_mdns_resolve</code>	Discover mDNS service

Next: [Chapter 12 – Filesystem](#)

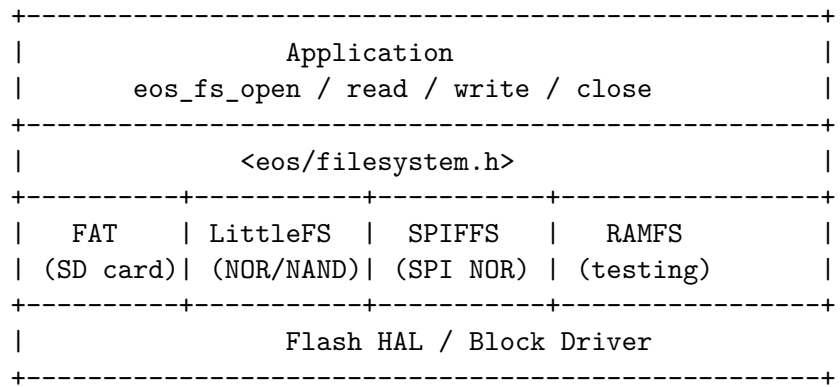
Chapter 13

Chapter 12: Filesystem

Author: Srikanth Patchava & EmbeddedOS Contributors

13.1 12.1 Overview

The EoS filesystem layer (eos/filesystem.h) provides a unified file API that abstracts over multiple storage backends – FAT, LittleFS, SPIFFS, and RAM disk. Applications use a single set of calls regardless of the underlying flash or SD card hardware, gated by EOS_ENABLE_FILESYSTEM.



13.2 12.2 Filesystem Types

Enum	Backend	Best For
EOS_FS_FAT	FAT12/16/32	SD cards, large media
EOS_FS_LITTLEFS	LittleFS	NOR/NAND flash, wear leveling
EOS_FS_SPIFFS	SPIFFS	Small SPI NOR flash
EOS_FS_RAMFS	RAM disk	Testing, volatile scratch

13.3 12.3 Configuration

```
typedef struct {
    eos_fs_type_t type;          /* Filesystem type */
    uint8_t flash_id;           /* HAL flash device ID */
    uint32_t base_addr;         /* Start address in flash */
    uint32_t size;              /* Partition size in bytes */
} eos_fs_config_t;
```

13.3.1 12.3.1 Initialization Example

```
#include <eos/filesystem.h>

void fs_setup(void)
{
    eos_fs_config_t cfg = {
        .type      = EOS_FS_LITTLEFS,
        .flash_id  = 0,           /* Internal flash */
        .base_addr = 0x00100000,  /* 1 MB offset */
        .size      = 0x00080000,  /* 512 KB partition */
    };

    int rc = eos_fs_init(&cfg);
    if (rc != 0) {
        printf("FS init failed: %d\n", rc);
        eos_fs_format();
        eos_fs_init(&cfg);
    }
}
```

13.4 12.4 Open Flags

Flags can be OR-ed together to control file open behavior:

Flag	Value	Description
EOS_O_READ	0x01	Open for reading
EOS_O_WRITE	0x02	Open for writing
EOS_O_CREATE	0x04	Create file if it does not exist
EOS_O_APPEND	0x08	Append writes to end of file
EOS_O_TRUNC	0x10	Truncate file to zero length

13.5 12.5 File Operations

13.5.1 12.5.1 Write and Read

```
void file_rw_example(void)
{
    /* Create and write */
    eos_file_t fd = eos_fs_open("/config.json",
                                EOS_O_WRITE | EOS_O_CREATE | EOS_O_TRUNC);
    if (fd == EOS_FILE_INVALID) return;

    const char *data = "{\"interval\":5000}";
    eos_fs_write(fd, data, strlen(data));
    eos_fs_sync(fd); /* Flush to flash */
    eos_fs_close(fd);

    /* Read back */
    fd = eos_fs_open("/config.json", EOS_O_READ);
    char buf[128];
    int n = eos_fs_read(fd, buf, sizeof(buf) - 1);
    if (n > 0) {
        buf[n] = 0;
        printf("Config: %s\n", buf);
    }
    eos_fs_close(fd);
}
```

13.5.2 12.5.2 Seek and Tell

```
void seek_example(eos_file_t fd)
{
    /* Jump to byte 100 from start */
    eos_fs_seek(fd, 100, EOS_SEEK_SET);

    /* Get current position */
    uint32_t pos;
    eos_fs_tell(fd, &pos);
    printf("Position: %u\n", pos);

    /* Jump to 50 bytes before end */
    eos_fs_seek(fd, -50, EOS_SEEK_END);
}
```

13.5.3 12.5.3 Truncate

```
/* Shrink a log file to 1 KB */
eos_file_t fd = eos_fs_open("/log.txt", EOS_O_WRITE);
```

```

eos_fs_truncate(fd, 1024);
eos_fs_close(fd);

```

13.6 12.6 Directory Operations

```

void directory_example(void)
{
    /* Create a directory */
    eos_fs_mkdir("/data");

    /* List directory contents */
    eos_dir_t dir = eos_fs_opendir("/data");
    if (dir == EOS_DIR_INVALID) return;

    eos_dirent_t entry;
    while (eos_fs_readdir(dir, &entry) == 0) {
        printf("%s %s %u bytes\n",
            entry.is_dir ? "[DIR]" : "[FILE]",
            entry.name,
            entry.size);
    }
    eos_fs_closedir(dir);
}

```

13.6.1 12.6.1 Directory Entry Structure

```

typedef struct {
    char    name[EOS_PATH_MAX]; /* File/directory name */
    uint32_t size;              /* Size in bytes */
    bool    is_dir;             /* true if directory */
} eos_dirent_t;

```

13.7 12.7 Path Operations

```

/* Check existence */
if (eos_fs_exists("/config.json")) {
    printf("Config file found\n");
}

/* Rename / move */
eos_fs_rename("/config.json", "/config.json.bak");

/* Delete */
eos_fs_remove("/old_log.txt");

```

13.8 12.8 Filesystem Statistics

```
eos_fs_stat_t stat;
eos_fs_stat(&stat);

printf("Total: %u KB\n", stat.total_bytes / 1024);
printf("Used: %u KB\n", stat.used_bytes / 1024);
printf("Free: %u KB\n", stat.free_bytes / 1024);

typedef struct {
    uint32_t total_bytes;    /* Total filesystem capacity */
    uint32_t used_bytes;     /* Bytes in use */
    uint32_t free_bytes;     /* Bytes available */
} eos_fs_stat_t;
```

13.9 12.9 System Limits

Constant	Value	Description
EOS_PATH_MAX	128	Maximum path length in bytes
EOS_FILE_MAX	8	Maximum simultaneously open fds

13.10 12.10 Configuration Persistence Pattern

A common pattern for storing device configuration:

```
#include <eos/filesystem.h>
#include <eos/crypto.h>

typedef struct {
    uint32_t magic;          /* 0xCOFFEE01 */
    uint32_t version;        /* Config schema version */
    uint32_t interval_ms;    /* Sensor polling interval */
    char      mqtt_host[64]; /* MQTT broker hostname */
    uint16_t mqtt_port;      /* MQTT broker port */
    uint32_t crc;            /* CRC-32 of preceding fields */
} device_config_t;

int config_save(const device_config_t *cfg)
{
    eos_file_t fd = eos_fs_open("/config.bin",
                                EOS_O_WRITE | EOS_O_CREATE | EOS_O_TRUNC);
    if (fd == EOS_FILE_INVALID) return -1;

    eos_fs_write(fd, cfg, sizeof(*cfg));
    eos_fs_sync(fd);
    eos_fs_close(fd);
}
```

```

    return 0;
}

int config_load(device_config_t *cfg)
{
    if (!eos_fs_exists("/config.bin")) return -1;

    eos_file_t fd = eos_fs_open("/config.bin", EOS_O_READ);
    if (fd == EOS_FILE_INVALID) return -1;

    int n = eos_fs_read(fd, cfg, sizeof(*cfg));
    eos_fs_close(fd);

    if (n != sizeof(*cfg) || cfg->magic != 0xC0FFEE01) {
        return -2;    /* Corrupt or wrong format */
    }

    /* Verify CRC */
    uint32_t expected = eos_crc32(0, cfg,
                                   sizeof(*cfg) - sizeof(uint32_t));
    if (expected != cfg->crc) return -3;

    return 0;
}

```

13.11 12.11 Filesystem Type Comparison

Feature	FAT	LittleFS	SPIFFS	RAMFS
Wear leveling	No	Yes	Yes	N/A
Directories	Yes	Yes	No	Yes
Power-safe	No	Yes (CoW)	Partial	No
Max file size	4 GB	Partition	Partition	RAM
RAM overhead	~4 KB	~2 KB	~1 KB	Minimal
Best for	SD cards	NOR flash	Small SPI	Testing

13.12 12.12 Data Logging Example

```

void log_sensor_reading(float temperature, float humidity)
{
    eos_file_t fd = eos_fs_open("/log.csv",
                                EOS_O_WRITE | EOS_O_CREATE | EOS_O_APPEND);
    if (fd == EOS_FILE_INVALID) return;

    char line[80];

```

```

int len = snprintf(line, sizeof(line), "%.2f,%.2f\n",
                    temperature, humidity);
eos_fs_write(fd, line, len);
eos_fs_close(fd);

/* Check if log is getting too large */
eos_fs_stat_t st;
eos_fs_stat(&st);
if (st.free_bytes < 4096) {
    /* Rotate: delete old log, keep recent */
    eos_fs_remove("/log.csv.old");
    eos_fs_rename("/log.csv", "/log.csv.old");
}
}

```

13.13 12.13 API Reference Summary

Function	Description
<code>eos_fs_init</code>	Initialize filesystem
<code>eos_fs_deinit</code>	Unmount and release resources
<code>eos_fs_format</code>	Format the filesystem
<code>eos_fs_stat</code>	Get capacity/usage statistics
<code>eos_fs_open</code>	Open a file with flags
<code>eos_fs_close</code>	Close a file descriptor
<code>eos_fs_read</code>	Read data from file
<code>eos_fs_write</code>	Write data to file
<code>eos_fs_seek</code>	Seek to position in file
<code>eos_fs_tell</code>	Get current file position
<code>eos_fs_truncate</code>	Set file size
<code>eos_fs_sync</code>	Flush data to storage
<code>eos_fs_mkdir</code>	Create a directory
<code>eos_fs_opendir</code>	Open directory for listing
<code>eos_fs_readdir</code>	Read next directory entry
<code>eos_fs_closedir</code>	Close directory handle
<code>eos_fs_remove</code>	Delete a file or empty directory
<code>eos_fs_rename</code>	Rename or move a file
<code>eos_fs_exists</code>	Check if a path exists

Next: [Chapter 13 – OTA Updates](#)

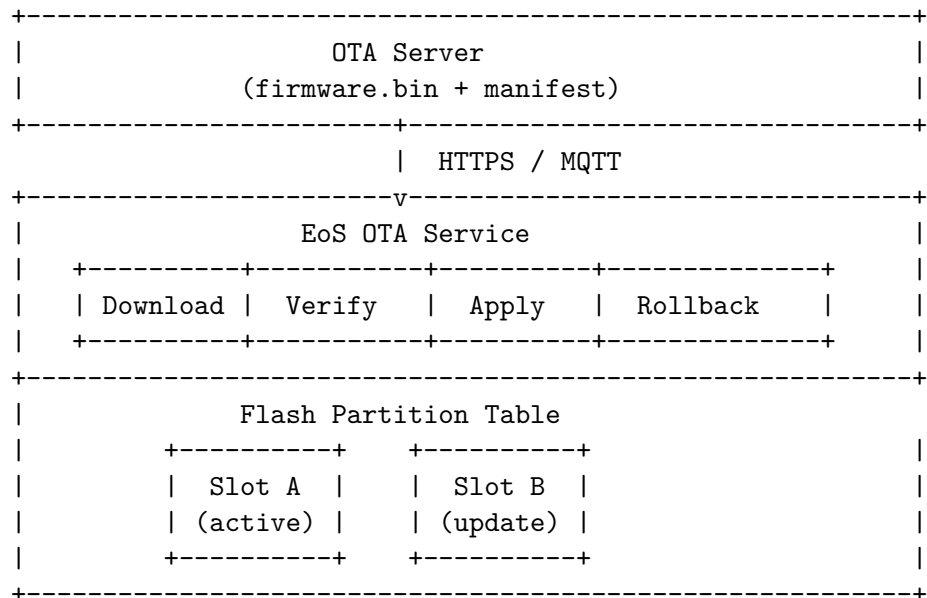
Chapter 14

Chapter 13: OTA Updates

Author: Srikanth Patchava & EmbeddedOS Contributors

14.1 13.1 Overview

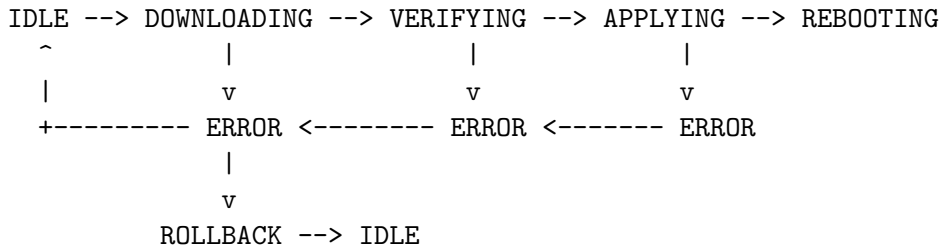
The EoS OTA (Over-the-Air) update service (`eos/ota.h`) provides a complete firmware update framework with download management, SHA-256 verification, A/B slot management, and automatic rollback. It is gated by the `EOS_ENABLE_OTA` configuration flag.



14.2 13.2 OTA State Machine

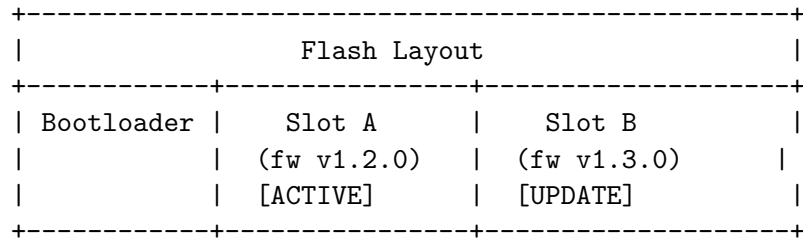
The update process follows a well-defined state machine:

State	Value	Description
EOS_OTA_STATE_IDLE	0	No update in progress
EOS_OTA_STATE_DOWNLOADING	1	Receiving firmware chunks
EOS_OTA_STATE_VERIFYING	2	Validating hash/signature
EOS_OTA_STATE_APPLYING	3	Writing to update slot
EOS_OTA_STATE_REBOOTING	4	About to reboot into new fw
EOS_OTA_STATE_ERROR	5	Error occurred



14.3 13.3 A/B Slot Management

EoS uses a dual-slot (A/B) partitioning scheme for safe updates:



Enum	Value	Description
EOS_OTA_SLOT_A	0	First firmware slot
EOS_OTA_SLOT_B	1	Second firmware slot

14.3.1 13.3.1 Slot Operations

```

/* Query which slot is currently running */
eos_ota_slot_t active = eos_ota_get_active_slot();
printf("Running from Slot %c\n",
       active == EOS_OTA_SLOT_A ? 'A' : 'B');

/* After successful boot, mark current slot as valid */
eos_ota_mark_slot_valid(active);

/* Force boot from a specific slot (e.g., for recovery) */
eos_ota_set_active_slot(EOS_OTA_SLOT_A);

```

14.4 13.4 Update Source Configuration

```
typedef struct {
    char    url[256];           /* Firmware download URL      */
    char    version[32];       /* Target version string      */
    uint32_t expected_size;    /* Expected size in bytes     */
    uint8_t expected_sha256[32]; /* Expected SHA-256 digest    */
    bool    use_tls;           /* Use HTTPS?                 */
} eos_ota_source_t;
```

14.5 13.5 Update Status

```
typedef struct {
    eos_ota_state_t state;      /* Current OTA state          */
    uint32_t    bytes_received; /* Downloaded so far          */
    uint32_t    total_bytes;    /* Total firmware size        */
    uint8_t     progress_pct;   /* 0-100 progress percentage */
    eos_ota_slot_t active_slot; /* Currently running slot     */
    eos_ota_slot_t update_slot; /* Slot being written to      */
    char        current_version[32]; /* Running fw version        */
    char        update_version[32]; /* New fw version             */
} eos_ota_status_t;
```

14.6 13.6 Complete OTA Workflow

14.6.1 13.6.1 Check for Updates

```
#include <eos/ota.h>

int check_and_update(void)
{
    eos_ota_init();

    eos_ota_source_t source = {
        .url          = "https://fw.example.com/v1.3.0/firmware.bin",
        .version       = "1.3.0",
        .expected_size = 262144,
        .use_tls       = true,
    };
    /* Set expected_sha256 from manifest */
    memcpy(source.expected_sha256, manifest_hash, 32);

    bool available = false;
    int rc = eos_ota_check_update(&source, &available);
    if (rc != 0 || !available) {
        printf("No update available\n");
    }
}
```



```

        return -1;
    }

    printf("Update available: v%s\n", source.version);
    return 0;
}

```

14.6.2 13.6.2 Download and Apply

```

int perform_update(const eos_ota_source_t *source)
{
    /* Begin the update -- sets state to DOWNLOADING */
    int rc = eos_ota_begin(source);
    if (rc != 0) return rc;

    /* Download in chunks (typically via HTTP GET) */
    eos_http_response_t resp;
    rc = eos_http_get(source->url, &resp);
    if (rc != 0) {
        eos_ota_abort();
        return rc;
    }

    /* Feed chunks to OTA writer */
    size_t offset = 0;
    while (offset < resp.body_len) {
        size_t chunk = (resp.body_len - offset > 4096)
            ? 4096 : resp.body_len - offset;
        eos_ota_write_chunk(resp.body + offset, chunk);
        offset += chunk;
    }
    eos_http_response_free(&resp);

    /* Finalize download */
    rc = eos_ota_finish();
    if (rc != 0) return rc;

    /* Verify integrity (SHA-256) */
    rc = eos_ota_verify();
    if (rc != 0) {
        printf("Verification failed -- aborting\n");
        eos_ota_abort();
        return rc;
    }

    /* Apply: set update slot as active */
    rc = eos_ota_apply();
}

```

```

    if (rc != 0) return rc;

    printf("Update applied -- rebooting...\n");
    /* System reboot triggered by eos_ota_apply */
    return 0;
}

```

14.6.3 13.6.3 Progress Monitoring

```

void progress_handler(uint8_t pct, void *ctx)
{
    printf("OTA progress: %u%%\n", pct);
    /* Update LED, display, or MQTT telemetry */
}

/* Register callback */
eos_ota_set_progress_callback(progress_handler, NULL);

```

14.6.4 13.6.4 Status Query

```

eos_ota_status_t status;
eos_ota_get_status(&status);

printf("State:      %d\n", status.state);
printf("Progress: %u%%\n", status.progress_pct);
printf("Current:  v%s (Slot %c)\n",
       status.current_version,
       status.active_slot == EOS_OTA_SLOT_A ? 'A' : 'B');
printf("Update:   v%s (Slot %c)\n",
       status.update_version,
       status.update_slot == EOS_OTA_SLOT_A ? 'A' : 'B');

```

14.7 13.7 Rollback

If the new firmware fails self-tests after reboot, the system can roll back to the previous version:

```

void first_boot_check(void)
{
    /* Run self-tests */
    if (selftest_passed()) {
        /* Mark current slot as valid */
        eos_ota_slot_t active = eos_ota_get_active_slot();
        eos_ota_mark_slot_valid(active);
        printf("Boot validated\n");
    } else {
        printf("Self-test failed -- rolling back\n");
    }
}

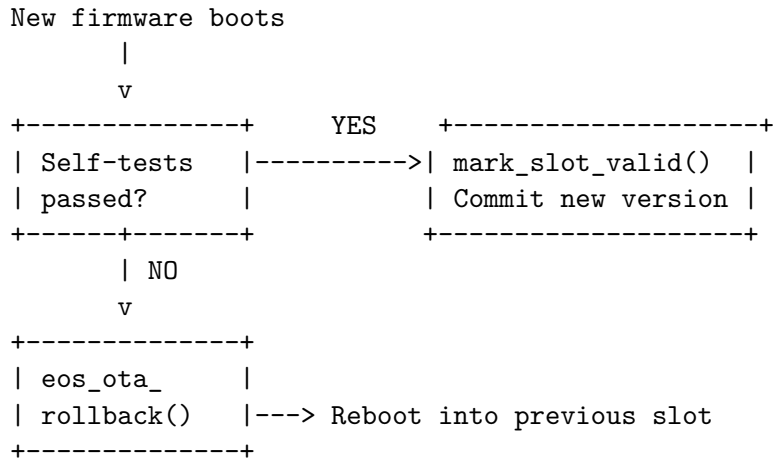
```

```

    eos_ota_rollback();
    /* Reboots into previous slot */
}
}

```

14.7.1 13.7.1 Rollback Safety Flow



14.8 13.8 OTA with MQTT Trigger

```

void on_ota_command(const char *topic, const uint8_t *payload,
                    size_t len, void *ctx)
{
    /* Parse JSON manifest */
    eos_ota_source_t source;
    parse_ota_manifest(payload, len, &source);

    /* Perform update in background task */
    perform_update(&source);
}

void setup_ota_listener(eos_mqtt_handle_t mqtt)
{
    eos_mqtt_subscribe(mqtt, "device/ota/update", 1,
                       on_ota_command, NULL);
}

```

14.9 13.9 Security Considerations

1. **Always use TLS** – Set `use_tls = true` for production downloads to prevent man-in-the-middle attacks.
2. **Verify before apply** – Never call `eos_ota_apply()` without a successful `eos_ota_verify()` return.

3. **Anti-rollback** – Maintain a monotonic version counter in OTP memory to prevent downgrade attacks.
4. **Signed manifests** – Verify the update manifest signature before trusting the `expected_sha256` field.

14.10 13.10 API Reference Summary

Function	Description
<code>eos_ota_init</code>	Initialize OTA subsystem
<code>eos_ota_deinit</code>	Shutdown OTA subsystem
<code>eos_ota_check_update</code>	Check if update is available
<code>eos_ota_begin</code>	Start a new update
<code>eos_ota_write_chunk</code>	Write firmware data chunk
<code>eos_ota_finish</code>	Finalize download
<code>eos_ota_abort</code>	Abort current update
<code>eos_ota_verify</code>	Verify firmware integrity
<code>eos_ota_apply</code>	Apply update and reboot
<code>eos_ota_rollback</code>	Rollback to previous firmware
<code>eos_ota_get_status</code>	Query current OTA status
<code>eos_ota_set_progress_callback</code>	Register progress callback
<code>eos_ota_get_active_slot</code>	Get currently active slot
<code>eos_ota_set_active_slot</code>	Force a specific slot active
<code>eos_ota_mark_slot_valid</code>	Mark slot as valid after boot

Next: [Chapter 14 – Sensor and Motor Frameworks](#)

Chapter 15

Chapter 14: Sensor & Motor Frameworks

Author: Srikanth Patchava & EmbeddedOS Contributors

15.1 14.1 Overview

EoS provides two complementary frameworks for physical-world interaction: the **Sensor Framework** (`eos/sensor.h`) for uniform sensor data acquisition and filtering, and the **Motor Control Framework** (`eos/motor_ctrl.h`) for closed-loop motor control with PID regulation and trajectory planning.

Application

Sensor Framework	Motor Control Framework
register / read / filter	PID / trajectory / encoder
<eos/sensor.h>	<eos/motor_ctrl.h>

HAL Drivers

ADC / I2C / SPI	PWM / Encoder / GPIO
-----------------	----------------------

Chapter 16

Part I: Sensor Framework

16.1 14.2 Sensor Types

EoS supports 14 built-in sensor types via `eos_sensor_type_t`:

Enum	ID	Description
<code>EOS_SENSOR_TEMPERATURE</code>	0	Temperature (°C)
<code>EOS_SENSOR_HUMIDITY</code>	1	Relative humidity (%)
<code>EOS_SENSOR_PRESSURE</code>	2	Barometric pressure
<code>EOS_SENSOR_LIGHT</code>	3	Ambient light (lux)
<code>EOS_SENSOR_PROXIMITY</code>	4	Distance / proximity
<code>EOS_SENSOR_ACCELEROMETER</code>	5	Acceleration (m/s ²)
<code>EOS_SENSOR_GYROSCOPE</code>	6	Angular velocity
<code>EOS_SENSOR_MAGNETOMETER</code>	7	Magnetic field
<code>EOS_SENSOR_HEART_RATE</code>	8	Heart rate (BPM)
<code>EOS_SENSOR_DUST</code>	9	Dust / particulate
<code>EOS_SENSOR_GAS</code>	10	Gas concentration
<code>EOS_SENSOR_VOLTAGE</code>	11	Voltage (V)
<code>EOS_SENSOR_CURRENT</code>	12	Current (A)
<code>EOS_SENSOR_CUSTOM</code>	13	User-defined sensor

Maximum registered sensors: `EOS_SENSOR_MAX` = 16.

16.2 14.3 Filter Types

The framework provides five built-in digital filters:

Enum	ID	Algorithm
<code>EOS_FILTER_NONE</code>	0	No filtering — raw value
<code>EOS_FILTER_AVERAGE</code>	1	Moving average
<code>EOS_FILTER_MEDIAN</code>	2	Median filter (spike rejection)
<code>EOS_FILTER_LOWPASS</code>	3	IIR low-pass filter

Enum	ID	Algorithm
EOS_FILTER_KALMAN	4	1-D Kalman filter

16.3 14.4 Sensor Registration

Each sensor is registered with a configuration struct and a read callback:

```
typedef int (*eos_sensor_read_fn)(uint8_t id, float *raw_value);

typedef struct {
    uint8_t      id;                /* Unique sensor ID */
    const char    *name;            /* Human-readable name */
    eos_sensor_type_t type;         /* Sensor category */
    eos_filter_type_t filter;       /* Default filter */
    uint8_t      filter_window;    /* Window size (1-32) */
    uint32_t      sample_interval_ms; /* Polling period */
    eos_sensor_read_fn read_fn;    /* Hardware read func */
} eos_sensor_config_t;
```

16.3.1 14.4.1 Registration Example

```
#include <eos/sensor.h>

/* Hardware-specific read function */
int read_bme280_temp(uint8_t id, float *value)
{
    *value = bme280_get_temperature(); /* HAL driver call */
    return 0;
}

int read_bme280_hum(uint8_t id, float *value)
{
    *value = bme280_get_humidity();
    return 0;
}

void sensors_setup(void)
{
    eos_sensor_init();

    eos_sensor_config_t temp_cfg = {
        .id      = 0,
        .name     = "BME280-Temp",
        .type     = EOS_SENSOR_TEMPERATURE,
        .filter   = EOS_FILTER_KALMAN,
        .filter_window = 8,
```

```

        .sample_interval_ms = 1000,
        .read_fn             = read_bme280_temp,
    };
    eos_sensor_register(&temp_cfg);

    eos_sensor_config_t hum_cfg = {
        .id                 = 1,
        .name                = "BME280-Hum",
        .type                = EOS_SENSOR_HUMIDITY,
        .filter              = EOS_FILTER_AVERAGE,
        .filter_window       = 4,
        .sample_interval_ms = 2000,
        .read_fn             = read_bme280_hum,
    };
    eos_sensor_register(&hum_cfg);
}

```

16.4 14.5 Reading Sensor Data

```

typedef struct {
    float    value;           /* Current reading          */
    float    min;            /* Minimum observed value   */
    float    max;            /* Maximum observed value   */
    uint32_t timestamp_ms;   /* Timestamp of this reading */
    bool     valid;          /* true if data is usable   */
} eos_sensor_reading_t;

```

16.4.1 14.5.1 Raw vs Filtered Reads

```

eos_sensor_reading_t reading;

/* Raw value - direct from hardware */
eos_sensor_read(0, &reading);
printf("Raw temp: %.2f°C\n", reading.value);

/* Filtered value - processed through configured filter */
eos_sensor_read_filtered(0, &reading);
printf("Filtered temp: %.2f°C (min=%.2f max=%.2f)\n",
        reading.value, reading.min, reading.max);

```

16.5 14.6 Calibration

```

typedef struct {
    float offset;           /* value = raw * scale + offset */
    float scale;

```



```
    bool calibrated;
} eos_sensor_calib_t;
```

16.5.1 14.6.1 Manual Calibration

```
/* Apply known offset and scale */
eos_sensor_calib_t cal = {
    .offset = -0.5f,    /* Subtract 0.5°C bias */
    .scale  = 1.02f,    /* Scale correction */
};
eos_sensor_calibrate(0, &cal);

/* Read back current calibration */
eos_sensor_calib_t current;
eos_sensor_get_calibration(0, &current);
printf("Offset=%.2f Scale=%.2f Calibrated=%d\n",
       current.offset, current.scale, current.calibrated);
```

16.5.2 14.6.2 Auto-Calibration

```
/* Let the framework compute calibration from baseline readings */
eos_sensor_auto_calibrate(0);
```

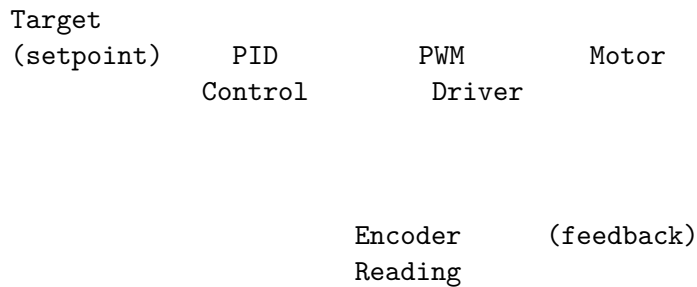
16.6 14.7 Runtime Filter Changes

```
/* Switch sensor 0 to median filter with window size 5 */
eos_sensor_set_filter(0, EOS_FILTER_MEDIAN, 5);
```

Chapter 17

Part II: Motor Control Framework

17.1 14.8 Motor Control Architecture



17.2 14.9 PID Parameters

```
typedef struct {
    float kp;           /* Proportional gain */
    float ki;           /* Integral gain */
    float kd;           /* Derivative gain */
    float integral_max; /* Anti-windup limit */
    float output_min;   /* Minimum output clamp */
    float output_max;   /* Maximum output clamp */
} eos_pid_params_t;
```

17.3 14.10 Motor Controller Configuration

```
typedef struct {
    uint8_t motor_id; /* HAL motor ID */
    eos_pid_params_t pid_speed; /* PID for speed loop */
    eos_pid_params_t pid_position; /* PID for position loop */
    uint16_t encoder_cpr; /* Counts per revolution */
}
```

```

    uint32_t      control_rate_hz; /* PID update frequency */
} eos_motor_ctrl_config_t;

```

17.3.1 14.10.1 Configuration Example

```

#include <eos/motor_ctrl.h>

void motor_setup(void)
{
    eos_motor_ctrl_init();

    eos_motor_ctrl_config_t cfg = {
        .motor_id      = 0,
        .pid_speed      = { .kp=1.0, .ki=0.1, .kd=0.01,
                           .integral_max=100, .output_min=-100,
                           .output_max=100 },
        .pid_position   = { .kp=2.0, .ki=0.05, .kd=0.1,
                           .integral_max=500, .output_min=-200,
                           .output_max=200 },
        .encoder_cpr    = 1024, /* Quadrature encoder */
        .control_rate_hz = 1000, /* 1 kHz PID loop */
    };
    eos_motor_ctrl_configure(&cfg);
}

```

17.4 14.11 Speed Control

```

/* Set target speed: positive = forward, negative = reverse */
eos_motor_ctrl_set_speed(0, 500); /* 500 RPM forward */
eos_motor_ctrl_set_speed(0, -300); /* 300 RPM reverse */
eos_motor_ctrl_set_speed(0, 0); /* Stop */

```

17.5 14.12 Position Control

```

/* Move to absolute position (encoder counts) */
eos_motor_ctrl_set_position(0, 10240); /* 10 revolutions */

/* Move relative to current position */
eos_motor_ctrl_move_relative(0, 1024); /* +1 revolution */
eos_motor_ctrl_move_relative(0, -512); /* -0.5 revolutions */

```

17.6 14.13 Trajectory Planning

Multi-segment trajectories enable smooth motion profiles:

```
typedef struct {
    int32_t target_position;    /* End position (counts) */
    int16_t max_speed;         /* Maximum speed (RPM) */
    int16_t acceleration;      /* Accel (steps/sec2) */
    uint32_t duration_ms;      /* Segment duration */
} eos_trajectory_t;
```

17.6.1 14.13.1 Trajectory Example

```
eos_trajectory_t path[] = {
    { .target_position = 2048, .max_speed = 200,
      .acceleration = 100, .duration_ms = 2000 },
    { .target_position = 4096, .max_speed = 500,
      .acceleration = 200, .duration_ms = 1500 },
    { .target_position = 0, .max_speed = 300,
      .acceleration = 150, .duration_ms = 3000 },
};

eos_motor_ctrl_run_trajectory(0, path, 3);
```

Speed

Time

Seg 1 Seg 2 Seg 3

17.7 14.14 Motor Status

```
typedef struct {
    int32_t current_position; /* Encoder position */
    int16_t current_speed;    /* Measured speed (RPM) */
    int32_t target_position;  /* Commanded position */
    int16_t target_speed;     /* Commanded speed */
    float pid_output;         /* Current PID output */
    bool in_motion;           /* Motor is moving */
    bool stalled;             /* Stall detected */
} eos_motor_ctrl_status_t;
```

```
eos_motor_ctrl_status_t st;
eos_motor_ctrl_get_status(0, &st);

printf("Pos: %d / %d Speed: %d / %d PID: %.2f\n",
```

```

        st.current_position, st.target_position,
        st.current_speed, st.target_speed,
        st.pid_output);

if (st.stalled) {
    printf("WARNING: Motor stall detected!\n");
    eos_motor_ctrl_emergency_stop(0);
}

```

17.8 14.15 Emergency Stop and Safety

```

/* Graceful stop - decelerates to zero */
eos_motor_ctrl_stop(0);

/* Emergency stop - immediate halt, disables PWM */
eos_motor_ctrl_emergency_stop(0);

/* Reset encoder position to zero (homing) */
eos_motor_ctrl_reset_position(0);

```

17.9 14.16 PID Tuning at Runtime

```

/* Adjust speed PID gains without re-initialization */
eos_pid_params_t new_pid = {
    .kp = 1.5, .ki = 0.2, .kd = 0.02,
    .integral_max = 150,
    .output_min = -100, .output_max = 100,
};
eos_motor_ctrl_set_pid_speed(0, &new_pid);

/* Adjust position PID separately */
eos_motor_ctrl_set_pid_position(0, &new_pid);

```

17.10 14.17 Control Loop Integration

The `eos_motor_ctrl_update()` function must be called periodically at the configured `control_rate_hz`. Typical integration points:

```

/* Option 1: Timer ISR */
void timer_isr(void)
{
    eos_motor_ctrl_update();
}

/* Option 2: RTOS task */

```

```

void motor_task(void *arg)
{
    while (1) {
        eos_motor_ctrl_update();
        eos_task_delay_ms(1);    /* 1 kHz */
    }
}

```

17.11 14.18 Combined Sensor + Motor Example

```

void temperature_fan_controller(void)
{
    eos_sensor_init();
    eos_motor_ctrl_init();

    /* Register temperature sensor */
    eos_sensor_config_t tcfg = {
        .id = 0, .name = "Heatsink",
        .type = EOS_SENSOR_TEMPERATURE,
        .filter = EOS_FILTER_LOWPASS,
        .filter_window = 4,
        .sample_interval_ms = 500,
        .read_fn = read_ntc_temp,
    };
    eos_sensor_register(&tcfg);

    /* Configure fan motor */
    eos_motor_ctrl_config_t mcfg = {
        .motor_id = 0,
        .pid_speed = { .kp=0.5, .ki=0.1, .kd=0.0,
                      .integral_max=100,
                      .output_min=0, .output_max=100 },
        .encoder_cpr = 2,
        .control_rate_hz = 100,
    };
    eos_motor_ctrl_configure(&mcfg);

    /* Control loop */
    while (1) {
        eos_sensor_reading_t temp;
        eos_sensor_read_filtered(0, &temp);

        int16_t fan_speed = 0;
        if (temp.value > 50.0f) fan_speed = (int16_t)(temp.value * 10);
        if (fan_speed > 1000)    fan_speed = 1000;
    }
}

```

```

    eos_motor_ctrl_set_speed(0, fan_speed);
    eos_task_delay_ms(500);
}
}

```

17.12 14.19 API Reference Summary

17.12.1 Sensor API

Function	Description
<code>eos_sensor_init</code>	Initialize sensor framework
<code>eos_sensor_deinit</code>	Shutdown sensor framework
<code>eos_sensor_register</code>	Register a sensor
<code>eos_sensor_unregister</code>	Unregister a sensor
<code>eos_sensor_read</code>	Read raw sensor value
<code>eos_sensor_read_filtered</code>	Read filtered sensor value
<code>eos_sensor_calibrate</code>	Apply calibration
<code>eos_sensor_get_calibration</code>	Read current calibration
<code>eos_sensor_auto_calibrate</code>	Auto-calibrate sensor
<code>eos_sensor_set_filter</code>	Change filter type/window
<code>eos_sensor_get_count</code>	Get registered sensor count
<code>eos_sensor_get_info</code>	Get sensor configuration

17.12.2 Motor Control API

Function	Description
<code>eos_motor_ctrl_init</code>	Initialize motor framework
<code>eos_motor_ctrl_deinit</code>	Shutdown motor framework
<code>eos_motor_ctrl_configure</code>	Configure a motor controller
<code>eos_motor_ctrl_remove</code>	Remove motor configuration
<code>eos_motor_ctrl_set_speed</code>	Set target speed (closed-loop)
<code>eos_motor_ctrl_set_position</code>	Set target position
<code>eos_motor_ctrl_move_relative</code>	Move by delta
<code>eos_motor_ctrl_run_trajectory</code>	Execute trajectory segments
<code>eos_motor_ctrl_stop</code>	Graceful stop
<code>eos_motor_ctrl_emergency_stop</code>	Immediate halt
<code>eos_motor_ctrl_reset_position</code>	Zero encoder position
<code>eos_motor_ctrl_set_pid_speed</code>	Tune speed PID
<code>eos_motor_ctrl_set_pid_position</code>	Tune position PID
<code>eos_motor_ctrl_get_status</code>	Get motor status
<code>eos_motor_ctrl_update</code>	Run one PID iteration

Next: [Chapter 15 — Compatibility Layers](#)

Chapter 18

Chapter 15: Compatibility Layers

Author: Srikanth Patchava & EmbeddedOS Contributors

18.1 15.1 Overview

EoS provides three compatibility layers that map well-known APIs onto native EoS kernel primitives. This enables porting legacy applications and leveraging existing codebases without rewriting them from scratch.

+-----+ Legacy / Ported Application +-----+			
POSIX Layer	VxWorks Layer	Linux IPC Layer	
pthreads, I/O,	taskSpawn,	AF_UNIX, eventfd,	
signals, IPC	semaphores, ISR	epoll, pipe	
+-----+			
EoS Kernel API			
Tasks - Mutexes - Semaphores - Queues - IRQs			
+-----+			

Layer	Headers	Coverage
POSIX	posix_threads.h, posix_io.h, posix_ipc.h, posix_signals.h, posix_sync.h, posix_time.h	Threads, file I/O, mqueues, pipes, shm, signals
VxWorks	vxworks_task.h, vxworks_sem.h, vxworks_int.h, vxworks_wd.h, vxworks_msgq.h	Tasks, semaphores, ISR, watchdog, message queues
Linux IPC	linux_ipc.h, linux_sysv_ipc.h	AF_UNIX sockets, eventfd, epoll, pipe, SysV IPC

Chapter 19

Part I: POSIX Compatibility

19.1 15.2 POSIX Threads (pthreads)

The pthreads layer (posix_threads.h) maps the standard pthread API onto EoS kernel task primitives.

19.1.1 15.2.1 Thread Attributes

```
typedef uint8_t pthread_t;

typedef struct {
    uint8_t  priority;      /* EoS task priority (0-31) */
    uint32_t stack_size;   /* Stack size in bytes      */
    int      detach_state; /* JOINABLE or DETACHED    */
} pthread_attr_t;

#define PTHREAD_CREATE_JOINABLE 0
#define PTHREAD_CREATE_DETACHED 1
#define PTHREAD_STACK_MIN      256
```

19.1.2 15.2.2 Thread Creation and Join

```
#include <eos/posix_threads.h>

void *worker(void *arg)
{
    int id = *(int *)arg;
    printf("Worker %d running\n", id);
    return NULL;
}

void posix_threads_example(void)
{
```

```

pthread_attr_t attr;
pthread_attr_init(&attr);

struct sched_param param = { .sched_priority = 10 };
pthread_attr_setschedparam(&attr, &param);
pthread_attr_setstacksize(&attr, 2048);

pthread_t t1, t2;
int id1 = 1, id2 = 2;

pthread_create(&t1, &attr, worker, &id1);
pthread_create(&t2, &attr, worker, &id2);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

pthread_attr_destroy(&attr);
}

```

19.1.3 15.2.3 Mapping Table: pthreads to EoS Kernel

POSIX Function	EoS Kernel Equivalent
pthread_create	eos_task_create
pthread_join	eos_task_join
pthread_exit	eos_task_delete(self)
pthread_self	eos_task_current
pthread_yield	eos_task_yield
pthread_attr_setschedparam	Priority mapping
pthread_attr_setstacksize	Stack size parameter

19.2 15.3 POSIX File I/O

The I/O layer (posix_io.h) provides open/close/read/write/lseek mapped to eos_fs_* with file descriptors 0/1/2 reserved for console I/O.

19.2.1 15.3.1 File Descriptor Model

```

fd 0 --- stdin  (console read)
fd 1 --- stdout (console write)
fd 2 --- stderr (console write)
fd 3..15 -- filesystem files (EOS_POSIX_FD_MAX = 16)

```

19.2.2 15.3.2 I/O Operations

```

#include <eos/posix_io.h>

```

```

void posix_io_example(void)
{
    eos_posix_io_init();

    /* Open and write */
    int fd = open("/data/log.txt",
                  EOS_O_WRONLY_IO | EOS_O_CREAT_IO | EOS_O_TRUNC_IO);
    write(fd, "Hello POSIX\n", 12);
    close(fd);

    /* Read back */
    fd = open("/data/log.txt", EOS_O_RDONLY_IO);
    char buf[64];
    ssize_t n = read(fd, buf, sizeof(buf));
    close(fd);

    /* File info */
    struct eos_stat st;
    stat("/data/log.txt", &st);
    printf("Size: %u bytes\n", st.st_size);
}

```

19.2.3 15.3.3 Open Flags

Flag	Value	Description
EOS_O_RDONLY_IO	0x0000	Read only
EOS_O_WRONLY_IO	0x0001	Write only
EOS_O_RDWR_IO	0x0002	Read and write
EOS_O_CREAT_IO	0x0100	Create if not exists
EOS_O_TRUNC_IO	0x0200	Truncate to zero
EOS_O_APPEND_IO	0x0400	Append mode

19.3 15.4 POSIX IPC

19.3.1 15.4.1 Message Queues

```

#include <eos/posix_ipc.h>

void mq_example(void)
{
    struct mq_attr attr = {
        .mq_maxmsg = 10,
        .mq_msgsize = 64,
    };
}

```

```

mqd_t mq = mq_open("/sensor_queue",
                  EOS_O_CREAT | EOS_O_RDWR, 0, &attr);

/* Send */
mq_send(mq, "temperature=23.5", 17, 0);

/* Receive */
char buf[64];
unsigned int prio;
ssize_t n = mq_receive(mq, buf, sizeof(buf), &prio);

mq_close(mq);
mq_unlink("/sensor_queue");
}

```

19.3.2 15.4.2 Pipes

```

void pipe_example(void)
{
    int pipefd[2];
    pipe(pipefd); /* pipefd[0] = read, pipefd[1] = write */

    write(pipefd[1], "data", 4);

    char buf[8];
    read(pipefd[0], buf, sizeof(buf));
}

```

19.3.3 15.4.3 Shared Memory

```

void shm_example(void)
{
    int fd = shm_open("/shared_data", EOS_O_CREAT | EOS_O_RDWR, 0);
    void *ptr = shm_mmap(fd, 4096);

    /* Use shared memory region */
    memcpy(ptr, "shared data", 12);

    shm_unlink("/shared_data");
}

```

Chapter 20

Part II: VxWorks Compatibility

20.1 15.5 VxWorks Task API

The VxWorks layer (vxworks_task.h) maps the classic VxWorks task management API onto EoS kernel tasks.

20.1.1 15.5.1 Task Spawning

```
#include <eos/vxworks_task.h>

int my_task(int arg1, int arg2, int arg3, int arg4, int arg5,
            int arg6, int arg7, int arg8, int arg9, int arg10)
{
    printf("VxWorks task running: arg1=%d\n", arg1);
    return 0;
}

void vxworks_task_example(void)
{
    TASK_ID tid = taskSpawn(
        "tMyTask",    /* name */
        100,          /* priority */
        VX_FP_TASK,   /* options */
        4096,         /* stack size */
        my_task,       /* entry point */
        42, 0, 0, 0, 0, 0, 0, 0, 0, 0 /* 10 args */
    );

    if (tid == TASK_ID_ERROR) {
        printf("Task spawn failed\n");
    }
}
```

20.1.2 15.5.2 Task Operations

```
/* Delay by ticks (10 ms per tick default) */
taskDelay(100); /* 1 second delay */

/* Get own task ID */
TASK_ID me = taskIdSelf();

/* Suspend / Resume */
taskSuspend(tid);
taskResume(tid);

/* Look up by name */
TASK_ID found = taskNameToId("tMyTask");

/* Lock / unlock preemption */
taskLock();
/* critical section */
taskUnlock();

/* Delete a task */
taskDelete(tid);
```

20.1.3 15.5.3 Mapping Table: VxWorks to EoS Kernel

VxWorks Function	EoS Kernel Equivalent
taskSpawn	eos_task_create
taskDelete	eos_task_delete
taskSuspend	eos_task_suspend
taskResume	eos_task_resume
taskDelay	eos_task_delay_ms
taskIdSelf	eos_task_current
taskLock	eos_irq_disable
taskUnlock	eos_irq_enable

20.2 15.6 VxWorks Semaphores

```
#include <eos/vxworks_sem.h>

void vxworks_sem_example(void)
{
    /* Binary semaphore (initially full) */
    SEM_ID bin = semBCreate(SEM_Q_PRIORITY, SEM_FULL);

    /* Mutex semaphore */
```

```

SEM_ID mtx = semMCreate(SEM_Q_PRIORITY | SEM_INVERSION_SAFE);

/* Counting semaphore (initial count = 5) */
SEM_ID cnt = semCCreate(SEM_Q_FIFO, 5);

/* Take with timeout (100 ticks = 1 second) */
STATUS rc = semTake(mtx, 100);
if (rc == OK) {
    /* critical section */
    semGive(mtx);
}

/* Take with no wait */
rc = semTake(bin, NO_WAIT);

/* Take blocking forever */
semTake(cnt, WAIT_FOREVER);
semGive(cnt);

/* Cleanup */
semDelete(bin);
semDelete(mtx);
semDelete(cnt);
}

```

20.2.1 15.6.1 Semaphore Types

Factory	Type	EoS Mapping	Use Case
semBCreate	Binary	eos_sem (max=1)	Event signaling
semMCreate	Mutex	eos_mutex	Mutual exclusion
semCCreate	Counting	eos_sem (max=N)	Resource pools

Chapter 21

Part III: Linux IPC Compatibility

21.1 15.7 AF_UNIX Socket Emulation

The Linux IPC layer (`linux_ipc.h`) emulates AF_UNIX sockets, eventfd, epoll, and pipes using EoS kernel queues and semaphores.

21.1.1 15.7.1 Socket Address

```
typedef struct {
    uint16_t sun_family;           /* EOS_AF_UNIX */
    char      sun_path[EOS_UNIX_SOCKET_NAME_MAX]; /* Socket name */
} eos_sockaddr_un_t;
```

21.1.2 15.7.2 Stream Socket Example

```
#include <eos/linux_ipc.h>

void unix_server(void)
{
    eos_linux_ipc_init();

    int fd = eos_unix_socket(EOS_AF_UNIX, EOS SOCK_STREAM, 0);

    eos_sockaddr_un_t addr = { .sun_family = EOS_AF_UNIX };
    strncpy(addr.sun_path, "/tmp/eos.sock", EOS_UNIX_SOCKET_NAME_MAX);

    eos_unix_bind(fd, &addr);
    eos_unix_listen(fd, EOS_UNIX_SOCKET_BACKLOG);

    eos_sockaddr_un_t client_addr;
    int client = eos_unix_accept(fd, &client_addr);

    char buf[256];
```



```

int n = eos_unix_recv(client, buf, sizeof(buf), 0);
eos_unix_send(client, "ACK", 3, 0);

eos_unix_close(client);
eos_unix_close(fd);
}

```

21.2 15.8 eventfd Emulation

```

void eventfd_example(void)
{
    int efd = eos_eventfd_create(0, 0);

    /* Signal from ISR or another task */
    eos_eventfd_write(efd, 1);

    /* Wait for signal */
    uint64_t val;
    eos_eventfd_read(efd, &val);
    printf("Event received: %llu\n", val);

    eos_eventfd_close(efd);
}

```

Flag	Description
EOS_EFD_NONBLOCK	Non-blocking read/write
EOS_EFD_SEMAPHORE	Semaphore mode (decrement by 1)

21.3 15.9 epoll Emulation

```

void epoll_example(void)
{
    int epfd = eos_epoll_create(10);

    int sock_fd = eos_unix_socket(EOS_AF_UNIX, EOS SOCK_STREAM, 0);

    eos_epoll_event_t ev = {
        .events = EOS_EPOLLIN,
        .data.fd = sock_fd,
    };
    eos_epoll_ctl(epfd, EOS_EPOLL_CTL_ADD, sock_fd, &ev);

    /* Wait for events */
    eos_epoll_event_t events[4];
}

```

```

int n = eos_epoll_wait(epfd, events, 4, 5000);

for (int i = 0; i < n; i++) {
    if (events[i].events & EOS_EPOLLIN) {
        printf("Data ready on fd %d\n", events[i].data.fd);
    }
}

eos_epoll_close(epfd);
}

```

21.3.1 15.9.1 epoll Events

Event	Description
EOS_EPOLLIN	Ready for reading
EOS_EPOLLOUT	Ready for writing
EOS_EPOLLERR	Error condition
EOS_EPOLLHUP	Hang up
EOS_EPOLLET	Edge-triggered mode

21.4 15.10 Pipe Emulation

```

void linux_pipe_example(void)
{
    int pipefd[2];
    eos_pipe(pipefd);

    /* Non-blocking pipe */
    int nb_pipefd[2];
    eos_pipe2(nb_pipefd, EOS_O_NONBLOCK);

    /* Write end to Read end */
    eos_unix_send(pipefd[1], "hello", 5, 0);

    char buf[16];
    eos_unix_recv(pipefd[0], buf, sizeof(buf), 0);
}

```

21.5 15.11 Configuration Constants

Constant	Default	Description
EOS_LINUX_IPC_MAX_FDS	32	Max file descriptors
EOS_UNIX_SOCKET_NAME_MAX	64	Max socket path length
EOS_UNIX_SOCKET_BACKLOG	4	Default listen backlog

Constant	Default	Description
EOS_UNIX_SOCKET_MSG_SIZE	256	Max message size
EOS_UNIX_SOCKET_QUEUE_DEPTH	16	Internal queue depth
EOS_EPOLL_MAX_EVENTS	16	Max epoll events
EOS_PIPE_QUEUE_DEPTH	64	Pipe internal queue depth

21.6 15.12 Porting Strategy

```

+-----+
| Existing Application |
| (POSIX / VxWorks / Linux) |
+-----+
|
+-----v-----+
| Step 1: Include EoS compat |
| headers instead of OS ones |
+-----+
|
+-----v-----+
| Step 2: Link compat layer |
| (posix / vxworks / linux) |
+-----+
|
+-----v-----+
| Step 3: Compile and test |
| -- most code works as-is |
+-----+

```

21.7 15.13 API Reference Summary

21.7.1 POSIX APIs

Function	Header	Description
pthread_create	posix_threads.h	Create thread
pthread_join	posix_threads.h	Wait for thread
pthread_exit	posix_threads.h	Exit current thread
pthread_self	posix_threads.h	Get thread ID
pthread_yield	posix_threads.h	Yield CPU
open	posix_io.h	Open file
close	posix_io.h	Close file descriptor
read	posix_io.h	Read from fd
write	posix_io.h	Write to fd
lseek	posix_io.h	Seek in file
stat / fstat	posix_io.h	File information
mq_open	posix_ipc.h	Open message queue
mq_send	posix_ipc.h	Send message

Function	Header	Description
mq_receive	posix_ipc.h	Receive message
pipe	posix_ipc.h	Create pipe
shm_open	posix_ipc.h	Open shared memory

21.7.2 VxWorks APIs

Function	Header	Description
taskSpawn	vxworks_task.h	Create task
taskDelete	vxworks_task.h	Delete task
taskSuspend	vxworks_task.h	Suspend task
taskResume	vxworks_task.h	Resume task
taskDelay	vxworks_task.h	Delay in ticks
taskIdSelf	vxworks_task.h	Get current task ID
semBCreate	vxworks_sem.h	Create binary semaphore
semMCreate	vxworks_sem.h	Create mutex semaphore
semCCreate	vxworks_sem.h	Create counting semaphore
semTake	vxworks_sem.h	Acquire semaphore
semGive	vxworks_sem.h	Release semaphore
semDelete	vxworks_sem.h	Delete semaphore

21.7.3 Linux IPC APIs

Function	Header	Description
eos_unix_socket	linux_ipc.h	Create AF_UNIX socket
eos_unix_bind	linux_ipc.h	Bind socket to path
eos_unix_listen	linux_ipc.h	Listen for connections
eos_unix_accept	linux_ipc.h	Accept connection
eos_unix_connect	linux_ipc.h	Connect to socket
eos_unix_send	linux_ipc.h	Send data
eos_unix_recv	linux_ipc.h	Receive data
eos_unix_close	linux_ipc.h	Close socket
eos_eventfd_create	linux_ipc.h	Create eventfd
eos_eventfd_write	linux_ipc.h	Signal eventfd
eos_eventfd_read	linux_ipc.h	Wait on eventfd
eos_epoll_create	linux_ipc.h	Create epoll instance
eos_epoll_ctl	linux_ipc.h	Add/modify/delete fd
eos_epoll_wait	linux_ipc.h	Wait for events
eos_pipe	linux_ipc.h	Create pipe
eos_pipe2	linux_ipc.h	Create pipe with flags
eos_linux_ipc_init	linux_ipc.h	Init IPC subsystem
eos_linux_ipc_deinit	linux_ipc.h	Shutdown IPC subsystem

Next: [Chapter 16 – UI Framework](#)

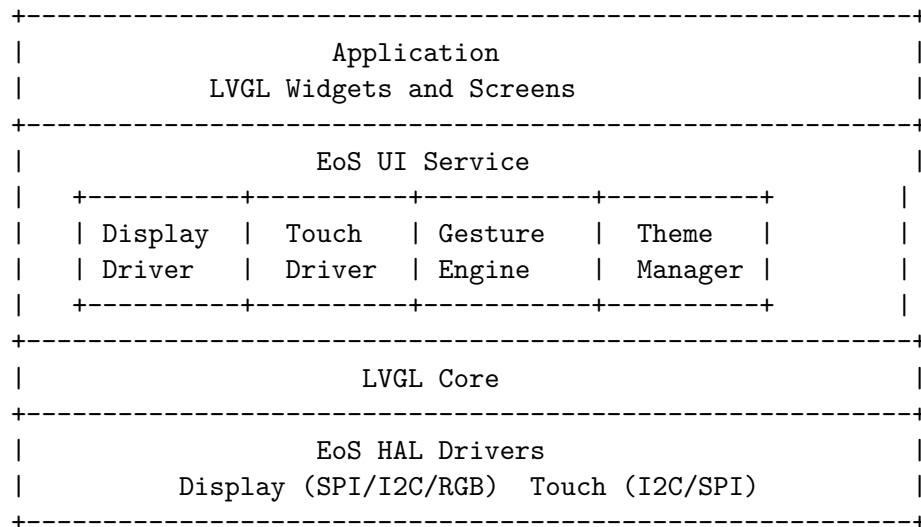
Chapter 22

Chapter 16: UI Framework

Author: Srikanth Patchava & EmbeddedOS Contributors

22.1 16.1 Overview

The EoS UI service (eos/ui.h) provides an integration layer between the LVGL graphics library and EoS HAL display/touch drivers. It handles framebuffer management, input device routing, gesture recognition, and theme switching – all gated by the `EOS_ENABLE_UI` configuration flag.



22.2 16.2 Configuration

22.2.1 16.2.1 Framebuffer Modes

Mode	Enum	Description
Full	<code>EOS_UI_FB_FULL</code>	Full-screen buffer (one or two)
Partial	<code>EOS_UI_FB_PARTIAL</code>	Band/stripe rendering

Mode	Enum	Description
Direct	EOS_UI_FB_DIRECT	No intermediate buffer

22.2.2 16.2.2 Themes

Theme	Enum	Description
Light	EOS_UI_THEME_LIGHT	Default light color scheme
Dark	EOS_UI_THEME_DARK	Dark mode color scheme

22.2.3 16.2.3 Display Rotation

Rotation	Enum	Degrees
Normal	EOS_UI_ROTATE_0	0 degrees
Landscape	EOS_UI_ROTATE_90	90 degrees CW
Inverted	EOS_UI_ROTATE_180	180 degrees
Landscape Inv.	EOS_UI_ROTATE_270	270 degrees CW

22.2.4 16.2.4 Configuration Structure

```
typedef struct {
    uint16_t    screen_width;    /* Display width in px */
    uint16_t    screen_height;   /* Display height in px */
    uint8_t     color_depth;     /* 1, 16, or 24 bits */
    uint8_t     display_id;      /* HAL display device ID */
    uint8_t     touch_id;        /* HAL touch device ID */
    eos_ui_fb_mode_t fb_mode;     /* Framebuffer strategy */
    eos_ui_rotation_t rotation;   /* Screen rotation */
    uint16_t    dpi;             /* Dots per inch (0=auto) */
    bool        double_buffer;   /* Use two framebuffers? */
} eos_ui_config_t;
```

22.3 16.3 Initialization

22.3.1 16.3.1 Basic Setup

```
#include <eos/ui.h>

void ui_setup(void)
{
    eos_ui_config_t cfg = {
        .screen_width   = 320,
        .screen_height  = 240,
        .color_depth    = 16,    /* RGB565 */
    }
```

```

        .display_id      = 0,          /* Primary display      */
        .touch_id        = 0,          /* Primary touch panel  */
        .fb_mode         = EOS_UI_FB_FULL,
        .rotation        = EOS_UI_ROTATE_0,
        .dpi              = 0,          /* LVGL default        */
        .double_buffer    = true,       /* Smooth rendering     */
    };

    int rc = eos_ui_init(&cfg);
    if (rc != 0) {
        printf("UI init failed: %d\n", rc);
        return;
    }

    /* Set dark theme */
    eos_ui_set_theme(EOS_UI_THEME_DARK);
}

```

22.3.2 16.3.2 Main Loop Integration

```

void ui_main_loop(void)
{
    while (1) {
        uint32_t sleep_ms = eos_ui_task_handler();
        eos_task_delay_ms(sleep_ms);
    }
}

```

22.3.3 16.3.3 RTOS Task Integration

```

void ui_task(void *arg)
{
    eos_ui_config_t cfg = { /* ... */ };
    eos_ui_init(&cfg);

    /* Create LVGL UI elements here */
    create_main_screen();

    while (1) {
        uint32_t sleep_ms = eos_ui_task_handler();
        eos_task_delay_ms(sleep_ms > 0 ? sleep_ms : 5);
    }
}

```


22.4 16.4 Display and Input Accessors

After initialization, you can access the underlying LVGL objects:

```
/* Get LVGL display object for advanced configuration */
lv_display_t *disp = eos_ui_get_display();

/* Get LVGL input device for cursor configuration */
lv_indev_t *indev = eos_ui_get_indev();
```

22.5 16.5 Theme Switching

```
/* Switch to dark mode at runtime */
eos_ui_set_theme(EOS_UI_THEME_DARK);

/* Switch back to light mode */
eos_ui_set_theme(EOS_UI_THEME_LIGHT);
```

22.6 16.6 Gesture Recognition

The gesture engine processes raw touch events and recognizes common gestures. It supports nine gesture types as a bitmask:

22.6.1 16.6.1 Gesture Types

Gesture	Bitmask	Description
EOS_GESTURE_NONE	0x000	No gesture
EOS_GESTURE_TAP	0x001	Single tap
EOS_GESTURE_DOUBLE_TAP	0x002	Double tap
EOS_GESTURE_LONG_PRESS	0x004	Long press and hold
EOS_GESTURE_SWIPE_LEFT	0x008	Swipe left
EOS_GESTURE_SWIPE_RIGHT	0x010	Swipe right
EOS_GESTURE_SWIPE_UP	0x020	Swipe up
EOS_GESTURE_SWIPE_DOWN	0x040	Swipe down
EOS_GESTURE_PINCH_IN	0x080	Pinch inward (zoom out)
EOS_GESTURE_PINCH_OUT	0x100	Pinch outward (zoom in)

22.6.2 16.6.2 Gesture Event Structure

```
typedef struct {
    eos_gesture_type_t type;           /* Which gesture detected */
    int16_t start_x;                  /* Touch start X coordinate */
    int16_t start_y;                  /* Touch start Y coordinate */
    int16_t end_x;                    /* Touch end X coordinate */
    int16_t end_y;                    /* Touch end Y coordinate */
}
```

```

        uint32_t          duration_ms; /* Gesture duration          */
    } eos_gesture_event_t;

```

22.6.3 16.6.3 Registering Gesture Callbacks

```

void on_swipe(const eos_gesture_event_t *event, void *ctx)
{
    switch (event->type) {
    case EOS_GESTURE_SWIPE_LEFT:
        printf("Swipe left: (%d,%d)->(%d,%d) %ums\n",
            event->start_x, event->start_y,
            event->end_x, event->end_y,
            event->duration_ms);
        navigate_next_screen();
        break;
    case EOS_GESTURE_SWIPE_RIGHT:
        navigate_prev_screen();
        break;
    default:
        break;
    }
}

void on_tap(const eos_gesture_event_t *event, void *ctx)
{
    if (event->type == EOS_GESTURE_LONG_PRESS) {
        show_context_menu(event->start_x, event->start_y);
    }
}

void setup_gestures(void)
{
    /* Register swipe callbacks */
    eos_ui_on_gesture(
        EOS_GESTURE_SWIPE_LEFT | EOS_GESTURE_SWIPE_RIGHT,
        on_swipe, NULL
    );

    /* Register tap and long-press */
    eos_ui_on_gesture(
        EOS_GESTURE_TAP | EOS_GESTURE_LONG_PRESS,
        on_tap, NULL
    );
}

```

22.6.4 16.6.4 Gesture Configuration

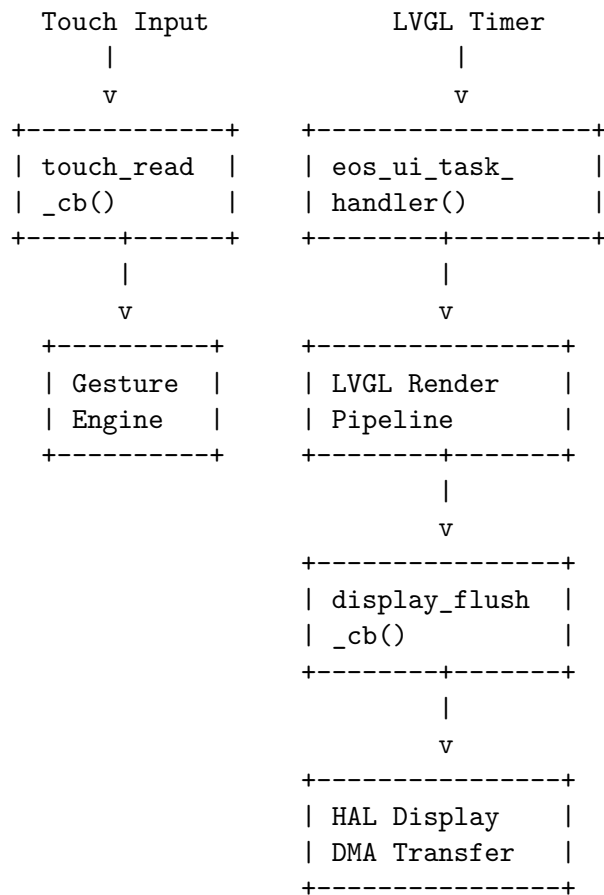
Fine-tune gesture detection thresholds:

```
typedef struct {
    uint16_t swipe_min_distance;    /* px: min drag for swipe */
    uint32_t long_press_time_ms;    /* ms: hold time for long-press */
    uint32_t double_tap_time_ms;    /* ms: max interval for double */
    uint16_t pinch_ratio_pct;       /* %: distance change for pinch */
} eos_gesture_config_t;

eos_gesture_config_t gcfg = {
    .swipe_min_distance = 50,        /* 50 px minimum swipe */
    .long_press_time_ms = 800,       /* 800 ms long press */
    .double_tap_time_ms = 300,       /* 300 ms double-tap gap */
    .pinch_ratio_pct = 20,           /* 20% distance change */
};

eos_ui_configure_gestures(&gcfg);
```

22.7 16.7 Display Rendering Pipeline



22.8 16.8 Internal Driver Callbacks

These functions are called internally by the display and touch drivers. Application code should not call them directly:

```
/* Called by LVGL when a display area needs flushing */
void eos_ui_display_flush_cb(lv_display_t *disp,
                             const void *area,
                             uint8_t *px_map);

/* Called by LVGL to poll touch input */
void eos_ui_touch_read_cb(lv_indev_t *indev, void *data);
```

22.9 16.9 LVGL Widget Examples

Once the EoS UI layer is initialized, use standard LVGL APIs to create widgets. The EoS layer manages the display driver – you focus on UI:

22.9.1 16.9.1 Button with Label

```
void create_button(void)
{
    lv_obj_t *btn = lv_button_create(lv_screen_active());
    lv_obj_set_size(btn, 120, 50);
    lv_obj_center(btn);

    lv_obj_t *label = lv_label_create(btn);
    lv_label_set_text(label, "Press Me");
    lv_obj_center(label);

    lv_obj_add_event_cb(btn, btn_event_handler,
                        LV_EVENT_CLICKED, NULL);
}

void btn_event_handler(lv_event_t *e)
{
    printf("Button clicked!\n");
}
```

22.9.2 16.9.2 Sensor Dashboard

```
void create_dashboard(void)
{
    /* Temperature gauge */
    lv_obj_t *meter = lv_meter_create(lv_screen_active());
    lv_obj_set_size(meter, 200, 200);
    lv_obj_center(meter);
```

```

lv_meter_scale_t *scale = lv_meter_add_scale(meter);
lv_meter_set_scale_range(meter, scale, 0, 100, 270, 135);

lv_meter_indicator_t *indic =
    lv_meter_add_needle_line(meter, scale, 4,
                             lv_palette_main(LV_PALETTE_RED), -10);

/* Update periodically */
lv_meter_set_indicator_value(meter, indic, 42);
}

```

22.10 16.10 Memory Considerations

Configuration	RAM Usage (320x240 16-bit)
Single buffer, full	~150 KB (1 framebuffer)
Double buffer, full	~300 KB (2 framebuffers)
Partial (10 lines)	~6.4 KB per buffer
Direct mode	0 KB (renders to display RAM)

22.10.1 16.10.1 Choosing Framebuffer Mode

Available RAM?

```

      |
+-----+-----+
|         |         |
|<64KB   |>256KB   |
|         |         |
v         v
Partial   Full + Double Buffer
or Direct (smoothest)

```

22.11 16.11 Complete Application Example

```

#include <eos/ui.h>
#include <eos/sensor.h>
#include <lvgl.h>

static lv_obj_t *temp_label;
static lv_obj_t *hum_label;

void create_main_screen(void)
{
    lv_obj_t *scr = lv_screen_active();

```

```

    /* Title */
    lv_obj_t *title = lv_label_create(scr);
    lv_label_set_text(title, "EoS Sensor Monitor");
    lv_obj_align(title, LV_ALIGN_TOP_MID, 0, 10);

    /* Temperature display */
    temp_label = lv_label_create(scr);
    lv_label_set_text(temp_label, "Temp: --.- C");
    lv_obj_align(temp_label, LV_ALIGN_CENTER, 0, -20);

    /* Humidity display */
    hum_label = lv_label_create(scr);
    lv_label_set_text(hum_label, "Hum: --.-%");
    lv_obj_align(hum_label, LV_ALIGN_CENTER, 0, 20);
}

void update_display(float temp, float hum)
{
    char buf[32];
    snprintf(buf, sizeof(buf), "Temp: %.1f C", temp);
    lv_label_set_text(temp_label, buf);

    snprintf(buf, sizeof(buf), "Hum: %.1f%%", hum);
    lv_label_set_text(hum_label, buf);
}

void app_main(void)
{
    /* Initialize UI */
    eos_ui_config_t cfg = {
        .screen_width  = 320,
        .screen_height = 240,
        .color_depth   = 16,
        .display_id    = 0,
        .touch_id       = 0,
        .fb_mode        = EOS_UI_FB_FULL,
        .rotation       = EOS_UI_ROTATE_0,
        .double_buffer  = true,
    };
    eos_ui_init(&cfg);
    eos_ui_set_theme(EOS_UI_THEME_DARK);

    /* Set up gesture navigation */
    eos_ui_on_gesture(
        EOS_GESTURE_SWIPE_LEFT | EOS_GESTURE_SWIPE_RIGHT,
        on_swipe, NULL
    );
}

```

```

    /* Create UI */
    create_main_screen();

    /* Main loop */
    while (1) {
        eos_sensor_reading_t temp, hum;
        eos_sensor_read_filtered(0, &temp);
        eos_sensor_read_filtered(1, &hum);
        update_display(temp.value, hum.value);

        eos_ui_task_handler();
        eos_task_delay_ms(33);    /* ~30 FPS */
    }
}

```

22.12 16.12 API Reference Summary

Function	Description
<code>eos_ui_init</code>	Initialize LVGL + display + touch
<code>eos_ui_deinit</code>	Tear down UI layer
<code>eos_ui_task_handler</code>	Run one LVGL timer iteration
<code>eos_ui_get_display</code>	Get LVGL display object
<code>eos_ui_get_indev</code>	Get LVGL input device object
<code>eos_ui_set_theme</code>	Switch light/dark theme
<code>eos_ui_on_gesture</code>	Register gesture callback
<code>eos_ui_configure_gestures</code>	Set gesture thresholds
<code>eos_ui_display_flush_cb</code>	Internal: display flush callback
<code>eos_ui_touch_read_cb</code>	Internal: touch read callback

End of Part III: Services

Chapter 23

Chapter 17: eBoot — The EoS Bootloader

Author: Srikanth Patchava & EmbeddedOS Contributors

23.1 17.1 Introduction

Every embedded system begins its life in the bootloader. Before the RTOS kernel initializes, before drivers enumerate peripherals, and before applications run, the bootloader establishes the foundation of trust and control that makes everything else possible.

eBoot is the production-grade boot platform for EoS. It supports **83 board ports** across **73 architecture families**, with clean separation between boot logic, hardware abstraction, and firmware management. eBoot handles everything from the first instruction after reset to handing control to the operating system — including secure boot verification, firmware updates, and automatic rollback on failure.

23.2 17.2 Architecture Overview

eBoot uses a two-stage boot architecture:

Reset Vector

Stage-0	Stage-1	EoS Kernel
Minimal	Full boot	Application
HW init	logic	Runtime

23.2.1 Stage-0: Minimal Hardware Initialization

Stage-0 runs directly from the reset vector. Its responsibilities are minimal and architecture-specific:

- CPU clock configuration
- Stack pointer initialization
- Minimal memory controller setup (if external RAM is needed)
- Verification and jump to Stage-1

Stage-0 is intentionally small (typically under 4 KB) to minimize the attack surface and to fit in on-chip ROM or protected flash regions.

23.2.2 Stage-1: Full Boot Logic

Stage-1 contains the bulk of eBoot functionality:

- A/B slot management and boot policy evaluation
- Secure boot chain verification (SHA-256, CRC-32, Ed25519 stubs)
- Firmware update pipeline (stream-based)
- Boot configuration storage (in flash or EEPROM)
- Handoff to the operating system

23.3 17.3 Key Features

Category	Capabilities
Boot Management	Staged boot (stage-0 + stage-1), A/B slots with automatic rollback, boot policy engine
Secure Boot	Self-contained SHA-256, CRC-32, Ed25519 signature stubs, anti-rollback counters
Firmware Update	Stream-based update pipeline, delta updates, integrity verification
Board Support	83 board ports across ARM Cortex-M/A/R, RISC-V, Xtensa, x86, and more
Integration	Seamless integration with EoS kernel and ebuild build system
Diagnostics	Boot log, failure counters, watchdog integration

23.4 17.4 Building eBoot

23.4.1 Native Build (Host Testing)

```
git clone https://github.com/embeddedos-org/eboot.git
cd eboot
```

```
cmake -B build -DEBLDR_BUILD_TESTS=ON
cmake --build build
```

```
cd build && ctest
```

23.4.2 Cross-Compilation for STM32F4

```
cmake -B build-arm -DEBLDR_BOARD=stm32f4 \  
  -DCMAKE_TOOLCHAIN_FILE=toolchains/arm-none-eabi.cmake  
cmake --build build-arm
```

23.4.3 Flashing with eFlash

eBoot includes a unified flashing tool written in Python:

```
python tools/eflash.py flash \  
  --board stm32f4 \  
  --image build-arm/eboot_firmware.bin \  
  --verify --reset
```

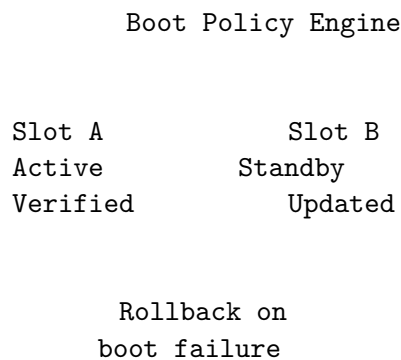
Alternatively, use the CMake flash target:

```
cmake --build build-arm --target flash
```

23.5 17.5 A/B Slot Management

eBoot implements A/B (dual-slot) firmware management, a pattern proven in Android and ChromeOS. The system maintains two complete firmware images and tracks which slot is active.

23.5.1 Slot State Machine



23.5.2 Boot Policy Rules

1. **Try active slot first** — if verification passes, boot it
2. **Fall back to standby** — if active fails integrity or crash counter exceeds threshold
3. **Automatic rollback** — if a newly updated slot fails three consecutive boots, revert
4. **Anti-rollback** — monotonic counter prevents downgrade attacks

23.5.3 Configuration Storage

Boot metadata is stored in a reserved flash sector:

```
typedef struct {
    uint8_t active_slot;           // 0 = A, 1 = B
    uint8_t boot_attempts[2];     // per-slot failure counters
    uint32_t rollback_counter;    // anti-rollback monotonic counter
    uint32_t crc32;               // metadata integrity check
} eboot_metadata_t;
```

23.6 17.6 Secure Boot Chain

eBoot implements a progressive trust chain:

1. **Stage-0 verifies Stage-1** — CRC-32 integrity check (fast, minimal code)
2. **Stage-1 verifies firmware** — SHA-256 hash comparison against signed manifest
3. **Ed25519 signature stubs** — pluggable public-key verification for production deployments

23.6.1 Hash Verification Example

```
#include "eboot/crypto.h"

int verify_firmware(const uint8_t *image, size_t len,
                   const uint8_t expected_hash[32]) {
    uint8_t computed[32];
    eboot_sha256(image, len, computed);
    return eboot_secure_compare(computed, expected_hash, 32);
}
```

The cryptographic primitives are self-contained — eBoot does not depend on external libraries. This is critical for the boot environment where the OS and its libraries are not yet available.

23.7 17.7 Firmware Update Pipeline

eBoot supports stream-based firmware updates, enabling over-the-air (OTA) updates even on memory-constrained devices.

23.7.1 Update Flow

Download	Verify	Write to	Update
stream	integrity	standby	metadata
(chunks)	per-chunk	slot	& reboot

Key design decisions:

- **Chunk-based processing** — firmware is received and verified in chunks, never requiring the full image in RAM
 - **Power-fail safe** — interrupted updates leave the active slot untouched
 - **Verify-before-commit** — the standby slot is fully written and verified before updating the boot metadata
-

23.8 17.8 Board Port Structure

Each board port provides a minimal set of functions that eBoot needs:

```
boards/
  stm32f4/
    board.c      # Clock, GPIO, flash driver
    board.h      # Pin definitions, memory map
    linker.ld    # Linker script
  nrf52840/
    board.c
    board.h
    linker.ld
  rpi4/
    board.c
    board.h
    linker.ld
  ...           # 80+ more board ports
```

A board port must implement the `eboot_board_ops_t` interface:

```
typedef struct {
    int  (*init)(void);
    void (*deinit)(void);
    int  (*flash_read)(uint32_t addr, uint8_t *buf, size_t len);
    int  (*flash_write)(uint32_t addr, const uint8_t *buf, size_t len);
    int  (*flash_erase)(uint32_t addr, size_t len);
    void (*reset)(void);
    void (*uart_putc)(char c);
} eboot_board_ops_t;
```

23.9 17.9 Integration with EoS and ebuild

eBoot is designed to work as part of the EoS toolchain:

ebuild eBoot (bootloader) EoS (kernel + apps)

build flash

configs binary firmware

When using `ebuild`, the bootloader is automatically included in the firmware image. The build system handles:

- Selecting the correct eBoot board port based on the target hardware
 - Linking the bootloader and application firmware into a single flashable image
 - Generating the boot metadata with correct slot information
-

23.10 17.10 Debugging and Diagnostics

23.10.1 Boot Log

eBoot outputs diagnostic information over UART during boot:

```
[eBoot v1.0] Stage-0 init OK
[eBoot] Flash: 1MB @ 0x08000000
[eBoot] Slot A: valid (SHA-256 OK), attempts=0
[eBoot] Slot B: valid (SHA-256 OK), attempts=0
[eBoot] Policy: boot Slot A (active)
[eBoot] Jumping to 0x08040000...
```

23.10.2 Failure Recovery

eBoot maintains boot attempt counters per slot. If a slot fails to boot successfully three times in a row (detected via watchdog timeout or explicit failure flag), eBoot automatically falls back to the other slot:

```
[eBoot] Slot A: boot failed (attempt 3/3)
[eBoot] Rollback: switching active to Slot B
[eBoot] Jumping to 0x08080000...
```

23.11 17.11 Testing

eBoot includes comprehensive tests that run on the host machine:

```
cmake -B build -DEBLDR_BUILD_TESTS=ON
cmake --build build
cd build && ctest --output-on-failure
```

Test categories include:

Category	Coverage
Crypto	SHA-256, CRC-32, secure compare
Slot management	A/B switching, rollback, anti-rollback
Flash driver	Read, write, erase, boundary conditions
Update pipeline	Chunked update, power-fail simulation

Category	Coverage
Boot policy	Policy evaluation, failure counters

23.12 17.12 Summary

eBoot provides the critical foundation layer for EoS-based systems. Its two-stage architecture, A/B slot management, and secure boot chain ensure that devices boot reliably and securely — even in the face of failed updates or compromised firmware. With 83 board ports and seamless integration with ebuild, eBoot scales from tiny microcontrollers to application processors.

Key takeaways:

- Two-stage boot architecture minimizes the trusted code base
- A/B slots with automatic rollback ensure update safety
- Stream-based firmware updates work on memory-constrained devices
- Self-contained crypto — no external dependencies in the boot path
- 83 board ports across 73 architecture families

Next: Chapter 18 — ebuild Build System

Chapter 24

Chapter 18: ebuild — The EoS Build System

Author: Srikanth Patchava & EmbeddedOS Contributors

24.1 18.1 Introduction

Building firmware for embedded systems has traditionally been a fragmented, error-prone process. Each target architecture requires its own toolchain, linker scripts, and build configuration. Multiply this by dozens of supported boards and the complexity becomes overwhelming.

ebuild solves this with a single command: from hardware schematic to deployable firmware. It reads KiCad/Eagle schematics or board descriptions, auto-generates all build configurations, and compiles firmware for 73+ embedded targets.

Note: ebuild is the EmbeddedOS Build Tool — not related to Gentoo’s ebuild package format.

24.2 18.2 Architecture

ebuild operates as a pipeline that transforms hardware descriptions into firmware binaries. The pipeline has four stages:

1. **Hardware Description** — KiCad schematic, YAML, CSV, or plain text input
2. **Analyze & Parse** — extract and classify components
3. **Generate Configs** — produce 9 configuration files
4. **Build Firmware** — compile the final binary

24.2.1 Supported Input Formats

Format	Description	Parser
.kicad_sch	KiCad schematic (S-expression format)	Real parser
.yaml	Board description file	YAML parser
.csv	Bill of Materials (Reference, Value columns)	CSV parser
.txt	Plain text description	NLP parser
.eagle	Eagle schematic	XML parser

24.2.2 Generated Configuration Files

When ebuild analyzes a hardware description, it generates **9 configuration files** that fully describe the build:

1. eos_config.h — Feature flags and peripheral enables
2. hal_config.h — HAL peripheral configuration
3. board.h — Pin assignments and memory map
4. linker.ld — Linker script for the target MCU
5. CMakeLists.txt — Build system configuration
6. openocd.cfg — Debug probe configuration
7. flash.sh — Flashing script
8. product_profile.h — EoS product profile header
9. README.md — Auto-generated board documentation

24.3 18.3 Installation

```
# Linux / macOS
git clone https://github.com/embeddedos-org/ebuild.git
cd ebuild
./install.sh          # or: pip install -e .
ebuild --version      # ebuild, version 0.1.0

# Windows
git clone https://github.com/embeddedos-org/ebuild.git
cd ebuild
install.bat
```

24.4 18.4 Hands-On Examples

24.4.1 Hello World

```
cd examples/hello_world
ebuild info && ebuild build && ./_build/hello
# → "Hello from EoS Build System!"
```


24.4.2 Multi-Target Build

```
cd examples/multi_target
ebuild build && ./_build/myapp
# → add=17, subtract=7, multiply=60, factorial=120
```

24.4.3 Hardware Analysis

The most powerful feature of ebuild is automatic hardware analysis:

```
# From a KiCad schematic (real S-expression parser)
ebuild analyze \
  --file hardware/board/sample_stm32f4_sensor.kicad_sch \
  --output-dir /tmp/configs

# From a YAML board description
ebuild analyze \
  --file hardware/board/sample_iot_gateway.yaml \
  --output-dir /tmp/iot

# From a CSV BOM
ebuild analyze --file /tmp/bom.csv --output-dir /tmp/bom_out

# From plain text
ebuild analyze --file desc.txt --output-dir /tmp/text_out
```

24.5 18.5 The Analyze Pipeline

When ebuild processes a hardware description, it follows a multi-step pipeline:

24.5.1 Step 1: Component Extraction

The parser identifies components from the input format and extracts: - Part numbers (e.g., STM32F407VGT6, BME280, W25Q128) - Pin connections and nets - Power rails and decoupling

24.5.2 Step 2: Component Classification

Each component is classified into categories:

Category	Examples
MCU	STM32F407, nRF52840, ESP32
Sensor	BME280, MPU6050, MAX30102
Memory	W25Q128 (SPI Flash), IS42S16400J (SDRAM)
Communication	SX1276 (LoRa), ENC28J60 (Ethernet)
Power	TPS63020 (Buck-Boost), MCP1700 (LDO)
Display	SSD1306 (OLED), ILI9341 (TFT)

Category	Examples
Actuator	DRV8833 (Motor Driver), Servo

24.5.3 Step 3: Configuration Generation

Based on the classified components, ebuild generates all 9 configuration files with correct peripheral enables, pin assignments, memory layout, and appropriate EoS product profile selection.

24.6 18.6 Build System Configuration

ebuild uses a `ebuild.toml` project file:

```
[project]
name = "my-sensor-node"
version = "1.0.0"

[target]
board = "stm32f407_discovery"
profile = "iot"

[dependencies]
eos = { version = ">=0.5.0", features = ["hal", "rtos", "net"] }
eboot = { version = ">=1.0.0" }

[build]
optimization = "size"
lto = true
strip = true
```

24.6.1 Build Commands

Command	Description
<code>ebuild init</code>	Create a new project from template
<code>ebuild info</code>	Display project and toolchain info
<code>ebuild build</code>	Compile the project
<code>ebuild clean</code>	Remove build artifacts
<code>ebuild flash</code>	Flash firmware to target
<code>ebuild debug</code>	Start GDB debug session
<code>ebuild analyze</code>	Analyze hardware and generate configs
<code>ebuild test</code>	Run unit tests
<code>ebuild package</code>	Create distributable firmware package

24.7 18.7 Cross-Compilation Support

ebuild manages cross-compilation toolchains for all EoS-supported architectures:

Architecture	Toolchain	Targets
ARM Cortex-M	arm-none-eabi-gcc	STM32, nRF52, SAM, RP2040
ARM Cortex-A	aarch64-linux-gnu-gcc	RPi 4/5, Jetson, i.MX8
ARM Cortex-R	arm-none-eabi-gcc	TMS570, Cortex-R5/R52
RISC-V	riscv64-linux-gnu-gcc	SiFive, ESP32-C3, K210
Xtensa	xtensa-esp32-elf-gcc	ESP32, ESP32-S3
x86_64	gcc / clang	QEMU, native host

ebuild automatically selects the correct toolchain based on the target board and verifies that it is installed before building.

24.8 18.8 Integration with EoS Ecosystem

ebuild is the glue that binds the EoS ecosystem together. It combines hardware analysis, EoS kernel, eBoot bootloader, and application code into a single firmware binary.

24.8.1 Typical Workflow

```
# 1. Analyze your hardware
ebuild analyze --file my_board.kicad_sch --output-dir config/

# 2. Build EoS + eBoot + your application
ebuild build --target all

# 3. Flash the complete firmware
ebuild flash --verify --reset

# 4. Debug over SWD/JTAG
ebuild debug
```

24.9 18.9 SDK Generation

ebuild can generate a complete SDK package for distribution:

```
ebuild package --sdk --output my-board-sdk.tar.gz
```

The SDK includes:

- Pre-built EoS libraries for the target
- Header files for the HAL and kernel APIs

- Toolchain configuration files
- Example projects
- Documentation

This enables third-party developers to build applications for a specific board without needing the full EoS source tree.

24.10 18.10 Continuous Integration

`ebuild` integrates with CI/CD pipelines. A typical GitHub Actions workflow builds firmware across multiple boards in a matrix strategy, running `ebuild setup`, `ebuild build`, and `ebuild test` for each target board (e.g., `stm32f4`, `nrf52840`, `rpi4`, `esp32`).

24.11 18.11 Summary

`ebuild` transforms the traditionally painful process of embedded firmware building into a streamlined, automated pipeline. Its ability to read hardware schematics and auto-generate complete build configurations eliminates an entire class of manual, error-prone work.

Key takeaways:

- One command from hardware schematic to firmware binary
 - Supports 5 input formats (KiCad, YAML, CSV, text, Eagle)
 - Generates 9 configuration files automatically
 - Manages cross-compilation for 73+ targets
 - Integrates eBoot, EoS kernel, and applications into a single build
 - SDK generation for third-party distribution
-

Next: Chapter 19 — eIPC Inter-Process Communication

Chapter 25

Chapter 19: eIPC — Embedded Inter-Process Communication

Author: Srikanth Patchava & EmbeddedOS Contributors

25.1 19.1 Introduction

As embedded systems grow in complexity, the need for structured communication between software components becomes critical. The **eIPC** (Embedded Inter-Process Communication) framework provides secure, real-time IPC for the EoS ecosystem — specifically designed as the communication bridge between **eNI** (Neural Interface) and **eAI** (AI Layer).

ENI EIPC EAI

eIPC is a standalone, cross-platform, security-enhanced IPC framework written in pure Go with zero external dependencies.

25.2 19.2 Architecture

eIPC follows a layered architecture with clean separation of concerns:

Application		
core/ Message Router Endpoint	services/ Broker Registry Policy Audit Health	security/ Authenticator Capability HMAC Integrity ReplayTracker Keyring
protocol/		

Frame encoding, versioning, compat

transport/

TCP Unix Socket Named Pipe Shared Mem

25.2.1 Core Components

Component	Responsibility
Message	Typed message structure with headers and payload
Router	Routes messages between endpoints by topic/pattern
Endpoint	Named connection point for sending/receiving messages
Broker	Central message broker with pub/sub and request/reply

25.2.2 Security Components

Component	Responsibility
Authenticator	Peer authentication using shared secrets
Capability	Action-level authorization (safe/controlled/restricted)
HMAC Integrity	Message integrity verification using HMAC-SHA256
ReplayTracker	Prevents message replay attacks with nonces
Keyring	Secure key storage and rotation

25.3 19.3 Key Features

- **Real-time capable** — bounded queues, priority lanes, timeout-aware delivery
- **Security-enhanced** — peer auth, capability authorization, HMAC integrity, replay protection
- **Cross-platform** — Linux (amd64/arm64/armv7), macOS (amd64/arm64), Windows (amd64/arm64)
- **Pluggable transports** — TCP, Unix domain sockets, Windows named pipes, shared memory
- **Auditable** — JSON-line audit logging with full request tracing
- **Policy engine** — three-tier action classification
- **Zero external dependencies** — pure Go standard library
- **LTS-friendly** — versioned protocol with compatibility guarantees

25.4 19.4 Transport Layer

eIPC supports multiple transport backends, selected based on deployment requirements:

Transport	Use Case	Latency	Security
TCP	Cross-machine, networked	Medium	TLS-ready
Unix Socket	Same-machine, Linux/macOS	Low	File ACL
Named Pipe	Same-machine, Windows	Low	ACL
Shared Memory	Ultra-low-latency, local	Lowest	Process

Transport selection is transparent to application code — the same message API works across all transports.

25.5 19.5 The Policy Engine

eIPC classifies all actions into three tiers:

Tier 1: SAFE

Read-only queries, health checks, status

→ No authorization required

Tier 2: CONTROLLED

Configuration changes, model switching

→ Requires authenticated peer

Tier 3: RESTRICTED

System shutdown, security config changes

→ Requires admin capability + audit log

This classification is enforced at the broker level before message routing.

25.6 19.6 Server Implementation

The eIPC server (`cmd/eipc-server/main.go`) demonstrates the complete framework in action. It initializes all components and serves as the IPC bridge between eNI and eAI.

25.6.1 Initialization Flow

```
func main() {
    // 1. Load configuration
    cfg := config.Load()

    // 2. Initialize security services
    authenticator := auth.NewAuthenticator(cfg.Security)
    capabilities := capability.NewManager(cfg.Capabilities)
```

```

// 3. Initialize transport
listener := tcp.NewListener(cfg.ListenAddr)

// 4. Initialize services
broker := core.NewBroker()
registry := registry.NewRegistry()
auditor := audit.NewAuditor(cfg.AuditLog)
healthSvc := health.NewService()

// 5. Start serving
server := NewServer(listener, broker, authenticator, capabilities, auditor)
server.Serve(ctx)
}

```

25.6.2 EAI Backend Integration

The server acts as a proxy between eNI clients and the eAI backend. When a chat or completion request arrives via eIPC, the server forwards it to the local eAI HTTP endpoint:

```

const eaiBackendURL = "http://127.0.0.1:8090"

func forwardToEAI(ctx context.Context, endpoint string,
    payload interface{}) (io.ReadCloser, string, error) {
    body, _ := json.Marshal(payload)
    req, _ := http.NewRequestWithContext(ctx, http.MethodPost,
        eaiBackendURL+endpoint, bytes.NewReader(body))
    req.Header.Set("Content-Type", "application/json")
    resp, err := http.DefaultClient.Do(req)
    // ...
    return resp.Body, resp.Header.Get("Content-Type"), nil
}

```

25.6.3 Message Types

Endpoint	Direction	Description
/v1/chat	eNI → eAI	Conversational AI requests
/v1/complete	eNI → eAI	Text completion requests
/v1/health	Any → Server	Health check (safe tier)
/v1/status	Any → Server	System status and metrics

25.7 19.7 Protocol Format

eIPC uses a binary frame protocol for efficient wire encoding:

Magic	Version	Length	Headers	Payload
4 bytes	2 bytes	4 bytes	variable	variable

25.7.1 Protocol Versioning

The protocol includes a version field for forward compatibility. The server negotiates the highest mutually supported version during the handshake:

- **v1.0** — Initial protocol with basic message routing
- **v1.1** — Added HMAC integrity and replay protection
- **v1.2** — Added priority lanes and bounded queues

25.8 19.8 Security Model

25.8.1 Authentication

Peers authenticate using HMAC-based challenge-response:

1. Client connects and sends a hello message
2. Server responds with a random challenge nonce
3. Client computes HMAC-SHA256(challenge, shared_secret) and sends it
4. Server verifies and grants a session token

25.8.2 Authorization

After authentication, every message is checked against the capability manager:

```
type CapabilityManager interface {
    Check(peer PeerID, action Action) (bool, error)
    Grant(peer PeerID, capability Capability) error
    Revoke(peer PeerID, capability Capability) error
}
```

25.8.3 Audit Trail

All controlled and restricted actions are logged to a JSON-line audit file:

```
{"ts": "2026-04-15T10:30:00Z", "peer": "eni-001", "action": "model.switch",
  "tier": "controlled", "result": "allowed", "latency_ms": 2}
```

25.9 19.9 Health Monitoring

eIPC includes a built-in health service that tracks:

- **Transport health** — connection count, bytes transferred, errors
- **Broker health** — message queue depth, routing latency

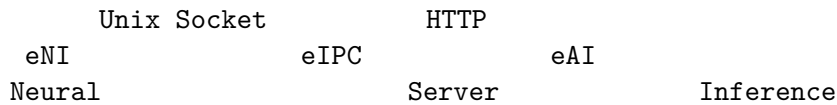
- **Peer health** — connected peers, authentication status
- **Backend health** — eAI endpoint availability and response times

The health endpoint is always classified as “safe” tier — no authentication required. This enables monitoring systems to check eIPC health without credentials.

25.10 19.10 Deployment Patterns

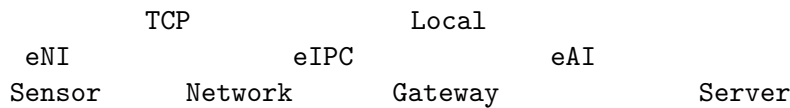
25.10.1 Pattern 1: Local Bridge (Most Common)

eNI and eAI run on the same device, communicating via Unix socket:



25.10.2 Pattern 2: Distributed

eNI runs on an edge sensor, eAI runs on a gateway:



25.11 19.11 Building and Testing

```

git clone https://github.com/embeddedos-org/eipc.git
cd eipc

# Build
go build ./...

# Run tests
go test ./... -v -race

# Run the server
go run cmd/eipc-server/main.go --config config.yaml

# Cross-compile for ARM
GOOS=linux GOARCH=arm64 go build -o eipc-server-arm64 cmd/eipc-server/main.go
  
```

25.12 19.12 Summary

eIPC provides the secure communication backbone for the EoS AI stack. By combining real-time message routing with layered security (authentication, authorization, integrity, and audit), eIPC ensures that neural interface data flows safely to the AI inference engine.

Key takeaways:

- Purpose-built IPC bridge between eNI and eAI
- Three-tier policy engine (safe / controlled / restricted)
- Four transport backends (TCP, Unix, Named Pipe, Shared Memory)
- HMAC-SHA256 integrity with replay protection
- Pure Go, zero external dependencies
- Cross-platform: Linux, macOS, Windows (amd64/arm64/armv7)

Next: Chapter 20 — eAI Embedded AI Layer

Chapter 26

Chapter 20: eAI — Embedded AI Layer

Author: Srikanth Patchava & EmbeddedOS Contributors

26.1 20.1 Introduction

Artificial intelligence is no longer confined to the cloud. Modern embedded systems — from autonomous vehicles to medical devices — demand on-device inference with zero cloud dependency, deterministic latency, and safety guarantees that cloud-based solutions cannot provide.

eAI is a C11 embedded AI framework that brings LLM inference, autonomous agents, and adaptive learning directly to resource-constrained devices. It runs entirely on-device with no cloud requirement, while supporting optional hybrid connectivity for model updates and federated learning.

26.2 20.2 Design Principles

Principle	Description
Offline-first	Full autonomy without cloud connectivity
Resource-aware	Quantized models, power-aware scheduling, memory-conscious
Security by default	8-layer defense architecture from boot to runtime
Two-tier design	Choose the right footprint for your hardware

26.3 20.3 Two-Tier Architecture

eAI ships as two distinct product tiers sharing a common foundation:

26.3.1 EAI-Min — Lightweight Runtime

For MCUs, SBCs, and battery-powered edge devices (Cortex-M7, RPi, nRF5340):

Module	Description
Runtime	Single-backend inference engine
Quantizer	INT4/INT8 model quantization
Scheduler	Power-aware task scheduling
HAL Bridge	Direct integration with EoS HAL

Typical footprint: 256 KB Flash, 64 KB RAM minimum.

26.3.2 EAI-Full — Complete AI Platform

For edge servers, gateways, and application processors (Jetson, i.MX8, x86):

Module	Description
Multi-backend	GGML, TensorRT, ONNX Runtime, custom engines
Agent Framework	Autonomous agent loop with tool calling
Memory System	Context management with vector store
RAG Pipeline	Retrieval-augmented generation
Fine-tuning	On-device model adaptation
Federation	Federated learning across device fleets

26.4 20.4 Inference Engine

The inference engine is the core of eAI. It supports multiple backends through a unified API:

```
#include "eai/inference.h"

// Load a quantized model
eai_model_t *model = eai_model_load("model.q4", EAI_BACKEND_GGML);

// Run inference
eai_result_t result;
eai_infer(model, "Analyze sensor reading: temp=72.3F", &result);

printf("Response: %s\n", result.text);
printf("Latency: %u ms\n", result.latency_ms);
printf("Tokens/s: %.1f\n", result.tokens_per_second);

eai_model_free(model);
```

26.4.1 Supported Backends

Backend	Precision	Hardware	Use Case
GGML	Q4, Q8, F16	CPU (any arch)	General-purpose
TensorRT	INT8, FP16	NVIDIA GPU/DLA	High-throughput
ONNX	FP32, INT8	CPU, NPU	Model portability
Custom	Configurable	FPGA, DSP	Specialized

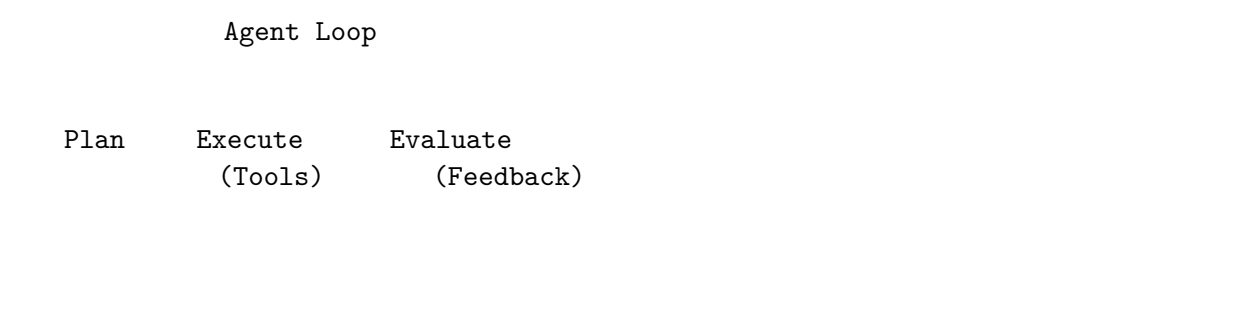
26.4.2 Quantization Pipeline

```
# Quantize a model for deployment
eai-quantize \
  --input model.gguf \
  --output model.q4 \
  --method q4_k_m \
  --calibration-data samples.jsonl
```

eAI supports progressive quantization: start with Q8 for accuracy, then move to Q4 for deployment on smaller devices.

26.5 20.5 Agent Framework

EAI-Full includes an autonomous agent framework for complex, multi-step tasks:



26.5.1 Tool Integration

Agents can call tools to interact with the physical world:

Tool Category	Examples
Sensor Reading	Temperature, humidity, accelerometer, GPS
Actuator Control	Motor, servo, relay, LED
Communication	MQTT publish, HTTP request, BLE broadcast
Data Processing	FFT, filtering, anomaly detection
System	Reboot, update, configure, log

26.5.2 Agent Configuration

```
# agent.yaml
name: "sensor-monitor"
model: "model.q4"
tools:
  - sensor_read
  - actuator_control
  - mqtt_publish
loop:
  max_iterations: 10
  timeout_seconds: 30
  eval_threshold: 0.8
memory:
  type: "sliding_window"
  context_tokens: 2048
```

26.6 20.6 Security Architecture

eAI implements an 8-layer defense-in-depth architecture:

Layer	Name	Protection
1	Secure Boot	Verified boot chain (via eBoot)
2	Model Integrity	SHA-256 verification before loading
3	Memory Isolation	Separate heaps for model and application
4	Input Validation	Prompt sanitization and length limits
5	Output Filtering	Response safety classification
6	Access Control	Capability-based API authorization
7	Audit Logging	All inference requests logged
8	Update Security	Signed model updates via OTA

26.7 20.7 Power-Aware Scheduling

On battery-powered devices, eAI manages inference scheduling to balance responsiveness with power consumption:

```
eai_power_config_t power_cfg = {
  .mode = EAI_POWER_ADAPTIVE,
  .battery_threshold_low = 20,      // percent
  .battery_threshold_critical = 5,
  .idle_timeout_ms = 5000,
  .max_inference_rate = 10,        // per minute
};
```

```
eai_power_configure(&power_cfg);
```

26.7.1 Power Modes

Mode	Behavior
Full	Unrestricted inference, maximum performance
Balanced	Throttled inference rate, medium power
Low	Only critical inferences, model stays loaded
Critical	Model unloaded, only cached responses served

26.8 20.8 Integration with EoS

eAI integrates directly with the EoS HAL for sensor access and actuator control:

```
#include "eai/hal_bridge.h"
#include "eos/hal.h"

// Read sensor data through EoS HAL
float temperature = eos_sensor_read(SENSOR_TEMPERATURE);

// Feed to eAI inference
char prompt[128];
snprintf(prompt, sizeof(prompt),
         "Temperature is %.1f°C. Is this within normal range?",
         temperature);

eai_result_t result;
eai_infer(model, prompt, &result);

// Act on the result through EoS HAL
if (result.classification == EAI_CLASS_ANOMALY) {
    eos_gpio_write(ALARM_PIN, true);
    eos_mqtt_publish("alerts/temperature", result.text);
}
```

26.9 20.9 Model Management

26.9.1 Model Formats

Format	Extension	Description
GGUF	.gguf	GGML universal format
EAI Q4	.q4	eAI quantized INT4

Format	Extension	Description
EAI Q8	.q8	eAI quantized INT8
ONNX	.onnx	Open Neural Network Exchange
TensorRT	.engine	NVIDIA TensorRT compiled

26.9.2 Over-the-Air Model Updates

```
eai_ota_config_t ota = {
    .url = "https://models.example.com/v2/sensor-monitor.q4",
    .verify_signature = true,
    .public_key = embedded_public_key,
    .max_download_size = 50 * 1024 * 1024,
    .apply_on_idle = true,
};
eai_ota_update(&ota);
```

26.10 20.10 Building eAI

```
git clone https://github.com/embeddedos-org/eai.git
cd eai

# Build EAI-Min (lightweight)
cmake -B build -DEAI_TIER=min
cmake --build build

# Build EAI-Full (complete)
cmake -B build -DEAI_TIER=full -DEAI_BACKEND_GGML=ON
cmake --build build

# Cross-compile for ARM
cmake -B build-arm \
    -DCMAKE_TOOLCHAIN_FILE=toolchains/aarch64-linux-gnu.cmake \
    -DEAI_TIER=full
cmake --build build-arm

# Run tests
ctest --test-dir build --output-on-failure
```

26.11 20.11 Compliance

eAI aligns with multiple safety and quality standards:

Standard	Relevance
ISO/IEC 25000	Software quality model
ISO/IEC 15288	Systems engineering lifecycle
ISO/IEC 20243	Supply chain integrity
IEC 61508	Functional safety (industrial)
ISO 26262	Functional safety (automotive)
DO-178C	Software assurance (aerospace)

26.12 20.12 Summary

eAI brings production-grade AI inference to embedded devices without cloud dependency. Its two-tier architecture (EAI-Min for MCUs, EAI-Full for edge servers) ensures the right AI footprint for every device class.

Key takeaways:

- C11 framework — runs on everything from Cortex-M7 to Jetson Orin
- Multiple inference backends (GGML, TensorRT, ONNX)
- Autonomous agent framework with tool calling
- 8-layer security architecture
- Power-aware scheduling for battery devices
- Zero cloud dependency with optional hybrid connectivity

Next: Chapter 21 — eNI Neural Interface

Chapter 27

Chapter 21: eNI — Embedded Neural Interface

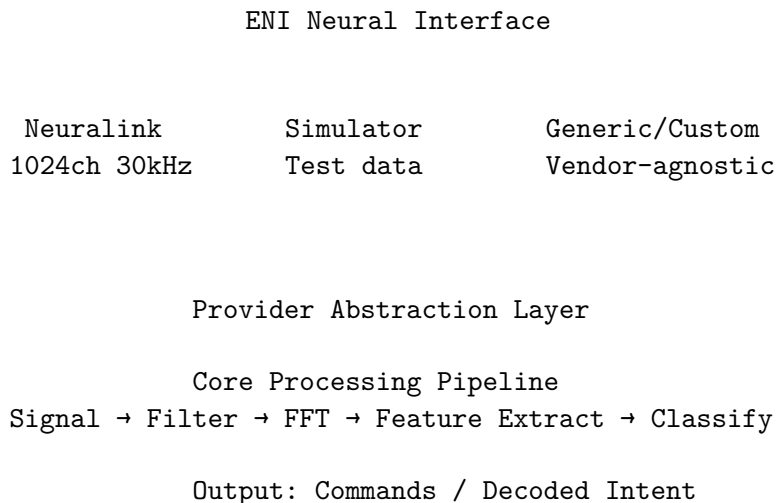
Author: Srikanth Patchava & EmbeddedOS Contributors

27.1 21.1 Introduction

Brain-computer interfaces (BCIs) represent the next frontier of human-machine interaction. The **eNI** (Embedded Neural Interface) provides a standardized, vendor-neutral framework to integrate BCIs, neural decoders, and assistive input systems into the EoS platform.

eNI bridges the gap between raw neural signals and actionable embedded system commands — enabling applications from medical prosthetics to direct neural control of drones and vehicles.

27.2 21.2 Architecture



27.2.1 Providers

eNI uses a provider abstraction to support multiple BCI hardware backends:

Provider	Channels	Sample Rate	Description
Neuralink	1024	30 kHz	Implanted neural threads
Simulator	64-1024	256 Hz	Synthetic test data generation
Generic	Variable	Variable	Vendor-agnostic BCI devices

27.3 21.3 Core Processing Pipeline

The signal processing pipeline transforms raw neural data into actionable commands through four stages.

27.3.1 Stage 1: Signal Acquisition

Raw samples are read from the provider at the hardware sample rate. eNI supports up to ENI_MAX_CHANNELS (1024) simultaneous channels.

27.3.2 Stage 2: Digital Signal Processing

The DSP module applies configurable filter chains:

- **Bandpass filtering** — isolate EEG frequency bands
- **Notch filtering** — remove power-line interference (50/60 Hz)
- **FFT analysis** — frequency domain decomposition (up to 512-point FFT)

27.3.3 EEG Frequency Bands

Band	Frequency Range	Associated State
Delta	0.5 – 4.0 Hz	Deep sleep
Theta	4.0 – 8.0 Hz	Drowsiness, meditation
Alpha	8.0 – 13.0 Hz	Relaxed, eyes closed
Beta	13.0 – 30.0 Hz	Active thinking, focus
Gamma	30.0 – 100.0 Hz	High-level cognition, perception

27.3.4 Stage 3: Feature Extraction

Features extracted from the processed signals include: - Band power ratios (alpha/beta for attention scoring) - Event-related potentials (ERPs) - Spike sorting (for invasive BCIs like Neuralink) - Coherence between channel pairs

27.3.5 Stage 4: Classification

The classifier maps features to discrete commands or continuous control signals. eNI supports multiple classification backends:

- Rule-based thresholding (for simple use cases)
 - Linear discriminant analysis (LDA)
 - Neural network classifiers (via eAI integration)
-

27.4 21.4 Python SDK

eNI includes a Python SDK for rapid prototyping and research. The SDK provides ctypes bindings to the native `libeni_common` library, with fallback to pure-Python synthetic data generation.

27.4.1 ENIProvider Class

```
from eni.core import ENIProvider, EEG_BANDS

# Create a provider (simulator for development)
provider = ENIProvider(provider_type="simulator")
provider.connect()

# Read neural data
data = provider.read_data(num_samples=256)
print(f"Shape: {data.shape}") # (256, 64)

# Compute band powers
powers = provider.compute_band_powers(data, sample_rate=256)
print(f"Alpha power: {powers['alpha']:.3f}")
print(f"Beta power: {powers['beta']:.3f}")

# Get attention score (beta/alpha ratio)
attention = powers['beta'] / powers['alpha']
print(f"Attention score: {attention:.2f}")

provider.disconnect()
```

27.4.2 Key SDK Constants

```
ENI_OK = 0
ENI_MAX_CHANNELS = 1024
ENI_DSP_MAX_FFT_SIZE = 512

EEG_BANDS = {
    "delta": (0.5, 4.0),
    "theta": (4.0, 8.0),
```

```

    "alpha": (8.0, 13.0),
    "beta": (13.0, 30.0),
    "gamma": (30.0, 100.0),
}

```

27.4.3 Library Loading

The Python SDK automatically searches for the native library in standard locations. Platform-specific library names are resolved (`libeni_common.so` on Linux, `libeni_common.dylib` on macOS, `eni_common.dll` on Windows). If the native library is not found, the provider falls back to generating synthetic neural data using NumPy — enabling development without hardware.

27.5 21.5 Native C API

The C API is designed for direct integration in embedded firmware:

```

#include "eni/core.h"

// Initialize provider
eni_provider_t *provider = eni_provider_create(ENI_PROVIDER_NEURALINK);
eni_provider_connect(provider);

// Read data
float buffer[ENI_MAX_CHANNELS * 256];
size_t samples = eni_provider_read(provider, buffer, 256);

// Process with DSP
eni_dsp_config_t dsp = {
    .filter_type = ENI_FILTER_BANDPASS,
    .low_freq = 8.0f,
    .high_freq = 30.0f,
    .sample_rate = 30000.0f,
};
eni_dsp_process(buffer, samples, &dsp);

// Classify
eni_command_t cmd;
eni_classify(buffer, samples, &cmd);

// Cleanup
eni_provider_disconnect(provider);
eni_provider_destroy(provider);

```

27.6 21.6 EIPC Integration

eNI communicates with eAI through the eIPC framework. This enables neural signals to drive AI inference:

eNI (Neural Input) --[eIPC]--> eIPC Server --[eIPC]--> eAI (AI Engine)

Enable eIPC integration at build time:

```
cmake -B build -DENI_EIPC_ENABLED=ON
```

27.7 21.7 Build Configurations

eNI supports flexible build configurations:

```
# Full build (minimal + framework + Neuralink provider)
cmake -B build -DENI_BUILD_TESTS=ON
cmake --build build

# Minimal only (MCU targets)
cmake -B build -DENI_BUILD_MIN=ON -DENI_BUILD_FRAMEWORK=OFF

# Without Neuralink provider
cmake -B build -DENI_PROVIDER_NEURALINK=OFF

# Cross-compile for ARM
cmake -B build-arm \
  -DCMAKE_C_COMPILER=aarch64-linux-gnu-gcc \
  -DCMAKE_SYSTEM_NAME=Linux
cmake --build build-arm
```

27.8 21.8 Applications

Domain	Application
Medical	Prosthetic limb control, neural rehabilitation
Assistive	Communication aids for locked-in patients
Drone Control	Direct neural flight control via eNI to eAI
Vehicle	Attention monitoring, drowsiness detection
Gaming/XR	Thought-based interaction in VR/AR
Industrial	Hands-free equipment operation

27.9 21.9 Simulator Mode

For development without BCI hardware, eNI includes a full-featured simulator that generates statistically representative neural data:

```
provider = ENIProvider(provider_type="simulator")
provider.connect()

# Simulator generates realistic synthetic EEG data
data = provider.read_data(num_samples=1024)

# Features: realistic frequency band distributions,
# configurable noise levels, optional event injection
```

The simulator enables full pipeline development and testing without any BCI hardware.

27.10 21.10 Summary

eNI provides the neural interface layer that connects brain-computer interfaces to the EoS embedded platform. Its provider abstraction supports everything from research-grade Neuralink implants to consumer EEG headsets, while the processing pipeline handles the complex signal processing needed to extract actionable commands from raw neural data.

Key takeaways:

- Vendor-neutral BCI framework with Neuralink, Simulator, and Generic providers
 - Complete DSP pipeline: filtering, FFT, feature extraction, classification
 - Python SDK for rapid prototyping with NumPy integration
 - Native C API for embedded deployment (as small as 32 KB Flash)
 - eIPC integration for neural-to-AI communication
 - Supports up to 1024 channels at 30 kHz sample rate
-

Next: Chapter 22 — EoSim Simulation Platform

Chapter 28

Chapter 22: EoSim — Multi-Architecture Simulation Platform

Author: Srikanth Patchava & EmbeddedOS Contributors

28.1 22.1 Introduction

Hardware availability is often the bottleneck in embedded development. Boards are expensive, shared among teams, and sometimes not yet manufactured. **EoSim** removes this constraint by providing simulation and validation for **52+ platforms** across **12 architectures** — enabling development, testing, and CI before hardware is ready.

28.2 22.2 Supported Platforms

EoSim supports an extensive range of platforms:

28.2.1 ARM Cortex-M (MCU)

Platform	SoC	Description
stm32f4	STM32F407	Cortex-M4F, 168 MHz, FPU
stm32h7	STM32H743	Cortex-M7, 480 MHz, Dual-core
nrf52	nRF52840	Cortex-M4F, BLE 5.0
nrf5340	nRF5340	Cortex-M33, Dual-core, BLE 5.3
rp2040	RP2040	Dual Cortex-M0+, PIO
s32k344	S32K344	Cortex-M7, Automotive CAN-FD
samd51	ATSAMD51	Cortex-M4F, Arduino/Adafruit

28.2.2 ARM Cortex-A (Application Processors)

Platform	SoC	Description
raspi4	BCM2711	Cortex-A72, 4-core, 8 GB
raspi5	BCM2712	Cortex-A76, 4-core
jetson-nano	Tegra X1	Cortex-A57, 128 CUDA
jetson-orin	Orin	Cortex-A78AE, 40 TOPS
imx8m	i.MX8M	Cortex-A53, NPU, 4K

28.2.3 RISC-V

Platform	SoC	Description
riscv64	Generic	RISC-V 64-bit Linux
sifive_u	FU740	RISC-V U74, Linux-capable
esp32c3	ESP32-C3	RISC-V, Wi-Fi/BLE

28.2.4 Other Architectures

Platform	Arch	Description
x86_64	x86_64	Generic Linux on QEMU q35
esp32	Xtensa	Xtensa LX6, Wi-Fi/BLE
aurix-tc3xx	TriCore	Automotive safety
mipsel	MIPS	MIPS little-endian

28.3 22.3 Simulation Engines

EoSim supports four simulation backends:

Engine	Type	Speed	Fidelity	Best For
EoSim native	Python simulation	Fast	Medium	Rapid prototyping, CI
Renode	Deterministic sim	Medium	High	Peripheral-accurate
QEMU	Binary emulation	Medium	High	Full firmware testing
HIL	Hardware-in-loop	Real	Exact	Final validation

28.3.1 Engine Selection

EoSim automatically selects the best engine based on the platform definition, but you can override it:

```
eosim run stm32f4 --engine renode      # Use Renode for peripheral accuracy
eosim run raspi4 --engine qemu         # Use QEMU for full Linux emulation
```

```
eosim run nrf52 --engine native      # Use EoSim native for speed
```

28.4 22.4 CLI Reference

```
# List all available platforms
eosim list

# Filter by architecture
eosim list --arch arm-cortex-m

# Show platform details
eosim info stm32f4

# Run a simulation
eosim run arm64-linux

# Run with custom firmware
eosim run stm32f4 --firmware build/firmware.bin

# Run validation tests
eosim test raspi4

# Validate a platform config file
eosim validate platforms/stm32f4/config.yaml

# Check environment health
eosim doctor

# Search platforms
eosim search "bluetooth"

# Platform statistics
eosim stats
```

28.5 22.5 Platform Definition Format

Each platform is defined by a YAML configuration file:

```
name: stm32f4
display_name: STM32F4 Discovery
arch: arm-cortex-m4
engine: renode
```

```

simulation:
  machine: STM32F4Discovery
  cpu: cortex-m4
  frequency_mhz: 168

peripherals:
  gpio: [GPIOA, GPIOB, GPIOC, GPIOD]
  uart: [USART1, USART2]
  spi: [SPI1, SPI2]
  i2c: [I2C1]
  timer: [TIM1, TIM2, TIM3, TIM4]
  adc: [ADC1]

memory:
  flash_kb: 1024
  sram_kb: 192

boot:
  entry_point: 0x08000000
  stack_pointer: 0x20030000

runtime:
  headless: true
  uart: sysbus.usart2

```

Platform definitions also include Renode `.repl` (platform description) and `.resc` (script) files for high-fidelity simulation.

28.6 22.6 GUI Dashboard

EoSim includes an optional Tkinter-based GUI for interactive simulation:

```
eosim gui stm32f4
```

28.6.1 GUI Panels

Panel	Description
GPIO Panel	Pin visualizer with click-to-toggle
UART Terminal	Real-time serial output
CPU State	Registers, PC, SP, flags, clock
Memory View	Hex dump of RAM/Flash regions
Peripheral Regs	TIM, ADC, SPI, I2C configuration registers
3D Renderers	Drone, robot, vehicle, aircraft visualization

The GUI provides a visual development experience similar to debugging on real hardware, but without the hardware.

28.7 22.7 CI Integration

EoSIm is designed to run in CI pipelines for automated validation:

```
# .github/workflows/firmware-test.yml
name: Firmware Validation
on: [push, pull_request]

jobs:
  simulate:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        platform: [stm32f4, nrf52, raspi4, esp32, riscv64]
    steps:
      - uses: actions/checkout@v4
      - name: Install EoSIm
        run: pip install eosim
      - name: Run simulation tests
        run: eosim test ${matrix.platform} --timeout 300
```

28.8 22.8 Cluster Simulation

For multi-device systems, EoSIm supports cluster simulation — running multiple interconnected device instances simultaneously:

```
eosim cluster run \
  --nodes "sensor1:nrf52,sensor2:nrf52,gateway:raspi4" \
  --network ble-mesh \
  --duration 60
```

This is invaluable for testing IoT mesh networks, distributed sensor systems, and multi-device protocols.

28.9 22.9 Specialty Simulators

Beyond embedded hardware, EoSIm includes domain-specific simulators:

Simulator	Domain	Description
aerodynamics-sim	Aerospace	Aerodynamics simulation
physiology-sim	Medical	Physiological modeling
weather-sim	Environmental	Weather modeling
finance-sim	Financial	Financial market simulation
gaming-sim	Gaming	Game engine simulation

These simulators enable domain-specific validation of EoS-based systems.

28.10 22.10 Architecture

```

eosim/
  eosim/
    cli/          # Click-based CLI entry point
    core/         # Core simulation logic
    engine/       # Backend engines (native, Renode)
    gui/          # Tkinter GUI dashboard
    integrations/ # External tool integrations
    artifacts/    # Simulation artifact management
    platforms/    # Platform abstraction layer
  platforms/     # 52+ platform definitions (YAML + Renode)
  tests/         # Unit + integration + scenario tests
  examples/      # Demo scenarios

```

28.11 22.11 Summary

EoSim enables embedded development without hardware constraints. With 52+ platform definitions, four simulation engines, and CI-native design, it ensures that firmware is validated early and often.

Key takeaways:

- 52+ platforms across 12 architectures
- Four simulation engines: native Python, Renode, QEMU, HIL
- Interactive GUI with GPIO, UART, CPU, and memory panels
- CI-native: runs in GitHub Actions, GitLab CI, Jenkins
- Cluster simulation for multi-device systems
- Domain-specific specialty simulators

Next: Chapter 23 — EoStudio Design Suite

Chapter 29

Chapter 23: EoStudio — Cross-Platform Design Suite

Author: Srikanth Patchava & EmbeddedOS Contributors

29.1 23.1 Introduction

Embedded development requires more than just a code editor. From 3D modeling of enclosures to PCB layout, from UI/UX prototyping to game design — modern embedded products demand a comprehensive design toolkit.

EoStudio is a unified design tool suite for the EmbeddedOS ecosystem, combining 12 specialized editors, 30+ code generators, and LLM-powered AI assistance — all running on Windows, Linux, and EoS.

29.2 23.2 The Twelve Editors

Editor	CLI Flag	Use Case
3D Modeler	<code>--editor 3d</code>	Mesh modeling, materials, lighting
CAD Designer	<code>--editor cad</code>	Parametric design, assemblies
Image Editor	<code>--editor paint</code>	Layers, brushes, filters
Game Editor	<code>--editor game</code>	ECS, tilemaps, sprites
UI/UX Designer	<code>--editor ui</code>	Components, flows, prototyping
Product Designer	<code>--editor product</code>	BOM, 3D-print validation
Interior Designer	<code>--editor interior</code>	Floor plans, furniture
UML Modeler	<code>--editor uml</code>	Class, sequence, state diagrams
Simulation Editor	<code>--editor simulation</code>	Block diagrams, PID, signals
Database Designer	<code>--editor database</code>	ERD, schema, SQL/ORM codegen
Hardware Editor	<code>--editor hardware</code>	PCB layout, schematics, Gerber
IDE	<code>--editor ide</code>	Code editing, debugging, Git

29.3 23.3 Quick Start

```
# Clone and install
git clone https://github.com/embeddedos-org/EoStudio.git
cd EoStudio
pip install -e ".[all]"

# Launch the design suite
EoStudio launch

# Create a project from template
EoStudio new --template todo-app -o ./my-app

# Generate React code from a design
EoStudio codegen my-app/todo-app.eostudio --framework react -o ./output

# Ask the AI design agent
EoStudio ask "Design a responsive dashboard with charts"
```

29.4 23.4 Code Generation

EoStudio can generate production-ready code from visual designs across 30+ frameworks:

Target	Frameworks
Mobile	Flutter, React Native, Kotlin (Android), Swift (iOS)
Desktop	Electron, Tauri, tkinter, Qt, Compose Desktop
Web	React+FastAPI, Vue+Flask, Angular+Express, Svelte
Database	SQLite, PostgreSQL, MySQL, SQLAlchemy, Prisma
UML to Code	Python, Java, Kotlin, TypeScript, C++, C#
3D/CAD	OpenSCAD, STL, OBJ, glTF, DXF
Game Engines	Godot, Unity, Unreal
Firmware	EoS, Baremetal, FreeRTOS, Zephyr

29.4.1 Code Generation Example

```
# Generate a Flutter mobile app from a UI design
EoStudio codegen dashboard.eostudio --framework flutter -o ./flutter_app

# Generate OpenSCAD from a CAD design
EoStudio codegen bracket.eostudio --framework openscad -o ./scad_output

# Generate EoS firmware from a hardware design
EoStudio codegen sensor_board.eostudio --framework eos -o ./firmware
```


29.5 23.5 AI and LLM Integration

EoStudio deeply integrates AI assistance across all editors:

Feature	Description
Design Agent	Multi-domain Q&A, design brief generation
Smart Chat	Per-editor AI panel with context-aware prompts
AI Generator	Text-to-UI, text-to-3D, text-to-CAD generation
AI Simulator	Parameter suggestion, instability detection
Kids Tutor	Interactive lessons with quizzes

29.5.1 LLM Backend Configuration

EoStudio supports both local and cloud LLM backends:

```
# Option 1: Ollama (local, private)
ollama pull llama3
EoStudio ask "Design a login page"

# Option 2: OpenAI API
export EOSTUDIO_LLM_PROVIDER=openai
export OPENAI_API_KEY=sk-your-key
export EOSTUDIO_LLM_MODEL=gpt-4o
EoStudio ask --domain cad "Design an L-bracket with mounting holes"
```

29.6 23.6 Project Templates

EoStudio includes 5 built-in templates demonstrating real workflows:

Template	Workflow	Command
todo-app	UI Designer to React/Flutter	EoStudio new --template todo-app
mechanical-part	CAD Designer to OpenSCAD to STL	EoStudio new --template mechanical-part
game-platformer	Game Editor to Godot/Unity	EoStudio new --template game-platformer
iot-dashboard	Database + UI to Full-stack	EoStudio new --template iot-dashboard
simulation-pid	Simulation Editor to PID	EoStudio new --template simulation-pid

29.7 23.7 Architecture

```
eostudio/
+-- cli/          # Click CLI (10 commands)
+-- core/
```

```

|   +-- ai/                # LLMClient, DesignAgent, SmartChat, AIGenerator
|   +-- geometry/         # Vec2/3/4, Matrix4, Mesh, Bezier, NURBS, CSG
|   +-- rendering/        # Rasterizer, scene graph, camera, Phong lighting
|   +-- physics/          # Rigid body, collision, particles
|   +-- cad/              # Parametric design, constraints, assembly
|   +-- simulation/       # Block diagrams, PID, signals, ODE solver
|   +-- uml/              # 5 diagram types + code generation
|   +-- game/             # ECS, tilemap, sprites, scripting
|   +-- image/            # Layers, brushes, filters
|   +-- hardware/         # PCB, schematic, Gerber
|   +-- ui_flow/          # Component library, prototyping
|   +-- interior/         # Floor plans, furniture
+-- gui/
|   +-- editors/          # 12 visual editors
|   +-- widgets/          # Viewport, canvas, timeline, properties
|   +-- dialogs/          # Export, settings, AI chat
+-- codegen/              # 30+ framework code generators
+-- formats/              # .EoStudio, OBJ, STL, SVG, glTF, DXF
+-- plugins/              # Plugin system + EoSim integration
+-- templates/            # 5 project templates

```

29.8 23.8 Plugin System

EoStudio supports plugins for extending functionality:

```

from eostudio.plugins.plugin_base import Plugin, PluginHook

class MyPlugin(Plugin):
    def activate(self, context):
        self._hooks[PluginHook.POST_CODEGEN] = self._on_codegen
        return super().activate(context)

    def _on_codegen(self, data):
        # Post-process generated code
        return {"processed": True}

```

Plugins can hook into lifecycle events: - PRE_CODEGEN / POST_CODEGEN — modify code generation
- PRE_EXPORT / POST_EXPORT — modify file export - EDITOR_INIT — add custom panels to editors
- AI_RESPONSE — filter or augment AI responses

29.9 23.9 Platform Support

Platform	Status	Backend
Windows 10/11	Stable	tkinter
Ubuntu 22.04+	Stable	tkinter
Linux (other)	Stable	tkinter
macOS	Stable	macOS native

Platform	Status	Backend
EoS	Stable	Framebuffer
Browser	Beta	Web backend

29.10 23.10 Testing

```

pip install -e ".[dev,all]"
pytest -v
pytest --cov=eostudio --cov-report=term-missing
flake8 eostudio/ tests/ --max-line-length=120
mypy eostudio/ --ignore-missing-imports

```

Test coverage spans: AI modules, geometry, code generation, plugins, simulation, file formats, and integration testing.

29.11 23.11 Summary

EoStudio consolidates the diverse design tools needed for embedded product development into a single, AI-enhanced suite. From CAD modeling to firmware code generation, from UI prototyping to PCB layout, EoStudio provides a unified workflow that eliminates tool-switching overhead.

Key takeaways:

- 12 specialized editors in one unified application
- 30+ code generators targeting mobile, web, desktop, firmware
- LLM-powered AI assistance with Ollama and OpenAI backends
- Plugin system for extensibility
- Cross-platform: Windows, Linux, macOS, EoS, and browser
- 5 built-in project templates for rapid prototyping

Next: Chapter 24 — eRadar360 Hardware Design

Chapter 30

Chapter 24: eRadar360 — Advanced Driver Awareness System

Author: Srikanth Patchava & EmbeddedOS Contributors

30.1 24.1 Introduction

The **eRadar360** (marketed as “Aegis One”) is a next-generation driver awareness system that combines 360-degree phased-array radar, V2X communication, AI-powered false-alert suppression, and OBD-II vehicle integration into a single device.

Unlike traditional radar detectors that rely solely on frequency-band detection, eRadar360 is built from the ground up as a **threat intelligence platform** — every subsystem shares data through an on-device neural network.

Target Price: \$699 | **Dimensions:** 120mm x 85mm x 18mm | **Weight:** 185g

30.2 24.2 Key Specifications

Feature	Specification
Display	4” Samsung AMOLED, 480x800, 600 nits
Front Radar	SIW phased array, 8x8 elements, 24 dBi, +/-45 deg steering
Rear Radar	SIW phased array, 4x4 elements, 18 dBi, +/-30 deg steering
Radar Bands	Ka (33.4-36 GHz), K (24.05-24.25 GHz), Ku, X
Laser	5x Hamamatsu InGaAs APD array, 360 deg, 904nm + 1550nm
V2X	DSRC (5.9 GHz) + C-V2X (PC5), law enforcement BSM
AI Engine	6 TOPS NPU (RK3588S), 97% false-alert suppression
GPS	u-blox NEO-M9N, 1.5m CEP, cloud threat database
OBD-II	CAN bus: speed, cruise state, lane context
Connectivity	Bluetooth 5.3, Wi-Fi 6, USB-C

30.3 24.3 System Architecture

The eRadar360 is built around three main processing units:

30.3.1 RK3588S Main Processor

The Rockchip RK3588S serves as the central brain: - **CPU:** 4x Cortex-A76 @ 2.4 GHz + 4x Cortex-A55 @ 1.8 GHz - **NPU:** 6 TOPS (INT8) for radar signature fingerprinting - **GPU:** Mali-G610 MP4 for display rendering - **Memory:** 1 GB DDR4-3200 - **Storage:** 256 MB QSPI NOR Flash

30.3.2 AWR2944 Radar SoCs (x2)

Two TI AWR2944 chips handle front and rear radar: - 77 GHz FMCW radar - 4 TX + 4 RX channels each (8 total) - 20 MHz IF bandwidth, 0.75m range resolution - SPI interface @ 50 MHz to the RK3588S

30.3.3 STM32H7B3 Co-Processor

Handles real-time sensor fusion: - ARM Cortex-M7 @ 280 MHz - Laser ADC processing, OBD-II parsing, GPS data - I2C connection to RK3588S

30.4 24.4 Why SIW Phased Array?

Standard microstrip patch arrays (used in competing products) are fixed-beam. The Substrate Integrated Waveguide (SIW) phased array electronically steers +/-45 degrees. When the AI detects a signal at 20 degrees left, firmware focuses the beam for 3-4 dB extra gain — detecting a source at 2 miles instead of 1 mile.

30.5 24.5 Radar Performance

Parameter	X-Band	K-Band	Ku-Band	Ka-Band
Frequency Range	10.500-10.550	24.050-24.250	13.450-13.500	33.400-36.000
Front Gain	7 dBi	12 dBi	8 dBi	24 dBi
Beam Steering	Fixed	Fixed	Fixed	+/-45 deg
Detection Range	1.5 mi	2.0 mi	1.5 mi	3.0+ mi

30.6 24.6 V2X Communication

Modern smart city infrastructure broadcasts law enforcement activity on DSRC/C-V2X channels. The Autotalks TEKTON3 chipset receives:

- **BSM** (Basic Safety Messages) — vehicle position and movement
- **TIM** (Traveler Information Messages) — road conditions
- **SPaT** (Signal Phase and Timing) — traffic signal data
- **MAP** — intersection geometry

This means eRadar360 knows about a speed trap a mile before any radar signal is detectable.

30.7 24.7 AI False-Alert Suppression

The RK3588S NPU fingerprints radar signatures by pulse pattern and modulation — not just frequency band. This achieves 97% false-alert suppression without cloud dependency.

Common false-alert sources eliminated: - Automatic door openers (K-band) - Adaptive cruise control radar (Ka-band) - Blind-spot monitoring systems - Traffic flow sensors

30.8 24.8 Laser Detection

Parameter	Value
Sensors	5x Hamamatsu G12183-010K InGaAs APD
Wavelength	900-1700 nm (904 nm + 1550 nm)
Field of View	360 deg (5 sensors at 72 deg spacing)
Response Time	< 100 us (pulse detection)
Alert Latency	< 50 ms (end-to-end)

30.9 24.9 OBD-II Integration

The OBD-II interface provides vehicle context to the alert engine:

- **Speed** — suppress alerts when stationary
- **Cruise state** — adjust alert priority when cruise control manages speed
- **RPM/Throttle** — detect acceleration patterns
- **DTC codes** — vehicle health monitoring

This eliminates an entire category of irrelevant warnings.

30.10 24.10 Electrical Specifications

30.10.1 Power

Parameter	Typ	Max
Input Voltage (USB-C)	5.0 V	5.5 V
Input Voltage (OBD-II)	12.0 V	16.0 V
Power Consumption (active)	8.5 W	10.0 W
Power Consumption (sleep)	0.5 W	0.8 W

30.10.2 Internal Rails

Rail	Voltage	Source
VCC_5V	5.00 V	USB-C / OBD-II
VCC_3V3	3.30 V	TPS65219 BUCK1
VCC_1V8	1.80 V	TPS65219 BUCK2
VCC_0V85	0.85 V	TPS65219 BUCK3
VCC_1V1	1.10 V	TPS65219 BUCK4

30.11 24.11 PCB Design

The eRadar360 uses a **10-layer hybrid PCB** with Rogers 4003C for RF layers and FR4 for digital layers. This provides:

- Low-loss RF transmission for radar frequencies
- Controlled impedance for high-speed digital signals
- Thermal management via internal ground planes

30.11.1 Design Files

File	Description
pcb_stackup.txt	10-layer Rogers/FR4 stackup
bom.csv	Full BOM with MPN and pricing
eradar360_schematic.html	Interactive block schematic
antenna_design.md	SIW phased array design calculations
bring_up_guide.md	Board bring-up and test procedure

30.12 24.12 Environmental and Regulatory

Parameter	Value
Operating Temperature	-20 to +85 deg C
Storage Temperature	-40 to +100 deg C
Humidity	10-90% RH (non-condensing)
ESD (contact/air)	+/-8 kV / +/-15 kV
Regulatory (planned)	FCC Part 15/90, CE, IC, RoHS

30.13 24.13 Summary

eRadar360 represents a paradigm shift in driver awareness systems. By combining phased-array radar, V2X communication, AI processing, and vehicle integration, it delivers capabilities that no single-sensor detector can match.

Key takeaways:

- SIW phased-array radar with +/-45 deg electronic beam steering
- V2X reception detects threats before radar signals are visible
- 6 TOPS NPU achieves 97% false-alert suppression on-device
- OBD-II integration provides vehicle context for alert filtering
- 360-degree laser detection with < 50 ms alert latency
- \$699 target price with ~\$330 BOM cost

Next: Chapter 25 — eHealth365 Health Monitoring

Chapter 31

Chapter 25: eHealth365 — Wearable Health Monitoring

Author: Srikanth Patchava & EmbeddedOS Contributors

31.1 25.1 Introduction

The **eHealth365** is a two-device health monitoring ecosystem designed to cover approximately 90% of all health metrics using just a smart ring, a smart patch, and a mobile app. The system combines continuous biometric sensing with periodic blood chemistry analysis.

31.2 25.2 System Architecture

Smart Ring Pro
Finger - worn 24/7

Heart rate + HRV
SpO2 oxygen
Body temperature
Sleep stages
Ketones (breath)
Steps and activity
Stress score (HRV)

Smart Patch Pro
Upper arm - weekly

Blood glucose (CGM)
Electrolytes (sweat)
Hydration (bioZ)
Skin pH
Monthly blood labs
Lactate (workout)

Bluetooth LE

Mobile App

Single Health Hub

- AI food camera
- Digital twin
- Doctor sharing
- Deficiency alerts

31.3 25.3 The Two Devices

31.3.1 Smart Ring Pro — Vitality Hub

The ring monitors the autonomic nervous system and movement. It is designed for 24/7 wear with no screen and haptic feedback only.

Specification	Value
Form Factor	Titanium/ceramic ring
Battery Life	4-5 days
Charging	Wireless (Qi)
Sensors	PPG, thermistor, accelerometer
Connectivity	Bluetooth LE
Water Resistance	IP68

31.3.2 Smart Patch Pro — Chemistry Hub

The patch monitors blood and interstitial fluid chemistry. It adheres to the upper arm and is replaced weekly.

Specification	Value
Form Factor	Flexible adhesive puck
Battery Life	7 days
Replacement	Weekly patch, monthly cartridge
Sensors	CGM, sweat biosensors, bioimpedance
Connectivity	Bluetooth LE

31.4 25.4 Health Metrics Coverage

Metric	Device	Method	Schedule
Heart rate and HRV	Ring	Optical PPG	Continuous
SpO2 oxygen	Ring	Optical sensor	Continuous
Body temperature	Ring	Thermistor	Continuous
Sleep stages	Ring	Accelerometer + HRV	Nightly

Metric	Device	Method	Schedule
Steps and activity	Ring	Accelerometer	Continuous
Stress score	Ring	HRV analysis	Hourly
Ketones	Ring	Micro breath port	2x daily
Blood glucose	Patch	CGM micro-sensor	Every 15 min
Sodium, potassium	Patch	Sweat biosensor	Every 4 hrs
Magnesium, zinc	Patch	Sweat biosensor	Every 4 hrs
Hydration level	Patch	Bioimpedance	3x daily
Skin pH	Patch	pH sensor	Every 4 hrs
Lactate	Patch	Sweat sensor	During exercise
Calories burned	Patch	Temp + glucose	Continuous
Vitamins (A-K, B1-B12)	Patch	Monthly cartridge	Monthly
Minerals (Fe, Zn, Ca, Mg)	Patch	Monthly cartridge	Monthly
Calories in (nutrition)	App	AI food camera	Per meal

31.5 25.5 Hardware Design — Smart Ring Pro

The ring PCB uses a flexible circuit board that wraps around the inside of the titanium band:

31.5.1 Key Components

- **MCU:** Nordic nRF52840 (Cortex-M4F, BLE 5.0)
- **PPG Sensor:** MAX30102 (heart rate, SpO2)
- **Accelerometer:** LIS3DH (steps, sleep detection)
- **Thermistor:** NTC 10K (body temperature)
- **Haptic Motor:** LRA vibration motor
- **Battery:** 30 mAh LiPo (wireless charging)

31.5.2 Power Budget

Mode	Current	Duration	Daily mAh
Active PPG	3.5 mA	10 min/hr	14.0
Sleep track	0.8 mA	8 hrs	6.4
Idle + BLE	0.15 mA	Remaining	1.5
Total			~22 mAh

With a 30 mAh battery, this yields approximately 1.3 days. The actual 4-5 day battery life is achieved through aggressive duty cycling and on-demand sensor activation.

31.6 25.6 Hardware Design — Smart Patch Pro

The patch uses a flexible PCB with integrated biosensors:

31.6.1 Key Components

- **MCU:** Nordic nRF5340 (dual Cortex-M33, BLE 5.3)
- **CGM Sensor:** Custom glucose micro-needle array
- **Sweat Sensors:** Ion-selective electrodes (Na+, K+, Mg2+, Zn2+)
- **Bioimpedance:** AD5940 analog front-end
- **pH Sensor:** ISFET-based pH electrode
- **Blood Cartridge:** Microfluidic slot for monthly labs
- **Battery:** 150 mAh flexible LiPo

31.6.2 Monthly Blood Cartridge

The blood cartridge is a microfluidic device that performs a finger-prick blood test. It measures: - Complete vitamin panel (A, C, D, E, K, B1-B12) - Mineral levels (iron, zinc, calcium, magnesium) - Basic metabolic panel

31.7 25.7 Mobile App Architecture

The mobile app serves as the central health hub:

Module	Function
Dashboard	Real-time vitals, trends, scores
AI Food Camera	Photograph meals for calorie estimation
Digital Twin	3D body model with health overlays
Doctor Portal	Share reports with healthcare providers
Alert Engine	Deficiency alerts and recommendations
Data Export	FHIR/HL7 compatible health records

31.8 25.8 Sensing Strategy — The Life Flow Pattern

eHealth365 implements a 24-hour sensing schedule called “Life Flow” that balances data completeness with battery life:

Time Period	Ring Activity	Patch Activity
Morning	HR, temp, HRV baseline	Glucose, hydration check
Active/Work	Steps, stress monitoring	Glucose every 15 min
Exercise	Continuous HR, SpO2	Lactate, electrolytes
Meals	(App: food camera)	Post-meal glucose spike
Evening	Stress recovery, ketones	Electrolyte summary

Time Period	Ring Activity	Patch Activity
Sleep	Sleep staging, HR, SpO2	Overnight glucose trend

31.9 25.9 Data Architecture

Sensors --> Edge Processing (on-device) --> BLE --> Phone App --> Cloud (optional)

All health data is processed on-device first. The phone app stores data locally with optional cloud sync for: - Multi-device access - Doctor sharing portal - Long-term trend analysis (AI-powered) - Emergency alert routing

31.10 25.10 Pricing and Go-To-Market

Item	Price
Smart Ring Pro	\$299
Smart Patch Pro starter kit	\$199
Weekly patch refills	\$15/week
Monthly blood cartridge	\$25/month
App subscription	\$9.99/month
Total first year	~\$1,100

31.10.1 Phased Rollout

Phase	Timeline	Scope
MVP	Year 1-2	Ring + patch with HR, sleep, glucose
Growth	Year 2-3	Breath analyzer, food AI, electrolytes
Complete	Year 4-5	Blood cartridge, full vitamin/mineral panel

31.11 25.11 EoS Integration

Both devices run EoS firmware with the **wearable** product profile:

```
cmake -B build -DEOS_PRODUCT=wearable \
  -DCMAKE_TOOLCHAIN_FILE=toolchains/arm-none-eabi.cmake
```

The firmware leverages: - **EoS HAL** — sensor drivers (PPG, ADC, I2C, SPI) - **EoS Kernel** — task scheduling, power management - **eBoot** — secure boot and OTA firmware updates - **eAI (Min)** — on-device health scoring algorithms

31.12 25.12 Summary

eHealth365 demonstrates how EoS-powered wearable devices can provide comprehensive health monitoring through a carefully designed two-device ecosystem.

Key takeaways:

- Two devices cover ~90% of health metrics
- Smart Ring Pro for autonomic/movement monitoring (24/7 wear)
- Smart Patch Pro for blood/fluid chemistry (weekly replacement)
- Monthly blood cartridge for vitamin and mineral panels
- AI-powered food camera for nutrition tracking
- EoS firmware with wearable product profile

Next: Chapter 26 — ePAM Personal Air Mobility

Chapter 32

Chapter 26: ePAM — Personal Air Mobility

Author: Srikanth Patchava & EmbeddedOS Contributors

32.1 26.1 Introduction

The **ePAM** (Personal Air Mobility) project encompasses a four-product vehicle line powered by solar-hybrid propulsion. From a \$28K electric car to a \$5M air-space combo unit, all four vehicles share a common platform architecture and run EoS firmware for their flight controllers and ECUs.

32.2 26.2 The Four Vehicles

Vehicle	Type	Seats	Power	Altitude	Price Range
Eco Car	Ground vehicle	4-5	Solar + solid-state batt	Ground	\$28K-\$45K
Urban Drone	eVTOL aircraft	4	Solar + solid-state batt	0-500m	\$85K-\$120K
Space Shuttle	Suborbital craft	4	Solar + H2 fuel cell	100km	\$2M-\$4M
Combo Unit	Air+space hybrid	4	Solar + H2 hybrid	0-100km	\$5M-\$9M

32.3 26.3 Power Architecture

All four vehicles share a common solar-hybrid power architecture:

32.3.1 The Water/Solar Energy Loop

1. **Solar Collection** — vehicle surface coated in flexible perovskite solar cells
2. **Water Electrolysis** — excess solar energy splits on-board H₂O into H₂ + O₂

3. **Hydrogen Fuel Cell** — H2 powers electric motors, water vapor exhaust is recaptured
4. **Kinetic Regeneration** — descent/braking energy captured and stored

This creates a partially closed-loop energy system where water is continuously recycled between electrolysis and fuel cell operation.

32.4 26.4 Eco Car — Mass Market

The Eco Car is the entry point to the ePAM product line:

32.4.1 Manufacturing Cost Breakdown (\$19K-\$32K per unit)

Category	Cost Range	% of Total
Battery pack (75 kWh)	\$7,500-\$11K	~35%
Powertrain (dual PMSM motors)	\$2,800-\$4,500	~14%
Solar integration (6 sq m)	\$900-\$1,500	~5%
Body and chassis (recycled Al)	\$3,000-\$5,000	~16%
Electronics and infotainment	\$1,200-\$2,000	~6%
Interior (sustainable)	\$1,000-\$2,000	~6%
H2 fuel cell (optional)	\$2,000-\$3,500	~10%
Assembly and QC	\$1,600-\$2,500	~8%

Key cost driver: the solid-state battery adds a \$2K premium over Li-ion but eliminates the complex thermal management system, netting lower total system cost at scale.

32.5 26.5 Urban Drone — City Commuter

The Urban Drone is a 4-seat eVTOL for urban air mobility:

32.5.1 Key Specifications

Parameter	Value
Max Altitude	500m (1,640 ft)
Range	100 km (62 mi)
Max Speed	200 km/h (124 mph)
Propulsion	8 electric motors, tilt-rotor
Autonomy	Level 4 (AI-assisted)
Emergency	Ballistic parachute + autorotation

32.6 26.6 Space Shuttle — Tourism

The Space Shuttle provides suborbital space tourism experiences:

32.6.1 Key Specifications

Parameter	Value
Max Altitude	100 km (Karman line)
Mission Duration	2-3 hours total
Weightlessness	5-8 minutes
Propulsion	Hybrid rocket + H2 fuel cell
Life Support	Pressurized cabin, O2 from H2O
Safety	Launch escape system, triple backup

32.7 26.7 Combo Unit — Premium Fleet

The Combo Unit combines air and space capabilities in a single vehicle. It transitions between ground, air (eVTOL mode), and suborbital flight. Target market: government, defense, and premium fleet operators.

32.8 26.8 Business Model

ePAM generates revenue from multiple streams:

Revenue Stream	Description
Direct vehicle sales	All four product tiers
Fleet leasing (B2B)	Urban Drone and Combo for operators
Ride-share platform	Percentage of ride-share transactions
Space tourism tickets	Per-seat suborbital experiences
Software/OTA subs	Autonomous driving updates
Energy resale (V2G)	Vehicle-to-grid power selling

32.9 26.9 Go-To-Market Roadmap

Phase	Timeline	Milestones
Phase 1	2025-27	EcoCar production, Urban Drone prototype FAA/EASA certifications, seed fleet cities
Phase 2	2027-30	Urban Drone mass launch, ride-share platform

Phase	Timeline	Milestones
Phase 3	2030+	H2 fueling network, space R&D milestones Space shuttle launch, Combo unit fleet Global expansion, IPO or licensing

32.10 26.10 Regulatory Requirements

Category	Requirements
Ground	NHTSA, EPA, ECE (standard automotive)
Air (eVTOL)	FAA Part 135, EASA, urban air mobility laws
Space	FAA AST, Space Act agreements
Infrastructure	Vertiports, H2 fueling stations

32.11 26.11 EoS Integration

All ePAM vehicles run EoS firmware:

Vehicle	EoS Product Profile	Key Modules
Eco Car	ev	Motor control, battery mgmt
Urban Drone	drone	Flight control, GPS, sensors
Space Shuttle	aerospace	Life support, propulsion
Combo Unit	autonomous	Multi-mode transitions

The vehicles leverage: - **EoS HAL** — motor drivers, sensor interfaces, CAN bus - **eAI** — autonomous flight, obstacle avoidance, route planning - **eNI** — neural interface for pilot brain-computer control - **EoSIm** — flight simulation and digital twin of all models - **eBoot** — secure boot for safety-critical firmware

32.12 26.12 Summary

ePAM demonstrates the full ambition of the EoS platform — from a mass-market electric car to a suborbital space shuttle, all running common firmware.

Key takeaways:

- Four-product vehicle line sharing common platform architecture
- Solar-hybrid power with water/hydrogen closed-loop energy cycle
- Eco Car at \$28K-\$45K targets mass market adoption

- Urban Drone for city commuting with Level 4 autonomy
- EoS firmware powers flight controllers and vehicle ECUs
- Phased rollout: ground first, then air, then space

Next: Chapter 27 — eApps Marketplace

Chapter 33

Chapter 27: eApps — Unified Marketplace and App Store

Author: Srikanth Patchava & EmbeddedOS Contributors

33.1 27.1 Introduction

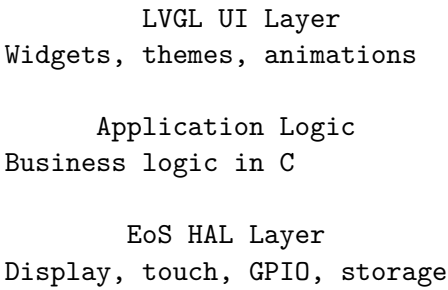
eApps is the unified marketplace, monorepo, and automated app store for the entire EoS ecosystem. It delivers **50 apps across 8 platform categories totaling 188 platform targets** — from native C+LVGL embedded apps to Flutter mobile apps to Chrome extensions.

33.2 27.2 App Categories

Category	Folder	Count	Technologies
Native Apps	apps/	46	C + LVGL (cross-platform)
Desktop Apps	desktop-apps/	1	Electron, Python, C/SDL2
Mobile Apps	mobile-apps/	32	Flutter (Android + iOS)
Web Apps	web-apps/	34	HTML5/JS/WASM PWA
Browser Extensions	browser-extensions/	20	WebExtensions Manifest V3
Dev Tools	dev-tools/	14	VS Code, JetBrains, Vim
CLI Tools	cli-tools/	22	Node.js, Python
Enterprise	enterprise/	16	Docker, Helm, MSI, MDM

33.3 27.3 Native App Architecture

The 46 native apps share a common architecture built on C and LVGL (Light and Versatile Graphics Library):



33.3.1 Building Native Apps

```
cd apps/calculator
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make
./calculator
```

33.3.2 Cross-Compilation

```
# Build for STM32F4
cmake .. -DCMAKE_TOOLCHAIN_FILE=../../toolchains/arm-none-eabi-stm32f4.cmake
make

# Build for Raspberry Pi
cmake .. -DCMAKE_TOOLCHAIN_FILE=../../toolchains/aarch64-linux-gnu.cmake
make
```

33.4 27.4 Featured Native Apps

App	Description
Calculator	Scientific calculator with graphing
File Manager	File browser with preview
Terminal	VT100 terminal emulator
Text Editor	Syntax-highlighted code editor
Settings	System configuration manager
Gallery	Image viewer with thumbnails
Music Player	Audio playback with visualizer
Clock	World clock, timer, stopwatch, alarm

App	Description
Weather	Weather display with forecasts
Camera	Camera capture with filters
Maps	Offline map viewer with GPS
Health	Health metrics dashboard
Home Control	Smart home device controller
System Monitor	CPU, memory, network status

33.5 27.5 Mobile Apps (Flutter)

The 32 mobile apps are built with Flutter for cross-platform deployment:

```
cd mobile-apps/ehealth
flutter pub get
flutter run          # Debug on connected device
flutter build apk    # Android release
flutter build ios     # iOS release
```

Mobile apps communicate with EoS devices over Bluetooth LE using a shared protocol library.

33.6 27.6 Web Apps (PWA)

The 34 web apps are progressive web applications deployable to any browser:

```
cd web-apps/dashboard
npm install
npm run dev      # Development server
npm run build    # Production build
```

Web apps are automatically deployed to GitHub Pages as PWAs with offline support via service workers.

33.7 27.7 Browser Extensions

20 browser extensions using WebExtensions Manifest V3:

```
cd browser-extensions/eos-connect
npm install
npm run build
# Load unpacked extension in Chrome/Firefox
```

33.8 27.8 Developer Tools

Tool	Platform	Description
VS Code Extension	VS Code	EoS syntax, debugging, flashing
JetBrains Plugin	IntelliJ	EoS project support
Vim Plugin	Vim/Neovim	EoS syntax highlighting

33.9 27.9 CLI Tools

22 command-line tools for development and automation:

```
# Install via npm
npm install -g @eos/cli-tools

# Or via pip
pip install eos-cli-tools
```

33.10 27.10 Enterprise Deployment

Enterprise tools support managed deployment:

Format	Description
Docker	Containerized services
Helm Charts	Kubernetes deployment
MSI	Windows managed installer
MDM	Mobile device management profiles

33.11 27.11 Live App Store

The eApps marketplace is hosted as a static GitHub Pages site:

<https://embeddedos-org.github.io/eApps/>

Features: - Categorized app browsing - Platform filtering - One-click download - Version history - User ratings and reviews

33.12 27.12 CI/CD Pipeline

Every app has automated CI/CD:

- **Native apps:** CMake build on Linux, Windows, macOS
 - **Mobile apps:** Flutter build + deploy to TestFlight/Play Store
 - **Web apps:** Build + deploy to GitHub Pages
 - **Extensions:** Package + publish to Chrome/Firefox stores
 - **CLI tools:** npm publish + PyPI publish
-

33.13 27.13 Summary

eApps provides the complete application ecosystem for EoS — from embedded native apps running on microcontrollers to enterprise Docker containers.

Key takeaways:

- 50 apps across 8 platform categories = 188 platform targets
 - Native apps built with C + LVGL for embedded displays
 - Flutter mobile apps for iOS and Android
 - PWA web apps with offline support
 - Automated CI/CD for all platforms
 - Live app store at embeddedos-org.github.io/eApps
-

Next: Chapter 28 — eDB Database

Chapter 34

Chapter 28: eDB — Unified Multi-Model Database

Author: Srikanth Patchava & EmbeddedOS Contributors

34.1 28.1 Introduction

eDB is a unified multi-model database that combines SQL, Document/NoSQL, and Key-Value storage in a single embedded engine. It includes a Python backend (FastAPI + SQLite), a React/TypeScript frontend with SQL editor and AI-powered query assistance, and a standalone browser version.

34.2 28.2 Three Data Models in One

Model	API	Use Case
SQL	db.sql.*	Structured data, relationships
Document	db.docs.*	Flexible JSON documents
Key-Value	db.kv.*	Cache, sessions, config

34.2.1 Usage Example

```
from edb.core.database import Database
from edb.core.models import ColumnDefinition, ColumnType, TableSchema

db = Database("my_app.db")

# SQL
schema = TableSchema(name="users", columns=[
    ColumnDefinition(name="id", col_type=ColumnType.INTEGER, primary_key=True),
```

```

    ColumnDefinition(name="name", col_type=ColumnType.TEXT),
])
db.sql.create_table(schema)
db.sql.insert("users", {"id": 1, "name": "Alice"})

# Documents
db.docs.insert("logs", {"event": "login", "user": "Alice"})

# Key-Value
db.kv.set("session:abc", {"user_id": 1}, ttl=3600)

```

34.3 28.3 Architecture

```

eDB/
  src/
    edb/                                # Python backend
      api/                              # FastAPI routes and dependencies
      auth/                             # JWT, users, RBAC
      ebot/                             # AI/NLP query interface
      query/                            # Query parser and planner
      security/                         # Encryption, audit, input validation
      config.py                         # Pydantic Settings
      cli.py                            # CLI entry point
    components/                         # React UI components
      TopBar.tsx
      TableList.tsx
      TableView.tsx                    # Data grid with inline editing
      QueryEditor.tsx                 # SQL query editor
      EBotSidebar.tsx                 # AI assistant sidebar
    hooks/                             # React hooks
  browser/
    edb.html                          # Standalone browser version
  tests/

```

34.4 28.4 Quick Start

34.4.1 Backend

```

git clone https://github.com/embeddedos-org/eDB.git
cd eDB
pip install -e ".[dev]"
edb init
edb admin create --username admin

```

```
edb serve --port 8000
# API docs at http://localhost:8000/docs
```

34.4.2 Frontend

```
npm install
npm run dev
# React UI at http://localhost:5178
```

34.4.3 Browser Standalone

Open `browser/edb.html` directly in any browser for a zero-install experience with `localStorage` persistence.

34.4.4 Interactive Shell

```
edb shell
# edb> SELECT * FROM users
# edb> .tables
# edb> .collections
```

34.5 28.5 Security Features

Feature	Description
JWT Authentication	Access and refresh tokens
RBAC	Admin, read_write, read_only roles
AES-256 Encryption	Field-level encryption at rest (GCM)
Audit Logging	Tamper-resistant logs with hash chain
Input Sanitization	SQL, NoSQL, and prompt injection defense

34.5.1 Required Environment Variables

Variable	Description
<code>EDB_JWT_SECRET</code>	Secret key for JWT signing
<code>EDB_ENCRYPTION_KEY</code>	Key for AES-256-GCM encryption
<code>EDB_CORS_ORIGINS</code>	Comma-separated allowed origins

34.6 28.6 eBot AI Query Assistant

eBot translates natural language queries into SQL or NoSQL operations:

User: "Show me all users who signed up last month"
eBot: `SELECT * FROM users WHERE created_at >= '2026-03-01'`

Current implementation uses rule-based NLP. The roadmap includes LLM-powered query generation via OpenAI or local models.

34.7 28.7 REST API

eDB exposes a full CRUD REST API via FastAPI:

Endpoint	Method	Description
<code>/api/tables</code>	GET	List all tables
<code>/api/tables</code>	POST	Create a table
<code>/api/tables/{name}</code>	GET	Get table data
<code>/api/tables/{name}</code>	DELETE	Drop a table
<code>/api/query</code>	POST	Execute SQL query
<code>/api/docs/{collection}</code>	POST	Insert document
<code>/api/kv/{key}</code>	GET	Get key-value pair
<code>/api/kv/{key}</code>	PUT	Set key-value pair
<code>/admin/audit</code>	GET	View audit logs

Auto-generated OpenAPI documentation is available at `/docs`.

34.8 28.8 React Frontend

The React frontend provides:

- **Table Navigator** — sidebar listing all tables and collections
- **Data Grid** — inline editing with validation
- **SQL Editor** — syntax-highlighted query editor with results
- **eBot Sidebar** — natural language query interface
- **Status Bar** — connection status, row count, query timing

34.9 28.9 Configuration

Configure via environment variables (prefix `EDB_`) or `.env` file:

```
EDB_DB_PATH=my_data.db
EDB_API_HOST=0.0.0.0
EDB_API_PORT=8000
EDB_JWT_SECRET=your-strong-secret-here
EDB_ENCRYPTION_KEY=your-encryption-key
EDB_CORS_ORIGINS=http://localhost:3000,https://yourdomain.com
```

34.10 28.10 Roadmap

- [x] Core multi-model engine (SQL, Document, KV)
 - [x] JWT authentication and RBAC
 - [x] AES-256 encryption at rest
 - [x] REST API (FastAPI)
 - [x] React frontend with SQL editor
 - [x] Browser standalone version
 - [] LLM-powered eBot
 - [] Graph data model
 - [] Multi-node clustering
 - [] GraphQL and gRPC interfaces
-

34.11 28.11 Summary

eDB provides a zero-dependency embedded database with three data models, enterprise security, and an AI-powered query assistant.

Key takeaways:

- Three data models (SQL, Document, KV) in one SQLite-backed engine
 - JWT auth, RBAC, AES-256 encryption, tamper-resistant audit logs
 - React frontend with inline editing and SQL editor
 - eBot natural language query assistant
 - Zero external database dependencies
 - Browser standalone version for zero-install usage
-

Next: Chapter 29 — eBrowser

Chapter 35

Chapter 29: eBrowser — Lightweight Embedded Web Browser

Author: Srikanth Patchava & EmbeddedOS Contributors

35.1 29.1 Introduction

eBrowser is a lightweight web browser designed for embedded and IoT devices. Built in C/C++ with SDL2 and LVGL, it provides web browsing capability on resource-constrained systems where full browsers like Chromium are impractical.

35.2 29.2 Quick Start

No prerequisites needed. The setup scripts auto-detect your OS and install everything (compiler, CMake, SDL2, LVGL):

```
# Linux / macOS
git clone --recursive https://github.com/embeddedos-org/eBrowser.git
cd eBrowser
chmod +x setup.sh
./setup.sh                # builds and opens the browser
./setup.sh https://example.com  # opens directly to a URL
```

```
:: Windows
git clone --recursive https://github.com/embeddedos-org/eBrowser.git
cd eBrowser
setup.bat
```

The scripts automatically install GCC/Clang, CMake, SDL2, and LVGL v9.2, then build and launch eBrowser.

35.3 29.3 Features

Feature	Description
HTML Rendering	Basic HTML5 parsing and layout
CSS Support	Core CSS properties and selectors
JavaScript	Lightweight JS engine for interactivity
Tab Management	Multiple tabs with tab bar
Bookmarks	Save and manage bookmarks
History	Browsing history with search
Downloads	File download manager
LVGL UI	Native LVGL widgets for browser chrome
Touch Support	Touch-optimized for embedded displays
Keyboard Nav	Full keyboard navigation support

35.4 29.4 Architecture

eBrowser UI (LVGL)
Address bar, tabs, bookmarks bar

Rendering Engine
HTML parser, CSS engine, layout

Network Stack
HTTP/HTTPS, TLS, DNS, caching

Platform Layer
SDL2 (desktop) / EoS HAL (embedded)

35.5 29.5 Building from Source

```
# Standard build
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
./ebrowser

# With custom SDL2 path
cmake .. -DSDL2_DIR=/opt/sdl2
```

```
# Cross-compile for embedded
cmake .. -DCMAKE_TOOLCHAIN_FILE=toolchains/aarch64-linux-gnu.cmake
```

35.6 29.6 Embedded Deployment

For embedded targets, eBrowser uses the EoS display HAL instead of SDL2:

```
cmake .. -DEBROWSER_BACKEND=eos \
  -DCMAKE_TOOLCHAIN_FILE=toolchains/arm-none-eabi-stm32f4.cmake \
  -DEOS_PRODUCT=hmi
```

This produces a firmware image that renders directly to a framebuffer or SPI/parallel display.

35.6.1 Memory Requirements

Configuration	Flash	RAM
Minimal (HTML)	512 KB	256 KB
Standard	1.5 MB	1 MB
Full (JS + CSS)	4 MB	4 MB

35.7 29.7 Configuration

eBrowser is configured via a YAML file or command-line flags:

```
# ebrowser.yaml
display:
  width: 800
  height: 480
  fullscreen: false

network:
  proxy: null
  timeout_seconds: 30
  max_connections: 6

rendering:
  font_size: 16
  dark_mode: false

cache:
  max_size_mb: 50
  location: /tmp/ebrowser_cache
```

35.8 29.8 Platform Support

Platform	Backend	Status
Linux x86_64	SDL2	Stable
Linux ARM64	SDL2	Stable
Windows	SDL2	Stable
macOS	SDL2	Stable
EoS (embedded)	Framebuffer	Stable
Raspberry Pi	SDL2/DRM	Stable

35.9 29.9 Summary

eBrowser brings web browsing to embedded devices where traditional browsers cannot run.

Key takeaways:

- Lightweight C/C++ browser with SDL2 and LVGL
 - Zero-prerequisite setup with auto-installing scripts
 - Cross-platform: desktop and embedded targets
 - Touch-optimized UI for embedded displays
 - Configurable memory footprint (256 KB to 4 MB RAM)
-

Next: Chapter 30 — eOffice Suite

Chapter 36

Chapter 30: eOffice — AI-Powered Office Productivity Suite

Author: Srikanth Patchava & EmbeddedOS Contributors

36.1 30.1 Introduction

eOffice is a complete open-source office productivity suite with built-in AI/LLM intelligence powered by **eBot**. It provides 12 applications covering word processing, spreadsheets, presentations, email, team messaging, and more — running on Windows, macOS, Linux, browsers, and as a PWA.

36.2 30.2 The Twelve Applications

App	Description	Equivalent	AI Features
eDocs	Word processor with rich text	Word	Spell check, grammar, translate
eSheets	Spreadsheet with formulas/charts	Excel	Formula suggestions, data analysis
eSlides	Presentation builder with themes	PowerPoint	Generate slides, talking points
eNotes	Notebooks with markdown	OneNote	Summarize notes, extract tasks
eMail	IMAP/SMTP email client	Outlook	Smart compose, rewrite tone
eDB	Visual database with SQL editor	Access	Generate SQL, explain queries
eDrive	Cloud file storage and sharing	OneDrive	Search files, summarize docs
eConnect	Team messaging with video calls	Teams	Summarize threads, draft messages
eCalendar	Calendar with scheduling	Calendar	Smart scheduling, conflict detect

App	Description	Equivalent	AI Features
eTasks	Task and project management	Planner	Priority suggestions, due dates
eDesign	Vector graphics editor	Designer	Auto-layout, color suggestions
eAdmin	IT administration console	Admin Center	Policy recommendations

36.3 30.3 Quick Start

```
git clone https://github.com/embeddedos-org/eOffice.git
cd eOffice
pip install -e ".[all]"

# Launch the suite
eoffice launch

# Launch a specific app
eoffice launch --app edocs

# Open a document
eoffice open document.edoc
```

36.4 30.4 AI Integration (eBot)

Every eOffice app has a built-in AI sidebar powered by eBot. eBot supports 33+ AI actions across all applications:

36.4.1 Document AI Actions

Action	Description
Spell Check	Fix spelling errors with context
Grammar Fix	Correct grammar and punctuation
Rewrite	Rewrite selected text in different style
Translate	Translate to/from 20+ languages
Summarize	Generate summary of document
Expand	Expand brief notes into full paragraphs

36.4.2 Spreadsheet AI Actions

Action	Description
Formula Suggest	Suggest formula based on description
Explain Formula	Explain what a formula does
Data Analysis	Statistical analysis of selected data
Chart Recommend	Suggest best chart type for data

36.4.3 Email AI Actions

Action	Description
Smart Compose	Auto-complete email drafts
Rewrite Tone	Change tone (formal, casual, friendly)
Summarize Thread	Summarize email conversation
Draft Reply	Generate reply based on context

36.5 30.5 Architecture

```

eoffice/
  apps/
    edocs/      # Word processor
    esheets/    # Spreadsheet
    eslides/    # Presentations
    enotes/     # Notebooks
    email/      # Email client
    econnect/   # Team messaging
    ecalendar/  # Calendar
    etasks/     # Task management
    edesign/    # Vector graphics
    eadmin/     # IT admin
  core/
    ebot/       # AI/LLM engine
    auth/       # Authentication
    storage/    # File storage abstraction
    sync/       # Real-time collaboration
  desktop/     # Electron desktop app
  web/         # React web client
  mobile/      # React Native mobile

```

36.6 30.6 Platform Support

Platform	Technology	Status
Windows 10/11	Electron	Stable
macOS	Electron	Stable
Linux	Electron	Stable
Web Browser	React	Stable
Chrome Extension	Manifest V3	Stable
PWA	Service Worker	Stable

36.7 30.7 File Formats

eOffice uses open file formats:

App	Native Format	Import/Export
eDocs	.edoc	.docx, .pdf, .html, .md, .txt
eSheets	.esheet	.xlsx, .csv, .tsv
eSlides	.eslide	.pptx, .pdf
eNotes	.enote	.md, .html

36.8 30.8 Security

- **End-to-end encryption** for eDrive and eConnect
- **2FA support** for account authentication
- **Role-based access** for team workspaces
- **Audit logging** for compliance
- **Data residency** options for enterprise

36.9 30.9 Testing

```
pip install -e ".[dev]"
pytest -v
pytest --cov=eoffice --cov-report=term-missing
```

75+ test cases covering all 12 applications, AI actions, file format conversion, and security modules.

36.10 30.10 Summary

eOffice provides a complete, AI-enhanced office suite as an open-source alternative to proprietary solutions.

Key takeaways:

- 12 office applications covering all productivity needs
- 33+ AI actions powered by eBot (local or cloud LLM)
- Cross-platform: desktop, web, mobile, PWA, Chrome extension
- Open file formats with import/export for Office formats
- End-to-end encryption and enterprise security

Next: Chapter 31 — eVera AI Assistant

Chapter 37

Chapter 31: eVera — Voice-First Multi-Agent AI Assistant

Author: Srikanth Patchava & EmbeddedOS Contributors

37.1 31.1 Introduction

eVera is a voice-first, multi-agent AI assistant that serves as the primary user interface for the EoS ecosystem. With **24+ specialized agents** and **183+ tools**, eVera can control devices, automate workflows, answer questions, and manage entire systems through natural language.

eVera runs on Windows, macOS, Linux, Android, and iOS.

37.2 31.2 Multi-Agent Architecture

eVera uses a multi-agent design where specialized agents handle different domains. A central orchestrator routes user requests to the appropriate agent based on intent classification.

37.2.1 Agent Categories

Category	Agents
System	Device control, settings, diagnostics
Productivity	Calendar, email, tasks, notes
Development	Code generation, debugging, deployment
Data	Database queries, analytics, visualization
Communication	Messaging, calls, notifications
IoT/Home	Smart home, sensors, automation rules
Creative	Image generation, music, design
Research	Web search, summarization, fact-checking

37.3 31.3 Voice Interface

eVera is voice-first by design:

User (voice): "Hey eVera, turn on the living room lights
and set temperature to 72"

eVera:

1. Speech-to-text (Whisper / on-device)
2. Intent classification → IoT agent
3. Tool call: `smart_home.set_device("lights", "living_room", "on")`
4. Tool call: `smart_home.set_temperature("living_room", 72)`
5. Text-to-speech response: "Done. Lights are on and
thermostat set to 72 degrees."

37.3.1 Speech Backends

Backend	Type	Latency	Privacy
Whisper	On-device	Low	Full
Google STT	Cloud	Medium	Partial
Azure Speech	Cloud	Medium	Partial

37.4 31.4 Tool System

eVera exposes 183+ tools that agents can call:

Tool Category	Count	Examples
File System	15	read, write, search, compress
Network	12	HTTP, MQTT, WebSocket, ping
Database	10	SQL query, document ops, KV ops
Smart Home	20	Lights, thermostat, locks, cameras
Calendar	8	Create event, check availability
Email	10	Send, read, search, draft
Code	15	Generate, review, test, deploy
System	12	Reboot, update, monitor, backup
Media	10	Play music, capture photo, record
Search	8	Web, local files, knowledge base
Math/Science	10	Calculate, convert, plot

37.5 31.5 Quick Start

```
git clone https://github.com/embeddedos-org/eVera.git
cd eVera
pip install -e ".[all]"

# Start eVera
evera start

# Start with voice mode
evera start --voice

# Ask a question
evera ask "What is the CPU temperature?"

# Run an automation
evera run automation/morning_routine.yaml
```

37.6 31.6 Automation Rules

eVera supports YAML-based automation rules:

```
name: morning_routine
trigger:
  time: "07:00"
  days: [mon, tue, wed, thu, fri]
actions:
  - agent: iot
    tool: smart_home.set_device
    args: { device: "coffee_maker", action: "on" }
  - agent: iot
    tool: smart_home.set_temperature
    args: { zone: "house", temp: 72 }
  - agent: productivity
    tool: calendar.get_today
    output: today_events
  - agent: communication
    tool: tts.speak
    args: { text: "Good morning. You have {{ today_events.count }} events today." }
```

37.7 31.7 Platform Support

Platform	Interface	Backend
Windows	Electron desktop	Python server
macOS	Electron desktop	Python server
Linux	Electron desktop	Python server
Android	React Native	On-device
iOS	React Native	On-device
Web	React webapp	Cloud/self-host
EoS (embedded)	LVGL + voice	On-device (eAI)

37.8 31.8 Security

- **Hardened by default** — input sanitization, rate limiting
- **Permission system** — tools require explicit user permission
- **Conversation encryption** — all data encrypted at rest
- **On-device option** — full local processing with Ollama/eAI
- **Audit trail** — all agent actions logged

37.9 31.9 LLM Backends

Backend	Type	Models
Ollama	Local	Llama 3, Mistral, Phi
OpenAI	Cloud	GPT-4o, GPT-4
eAI	Embedded	Quantized on-device models
Anthropic	Cloud	Claude

37.10 31.10 Summary

eVera is the conversational AI layer that makes the entire EoS ecosystem accessible through natural language.

Key takeaways:

- 24+ specialized agents covering all user-facing domains
- 183+ tools for device control, automation, and productivity
- Voice-first with on-device speech processing
- YAML-based automation rules
- Cross-platform: desktop, mobile, web, embedded
- Multiple LLM backends (local and cloud)

Next: Chapter 32 — eStocks Trading System

Chapter 38

Chapter 32: eStocks — Algorithmic Trading System

Author: Srikanth Patchava & EmbeddedOS Contributors

38.1 32.1 Introduction

eStocks is a comprehensive algorithmic trading system with **15 strategies**, **7 data sources**, **7-layer risk management**, and full production safety controls. It includes 288+ tests, thread-safe state persistence, and crash-recoverable operation.

38.2 32.2 Quick Start

```
# Setup (installs deps, validates system, tests connectivity)
python setup_trading.py

# Paper trade with real Yahoo Finance data (no broker needed)
python paper_trader.py --symbols AAPL,MSFT,GOOGL --strategy meta_ensemble

# Scan 15 stocks with all strategies
python paper_trader.py --scan-universe

# Run tests
python -m pytest tests/ -v
```

38.3 32.3 The 15 Trading Strategies

#	Strategy	Based On
1	Trend Following	EMA crossover + ADX + trailing stop
2	Breakout	Donchian channel breakout
3	Mean Reversion	RSI + Bollinger Bands
4	Factor	12-1 month momentum long/short
5	Darvas Box	Darvas box breakout
6	Triple Screen	Elder triple screen system
7	CAN SLIM	O'Neil CAN SLIM 7-criteria
8	Value	Graham fundamental value
9	ML (LSTM)	LSTM deep learning
10	RL (PPO)	PPO reinforcement learning
11	Self-Learning	Adaptive ML ensemble
12	Sentiment	News sentiment + technicals
13	Earnings	Earnings calendar trading
14	Sector Rotation	Sector ETF momentum
15	Meta Ensemble	All sources combined

All 15 strategies use all 7 data sources: Price, Volume, Fundamentals, News, Earnings, ML predictions, and Market Regime detection.

38.4 32.4 Seven-Layer Risk Management

Layer 7: PORTFOLIO HEAT	Max 20% equity at risk
Layer 6: POSITION CAP	Max 25% equity / 10K shares per position
Layer 5: CIRCUIT BREAKER	10% drawdown -> 24h pause
Layer 4: MONTHLY CAP	Elder 6% monthly loss limit
Layer 3: DAILY LIMIT	\$5,000/day hard stop
Layer 2: COOLDOWN	30-min pause after 3 consecutive losses
Layer 1: PER-TRADE RISK	2% of equity per trade

38.5 32.5 Production Safety Controls

Control	Threshold
Fat-finger protection	10K shares max
Price deviation	+/-10% limit
Short limits	5 positions / 30%
Liquidity filter	50K min daily volume
Market hours	Enforced
State persistence	SQLite WAL (thread-safe)

38.6 32.6 Platform Support

Platform	Language	Broker
TradingView	Pine Script	IB, TradeStation
thinkorswim	thinkScript	Charles Schwab
Interactive Brokers	Python (TWS)	Interactive Brokers
TradeStation	EasyLanguage	TradeStation

38.7 32.7 Architecture

```
stocks_plugin/  
  shared/  
    risk_manager.py           # 7-layer risk engine  
    strategy_enricher.py      # Multi-source data enrichment  
    trade_journal.py          # Psychology/discipline journal  
    data/public_data_fetcher.py # OHLCV, fundamentals, news  
    indicators/               # 35+ indicators, 14 patterns  
    backtesting/              # Multi-asset backtester  
    ml/                       # Sentiment, regime, LSTM, RL  
    strategies/examples/      # 15 registered strategies  
    tests/                   # 288+ tests  
    setup_trading.py          # One-command setup  
    paper_trader.py           # Paper trading simulator
```

38.8 32.8 Backtesting

The backtesting engine supports multi-asset testing with R-multiples and System Quality Number (SQN) analysis:

```
python -m stocks_plugin.shared.backtesting.runner \  
  --strategy trend_following \  
  --symbols AAPL,MSFT,GOOGL,AMZN \  
  --start 2020-01-01 \  
  --end 2025-12-31 \  
  --initial-capital 100000
```

Output includes: - Total return and CAGR - Maximum drawdown - Sharpe ratio and Sortino ratio
- Win rate and R-multiple distribution - SQN score

38.9 32.9 ML and AI Strategies

38.9.1 LSTM Deep Learning (Strategy 9)

Uses Long Short-Term Memory networks trained on historical price data to predict next-day price movements.

38.9.2 PPO Reinforcement Learning (Strategy 10)

Uses Proximal Policy Optimization to learn optimal trading policies through simulated market interaction.

38.9.3 Self-Learning Ensemble (Strategy 11)

An adaptive system that continuously retrains its model ensemble based on recent market conditions.

38.9.4 Sentiment Analysis (Strategy 12)

Analyzes news headlines and social media sentiment using NLP to generate trading signals.

38.10 32.10 CI/CD

- **Tests:** Python 3.10/3.11/3.12 matrix
 - **Security:** Bandit scan, hardcoded secrets check
 - **Strategy validation:** Verifies all 15 strategies register correctly
 - **Release:** Tag `v*` triggers automated GitHub release
-

38.11 32.11 Summary

eStocks provides a production-grade algorithmic trading system with comprehensive risk management and multiple strategy paradigms.

Key takeaways:

- 15 trading strategies spanning classical and ML approaches
 - 7-layer risk management with production safety controls
 - Paper trading simulator requiring no broker account
 - Multi-platform: TradingView, thinkorswim, IB, TradeStation
 - 288+ tests with thread-safe crash recovery
 - Backtesting with R-multiple and SQN analysis
-

Next: Chapter 33 — Cross-Compilation

Chapter 39

Chapter 33: Cross-Compilation for EoS

Author: Srikanth Patchava & EmbeddedOS Contributors

39.1 33.1 Introduction

EoS supports cross-compilation for multiple architectures from a single development host. This chapter covers the toolchain infrastructure, CMake integration, and practical workflows for targeting ARM, RISC-V, and other architectures.

39.2 33.2 Supported Toolchains

Toolchain File	Architecture	Targets
<code>arm-none-eabi-stm32f4.cmake</code>	ARM Cortex-M4	STM32F4 Discovery
<code>arm-none-eabi-r5.cmake</code>	ARM Cortex-R5	Safety MCUs (TMS570)
<code>arm-linux-gnueabihf.cmake</code>	ARM Cortex-A	RPi (32-bit), BeagleBone
<code>aaarch64-linux-gnu.cmake</code>	AArch64	RPi 4/5, Jetson, i.MX8
<code>riscv64-linux-gnu.cmake</code>	RISC-V 64	SiFive, StarFive

39.3 33.3 Toolchain File Anatomy

A CMake toolchain file sets compiler paths, CPU flags, and sysroot locations. Here is the STM32F4 toolchain:


```

set(CMAKE_SYSTEM_NAME Generic)
set(CMAKE_SYSTEM_PROCESSOR cortex-m4)

set(CMAKE_C_COMPILER arm-none-eabi-gcc)
set(CMAKE_CXX_COMPILER arm-none-eabi-g++)
set(CMAKE_ASM_COMPILER arm-none-eabi-gcc)
set(CMAKE_OBJCOPY arm-none-eabi-objcopy)

set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY)

# Cortex-M4 with hardware FPU
set(CPU_FLAGS "-mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpu=fpv4-sp-d16")

set(CMAKE_C_FLAGS_INIT
    "${CPU_FLAGS} -fdata-sections -ffunction-sections -fno-common -fstack-protector-strong"
)

set(CMAKE_EXE_LINKER_FLAGS_INIT
    "${CPU_FLAGS} -Wl,--gc-sections -specs=nano.specs -specs=nosys.specs -lnosys"
)

set(CMAKE_CROSSCOMPILING TRUE)
set(EOS_PLATFORM "rtos" CACHE STRING "Target platform" FORCE)

```

39.3.1 Key Elements

Setting	Purpose
CMAKE_SYSTEM_NAME Generic	Indicates bare-metal (no OS)
CPU_FLAGS	Architecture-specific compiler flags
-ffunction-sections	Enable linker garbage collection
-specs=nano.specs	Use newlib-nano for smaller footprint
-fstack-protector-strong	Stack buffer overflow detection
EOS_PLATFORM "rtos"	Selects EoS RTOS configuration

39.4 33.4 AArch64 Linux Toolchain

For application processors running Linux:

```

set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR aarch64)

set(CMAKE_C_COMPILER aarch64-linux-gnu-gcc)
set(CMAKE_CXX_COMPILER aarch64-linux-gnu-g++)

```

```

set(CMAKE_C_FLAGS_INIT      "-ffunction-sections -fdata-sections")
set(CMAKE_EXE_LINKER_FLAGS_INIT "-Wl,--gc-sections")

set(CMAKE_FIND_ROOT_PATH /usr/aarch64-linux-gnu)
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)

```

Key difference from bare-metal: CMAKE_SYSTEM_NAME is Linux and the sysroot is set to the cross-compilation environment.

39.5 33.5 RISC-V Toolchain

```

set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR riscv64)

set(CMAKE_C_COMPILER      riscv64-linux-gnu-gcc)
set(CMAKE_C_FLAGS_INIT "-march=rv64gc -mabi=lp64d -ffunction-sections -fdata-sections")

```

The `-march=rv64gc` flag enables the general-purpose RISC-V ISA with compressed instructions. The `-mabi=lp64d` flag uses the LP64 ABI with hardware double-precision float.

39.6 33.6 Cross-Compilation Workflow

39.6.1 Step 1: Install the Toolchain

```

# ARM bare-metal
sudo apt install gcc-arm-none-eabi

# AArch64 Linux
sudo apt install gcc-aarch64-linux-gnu

# RISC-V
sudo apt install gcc-riscv64-linux-gnu

```

39.6.2 Step 2: Configure with Toolchain File

```

cmake -B build-stm32 \
  -DCMAKE_TOOLCHAIN_FILE=toolchains/arm-none-eabi-stm32f4.cmake \
  -DEOS_PRODUCT=iot

```

39.6.3 Step 3: Build

```
cmake --build build-stm32
```

39.6.4 Step 4: Flash

```
# Using OpenOCD
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg \
  -c "program build-stm32/eos_firmware.bin 0x08000000 verify reset exit"

# Using ebuild
ebuild flash --board stm32f4 --image build-stm32/eos_firmware.bin
```

39.7 33.7 Multi-Architecture CI

The EoS CI pipeline cross-compiles for all supported architectures:

```
strategy:
  matrix:
    include:
      - arch: arm-cortex-m4
        toolchain: arm-none-eabi-stm32f4.cmake
        product: iot
      - arch: aarch64
        toolchain: aarch64-linux-gnu.cmake
        product: gateway
      - arch: riscv64
        toolchain: riscv64-linux-gnu.cmake
        product: robot
```

39.8 33.8 Linker Scripts

Each target requires a linker script defining the memory layout:

```
/* STM32F407 Memory Layout */
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 1024K
    SRAM (rwx) : ORIGIN = 0x20000000, LENGTH = 192K
    CCM (rwx) : ORIGIN = 0x10000000, LENGTH = 64K
}

SECTIONS
{
```

```

.text : {
    KEEP(*(.isr_vector))
    *(.text*)
    *(.rodata*)
} > FLASH

.data : {
    _sdata = .;
    *(.data*)
    _edata = .;
} > SRAM AT > FLASH

.bss : {
    _sbss = .;
    *(.bss*)
    *(COMMON)
    _ebss = .;
} > SRAM

_estack = ORIGIN(SRAM) + LENGTH(SRAM);
}

```

39.9 33.9 Binary Size Optimization

Technique	Savings	Flag/Setting
-Os optimization	15-30%	CMAKE_C_FLAGS_RELEASE "-Os"
Section garbage collection	10-20%	-ffunction-sections + -Wl,--gc-sections
Newlib-nano	50-80 KB	-specs=nano.specs
Link-time optimization	5-15%	-flto
Strip debug symbols	20-50%	arm-none-eabi-strip

39.10 33.10 Debugging Cross-Compiled Firmware

```

# Start GDB server
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg

# Connect GDB
arm-none-eabi-gdb build-stm32/eos_firmware.elf
(gdb) target remote :3333
(gdb) monitor reset halt
(gdb) load

```

```
(gdb) break main
(gdb) continue
```

39.11 33.11 Summary

EoS provides a clean, CMake-based cross-compilation infrastructure that supports bare-metal MCUs, Linux application processors, and RISC-V targets.

Key takeaways:

- CMake toolchain files for ARM Cortex-M/A/R, AArch64, and RISC-V
 - Bare-metal and Linux cross-compilation from a single build system
 - Linker scripts define memory layout per target
 - Binary size optimization techniques for constrained devices
 - Multi-architecture CI ensures all targets compile cleanly
-

Next: Chapter 34 — Testing and Quality

Chapter 40

Chapter 34: Testing and Quality Assurance

Author: Srikanth Patchava & EmbeddedOS Contributors

40.1 34.1 Introduction

Quality assurance is non-negotiable in embedded systems. A bug in a medical device or automotive controller can have life-threatening consequences. EoS implements a comprehensive testing strategy spanning unit tests, integration tests, fuzz testing, and CI validation across all supported platforms.

40.2 34.2 Test Infrastructure

The EoS test suite is located in the `tests/` directory:

Test File	Coverage Area
<code>test_hal.c</code>	Hardware Abstraction Layer
<code>test_kernel.c</code>	RTOS kernel (tasks, mutexes, queues)
<code>test_drivers.c</code>	Driver framework
<code>test_crypto.c</code>	Cryptographic services
<code>test_crypto_aes.c</code>	AES encryption
<code>test_crypto_ecc.c</code>	Elliptic curve cryptography
<code>test_crypto_rsa.c</code>	RSA operations
<code>test_crypto_sha512.c</code>	SHA-512 hashing
<code>test_filesystem.c</code>	Filesystem operations
<code>test_firmware.c</code>	Firmware management
<code>test_net.c</code>	Networking stack
<code>test_security.c</code>	Security services
<code>test_sensor.c</code>	Sensor subsystem
<code>test_motor_ctrl.c</code>	Motor control

Test File	Coverage Area
test_multicore.c	SMP/AMP multicore
test_power.c	Power management
test_ota.c	Over-the-air updates
test_os_services.c	OS-level services
test_config.c	Configuration system
test_profiles.c	Product profiles
test_driver_recovery.c	Driver fault recovery
test_ui_gesture.c	UI gesture recognition
test_graph.c	Graph data structures
test_board_configs.c	Board configuration validation
test_service.c	Service framework
test_package.c	Package management
test_debug.c	Debug facilities

40.3 34.3 Test Framework

EoS uses a lightweight, custom test framework designed for portability:

```
#define TEST(name) \
    static void name(void); \
    static void run_##name(void) { \
        printf(" %-50s ", #name); \
        name(); \
        tests_passed++; \
        printf("[PASS]\n"); \
    } \
    static void name(void)

#define ASSERT(cond) do { \
    if (!(cond)) { \
        printf("[FAIL] %s:%d: %s\n", __FILE__, __LINE__, #cond); \
        exit(1); \
    } \
} while(0)
```

This framework compiles on any C11 compiler with zero dependencies, making it suitable for both host and cross-compiled testing.

40.4 34.4 HAL Testing with Mock Backends

The HAL tests use mock backends that simulate hardware behavior:

```
static const eos_hal_backend_t mock_backend = {
    .name = "mock",
    .init = mock_init,
    .deinit = mock_deinit,
    .delay_ms = mock_delay,
    .get_tick_ms = mock_get_tick,
    .gpio_init = mock_gpio_init,
    .gpio_write = mock_gpio_write,
    .gpio_read = mock_gpio_read,
    .gpio_toggle = mock_gpio_toggle,
    .uart_init = mock_uart_init,
    .uart_write = mock_uart_write,
    .spi_init = mock_spi_init,
    .spi_transfer = mock_spi_transfer,
    .i2c_init = mock_i2c_init,
    .i2c_write = mock_i2c_write,
    .i2c_read = mock_i2c_read,
    .timer_init = mock_timer_init,
    .timer_start = mock_timer_start,
    .irq_disable = mock_irq_disable,
    .irq_enable = mock_irq_enable,
};
```

This enables full HAL testing on the host machine without hardware.

40.5 34.5 Kernel Testing

Kernel tests validate the RTOS primitives:

```
static void test_task_create(void) {
    eos_kernel_init();
    int h = eos_task_create("t1", test_entry, NULL, 5, 1024);
    assert(h >= 0);
    assert(eos_task_get_state(h) == EOS_TASK_READY);
    assert(strcmp(eos_task_get_name(h), "t1") == 0);
}

static void test_mutex(void) {
    eos_kernel_init();
    eos_mutex_handle_t m;
    assert(eos_mutex_create(&m) == EOS_KERN_OK);
    assert(eos_mutex_lock(m, 0) == EOS_KERN_OK);
    assert(eos_mutex_lock(m, 0) == EOS_KERN_OK); // recursive
    assert(eos_mutex_unlock(m) == EOS_KERN_OK);
    assert(eos_mutex_delete(m) == EOS_KERN_OK);
}
```



```

static void test_semaphore(void) {
    eos_kernel_init();
    eos_sem_handle_t s;
    assert(eos_sem_create(&s, 3, 5) == EOS_KERN_OK);
    assert(eos_sem_get_count(s) == 3);
    assert(eos_sem_wait(s, 0) == EOS_KERN_OK);
    assert(eos_sem_get_count(s) == 2);
}

static void test_queue(void) {
    eos_kernel_init();
    eos_queue_handle_t q;
    assert(eos_queue_create(&q, sizeof(int), 4) == EOS_KERN_OK);
    int val = 42;
    assert(eos_queue_send(q, &val, 0) == EOS_KERN_OK);
    int out = 0;
    assert(eos_queue_receive(q, &out, 0) == EOS_KERN_OK);
    assert(out == 42);
}

```

40.6 34.6 Fuzz Testing

The `tests/fuzz/` directory contains fuzz test harnesses for security- critical components:

- Crypto input fuzzing
- Network packet parsing
- Configuration file parsing
- Filesystem operation fuzzing

Fuzz tests use libFuzzer or AFL and run in the nightly CI pipeline.

40.7 34.7 Running Tests

```

# Build with tests enabled
cmake -B build -DEOS_BUILD_TESTS=ON
cmake --build build

# Run all tests
ctest --test-dir build --output-on-failure

# Run specific test
./build/test_kernel

```

```
# Run with verbose output
ctest --test-dir build -V
```

40.8 34.8 CI Pipeline

All pull requests must pass:

Check	Platform	What It Validates
Build (ubuntu-latest)	Linux	GCC build with all libraries
Build (windows-latest)	Windows	MSVC cross-platform check
Build (macos-latest)	macOS	Clang build verification
Product (robot)	Linux	Robot profile compiles
Product (gateway)	Linux	Gateway profile compiles
Product (medical)	Linux	Medical profile compiles
Product (automotive)	Linux	Automotive profile compiles
Product (iot)	Linux	IoT profile compiles
Product (aerospace)	Linux	Aerospace profile compiles

40.8.1 Nightly Regression

Additional nightly checks include: - Full test suite on all 3 OS platforms - All product profile builds
- Cross-compilation for AArch64, ARM hard-float, and RISC-V

40.9 34.9 Code Quality Standards

Standard	Requirement
C Standard	C11 (<code>-std=c11</code>)
Warnings	<code>-Wall -Wextra</code> with zero warnings
Platform guards	All OS-specific code guarded with <code>#ifdef</code>
Naming	<code>snake_case</code> with <code>eos_</code> prefix
Documentation	Doxygen-compatible <code>/** */</code> doc comments
Types	<code><stdint.h></code> types (<code>uint32_t</code> , etc.)

40.10 34.10 Test Coverage Goals

Module	Target Coverage	Current
HAL	90%	85%
Kernel	95%	92%

Module	Target Coverage	Current
Crypto	95%	90%
Networking	85%	80%
Drivers	80%	75%
Services	85%	80%

40.11 34.11 Summary

EoS implements a rigorous testing strategy appropriate for safety-critical embedded systems.

Key takeaways:

- 27 test files covering HAL, kernel, crypto, networking, and more
- Mock-based HAL testing enables host-machine validation
- Fuzz testing for security-critical components
- CI validates builds on Linux, Windows, macOS
- Product profile builds verified for 6 safety-critical profiles
- Nightly cross-compilation regression for all architectures

Next: Chapter 35 — Safety and Compliance

Chapter 41

Chapter 35: Safety and Compliance

Author: Srikanth Patchava & EmbeddedOS Contributors

41.1 35.1 Introduction

EoS targets safety-critical domains including automotive, aerospace, medical, and industrial systems. This chapter covers the safety documentation, compliance frameworks, and engineering practices that enable EoS to meet international safety standards.

41.2 35.2 Applicable Standards

Standard	Domain	Key Requirements
IEC 61508	Industrial	SIL 1-4 functional safety
ISO 26262	Automotive	ASIL A-D functional safety
DO-178C	Aerospace	DAL A-E software assurance
IEC 62304	Medical devices	Software lifecycle for medical SW
ISO 15288	Systems engineering	System lifecycle processes
ISO 25000	Software quality	SQuaRE quality model
ISO 27001	Information security	Security management system
FIPS 140-3	Cryptography	Cryptographic module validation

41.3 35.3 IEC 61508 — Industrial Functional Safety

EoS provides documentation for IEC 61508 compliance in the `docs/safety/iec61508/` directory:

Document	Purpose
overview.md	IEC 61508 applicability analysis
sil_checklist.md	SIL determination checklist
software_safety_manual.md	Software safety manual
validation_plan.md	Validation plan and procedures

41.3.1 SIL Levels and EoS

SIL Level	Target Failure Rate	EoS Support
SIL 1	$< 10^{-5}$ /hr	Full (standard configuration)
SIL 2	$< 10^{-6}$ /hr	Full (with safety profile)
SIL 3	$< 10^{-7}$ /hr	Partial (requires certification)
SIL 4	$< 10^{-8}$ /hr	Not targeted

41.4 35.4 ISO 26262 — Automotive Functional Safety

Automotive safety documentation in `docs/safety/iso26262/`:

Document	Purpose
overview.md	ISO 26262 applicability
safety_plan.md	Safety plan
hazard_analysis.md	Hazard analysis and risk assessment
safety_requirements.md	Safety requirements specification
software_architecture.md	Software architectural design
tool_classification.md	Development tool classification
verification_plan.md	Verification plan

41.4.1 ASIL Levels and EoS Product Profiles

ASIL Level	Products	EoS Profile
ASIL A	Infotainment	infotainment
ASIL B	Body electronics	automotive
ASIL C	Chassis systems	automotive
ASIL D	Powertrain, ADAS	cockpit

41.5 35.5 DO-178C — Aerospace Software Assurance

Aerospace documentation in `docs/safety/do178c/`:

Document	Purpose
overview.md	DO-178C applicability
software_development_plan.md	Software development plan
software_requirements.md	Software requirements
software_design.md	Software design documentation
verification_results.md	Verification results summary
configuration_management.md	Configuration management plan
plan_for_software_aspects.md	Plan for Software Aspects of Cert.

41.5.1 Design Assurance Levels

DAL	Failure Condition	EoS Profile
DAL A	Catastrophic	aerospace
DAL B	Hazardous/Severe-Major	aerospace
DAL C	Major	drone
DAL D	Minor	ground_control
DAL E	No Effect	Any profile

41.6 35.6 Safety-Critical Engineering Practices

41.6.1 Memory Safety

- Stack overflow detection via `-fstack-protector-strong`
- Static stack analysis with compiler warnings
- No dynamic memory allocation in safety-critical paths
- Memory protection unit (MPU) configuration for task isolation

41.6.2 Deterministic Behavior

- Worst-case execution time (WCET) analysis for critical tasks
- Bounded interrupt latency
- No unbounded loops in interrupt handlers
- Priority ceiling protocol for mutex operations

41.6.3 Defensive Programming

```
// All public APIs validate inputs
int eos_gpio_init(const eos_gpio_config_t *cfg) {
    if (!cfg) return EOS_ERR_INVALID;
    if (cfg->pin >= EOS_MAX_GPIO_PINS) return EOS_ERR_INVALID;
    // ...
}
```

41.6.4 Redundancy

- Watchdog timer for system health monitoring
 - A/B firmware slots for safe rollback (via eBoot)
 - CRC/SHA-256 integrity checks on critical data
 - Redundant sensor readings with voting
-

41.7 35.7 Coding Standards

For safety-critical profiles, EoS follows:

Standard	Application
MISRA C:2012	Automotive and industrial code
CERT C	Secure coding guidelines
CWE Top 25	Common weakness enumeration avoidance

41.7.1 Key Rules

- No `malloc()`/`free()` in safety-critical code
 - No recursion in interrupt context
 - All switch statements have `default` cases
 - All function return values are checked
 - No implicit type conversions that lose precision
-

41.8 35.8 Traceability

EoS maintains bidirectional traceability:

Requirements --> Design --> Implementation --> Tests --> Verification

Each safety requirement is traced to: 1. The design element that addresses it 2. The source code that implements it 3. The test case that verifies it 4. The verification evidence that confirms it

41.9 35.9 Configuration Management

- Git-based version control with signed commits
 - Tagged releases with semantic versioning
 - SBOM generation (SPDX and CycloneDX formats)
 - Change impact analysis for safety-relevant modifications
-

41.10 35.10 Summary

EoS provides the documentation and engineering practices needed to pursue certification under multiple international safety standards.

Key takeaways:

- Documentation for IEC 61508, ISO 26262, and DO-178C
- Product profiles mapped to safety integrity levels
- Memory safety, determinism, and defensive programming practices
- MISRA C:2012 and CERT C coding standards
- Bidirectional requirements traceability
- SBOM generation for supply chain compliance

Next: Chapter 36 — Contributing to EoS

Chapter 42

Chapter 36: Contributing to EoS

Author: Srikanth Patchava & EmbeddedOS Contributors

42.1 36.1 Introduction

EoS is an open-source project that welcomes contributions from the embedded systems community. This chapter covers everything you need to know to contribute effectively — from setting up your development environment to submitting your first pull request.

42.2 36.2 Prerequisites

Tool	Minimum Version	Notes
CMake	3.15+	Build system generator
GCC	10+	Or Clang 10+, or MSVC 2019+
Git	2.25+	Version control
Python	3.8+	Test scripts and tooling
Doxygen	1.9+	Optional — API documentation

42.3 36.3 Development Setup

```
git clone https://github.com/embeddedos-org/eos.git
cd eos
cmake -B build -DEOS_BUILD_TESTS=ON
cmake --build build
ctest --test-dir build --output-on-failure
```

To build a specific product profile:

```
cmake -B build -DEOS_PRODUCT=robot
cmake --build build
```

42.4 36.4 Contribution Workflow

1. **Fork** the repository on GitHub
 2. **Create a feature branch:** `git checkout -b feat/my-feature`
 3. **Make your changes** following the code guidelines
 4. **Build and test** locally on at least one OS
 5. **Commit** with conventional commit messages
 6. **Submit a pull request** against the `master` branch
-

42.5 36.5 Code Guidelines

42.5.1 C Style

- **Standard:** C11 (`-std=c11`)
- **Warnings:** `-Wall -Wextra` must compile clean (zero warnings)
- **Naming:** `snake_case` for all functions, types, and variables
- **Prefix:** All public symbols use the `eos_` prefix
- **Documentation:** Doxygen-compatible `/** */` doc comments
- **Include guards:** `#ifndef HEADER_NAME_H / #define / #endif`
- **Types:** Use `<stdint.h>` types (`uint32_t`, `int8_t`, etc.)

42.5.2 Platform Guards

All platform-specific code must be guarded:

```
#ifdef _WIN32
    // Windows-specific code
#elif defined(__APPLE__)
    // macOS-specific code
#else
    // Linux/POSIX code
#endif
```

42.5.3 Portability Rules

- No `__builtin_*` without MSVC fallback
 - No hardcoded Unix paths — use `getenv("TEMP")` on Windows
 - Always `#include <stddef.h>` when using `size_t`
 - Always `#include <stdlib.h>` when using `getenv()`, `malloc()`
-

42.6 36.6 Commit Messages

Follow Conventional Commits:

```
feat: add CAN bus driver for automotive profile
fix: datacenter popcount portability for MSVC
docs: add quickstart guide for nRF52
ci: add aerospace product build to CI matrix
chore: bump version to 0.6.0
```

42.7 36.7 CI Requirements

All pull requests must pass these checks before merging:

Check	What It Validates
Build (Linux)	GCC build with all libraries
Build (Windows)	MSVC cross-platform portability
Build (macOS)	Clang build verification
Product profiles (6x)	Robot, gateway, medical, automotive, iot, aerospace

42.7.1 How to Verify Locally

```
# Full build with tests
cmake -B build -DEOS_BUILD_TESTS=ON
cmake --build build --config Release
ctest --test-dir build --output-on-failure -C Release

# Product profiles
cmake -B build-robot -DEOS_PRODUCT=robot && cmake --build build-robot
cmake -B build-medical -DEOS_PRODUCT=medical && cmake --build build-medical
```

42.8 36.8 Adding New Features

42.8.1 New HAL Peripheral

1. Add type declarations to `hal/include/eos/hal_extended.h`
2. Add stub implementations to `hal/src/hal_extended_stubs.c`
3. Guard with `#if EOS_ENABLE_*` preprocessor block
4. Add tests to `tests/test_hal.c`

42.8.2 New Product Profile

1. Create `products/<name>.h` with `EOS_ENABLE_*` flags
2. Add `#elif` case to `include/eos/eos_config.h`

3. Add to CI matrix in `.github/workflows/ci.yml`
4. Test: `cmake -B build -DEOS_PRODUCT=<name> && cmake --build build`

42.8.3 New Service

1. Create directory under `services/<name>/`
 2. Add `CMakeLists.txt` with proper `target_link_libraries`
 3. Add header to `include/eos/`
 4. Add tests to `tests/test_<name>.c`
-

42.9 36.9 Pull Request Checklist

- ☐ Code compiles with zero warnings on GCC, Clang, and MSVC
 - ☐ All existing tests pass
 - ☐ New HAL peripherals include stubs
 - ☐ New services include a `CMakeLists.txt`
 - ☐ Platform-specific code has `#ifdef` guards
 - ☐ No GCC-only builtins without portable fallbacks
 - ☐ No hardcoded filesystem paths
 - ☐ Commit messages follow conventional commits format
-

42.10 36.10 Project Structure

```

eos/
  hal/           # Hardware Abstraction Layer (33 peripherals)
  kernel/        # RTOS kernel + multicore (SMP/AMP)
  core/          # OS core, config, logging
  include/eos/   # Global headers
  products/      # 41+ product profile headers
  services/      # Crypto, security, OTA, sensor, motor, filesystem
  drivers/       # Driver framework
  power/         # Power management
  net/           # Networking (sockets, HTTP, MQTT, mDNS)
  systems/       # Firmware assembly, rootfs
  toolchains/    # Cross-compilation configs
  tests/         # Unit test suites
  examples/      # Example applications
  docs/         # Documentation

```

42.11 36.11 Reporting Issues

Use GitHub Issues with appropriate labels:

- **bug** — something is broken
- **enhancement** — feature request
- **question** — need clarification

Include: OS, compiler version, product profile, and full error output. For build failures, attach the full CMake configure and build log.

42.12 36.12 License

By contributing, you agree that your contributions will be licensed under the MIT License.

42.13 36.13 Summary

Contributing to EoS follows established open-source practices with additional emphasis on cross-platform portability and safety-critical code quality.

Key takeaways:

- C11 with `-Wall -Wextra` zero-warning policy
 - All code must work on GCC, Clang, and MSVC
 - Platform-specific code requires `#ifdef` guards
 - Conventional commits for clear change history
 - CI validates builds on 3 OS platforms and 6 product profiles
 - New features need tests, stubs, and documentation
-

End of Part 7

Chapter 43

Appendix A: API Quick Reference

Author: Srikanth Patchava & EmbeddedOS Contributors

43.1 Top 50 EoS API Functions

#	Function	Header	Description
1	<code>eos_hal_init()</code>	<code>eos/hal.h</code>	Initialize the HAL with a backend
2	<code>eos_hal_deinit()</code>	<code>eos/hal.h</code>	Deinitialize the HAL
3	<code>eos_gpio_init()</code>	<code>eos/hal.h</code>	Configure a GPIO pin
4	<code>eos_gpio_write()</code>	<code>eos/hal.h</code>	Write digital value to GPIO pin
5	<code>eos_gpio_read()</code>	<code>eos/hal.h</code>	Read digital value from GPIO pin
6	<code>eos_gpio_toggle()</code>	<code>eos/hal.h</code>	Toggle GPIO pin state
7	<code>eos_gpio_set_irq()</code>	<code>eos/hal.h</code>	Set GPIO interrupt callback
8	<code>eos_uart_init()</code>	<code>eos/hal.h</code>	Initialize UART peripheral
9	<code>eos_uart_write()</code>	<code>eos/hal.h</code>	Write data to UART
10	<code>eos_uart_read()</code>	<code>eos/hal.h</code>	Read data from UART with timeout
11	<code>eos_spi_init()</code>	<code>eos/hal.h</code>	Initialize SPI peripheral
12	<code>eos_spi_transfer()</code>	<code>eos/hal.h</code>	Full-duplex SPI transfer
13	<code>eos_i2c_init()</code>	<code>eos/hal.h</code>	Initialize I2C peripheral
14	<code>eos_i2c_write()</code>	<code>eos/hal.h</code>	Write data to I2C device
15	<code>eos_i2c_read()</code>	<code>eos/hal.h</code>	Read data from I2C device
16	<code>eos_timer_init()</code>	<code>eos/hal.h</code>	Initialize hardware timer
17	<code>eos_timer_start()</code>	<code>eos/hal.h</code>	Start a timer
18	<code>eos_timer_stop()</code>	<code>eos/hal.h</code>	Stop a timer
19	<code>eos_delay_ms()</code>	<code>eos/hal.h</code>	Blocking delay in milliseconds
20	<code>eos_get_tick_ms()</code>	<code>eos/hal.h</code>	Get system tick in milliseconds
21	<code>eos_kernel_init()</code>	<code>eos/kernel.h</code>	Initialize the RTOS kernel
22	<code>eos_kernel_start()</code>	<code>eos/kernel.h</code>	Start the RTOS scheduler
23	<code>eos_task_create()</code>	<code>eos/kernel.h</code>	Create a new task
24	<code>eos_task_delete()</code>	<code>eos/kernel.h</code>	Delete a task
25	<code>eos_task_suspend()</code>	<code>eos/kernel.h</code>	Suspend a task

#	Function	Header	Description
26	<code>eos_task_resume()</code>	<code>eos/kernel.h</code>	Resume a suspended task
27	<code>eos_task_get_state()</code>	<code>eos/kernel.h</code>	Get current task state
28	<code>eos_task_get_name()</code>	<code>eos/kernel.h</code>	Get task name string
29	<code>eos_mutex_create()</code>	<code>eos/kernel.h</code>	Create a recursive mutex
30	<code>eos_mutex_lock()</code>	<code>eos/kernel.h</code>	Lock mutex with timeout
31	<code>eos_mutex_unlock()</code>	<code>eos/kernel.h</code>	Unlock a mutex
32	<code>eos_mutex_delete()</code>	<code>eos/kernel.h</code>	Delete a mutex
33	<code>eos_sem_create()</code>	<code>eos/kernel.h</code>	Create a counting semaphore
34	<code>eos_sem_wait()</code>	<code>eos/kernel.h</code>	Wait on semaphore with timeout
35	<code>eos_sem_post()</code>	<code>eos/kernel.h</code>	Post (signal) a semaphore
36	<code>eos_sem_get_count()</code>	<code>eos/kernel.h</code>	Get current semaphore count
37	<code>eos_queue_create()</code>	<code>eos/kernel.h</code>	Create a message queue
38	<code>eos_queue_send()</code>	<code>eos/kernel.h</code>	Send message to queue
39	<code>eos_queue_receive()</code>	<code>eos/kernel.h</code>	Receive message from queue
40	<code>eos_queue_is_empty()</code>	<code>eos/kernel.h</code>	Check if queue is empty
41	<code>eos_crypto_sha256()</code>	<code>eos/crypto.h</code>	Compute SHA-256 hash
42	<code>eos_crypto_aes_encrypt()</code>	<code>eos/crypto.h</code>	AES-256 encryption
43	<code>eos_crypto_aes_decrypt()</code>	<code>eos/crypto.h</code>	AES-256 decryption
44	<code>eos_net_socket_create()</code>	<code>eos/net.h</code>	Create a network socket
45	<code>eos_net_connect()</code>	<code>eos/net.h</code>	Connect socket to remote
46	<code>eos_net_send()</code>	<code>eos/net.h</code>	Send data over socket
47	<code>eos_net_recv()</code>	<code>eos/net.h</code>	Receive data from socket
48	<code>eos_fs_open()</code>	<code>eos/filesystem.h</code>	Open a file
49	<code>eos_fs_read()</code>	<code>eos/filesystem.h</code>	Read from a file
50	<code>eos_fs_write()</code>	<code>eos/filesystem.h</code>	Write to a file

43.2 Return Codes

Code	Value	Meaning
<code>EOS_OK</code>	0	Success
<code>EOS_ERR_INVALID</code>	-1	Invalid parameter
<code>EOS_ERR_TIMEOUT</code>	-2	Operation timed out
<code>EOS_ERR_NOMEM</code>	-3	Out of memory
<code>EOS_ERR_BUSY</code>	-4	Resource busy
<code>EOS_ERR_IO</code>	-5	I/O error
<code>EOS_KERN_OK</code>	0	Kernel operation OK
<code>EOS_KERN_INVALID</code>	-1	Invalid kernel param
<code>EOS_KERN_TIMEOUT</code>	-2	Kernel timeout
<code>EOS_KERN_EMPTY</code>	-3	Queue/resource empty

Chapter 44

Appendix B: Product Profiles

Author: Srikanth Patchava & EmbeddedOS Contributors

44.1 All EoS Product Profiles

EoS includes 48 product profile headers in the `products/` directory. Each profile enables the specific HAL peripherals, kernel features, and services needed for that product category.

#	Profile	Header File	Target Domain
1	adapter	<code>adapter.h</code>	Protocol adapter/bridge
2	aerospace	<code>aerospace.h</code>	Aerospace flight systems
3	ai_edge	<code>ai_edge.h</code>	AI edge inference device
4	automotive	<code>automotive.h</code>	Automotive ECU
5	autonomous	<code>autonomous.h</code>	Autonomous vehicle/robot
6	banking	<code>banking.h</code>	Banking/fintech terminal
7	cast_device	<code>cast_device.h</code>	Media casting device
8	cockpit	<code>cockpit.h</code>	Vehicle cockpit/dashboard
9	computer	<code>computer.h</code>	General-purpose computer
10	crypto_hw	<code>crypto_hw.h</code>	Hardware security module
11	desktop	<code>desktop.h</code>	Desktop computer
12	diagnostic	<code>diagnostic.h</code>	Diagnostic/test equipment
13	drone	<code>drone.h</code>	UAV/drone flight controller
14	ev	<code>ev.h</code>	Electric vehicle
15	fitness	<code>fitness.h</code>	Fitness tracker
16	gaming	<code>gaming.h</code>	Gaming console/handheld
17	gateway	<code>gateway.h</code>	IoT gateway
18	ground_control	<code>ground_control.h</code>	Ground control station
19	hmi	<code>hmi.h</code>	Human-machine interface
20	home_camera	<code>home_camera.h</code>	Smart home camera
21	industrial	<code>industrial.h</code>	Industrial controller
22	infotainment	<code>infotainment.h</code>	Vehicle infotainment
23	iot	<code>iot.h</code>	IoT sensor node

#	Profile	Header File	Target Domain
24	iptv_stb	iptv_stb.h	IPTV set-top box
25	medical	medical.h	Medical device
26	mobile	mobile.h	Mobile/smartphone
27	plc	plc.h	Programmable logic controller
28	pos	pos.h	Point-of-sale terminal
29	printer	printer.h	Printer controller
30	robot	robot.h	Robotics platform
31	router	router.h	Network router
32	satellite	satellite.h	Satellite system
33	security_cam	security_cam.h	Security camera
34	server	server.h	Edge server
35	smart_home	smart_home.h	Smart home hub
36	smart_speaker	smart_speaker.h	Smart speaker
37	smart_tv	smart_tv.h	Smart TV
38	space_comm	space_comm.h	Space communication system
39	telecom	telecom.h	Telecom equipment
40	telemedicine	telemedicine.h	Telemedicine device
41	thermostat	thermostat.h	Smart thermostat
42	tv_os	tv_os.h	TV operating system
43	vacuum	vacuum.h	Robotic vacuum cleaner
44	vbox_test	vbox_test.h	VirtualBox test environment
45	voice	voice.h	Voice assistant device
46	watch	watch.h	Smartwatch
47	wearable	wearable.h	Generic wearable device
48	xr_headset	xr_headset.h	XR/VR/AR headset

44.2 Usage

Select a product profile at build time:

```
cmake -B build -DEOS_PRODUCT=robot
cmake --build build
```

Each profile header defines `EOS_ENABLE_*` flags that control which HAL peripherals, kernel features, and services are compiled into the firmware.

Chapter 45

Appendix C: Pin Reference Tables

Author: Srikanth Patchava & EmbeddedOS Contributors

45.1 STM32F407VGT6 (LQFP-100)

45.1.1 Common Peripheral Pin Assignments

Function	Pin	AF	Notes
USART2_TX	PA2	AF7	Debug console
USART2_RX	PA3	AF7	Debug console
SPI1_SCK	PA5	AF5	SPI bus clock
SPI1_MISO	PA6	AF5	SPI bus data in
SPI1_MOSI	PA7	AF5	SPI bus data out
I2C1_SCL	PB6	AF4	I2C bus clock
I2C1_SDA	PB7	AF4	I2C bus data
LED_GREEN	PD12	GPIO	User LED (active high)
LED_ORANGE	PD13	GPIO	User LED (active high)
LED_RED	PD14	GPIO	User LED (active high)
LED_BLUE	PD15	GPIO	User LED (active high)
USER_BUTTON	PA0	GPIO	Blue user button
USB_DM	PA11	AF10	USB OTG FS
USB_DP	PA12	AF10	USB OTG FS
CAN1_RX	PD0	AF9	CAN bus receive
CAN1_TX	PD1	AF9	CAN bus transmit
ADC1_IN0	PA0	Analog	ADC channel 0
DAC1_OUT	PA4	Analog	DAC output
TIM1_CH1	PE9	AF1	PWM output
SWD_IO	PA13	AF0	Debug (SWDIO)
SWD_CLK	PA14	AF0	Debug (SWCLK)

45.1.2 Memory Map

Region	Start	End	Size
Flash	0x0800_0000	0x080F_FFFF	1 MB
SRAM1	0x2000_0000	0x2001_FFFF	128 KB
SRAM2	0x2002_0000	0x2002_FFFF	64 KB
CCM	0x1000_0000	0x1000_FFFF	64 KB
Peripherals	0x4000_0000	0x5FFF_FFFF	

45.2 nRF52840 (QFN-73)

45.2.1 Common Peripheral Pin Assignments

Function	Pin	Notes
UART_TX	P0.06	Default UART TX
UART_RX	P0.08	Default UART RX
SPI0_SCK	P0.27	SPI clock
SPI0_MOSI	P0.26	SPI data out
SPI0_MISO	P1.08	SPI data in
TWI0_SCL	P0.11	I2C clock
TWI0_SDA	P0.12	I2C data
LED1	P0.13	User LED 1
LED2	P0.14	User LED 2
LED3	P0.15	User LED 3
LED4	P0.16	User LED 4
BUTTON1	P0.11	User button 1
BUTTON2	P0.12	User button 2
USB_D+	-	Internal USB
USB_D-	-	Internal USB
NFC1	P0.09	NFC antenna 1
NFC2	P0.10	NFC antenna 2
RESET	P0.18	Active low reset

45.2.2 Memory Map

Region	Start	End	Size
Flash	0x0000_0000	0x000F_FFFF	1 MB
SRAM	0x2000_0000	0x2003_FFFF	256 KB
Peripherals	0x4000_0000	0x4001_FFFF	

45.3 Raspberry Pi 4 (BCM2711)

45.3.1 GPIO Header (40-pin)

Pin	GPIO	Function	Alt Functions
3	GPIO2	I2C1_SDA	SDA1
5	GPIO3	I2C1_SCL	SCL1
8	GPIO14	UART0_TX	TXD0
10	GPIO15	UART0_RX	RXD0
11	GPIO17	General GPIO	SPI1_CE1
12	GPIO18	PCM_CLK / PWM0	
13	GPIO27	General GPIO	
15	GPIO22	General GPIO	
16	GPIO23	General GPIO	
18	GPIO24	General GPIO	
19	GPIO10	SPI0_MOSI	
21	GPIO9	SPI0_MISO	
23	GPIO11	SPI0_SCLK	
24	GPIO8	SPI0_CE0	
26	GPIO7	SPI0_CE1	
29	GPIO5	General GPIO	
31	GPIO6	General GPIO	
32	GPIO12	PWM0	
33	GPIO13	PWM1	
35	GPIO19	PCM_FS / SPI1	
36	GPIO16	General GPIO	
37	GPIO26	General GPIO	
38	GPIO20	SPI1_MOSI	
40	GPIO21	SPI1_SCLK	

45.3.2 Power Pins

Pin(s)	Function
1, 17	3.3V Power
2, 4	5V Power
6,9,14,20,25,30,34,39	Ground

Chapter 46

Appendix D: Glossary

Author: Srikanth Patchava & EmbeddedOS Contributors

Term	Definition
ADC	Analog-to-Digital Converter — converts analog signals to digital values
AMP	Asymmetric Multi-Processing — cores run different code/OS instances
API	Application Programming Interface
ASIL	Automotive Safety Integrity Level (ISO 26262: A through D)
BCI	Brain-Computer Interface — direct neural signal input
BLE	Bluetooth Low Energy — low-power wireless protocol
BOM	Bill of Materials — list of components for manufacturing
BSP	Board Support Package — hardware-specific code for a target board
CAN	Controller Area Network — automotive/industrial serial bus
CGM	Continuous Glucose Monitor — real-time blood sugar sensor
CI/CD	Continuous Integration / Continuous Deployment
CRC	Cyclic Redundancy Check — error-detecting code
DAC	Digital-to-Analog Converter
DAL	Design Assurance Level (DO-178C: A through E)
DMA	Direct Memory Access — hardware-managed data transfer
DSP	Digital Signal Processing/Processor
EEPROM	Electrically Erasable Programmable Read-Only Memory
EoS	Embedded Operating System — the EmbeddedOS platform
eVTOL	Electric Vertical Take-Off and Landing aircraft
FFT	Fast Fourier Transform — frequency domain analysis
FMCW	Frequency-Modulated Continuous Wave — radar modulation technique
FPU	Floating Point Unit — hardware float acceleration
GPIO	General-Purpose Input/Output — digital pin interface
HAL	Hardware Abstraction Layer — platform-independent hardware API
HMAC	Hash-based Message Authentication Code
HRV	Heart Rate Variability — cardiac rhythm variation
I2C	Inter-Integrated Circuit — two-wire serial bus
IPC	Inter-Process Communication

Term	Definition
ISR	Interrupt Service Routine — interrupt handler function
JTAG	Joint Test Action Group — debug/programming interface
LDO	Low-Dropout Regulator — linear voltage regulator
LLM	Large Language Model — AI language model
LVGL	Light and Versatile Graphics Library — embedded GUI framework
MCU	Microcontroller Unit — single-chip embedded processor
MISRA	Motor Industry Software Reliability Association — C coding standard
MQTT	Message Queuing Telemetry Transport — IoT messaging protocol
MPU	Memory Protection Unit — hardware memory isolation
NPU	Neural Processing Unit — AI inference accelerator
NOR	NOR Flash — byte-addressable non-volatile memory
OBD-II	On-Board Diagnostics II — vehicle diagnostic standard
ONNX	Open Neural Network Exchange — model interchange format
OTA	Over-The-Air — wireless firmware/software update
PCB	Printed Circuit Board
PID	Proportional-Integral-Derivative — control loop algorithm
PMIC	Power Management IC — integrated power regulation
PPG	Photoplethysmography — optical heart rate sensing
PWM	Pulse Width Modulation — variable duty-cycle digital output
QSPI	Quad SPI — high-speed serial flash interface
RAG	Retrieval-Augmented Generation — AI technique combining search and LLM
RTOS	Real-Time Operating System — OS with deterministic timing guarantees
SBOM	Software Bill of Materials — component inventory for supply chain
SBC	Single-Board Computer (e.g., Raspberry Pi)
SIL	Safety Integrity Level (IEC 61508: 1 through 4)
SIW	Substrate Integrated Waveguide — PCB-integrated waveguide technology
SMP	Symmetric Multi-Processing — all cores run same OS instance
SPI	Serial Peripheral Interface — synchronous serial bus
SWD	Serial Wire Debug — ARM debug interface (2-wire JTAG)
TLS	Transport Layer Security — encrypted network communication
UART	Universal Asynchronous Receiver-Transmitter — serial communication
V2X	Vehicle-to-Everything — vehicular communication standard
WCET	Worst-Case Execution Time — maximum execution time analysis

Chapter 47

Appendix E: Bibliography and Standards References

Author: Srikanth Patchava & EmbeddedOS Contributors

47.1 Safety and Functional Safety

Standard	Title	Relevance to EoS
IEC 61508:2010	Functional Safety of E/E/PE Safety-Related Systems	Industrial safety (SIL 1-4)
ISO 26262:2018	Road Vehicles — Functional Safety	Automotive safety (ASIL A-D)
DO-178C	Software Considerations in Airborne Systems	Aerospace DAL A-E
DO-254	Design Assurance Guidance for Airborne Electronic HW	Aerospace hardware
IEC 62304:2006	Medical Device Software — Lifecycle Processes	Medical device software
ISO 14971:2019	Medical Devices — Risk Management	Medical risk assessment

47.2 Systems and Software Engineering

Standard	Title	Relevance to EoS
ISO/IEC/IEEE 15288:2023	Systems and Software Engineering — System Life Cycle	System lifecycle
ISO/IEC 12207:2017	Systems and Software Engineering — Software Life Cycle	Software lifecycle
ISO/IEC/IEEE 42010:2022	Architecture Description	Architecture documentation
IEEE 1003.1 (POSIX)	Portable Operating System Interface	OS API compatibility

47.3 Software Quality

Standard	Title	Relevance to EoS
ISO/IEC 25000:2014	Systems and Software Quality Requirements (SQuaRE)	Quality model framework
ISO/IEC 25010:2023	Quality Model	Quality characteristics
ISO/IEC 25040:2011	Quality Evaluation Process	Evaluation methodology

47.4 Security

Standard	Title	Relevance to EoS
ISO/IEC 27001:2022	Information Security Management Systems	Security management
ISO/IEC 15408 FIPS 140-3	Common Criteria for IT Security Evaluation Security Requirements for Cryptographic Modules	Security evaluation Crypto module validation
IEC 62443	Industrial Automation and Control Systems Security	Industrial cybersecurity

47.5 Coding Standards

Standard	Title	Relevance to EoS
MISRA C:2012	Guidelines for the Use of the C Language	Safety-critical C coding
CERT C	SEI CERT C Coding Standard	Secure C coding
CWE Top 25	Common Weakness Enumeration	Vulnerability avoidance

47.6 Communication and Connectivity

Standard	Title	Relevance to EoS
IEEE 802.11	Wireless LAN (Wi-Fi)	Wi-Fi connectivity
IEEE 802.15.4	Low-Rate Wireless PAN	Zigbee, Thread
Bluetooth 5.x	Bluetooth Core Specification	BLE communication
ISO 11898	Controller Area Network (CAN)	CAN bus automotive
SAE J1939	CAN for Commercial Vehicles	Heavy vehicle CAN
ISO 15765	Diagnostics on CAN (UDS)	OBD-II diagnostics

47.7 Embedded and Real-Time

Standard	Title	Relevance to EoS
ARINC 653	Avionics Application Software Standard Interface	Aerospace RTOS partitioning
AUTOSAR	AUTomotive Open System ARchitecture	Automotive SW architecture
OSEK/VDX	Open Systems for Automotive Electronics	Automotive RTOS standard

47.8 Supply Chain and Compliance

Standard	Title	Relevance to EoS
ISO/IEC 20243:2018	Open Trusted Technology Provider Standard (O-TTPS)	Supply chain integrity
SPDX	Software Package Data Exchange	License compliance
CycloneDX	Software Bill of Materials Standard	SBOM format
OpenChain (ISO 5230)	Open Source License Compliance	OSS compliance

47.9 Recommended Reading

1. **Elecia White**, *Making Embedded Systems*, 2nd Ed. (O'Reilly, 2024)
2. **Jean Labrosse**, *MicroC/OS-III: The Real-Time Kernel* (Micrium, 2016)
3. **Jack Ganssle**, *The Art of Designing Embedded Systems*, 2nd Ed. (Newnes, 2008)
4. **Philip Koopman**, *Better Embedded System Software* (Drumnadrochit, 2010)
5. **Nancy Leveson**, *Engineering a Safer World* (MIT Press, 2012)
6. **ARM Ltd.**, *ARM Cortex-M4 Technical Reference Manual*
7. **RISC-V Foundation**, *RISC-V ISA Specification, Volume I* (2024)